

- `AuthenticationMiddleware` adds the user attribute to the request object
- `MessageMiddleware` adds "flash" message support
- `XFrameOptionsMiddleware` protects against X-Frame-Options header clickjacking attacks

The middleware modules are loaded in the order that they appear in the `MIDDLEWARE` list. This makes sense because we want to call the middleware that deals with initial security issues first so that dangerous requests are rejected before further processing occurs. Django also comes with several other middleware modules that perform important functions, such as using `gzip` file compression, redirect configuration, and web cache configuration.

This chapter is devoted to discussing two important aspects of stateful application development that are implemented as middleware components – `SessionMiddleware` and `AuthenticationMiddleware`.

The `process_request` method of `SessionMiddleware` adds a session object as an attribute of the request object. The `process_request` method of `AuthenticationMiddleware` adds a user object as an attribute of the request object.

It is possible to write a Django project without these layers of the middleware stack if a project does not require user authentication or a means of preserving the state of individual interactions. However, most of the default middleware plays an important role in application security. If you don't have a good reason for changing the middleware components, it is best to maintain these initial settings. The Admin app requires `SessionMiddleware`, `AuthenticationMiddleware`, and `MessageMiddleware` to run, and the Django server will throw errors such as these if the Admin app is installed without them:

```
django.core.management.base.SystemCheckError: SystemCheckError: System check
identified some issues:

ERRORS:
?: (admin.E408) 'django.contrib.auth.middleware.AuthenticationMiddleware' must be
in MIDDLEWARE in order to use the admin application.
?: (admin.E409) 'django.contrib.messages.middleware.MessageMiddleware' must be in
MIDDLEWARE in order to use the admin application.
?: (admin.E410) 'django.contrib.sessions.middleware.SessionMiddleware' must be in
MIDDLEWARE in order to use the admin application.
```

Now that we know about the middleware modules, let's look at one way we can enable authentication in our project using the authentication app's views and templates.

- AuthenticationMiddleware는 요청 객체에 user 속성 idx_9078fd37을 추가합니다. MessageMiddleware는 "flash" 메시지 지원을 추가합니다.
- XFrameOptionsMiddleware는 X-Frame-Options 헤더 클릭재킹 공격으로부터 보호합니다.

미들웨어 모듈은 MIDDLEWARE 목록에 나타나는 순서대로 로드됩니다. 이는 초기 보안 문제를 처리하는 미들웨어를 먼저 호출하여 위험한 요청이 추가 처리 전에 거부되도록 하기 때문에 의미가 있습니다. Django는 gzip 파일 압축, 리디렉션 구성 및 웹 캐시 구성을 수행하는 여러 다른 미들웨어 모듈도 함께 제공합니다.

이 장에서는 미들웨어 구성 요소로 구현된 상태 유지 애플리케이션 개발의 두 가지 중요한 측면인 SessionMiddleware와 AuthenticationMiddleware에 대해 논의합니다.

SessionMiddleware의 process_request 메서드는 요청 객체의 속성으로 세션 객체를 추가합니다. AuthenticationMiddleware의 process_request 메서드는 요청 객체의 속성으로 사용자 객체를 추가합니다.

사용자 인증이나 개별 상호작용의 상태를 유지할 수단이 필요하지 않은 프로젝트라면 이러한 미들웨어 스택의 계층 없이 Django 프로젝트를 작성하는 것이 가능합니다. 그러나 대부분의 기본 미들웨어는 애플리케이션 보안에서 중요한 역할을 합니다. 미들웨어 구성 요소를 변경할 좋은 이유가 없다면 이러한 초기 설정을 유지하는 것이 가장 좋습니다. Admin 앱은 SessionMiddleware, AuthenticationMiddleware 및 MessageMiddleware가 실행되도록 요구하며, Admin 앱이 이들 없이 설치되면 다음과 같은 오류가 발생합니다:

```
django.core.management.base.SystemCheckError: SystemCheckError: 시스템 검사
문제를 확인했습니다:

오류:
?: (admin.E408) 'django.contrib.auth.middleware.AuthenticationMiddleware'는 관리
애플리케이션을 사용하기 위해 MIDDLEWARE에 있어야 합니다.
?: (admin.E409) 'django.contrib.messages.middleware.MessageMiddleware'는 관리 애플
리케이션을 사용하기 위해 MIDDLEWARE에 있어야 합니다.
?: (admin.E410) 'django.contrib.sessions.middleware.SessionMiddleware'는 관리 애플
리케이션을 사용하기 위해 MIDDLEWARE에 있어야 합니다.
```

미들웨어 모듈에 대해 알게 되었으니, 인증 앱의 뷰와 템플릿을 사용하여 프로젝트에서 인증을 활성화할 수 있는 한 가지 방법을 살펴보겠습니다.

Implementing authentication views and templates

We encountered the login form on the Admin app in *Chapter 4, Introduction to Django Admin*. This is the authentication entry point for staff users who have access to the Admin app. We also need to create a login capability for ordinary users who want to give book reviews. Fortunately, the authentication app comes with the tools to make this possible.

As we work through the forms and views of the authentication app, we will encounter a lot of flexibility in its implementation. We are free to implement our own login pages, define either very simple or fine-grained security policies at the view level, and authenticate against external authorities.

The authentication app exists to accommodate a lot of different approaches to authentication so that Django doesn't rigidly enforce a single mechanism. For a first-time user encountering the documentation, this can be quite bewildering. For the most part in this chapter, we will follow Django's defaults, but some of the important configuration options will be noted.

A Django project's settings object contains attributes for login behavior. `LOGIN_URL` specifies the URL of the login page. `"/accounts/login/"` is the default value. `LOGIN_REDIRECT_URL` specifies the path to where a successful login is redirected. The default path is `"/accounts/profile/"`.

The authentication app supplies standard forms and views for carrying out typical authentication tasks. These forms are located in `django.contrib.auth.forms`, while the views are located in `django.contrib.auth.views`.

The views are referenced by these URL patterns present in `django.contrib.auth.urls`:

```
urlpatterns = [
    path('login/', views.LoginView.as_view(),
         name='login'),
    path('logout/', views.LogoutView.as_view(),
         name='logout'),
    path('password_change/',
         views.PasswordChangeView.as_view(),
         name='password_change'),
    path('password_change/done/',
         views.PasswordChangeDoneView.as_view(),
         name='password_change_done'),
    path('password_reset/',
         views.PasswordResetView.as_view(),
         name='password_reset'),
```

인증 뷰 및 템플릿 구현

우리는 4장, Django Admin 소개에서 Admin 앱의 로그인 양식을 만났습니다. 이것은 Admin 앱에 접근할 수 있는 직원 사용자를 위한 idx_35185719 인증 진입점입니다. 우리는 또한 책 리뷰를 작성하고자 하는 일반 사용자를 위한 로그인 기능을 만들어야 합니다. 다행히도 인증 앱은 이를 가능하게 하는 도구를 제공합니다.

인증 앱의 양식과 뷰를 진행하면서 우리는 그 구현에서 많은 유연성을 경험할 것입니다. 우리는 자신의 로그인 페이지를 구현할 자유가 있으며, 뷰 수준에서 매우 간단하거나 세분화된 보안 정책을 정의하고 외부 권한에 대해 인증할 수 있습니다.

인증 앱은 Django가 단일 메커니즘을 엄격하게 강제하지 않도록 다양한 인증 접근 방식을 수용하기 위해 존재합니다. 문서를 처음 접하는 사용자에게는 이것이 꽤 혼란스러울 수 있습니다. 이 장에서는 대부분 Django의 기본값을 따르겠지만, 몇 가지 중요한 구성 옵션은 언급될 것입니다.

Django 프로젝트의 설정 객체는 로그인 동작에 대한 속성을 포함합니다. LOGIN_URL은 로그인 페이지의 URL을 지정합니다. '/accounts/login/'이 기본값입니다.

LOGIN_REDIRECT_URL은 성공적인 로그인이 리디렉션되는 경로를 지정합니다. 기본 경로는 '/accounts/profile/'입니다.

인증 앱은 일반적인 인증 작업을 수행하기 위한 표준 양식과 뷰를 제공합니다. 이러한 양식은 django.contrib.auth.forms에 위치하고, 뷰는 django.contrib.auth.views에 위치합니다.

뷰는 django.contrib.auth.urls에 있는 이러한 URL 패턴으로 참조됩니다:

```
urlpatterns = [
    path('login/', views.LoginView.as_view(),
         name='login'),
    path('logout/', views.LogoutView.as_view(),
         name='logout'),
    path('password_change/',
         views.PasswordChangeView.as_view(),
         name='password_change'),
    path('password_change/done/',
         views.PasswordChangeDoneView.as_view(),
         name='password_change_done'),
    path('password_reset/',
         views.PasswordResetView.as_view(),
         name='password_reset'),
```



```

    path('password_reset/done/',
         views.PasswordResetDoneView.as_view(),
         name='password_reset_done'),
    path('reset/<uidb64>/<token>/',
         views.PasswordResetConfirmView.as_view(),
         name='password_reset_confirm'),
    path('reset/done/',
         views.PasswordResetCompleteView.as_view(),
         name='password_reset_complete'),
]

```

If this style of views looks unfamiliar, it is because they are class-based views rather than the function-based views that we have previously encountered. We will learn more about class-based views in *Chapter 11, Advanced Templates and Class-Based Views*. For now, note that the authentication app makes use of class inheritance to group the functionality of views and prevent a lot of repetitive coding.

If we want to maintain the default URLs and views that are presupposed by the authentication app and Django settings, we can include the authentication app's URLs in our project's `urlpatterns`.

By taking this approach, we have saved a lot of work. We only need to include the authentication app's URLs in our `<project>/urls.py` file and assign it to the `'accounts'` namespace. Designating this namespace ensures that our reverse URLs correspond to the default template values of the views. The `authlib` views contain code references to templates named `password_reset_done` and `password_reset_complete`, so their paths need to be explicitly included in `urlpatterns` too:

```

from django.contrib import admin, auth
...

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/",
         include(("django.contrib.auth.urls", "auth"),
                 namespace="accounts")),
    path("accounts/password_reset/done/",
         auth.views.PasswordResetDoneView.as_view(),
         name="password_reset_done"),
    path("accounts/reset/done/",
         auth.views.PasswordResetCompleteView.as_view(),

```

```

    path('password_reset/done/',
          views.PasswordResetDoneView.as_view(),
          name='password_reset_done'),
    path('reset/<uidb64>/<token>/',
          views.PasswordResetConfirmView.as_view(),
          name='password_reset_confirm'),
    path('reset/done/',
          views.PasswordResetCompleteView.as_view(),
          name='password_reset_complete'),
    ...

```

이러한 스타일의 뷰가 낯설게 느껴진다면, 그것은 우리가 이전에 접했던 함수 기반 뷰가 아니라 클래스 기반 뷰이기 때문입니다. 클래스 기반 뷰에 대해서는 11장, 고급 템플릿 및 클래스 기반 뷰에서 더 배울 것입니다. 지금은 `idx_f638839bauthentication` 앱이 뷰의 기능을 그룹화하고 많은 반복 코딩을 방지하기 위해 클래스 상속을 사용한다는 점에 주목하세요.

인증 앱과 Django 설정에서 전제된 기본 URL 및 뷰를 유지하려면, 인증 앱의 URL을 프로젝트의 `urlpatterns`에 포함시킬 수 있습니다.

이 접근 방식을 취함으로써 우리는 많은 작업을 절약했습니다. 우리는 인증 앱의 URL을 `<project>/urls.py` 파일에 포함시키고 'accounts' 네임스페이스에 할당하기만 하면 됩니다. 이 네임스페이스를 지정하면 우리의 역 URL이 뷰의 기본 템플릿 값에 해당하도록 보장됩니다. `authlib` 뷰는 `password_reset_done` 및 `password_reset_complete`라는 템플릿에 대한 코드 참조를 포함하고 있으므로, 이들의 경로도 `urlpatterns`에 명시적으로 포함되어야 합니다:

```

from django.contrib import admin, auth
...

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/",
          include(("django.contrib.auth.urls", "auth"),
                  namespace="accounts")),
    path("accounts/password_reset/done/",
          auth.views.PasswordResetDoneView.as_view(),
          name="password_reset_done", ),
    path("accounts/reset/done/",
          auth.views.PasswordResetCompleteView.as_view(),

```

```
        name="password_reset_complete",),
    path("", reviews.views.index),
    path("book-search/", reviews.views.book_search,
        name="book_search"),
    path("", include("reviews.urls")),
]
```

Though the authentication app comes with forms and views, it lacks the templates needed to render these components as HTML. *Figure 9.1* lists the templates that we require to implement the authentication functionality in our project. Fortunately, the Admin app does implement a set of templates that we can utilize for our purposes.

Note

When we say *Django source code*, it's the directory where your Django installation resides. If you installed Django in a virtual environment (as detailed in *Chapter 1*), you can find these template files at <name of your virtual environment>/lib/python3.X/site-packages/django/contrib/admin/templates/registration/. Provided your virtual environment is activated and Django is installed in it, you can also retrieve the complete path to the site-packages directory by running the python `-c "import sys; print(sys.path)"` command in a terminal.

We could just copy the template files from the Django source code in the `django/contrib/admin/templates/registration` directory and `django/contrib/admin/templates/admin/login.html` into our project's `templates/registration` directory:

```

        name="password_reset_complete", ),
        path("", reviews.views.index),
        path("book-search/", reviews.views.book_search,
            name="book_search"),
        path("", include("reviews.urls")),
    ]

```

인증 `idx_e0c0b0bc` 앱은 양식과 뷰를 제공하지만, 이러한 구성 요소를 HTML로 렌더링하는 데 필요한 `idx_f33dd9c` 템플릿이 부족합니다. 그림 9.1은 프로젝트에서 인증 기능을 구현하는 데 필요한 템플릿을 나열합니다. 다행히도, Admin 앱은 우리의 목적을 위해 활용할 수 있는 템플릿 세트를 구현합니다.

참고

Django 소스 코드를 말할 때, 그것은 당신의 Django 설치가 위치한 디렉토리입니다. 만약 당신이 가상 환경에 Django를 설치했다면(1장에서 자세히 설명됨), `<name of your virtual environment>/lib/python3.X/site-packages/django/contrib/admin/templates/registration/`에서 이러한 템플릿 파일을 찾을 수 있습니다. 당신의 가상 환경이 활성화되어 있고 그 안에 Django가 설치되어 있다면, 터미널에서 `python-c "import sys; print(sys.path)"` 명령어를 실행하여 `site-packages` 디렉토리의 전체 경로를 가져올 수도 있습니다.

우리는 `django/contrib/admin/templates/registration` 디렉토리와 `django/contrib/admin/templates/admin/login.html`에서 Django 소스 코드의 템플릿 파일을 복사하여 우리의 프로젝트의 `templates/registration` 디렉토리에 넣을 수 있습니다:

Template Path

```
templates/registration/login.html
templates/registration/password_reset_email.html
templates/registration/password_change_form.html
templates/registration/password_change_done.html
templates/registration/password_reset_form.html
templates/registration/password_reset_done.html
templates/registration/password_reset_confirm.html
templates/registration/password_reset_complete.html
templates/registration/logged_out.html
```

Figure 9.1: Default paths for authentication templates

Note

We only need to copy the templates that are dependencies for the views, and we should avoid copying the `base.html` or `base_site.html` files.

This gives a promising result at first, but as they stand, the admin templates do not meet our precise needs, as we can see from the login page (*Figure 9.2*):

Template Path

```
templates/registration/login.html
templates/registration/password_reset_email.html
templates/registration/password_change_form.html
templates/registration/password_change_done.html
templates/registration/password_reset_form.html
templates/registration/password_reset_done.html
templates/registration/password_reset_confirm.html
templates/registration/password_reset_complete.html
templates/registration/logged_out.html
```

그림 9.1: 인증 템플릿의 기본 경로

노트

우리는 뷰에 대한 의존성인 템플릿만 복사하면 되며, base.html 또는 base_site.html 파일을 복사하는 것은 피해야 합니다.

이는 처음에는 유망한 결과를 주지만, 현재 상태로는 관리자 템플릿이 우리의 정확한 요구를 충족하지 못합니다. 로그인 페이지(그림 9.2)에서 볼 수 있습니다:

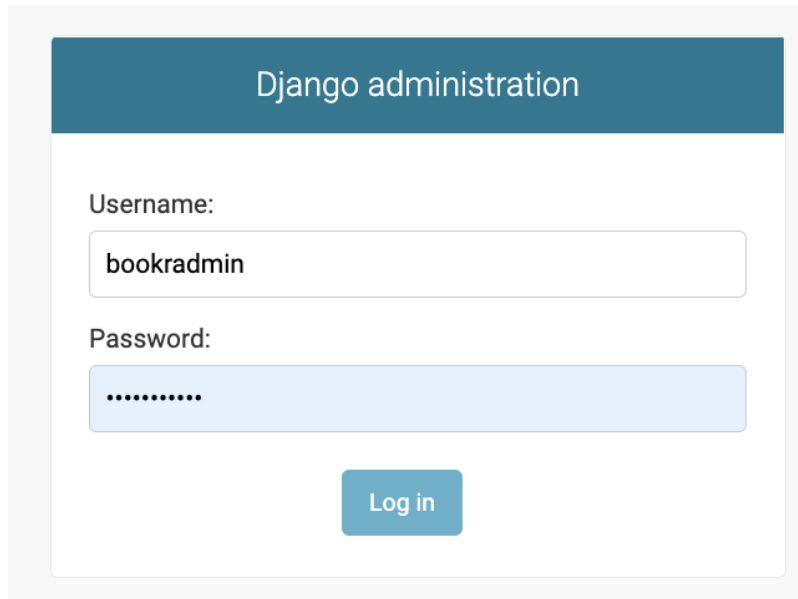


Figure 9.2: A first attempt at a user login screen

As these authentication pages inherit from the Admin app's `admin/base_site.html` template, they follow the style of the Admin app. We would prefer for these pages to follow the style of the bookr project that we have developed. We can do this by following these three steps on each Django template that we have copied from the Admin app to our project:

1. The first change that needs to be made is to replace the `{% extends "admin/base_site.html" %}` tag with `{% extends "base.html" %}`, as we want to inherit from the bookr project's base template.
2. Given that `template/base.html` only contains the `title`, `brand`, and `content` block definitions, we should examine the additional block substitutions from our templates in the bookr folder. We are not using the content from the `userlinks` and `breadcrumbs` blocks in our app's template layouts, so these blocks can be removed entirely. Some of these blocks, such as `content_title` and `reset_link`, contain HTML content that is relevant to our application, so they should be retained.

For example, the `password_change_done.html` template contains the `userlinks` and `breadcrumbs` blocks:

```
{% extends "admin/base_site.html" %}
{% load i18n %}
```

The image shows a Django administration login interface. At the top, there is a dark blue header with the text 'Django administration'. Below this, the form is white with a light gray border. It contains two input fields: 'Username:' with the value 'bookradmin' and 'Password:' with masked characters '.....'. A blue 'Log in' button is positioned below the password field.

그림 9.2: 사용자 로그인 화면에 대한 첫 번째 시도

이 인증 페이지들은 Admin 앱의 `admin/base_site.html` 템플릿을 상속받기 때문에 Admin 앱의 스타일을 따릅니다. 우리는 이 페이지들이 우리가 개발한 bookr 프로젝트의 스타일을 따르기를 원합니다. 이를 위해 Admin 앱에서 프로젝트로 복사한 각 Django 템플릿에서 다음 세 가지 단계를 따를 수 있습니다:

1. 첫 번째로 변경해야 할 것은 `{% extends "admin/base_site.html" %}` 태그를 `{% extends "base.html" %}`로 교체하는 것입니다. 우리는 bookr 프로젝트의 기본 템플릿을 상속받기를 원합니다.
2. `template/base.html`은 제목, 브랜드 및 콘텐츠 블록 정의만 포함하고 있으므로, bookr 폴더의 템플릿에서 추가 블록 대체를 검토해야 합니다. 우리는 앱의 템플릿 레이아웃에서 `userlinks` 및 `breadcrumbs` 블록의 콘텐츠를 사용하지 않으므로 이 블록들은 완전히 제거할 수 있습니다.

이 블록 중 일부는 `content_title` 및 `reset_link`와 같이 우리 애플리케이션과 관련된 HTML 콘텐츠를 포함하고 있으므로 유지해야 합니다.

예를 들어, `password_change_done.html` 템플릿은 `userlinks` 및 `breadcrumbs` 블록을 포함하고 있습니다:

```
{% extends "admin/base_site.html" %}
{% load i18n %}
```



```
{% block userlinks %}
    {% url 'django-admindocs-docroot' as docsroot %}{% if docsroot %}<a
href="{% docsroot %}">{% translate 'Documentation' %}</a> / {% endif %}{%
translate 'Change password' %} /
    <form id="logout-form" ... ></form>
    {% include "admin/color_theme_toggle.html" %}
{% endblock %}
{% block breadcrumbs %} <div class="breadcrumbs">
<a href="{% url 'admin:index' %}">Home</a>
> Password change</div> {% endblock %}
{% block content %}
<p>{% translate 'Your password was changed.' %}</p>
{% endblock %}
```

It will be simplified to this template in the bookr project:

```
{% extends "base.html" %}
{% load i18n %}

{% block content %}
<p>{% translate 'Your password was changed.' %}</p>
{% endblock %}
```

3. Likewise, there are reverse URL patterns that need to change to reflect the current path, so {% url 'login' %} gets replaced with {% url 'accounts:login' %}.

Given these considerations, the next exercise will focus on transforming the Admin app's login template into a login template for the bookr project.

Note

The `i18n` module is used for creating multilingual content. If you intend to develop multilingual content for your website, leave the `i18n` import, `translate` tags, and `translateblock` statements in the templates. For brevity, we will not be covering those in detail in this chapter.

```
{% block userlinks %}
    {% url 'django-admindocs-docroot' as docsroot %}{% if docsroot %}
    <ahref="{{ docsroot }}">{% translate '문서' %}</a> / {% endif %}{%

translate '비밀번호 변경' %} /
    <form id="logout-form" ... ></form>
    {% include "admin/color_theme_toggle.html" %}
{% endblock %}
{% block breadcrumbs %} <div class="breadcrumbs">
<a href="{% url 'admin:index' %}">홈</a>
> 비밀번호 변경</div> {% endblock %}
{% block content %}
<p>{% translate '비밀번호가 변경되었습니다.' %}</p>
{% endblock %}
```

이것은 bookr 프로젝트의 idx_84e1aab7 이 idx_c5110182 템플릿으로 단순화됩니다:

```
{% extends "base.html" %}
{% load i18n %}

{% block content %}
<p>{% translate '비밀번호가 변경되었습니다.' %}</p>
{% endblock %}
```

3. 마찬가지로, 현재 경로를 반영하기 위해 변경해야 할 역 URL 패턴이 있으며, {% url 'login' %}은 {% url 'accounts:login' %}으로 대체됩니다.

이러한 고려 사항을 감안할 때, 다음 연습은 Admin 앱의 로그인 템플릿을 bookr 프로젝트의 로그인 템플릿으로 변환하는 데 중점을 둘 것입니다.

참고

i18n 모듈은 다국어 콘텐츠를 생성하는 데 사용됩니다. 웹사이트에 대해 다국어 콘텐츠를 개발할 계획이라면 i18n 가져오기, 번역 태그 및 템플릿의 번역 블록 문을 남겨 두십시오. 간결함을 위해 이 장에서는 이를 자세히 다루지 않을 것입니다.

Exercise 9.01 — Repurposing the Admin app login template

We started this chapter without a login page for our project. By adding the URL patterns for authentication and copying the templates from the Admin app to our project, we can implement the functionality of a login page. But this login page is not satisfactory as it is directly copied from the Admin app and is disconnected from the Bookr design. In this exercise, we will follow the steps needed to repurpose the Admin app's login template for our project. The new login template will need to inherit its style and format directly from the bookr project's `templates/base.html` file:

1. Create a directory inside your project for `templates/registration`.

The Admin login template is located in the Django source directory at the `django/contrib/admin/templates/admin/login.html` path. It begins with an `extends` tag, a `load` tag, importing the `i18n` and `static` modules, and a series of block extensions that override the blocks defined in the child template, `django/contrib/admin/templates/admin/base.html`. A truncated snippet of the `login.html` file is shown in the following code block:

```
{% extends "admin/base_site.html" %}
{% load i18n static %}
{% block title %}{% if form.errors %}
{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block extrastyle %}...{% endblock %}
{% block bodyclass %}...{% endblock %}
{% block usertools %}{% endblock %}
{% block nav-global %}{% endblock %}
{% block content_title %}{% endblock %}
{% block breadcrumbs %}{% endblock %}
{% block content %}
{% if form.errors and not form.non_field_errors %}
... {% endif %}
<div id="content-main">
...
<form action="{{ app_path ...}}{% csrf_token %}
...
</form>
</div>
{% endblock %}
```

연습 9.01 – 관리자 앱 로그인 템플릿 재사용

우리는 프로젝트에 로그인 페이지 없이 이 `idx_ff625f1`장을 시작했습니다. 인증을 위한 URL 패턴을 추가하고 관리자 앱에서 템플릿을 프로젝트로 복사함으로써 로그인 페이지의 기능을 구현할 수 있습니다. 그러나 이 로그인 페이지는 관리자 앱에서 직접 복사된 것이기 때문에 만족스럽지 않으며 Bookr 디자인과 연결되어 있지 않습니다. 이 연습에서는 관리자 앱의 로그인 템플릿을 프로젝트에 맞게 재사용하는 데 필요한 단계를 따를 것입니다. 새로운 로그인 템플릿은 bookr 프로젝트의 `templates/base.html` 파일에서 스타일과 형식을 직접 상속받아야 합니다:

1. 프로젝트 내에 `templates/registration`을 위한 디렉토리를 만듭니다.

관리자 로그인 템플릿은 Django 소스 디렉토리의

`django/contrib/admin/templates/admin/login.html` 경로에 위치해 있습니다. 이 템플릿은 `extends` 태그, `i18n` 및 `static` 모듈을 가져오는 `load` 태그, 그리고 자식 템플릿인 `django/contrib/admin/templates/admin/base.html`에서 정의된 블록을 재정의하는 일련의 블록 확장으로 시작됩니다. `login.html` 파일의 잘린 스니펫은 다음 코드 블록에 표시됩니다:

```
{% extends "admin/base_site.html" %}
{% load i18n static %}
{% block title %}{% if form.errors %}{%
translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}{% block
extrastyle %}...{% endblock %}{% block
bodyclass %}...{% endblock %}{% block
usertools %}{% endblock %}{% block nav-
global %}{% endblock %}{% block
content_title %}{% endblock %}{% block
breadcrumbs %}{% endblock %}{% block
content %}
{% if form.errors and not form.non_field_errors %}
... {% endif %}
<div id="content-main">
...
<form action="{{ app_path ...}}{% csrf_token %}
...
</form>
</div>
{% endblock %}
```

2. Copy this Admin login template, `django/contrib/admin/templates/admin/login.html`, into `templates/registration` and begin editing the file using PyCharm.
3. As the login template you are editing is located at `templates/registration/login.html` and extends the base template (`templates/base.html`), replace the argument of the `extends` tag at the top of `templates/registration/login.html` with the following:

```
{% extends "base.html" %}
```

4. We don't need most of the contents of this file. Just retain the content block, which contains the login form. The remainder of the template will consist of loading the `i18n` and static tag libraries:

```
{% load i18n static %}
{% block title %}{% if form.errors %}{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block content %}
...
{% endblock %}
```

5. Now, you must replace the paths and reverse URL patterns in `templates/registration/login.html` with ones that are appropriate to your project. As you don't have an `app_path` variable defined in your template, it needs to be replaced with the reverse URL for the login, `'accounts:login'`. So, consider the following line:

```
<form action="{{ app_path }}" method="post"
id="login-form">
```

This line changes as follows:

```
<form action="{% url 'accounts:login' %}"
method="post" id="login-form">
```

No `'admin_password_reset'` has been defined in your project paths, so it will be replaced with `'accounts:password_reset'`.

2. 이 Admin 로그인 템플릿인 `django/contrib/admin/templates/admin/login.html`을 `templates/registration`으로 복사하고 PyCharm을 사용하여 파일을 편집하기 시작하세요. 편집 중
3. 인 로그인 템플릿은 `templates/registration/login.html`에 위치하고 기본 템플릿(`templates/base.html`)을 확장하므로, `templates/registration/login.html`의 맨 위에 있는 `extends` 태그의 인수를 다음으로 교체하세요:

```
{% extends "base.html" %}
```

4. 이 파일의 대부분의 내용은 필요하지 않습니다. 로그인 양식이 포함된 콘텐츠 블록만 유지하세요. 템플릿의 나머지 부분은 `i18n` 및 `static` 태그 라이브러리를 로드하는 것으로 구성됩니다:

```
{% load i18n static %}
{% block title %}{% if form.errors %}{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block content %}
...
{% endblock %}
```

5. 이제 `templates/` 내의 경로와 역방향 URL 패턴을 교체해야 합니다. `registration/login.html`을(를) `idx_b2239c3aproject`에 적합한 것으로 교체하세요. 템플릿에 `app_path` 변수가 정의되어 있지 않으므로 로그인에 대한 역 URL인 `'accounts:login'`으로 교체해야 합니다. 따라서 다음 줄을 고려하세요:

```
<form action="{{ app_path }}" method="post"
id="login-form">
```

이 줄은 다음과 같이 변경됩니다:

```
<form action="{% url 'accounts:login' %}"
method="post" id="login-form">
```

당신의 프로젝트 경로에 `'admin_password_reset'`이 정의되어 있지 않으므로, `'accounts:password_reset'`으로 대체됩니다.

Consider the following line:

```
{% url 'admin_password_reset' as password_reset_url %}
```

This changes to the following line:

```
{% url 'accounts:password_reset'
      as password_reset_url %}
```

Your login template will look as follows:

```
{% extends "base.html" %}
{% load i18n static %}
{% block title %}{% if form.errors %}{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block content %}
{% if form.errors and not form.non_field_errors %}
<p class="errornote">
{% if form.errors.items|length == 1 %}{% translate
"Please correct the error
below." %}{% else %}{% translate "Please correct
the errors below." %}{% endif %}
</p>
{% endif %}
```

You can find the complete code for this file in the Chapter09 folder in this book's GitHub repo.

6. As mentioned previously, to use the standard Django authentication views, we must add URL mappings to them. Open the `urls.py` file in the bookr project directory, ensure that `auth` is imported from `django.contrib`, and add the three paths to the URL patterns:

```
from django.contrib import admin, auth
...
urlpatterns = [
    ...
    path("accounts/",
         include(("django.contrib.auth.urls", "auth"),
                 namespace="accounts")),
    path("accounts/password_reset/done/",
```

다음 줄을 고려하세요:

```
{% url 'admin_password_reset' as password_reset_url %}
```

이것은 idx_89542682를 다음 줄로 변경합니다:

```
{% url 'accounts:password_reset'
as password_reset_url %}
```

귀하의 로그인 템플릿은 다음과 같이 보일 것입니다:

```
{% extends "base.html" %}
{% load i18n static %}
{% block title %}{% if form.errors %}{% translate "Error:" %} {% endif %}
{{ block.super }}{% endblock %}
{% block content %}
{% if form.errors and not form.non_field_errors %}
<p class="errornote">{% if form.errors.items|
length == 1 %}{% translate "오류를 수정해 주세요"
아래에." %}{% else %}{% translate "아래의 오류를
수정해 주세요." %}{% endif %}
</p>
{% endif %}
```

이 파일의 전체 코드는 이 책의 GitHub 저장소의 Chapter09 폴더에서 찾을 수 있습니다.

6. 앞서 언급했듯이, 표준 Django 인증 뷰를 사용하려면 URL 매핑을 추가해야 합니다. bookr 프로젝트 디렉토리의 urls.py 파일을 열고, django.contrib에서 auth가 가져와졌는지 확인한 후, URLpatterns에 세 개의 경로를 추가하세요:

```
from django.contrib import admin, auth
...
urlpatterns = [
    ...
    path("accounts/", include((
        django.contrib.auth.urls, "
        auth"
        namespace="accounts"))),
    path("accounts/password_reset/done/",
```



```
auth.views.PasswordResetDoneView.as_view(),
name="password_reset_done",),
path("accounts/reset/done/",
auth.views.PasswordResetCompleteView.as_view(),
name="password_reset_complete",),
...
]
```

7. Now, when you visit the login link at `http://127.0.0.1:8000/accounts/login/`, you will see this page:

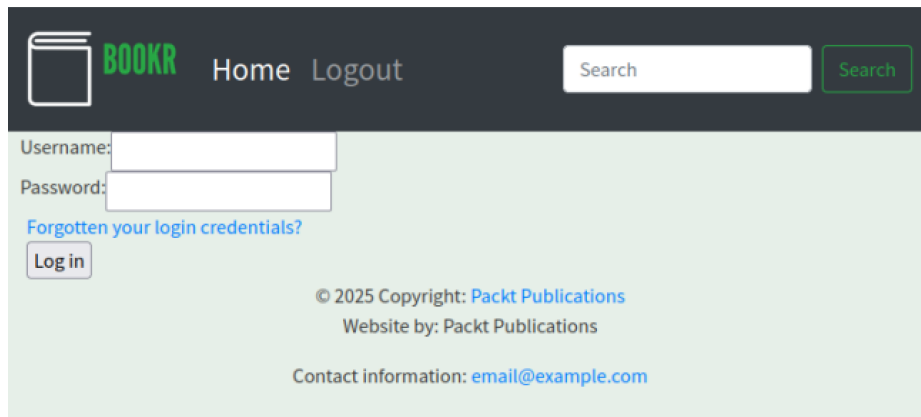


Figure 9.3: The Bookr login page

By completing this exercise, you have created the template required for non-admin authentication in your project.

Note

Before you proceed, you'll need to make sure the rest of the templates in the registration directory follow the bookr project's style; that is, they do not inherit from the Admin app's `admin/base_site.html` template. You've already seen this done with `password_change_done.html` and the `login.html` templates. Go ahead and apply what you've learned in this exercise (and the section before it) to the rest of the files in the registration directory. Alternatively, you may download the modified files from the Chapter09 folder in this book's GitHub repo.

Now that we have modified our project to enable authentication, we will learn about password and session storage in Django.

```

        auth.views.PasswordResetDoneView.as_view(),
        name="password_reset_done", ),
        path("accounts/reset/done/",
            auth.views.PasswordResetCompleteView.as_view(),
            name="password_reset_complete", ),
        ...
    ]

```

7. 이제 <http://127.0.0.1:8000/accounts/login/> 에 있는 로그인 링크를 방문하면 다음 페이지가 표시됩니다:

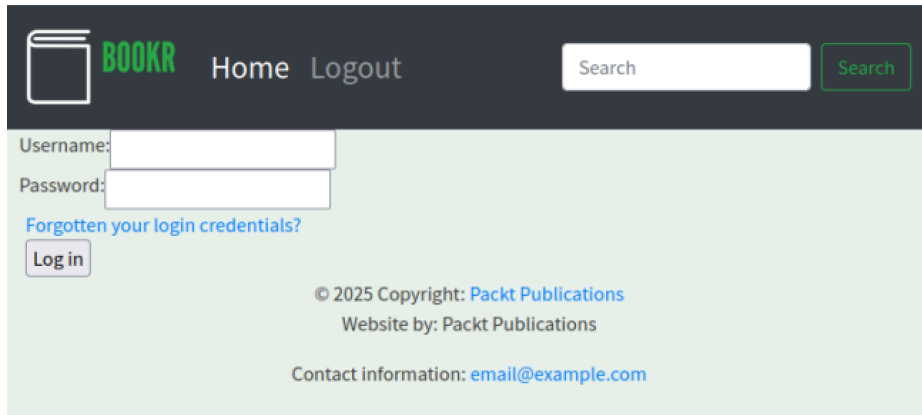


그림 9.3: Bookr 로그인 페이지

이 연습을 완료함으로써 프로젝트에서 비관리자 인증에 필요한 템플릿을 생성했습니다.

메모

진행하기 전에, 등록 디렉토리의 나머지 템플릿이 bookr 프로젝트의 스타일을 따르는지 확인해야 합니다. 즉, Admin 앱의 `admin/base_site.html` 템플릿을 상속받지 않아야 합니다. 이미 `password_change_done.html` 및 `login.html` 템플릿에서 이 작업이 수행된 것을 보았습니다. 이 연습(및 그 이전 섹션)에서 배운 내용을 등록 디렉토리의 나머지 파일에 적용해 보세요. 또는 이 책의 GitHub 리포지토리의 Chapter09 폴더에서 수정된 파일을 다운로드할 수 있습니다.

이제 인증을 가능하게 하기 위해 프로젝트를 수정했으므로, Django에서 비밀번호 및 세션 저장소에 대해 배울 것입니다.

Password storage in Django

Django does not store passwords in plain text form in the database. Instead, passwords are digested with a hashing algorithm, such as **PBKDF2/SHA256**, **BCrypt/SHA256**, or **Argon2**. As hashing algorithms are a one-way transformation, this prevents a user's password from being decrypted from the hash stored in the database. This often comes as a surprise to users who expect a system administrator to retrieve their forgotten password, but it is best practice in security design. So, if we query the database for the password, we will see something like this:

```
sqlite> select password from auth_user;
pbkdf2_sha256$870000$81tMmVyCdDp4Kl3fe2khaF$XopYZzmVFw04pZe7h014cy0sLBUzllYtG+uFsy9RG10=
pbkdf2_sha256$1000000$io5fNjVKzW74uvQB9L15LJ$V1NAajdh2r6dZhdvcqptmvWdL43N3Ce3qp19evf0H88=
pbkdf2_sha256$870000$UWL4u5af6xn637DmbUyY1j$jfDS1wfnYFwHejCkFF32E8yGnphK4tm0xALQ1ho/Gh4=
```

Figure 9.4 – Password hashes in a Django SQLite3 database

The components of this string are `<algorithm>$<iterations>$<salt>$<hash>`. As several hashing algorithms have been compromised over time and we sometimes need to work with mandated security requirements, Django is flexible enough to accommodate new algorithms and can maintain data encrypted in multiple algorithms.

The profile page and the `request.user` object

When a login is successful, the login view redirects to `/accounts/profile`. However, this path is not included in the existing `auth.url`, nor does the authentication app provide a template for it. To avoid a `Page not Found` error, a view and an appropriate URL pattern are required.

Each Django request has a `request.user` object. If the request is made by an unauthenticated user, `request.user` will be an `AnonymousUser` object. If the request is made by an authenticated user, then `request.user` will be a `User` object. This makes it easy to retrieve personalized user information in a Django view and render it in a template.

In the next exercise, we will add a profile page to our bookr project.

Exercise 9.02 – Adding a profile page

In this exercise, we will add a profile page to our project. To do so, we need to include the path to it in our URL patterns and also include it in our views and templates. The profile page will simply display the following attributes from the `request.user` object:

- `username`
- `first_name` and `last_name`
- `date_joined`

장고에서의 비밀번호 저장

장고는 데이터베이스에 `idx_6a8e7d1epasswords`를 일반 텍스트 형식으로 저장하지 않습니다. 대신, 비밀번호는 PBKDF2/SHA256, BCrypt/SHA256 또는 Argon2와 같은 해싱 알고리즘으로 처리됩니다. 해싱 알고리즘은 `idx_e017dbcf`일방향 변환이므로, 사용자의 비밀번호가 데이터베이스에 저장된 해시에서 복호화되는 것을 방지합니다. 이는 종종 시스템 관리자가 잊어버린 비밀번호를 검색할 수 있을 것이라고 기대하는 사용자에게 놀라움으로 다가옵니다. 그러나 이는 보안 설계의 모범 사례입니다. 따라서 비밀번호에 대해 데이터베이스를 쿼리하면 다음과 같은 내용을 볼 수 있습니다:

```
sqlite> select password from auth_user;
pbkdf2_sha256$870000$81tMmVyCdDp4Kl3fe2khaF$XopYZzmVFw04pZe7h014cy0sLBuZllYtG+uFsy9RG10=
pbkdf2_sha256$1000000$io5fNjVKzW74uvQB9L15LJ$V1NAajdh2r6dZhdvcqptmvWd143N3Ce3qp19evf0H88=
pbkdf2_sha256$870000$UWL4u5af6xn637DmbUyY1j$jdS1wfnYFwHejCkFF32E8yGnphK4tm0xALQ1ho/Gh4=
```

그림 9.4 - Django SQLite3 데이터베이스의 비밀번호 해시

이 문자열의 구성 요소는 `<algorithm>$<iterations>$<salt>$<hash>`입니다. 여러 해싱 알고리즘이 시간이 지남에 따라 `idx_df5da191`손상되었고, 때때로 우리는 의무화된 보안 요구 사항을 충족해야 하므로, 장고는 새로운 알고리즘을 수용할 수 있을 만큼 유연하며 여러 알고리즘으로 암호화된 데이터를 유지할 수 있습니다.

프로필 페이지와 request.user 객체

로그인이 `idx_453e06e0`성공하면, 로그인 뷰는 `/accounts/profile`로 리디렉션됩니다. 그러나 이 경로는 기존 `auth.url`에 포함되어 있지 않으며, 인증 앱에서도 이를 위한 템플릿을 제공하지 않습니다. 페이지를 찾을 수 없음 오류를 피하기 위해서는 뷰와 적절한 URL 패턴이 필요합니다.

각 Django 요청에는 `request.user` 객체가 있습니다. 요청이 인증되지 않은 `idx_9d6b3827` 사용자에 의해 이루어지면, `request.user`는 `AnonymousUser` 객체가 됩니다. 요청이 인증된 사용자에 의해 이루어지면, `request.user`는 `User` 객체가 됩니다. 이는 Django 뷰에서 개인화된 사용자 정보를 쉽게 검색하고 템플릿에 렌더링할 수 있게 해줍니다.

다음 연습에서는 우리의 `bookr` 프로젝트에 프로필 페이지를 추가할 것입니다.

연습 9.02 – 프로필 페이지 추가

이 연습에서는 프로젝트에 프로필 페이지를 추가할 것입니다. 이를 위해 URL 패턴에 해당 경로를 포함하고, 뷰와 템플릿에도 포함해야 합니다. 프로필 페이지는 요청된 `user` 객체에서 다음 속성을 단순히 표시합니다:

- 사용자 이름
- 이름과 성가입 날짜
-

- email
- last_login

Perform the following steps to complete this exercise:

1. Add `bookr/views.py` to the project. It needs a trivial profile function to define our view:

```
from django.shortcuts import render

def profile(request):
    return render(request, 'profile.html')
```

2. In the templates folder of your main bookr project folder, create a new file called `profile.html`. In this template, the attributes of the `request.user` object can easily be referenced by using a notation such as `{{ request.user.username }}`:

```
{% extends "base.html" %}
{% block title %}Bookr{% endblock %}
{% block content %}
<h2>Profile</h2>
<div>
  <p>
    Username: {{ request.user.username }} <br>
    Name: {{ request.user.first_name }}
    {{ request.user.last_name }}<br>
    Date Joined: {{ request.user.date_joined }} <br>
    Email: {{ request.user.email }}<br>
    Last Login: {{ request.user.last_login }}<br>
  </p>
</div>
{% endblock %}
```

We also added a block containing the profile details of the user. More importantly, we made sure that `profile.html` extends `base.html`.

3. Finally, this path needs to be added to the top of the `urlpatterns` list in `bookr/urls.py`. First, import the new views and then add a path linking the `accounts/profile/` URL to `bookr.views.profile`:

```
from bookr.views import profile
```

- 이메일
- 마지막 로그인

이 연습을 완료하려면 다음 단계를 수행하십시오:

1. bookr/views.py를 프로젝트에 추가하십시오. 우리의 뷰를 정의하기 위해 사소한 프로필 함수가 필요합니다:

```
from django.shortcuts import render

def profile(request):
    return render(request, 'profile.html')
```

2. 당신의 메인 bookr 프로젝트 폴더의 templates 폴더에 profile.html이라는 새 파일을 만드세요. 이 템플릿에서는 {{ request.user.username }}와 같은 표기법을 사용하여 request.user 객체의 속성을 쉽게 참조할 수 있습니다:

```
{% extends "base.html" %}
{% block title %}Bookr{% endblock %}
{% block content %}
<h2>프로필</h2>
<div>
  <p>
    사용자 이름: {{ request.user.username }} <br>
    이름: {{ request.user.first_name }}
    {{ request.user.last_name }}<br>
    가입 날짜: {{ request.user.date_joined }} <br>
    이메일: {{ request.user.email }}<br>
    마지막 로그인: {{ request.user.last_login }}<br>
  </p>
</div>
{% endblock %}
```

우리는 또한 사용자의 프로필 세부정보를 포함하는 블록을 추가했습니다. 더 중요한 것은, profile.html이 base.html을 확장하도록 했습니다.

3. 마침내, 이 경로 idx_b74d583f는 bookr/urls.py의 urlpatterns 목록 맨 위에 추가되어야 합니다. 먼저, 새로운 뷰를 가져온 다음 accounts/profile/URL을 bookr.views.profile에 연결하는 경로를 추가하세요:

```
from bookr.views import profile
```

```
urlpatterns = [...  
    path('accounts/profile/', profile,  
        name='profile'),  
    path("", reviews.views.index),  
    path("book-search/",  
        reviews.views.book_search,  
        name="book_search"),  
    path('', include('reviews.urls'))]
```

This is a good start on a user profile page. We created an account for Alice in the *CRUD operations using the Django admin app* section in *Chapter 4*. When Alice logs in and visits `http://localhost:8000/accounts/profile/`, it is rendered as shown in *Figure 9.5*. Remember, if the server needs to be started, use the `python manage.py runserver` command:

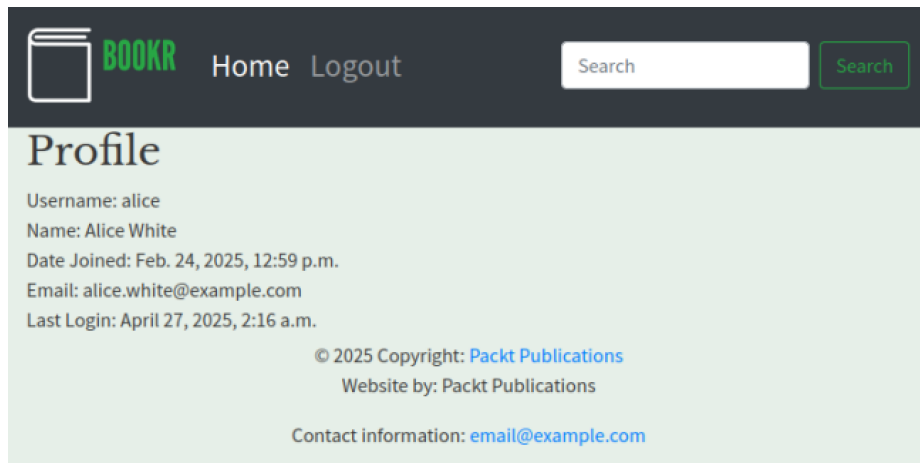


Figure 9.5: Alice visits her user profile

With that, we've seen how we can redirect a user to their profile page, once they've successfully logged in. Now, let's discuss how we can give content access to specific users only.

Authentication decorators and redirection

Now that we have learned how to allow ordinary users to log in to our project, we can discover how to restrict content to authenticated users. The authentication module comes with some useful decorators that can be used to secure views according to the current user's authentication or access.

```
urlpatterns = [...
    path('accounts/profile/', profile,
        name='profile'),
    path("", reviews.views.index),
    path("book-search/",
        reviews.views.book_search,
        name="book_search"),
    path('', include('reviews.urls'))]
```

사용자 프로필 페이지에 대한 좋은 시작입니다. 우리는 4장에서 Django 관리 앱 섹션을 사용하여 Alice를 위한 계정을 만들었습니다. Alice가 로그인하고 `http://localhost:8000/accounts/profile/`를 방문하면 그림 9.5와 같이 렌더링됩니다. 서버를 시작해야 하는 경우 `python manage.py runservercommand`를 사용하세요:

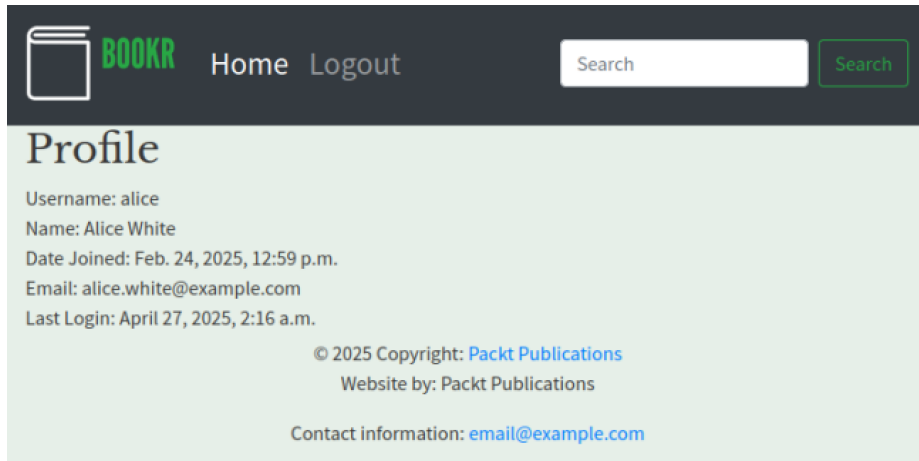


그림 9.5: 앨리스가 사용자 프로필을 방문합니다

이제 우리는 사용자가 성공적으로 로그인한 후 프로필 페이지로 리디렉션하는 방법을 보았습니다. 이제 특정 사용자에게만 콘텐츠 접근을 허용하는 방법에 대해 논의해 보겠습니다.

인증 데코레이터 및 리디렉션

이제 우리는 일반 사용자가 우리 `idx_86fb0990` 프로젝트에 로그인할 수 있도록 하는 방법을 배웠으므로 인증된 사용자에게 콘텐츠를 제한하는 방법을 알아볼 수 있습니다. 인증 모듈에는 현재 사용자의 인증 또는 접근에 따라 뷰를 보호하는 데 사용할 수 있는 유용한 데코레이터가 포함되어 있습니다.

At present, if Alice were to log out of Bookr, the profile page would still render and display empty details. Instead of this happening, it would be preferable for any unauthenticated visitor to be directed to the login screen:

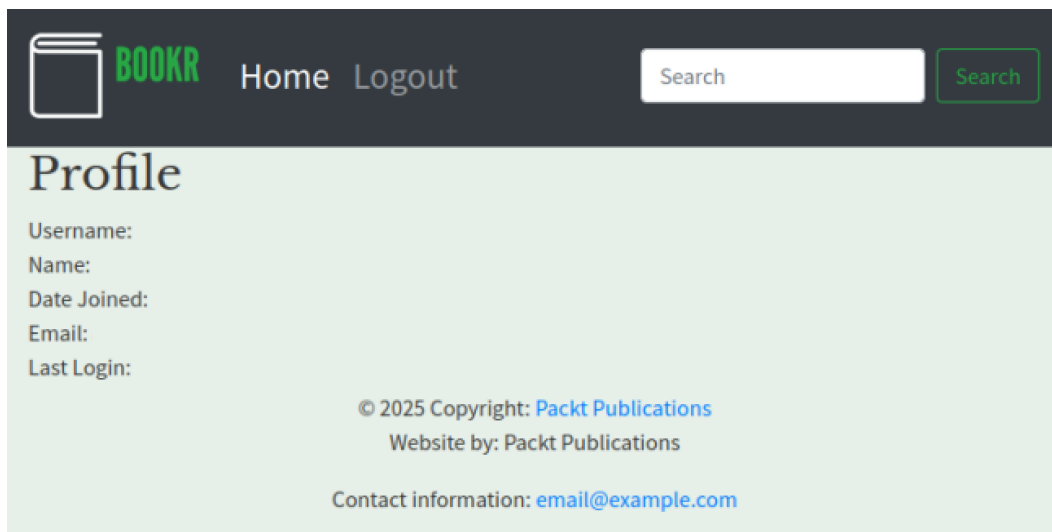


Figure 9.6: An unauthenticated user visits a user profile

The authentication app comes with useful decorators for adding authentication behavior to Django views. In this situation of securing our profile view, we can use the `login_required` decorator:

```
from django.contrib.auth.decorators import login_required

@login_required
def profile(request):
    ...
```

Now, if an unauthenticated user visits the `/accounts/profile` URL, they will be redirected to `http://localhost:8000/accounts/login/?next=/accounts/profile/`.

This URL takes the user to the login URL. The `next` parameter in the GET variables tells the login view where to redirect after a successful login. The default behavior is to redirect back to the current view, but this can be overridden by specifying the `login_url` argument to the

현재, 만약 앨리스가 Bookr에서 로그아웃을 한다면, 프로필 페이지는 여전히 렌더링되고 빈 세부정보를 표시할 것입니다. 이러한 일이 발생하는 대신, 인증되지 않은 방문자가 로그인 화면으로 리디렉션되는 것이 바람직합니다:

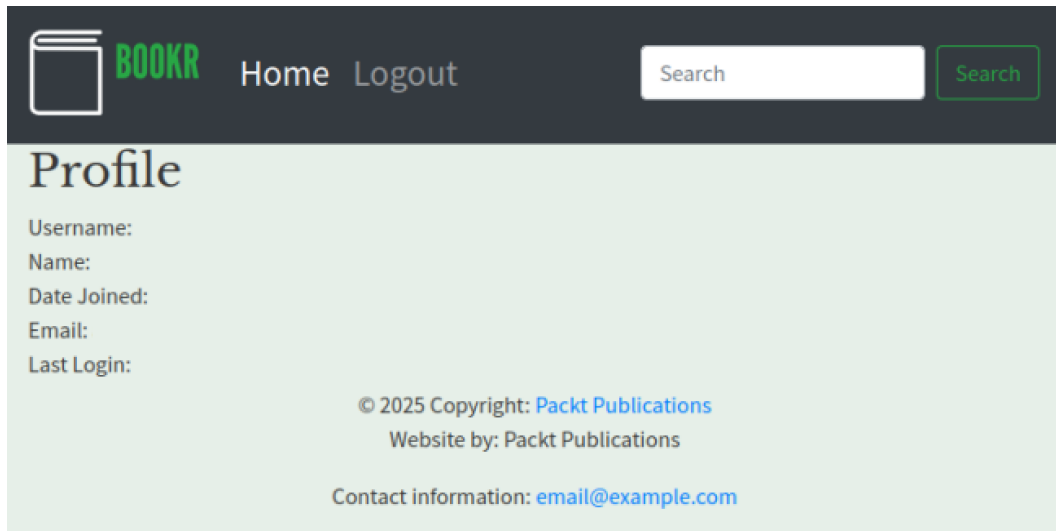


그림 9.6: 인증되지 않은 사용자가 사용자 프로필을 방문함

인증 앱 `idx_e3d62017`은 Django 뷰에 인증 `idx_b5bb16d3` 동작을 추가하기 위한 유용한 데코레이터를 제공합니다. 프로필 뷰를 보호하는 이 상황에서, 우리는 `login_required` 데코레이터를 사용할 수 있습니다:

```
from django.contrib.auth.decorators import login_required

@login_required
def profile(request):
    ...
```

이제 인증되지 않은 사용자가 `/accounts/profile` URL을 방문하면, 그들은 `http://localhost:8000/accounts/login/?next=/accounts/profile/`로 리디렉션될 것입니다.

이 URL은 사용자를 로그인 URL로 안내합니다. GET 변수의 다음 매개변수는 로그인 뷰가 성공적인 로그인 후 어디로 리디렉션할지를 알려줍니다. 기본 동작은 현재 뷰로 다시 리디렉션하는 것이지만, `login_url` 인수를 지정하여 이를 재정의할 수 있습니다.

login_required decorator. For example, if we had some need to redirect to a different page after login, we could have explicitly stated it in the decorator call like this:

```
@login_required(login_url='/accounts/profile2')
```

If we had rewritten our login view to expect the redirection URL to be specified in a different URL argument to 'next', we could explicate this in the decorator call with the redirect_field_name argument:

```
@login_required(redirect_field_name='redirect_to')
```

There are often situations where a URL should be restricted to users or groups holding a specific condition. Consider the case where we have a page for staff users to view any user profile. We don't want this URL to be accessible to all users, so we want to limit this URL to users or groups with the 'view_user' permission and forward the unauthorized requests to the login URL:

```
from django.contrib.auth.decorators import (login_required,
                                             permission_required)
...

@permission_required('view_group')
def user_profile(request, uid):
    user = get_object_or_404(User, id=uid)
    permissions = user.get_all_permissions()
    return render(request, 'user_profile.html',
                  {'user': user, 'permissions': permissions})
```

So with this decorator applied to our user_profile view, an unauthorized user visiting `http://localhost:8000/accounts/users/123/profile/` would be redirected to `http://localhost:8000/accounts/login/?next=/accounts/users/123/profile/`.

Since Django 5.1, an additional middleware component called `LoginRequiredMiddleware` is available. This can be activated by adding `"django.contrib.auth.middleware.LoginRequiredMiddleware"` to the `MIDDLEWARE` list in a project's `settings.py` file. This will enforce the login requirement by default for views in the project. In order to allow a view to be called by non-validated users, the `@login_not_required` decorator will need to be employed.

login_required 데코레이터. 예를 들어, 로그인 후 다른 페이지로 리디렉션할 필요가 있다면, 데코레이터 호출에서 다음과 같이 명시적으로 지정할 수 있습니다:

```
@login_required(login_url='/accounts/profile2')
```

로그인 뷰를 리디렉션 URL이 'next'의 다른 URL 인수로 지정되도록 다시 작성했다면, 데코레이터 호출에서 redirect_field_name 인수로 이를 명시할 수 있습니다:

```
@login_required(redirect_field_argument='redirect_to')
```

특정 조건을 가진 사용자나 그룹에게 URL을 제한해야 하는 상황이 자주 발생합니다. 직원 사용자가 모든 사용자 프로필을 볼 수 있는 페이지가 있다고 가정해 보겠습니다. 이 URL이 모든 사용자에게 접근 가능하지 않도록 하고, 'view_user' 권한이 있는 사용자나 그룹으로 제한하고, 권한이 없는 요청은 로그인 URL로 전달하고자 합니다:

```
from django.contrib.auth.decorators import (login_required,
                                             permission_required)
...

@permission_required('view_group')
def user_profile(request, uid):
    user = get_object_or_404(User, id=uid)
    permissions = user.get_all_permissions()
    return render(request, 'user_profile.html',
                  {'user': user, 'permissions': permissions})
```

따라서 이 데코레이터가 user_profile 뷰에 적용되면, 권한이 없는 사용자가 <http://localhost:8000/accounts/users/123/profile/>를 방문할 경우 <http://localhost:8000/accounts/login/?next=/accounts/users/123/profile/>로 리디렉션됩니다.

Django 5.1부터 LoginRequiredMiddleware라는 추가 미들웨어 구성 요소가 제공됩니다. 이는 프로젝트의 settings.py 파일의 MIDDLEWARE 목록에 "

django.contrib.auth.middleware.LoginRequiredMiddleware"를 추가하여 활성화할 수 있습니다. 이렇게 하면 프로젝트의 뷰에 대해 기본적으로 로그인 요구 사항이 적용됩니다. 비검증된 사용자가 호출할 수 있도록 하려면 @login_not_required 데코레이터를 사용해야 합니다.

Sometimes, though, we need to structure more subtle conditional permissions that don't fall into the scope of these two directors. For this purpose, Django provides a custom decorator that takes an arbitrary function as an argument. The `user_passes_test` decorator requires a `test_func` argument:

```
user_passes_test(test_func, login_url=None,
                  redirect_field_name='next')
```

Here's an example where we have a view, `veteran_features`, that is only available to users who have been registered on the site for more than a year:

```
from django.contrib.auth.decorators import (login_required,
                                             permission_required, user_passes_test)
from django.utils import timezone...

def veteran_user(user):
    now = timezone.now()
    if user.date_joined is None:
        return False
    return now - user.date_joined >
        datetime.timedelta(days=365)

@user_passes_test(veteran_user)
def veteran_features(request):
    user = request.user
    permissions = user.get_all_permissions()
    return render(request, 'veteran_profile.html',
                  {'user': user, 'permissions': permissions})
```

Sometimes, the logic in our views cannot be handled with one of these decorators, and we need to apply the redirect within the control flow of the view. We can do this using the `redirect_to_login` helper function. It takes the same arguments as the decorators, as shown in the following snippet:

```
redirect_to_login(next, login_url=None,
                  redirect_field_name='next')
```

We will consolidate our knowledge of authentication decorators by applying them to the bookr project in the following exercise.

때때로, 우리는 이러한 두 감독의 범위에 포함되지 않는 더 미묘한 조건부 권한을 구조화해야 할 필요가 있습니다. 이를 위해 Django는 임의의 함수를 인수로 받는 사용자 정의 데코레이터를 제공합니다. `user_passes_test` 데코레이터는 `test_func` 인수를 요구합니다:

```
user_passes_test(test_func, login_url=None,
                  redirect_field_name='next')
```

여기에는 사이트에 1년 이상 등록된 사용자만 사용할 수 있는 view인 `veteran_features`의 예가 있습니다:

```
from django.contrib.auth.decorators import (login_required,
                                             permission_required, user_passes_test)
from django.utils import timezone...

def veteran_user(user):
    now = timezone.now()
    if user.date_joined is None:
        return False
    return now - user.date_joined >
        datetime.timedelta(days=365)

@user_passes_test(veteran_user)
def veteran_features(request):
    user = request.user
    permissions = user.get_all_permissions()
    return render(request, 'veteran_profile.html',
                  {'user': user, 'permissions': permissions})
```

때때로, `idx_f74d0b980ur` 뷰의 논리는 이러한 `idx_7d8d3076` 데코레이터 중 하나로 처리할 수 없으며, 뷰의 제어 흐름 내에서 리디렉션을 적용해야 합니다. 우리는 `redirect_to_login` 헬퍼 함수를 사용하여 이를 수행할 수 있습니다. 이 함수는 다음 코드 조각에 표시된 것처럼 데코레이터와 동일한 인수를 사용합니다:

```
redirect_to_login(next, login_url=None,
                  redirect_field_name='next')
```

다음 연습에서 `bookrproject`에 인증 데코레이터를 적용하여 우리의 지식을 통합할 것입니다.

Exercise 9.03 – Adding authentication decorators to the views

Having learned about the flexibility of the authentication app's permission and authentication decorators, we will now set about putting them to use in the **Reviews** app. We need to ensure that only authenticated users can edit reviews and that only staff users can edit publishers. There are several ways of doing this, so we will attempt a few approaches. All the code in these steps is in the `reviews/views.py` file:

1. Your first instinct to solve this problem would be to think that the `publisher_edit` method needs an appropriate decorator to enforce that the user has `edit_publisher` permission. For this, you could easily do something like this:

```
from django.contrib.auth.decorators import \
    permission_required
...

@permission_required('edit_publisher')
def publisher_edit(request, pk=None):
    ...
```

2. Using this method is fine, and it's one way to add permissions checking to a view. You can also use a slightly more complicated but more flexible method. Instead of using a permission decorator to enforce permission rights on the `publisher_edit` method, you can create a test function that requires a staff user and apply this test function to `publisher_edit` with the `user_passes_test` decorator. Writing a test function allows more customization on how you validate users' access rights or permissions. If you made changes to your `views.py` file in *Step 1*, feel free to comment the decorator out (or delete it) and write the following test function instead:

```
from django.contrib.auth.decorators import \
    user_passes_test
...

def is_staff_user(user):
    return user.is_staff

@user_passes_test(is_staff_user)
...

```

연습 9.03 – 인증 데코레이터를 views에 추가하기

인증 앱의 권한 및 인증 데코레이터의 유연성에 대해 배운 후, 이제 이를 리뷰 앱에서 사용해 보겠습니다. 인증된 사용자만 리뷰를 편집할 수 있고, 직원 사용자만 출판사를 편집할 수 있도록 해야 합니다. 이를 수행하는 방법은 여러 가지가 있으므로 몇 가지 접근 방식을 시도해 보겠습니다. 이 단계의 모든 코드는 `reviews/views.py` 파일에 있습니다:

1. 이 문제를 해결하기 위한 첫 번째 직관은 `publisher_edit` 메서드에 사용자가 `edit_publisher` 권한을 가지고 있는지 강제하는 적절한 데코레이터가 필요하다는 것입니다. 이를 위해 다음과 같이 쉽게 할 수 있습니다:

```
from django.contrib.auth.decorators import \
    permission_required
...

@permission_required('edit_publisher')
def publisher_edit(request, pk=None):
    ...
```

2. 이 방법을 사용하는 것은 괜찮으며, 뷰에 권한 검사를 추가하는 한 가지 방법입니다. 조금 더 복잡하지만 더 유연한 방법을 사용할 수도 있습니다. `publisher_edit` 메서드에 권한 장식을 사용하여 권한을 강제하는 대신, 직원 사용자를 요구하는 테스트 함수를 만들고 이 테스트 함수를 `user_passes_test` 장식자로 `publisher_edit`에 적용할 수 있습니다. 테스트 함수를 작성하면 사용자의 접근 권한이나 권한을 검증하는 방법에 대해 더 많은 사용자 정의가 가능합니다. 1단계에서 `views.py` 파일을 변경했다면, 장식을 주식 처리하거나(삭제하거나) 대신 다음 테스트 함수를 작성해도 좋습니다:

```
from django.contrib.auth.decorators import \
    user_passes_test
...

def is_staff_user(user):
    return user.is_staff

@user_passes_test(is_staff_user)
...

```


3. Ensure that login is required for the `review_edit` and `book_media` functions by adding the appropriate decorator:

```
...

from django.contrib.auth.decorators import (
    login_required, user_passes_test)
...
@login_required
def review_edit(request, book_pk, review_pk=None):

@login_required
def book_media(request, pk):
...
```

4. In the `review_edit` method, add logic to the view that requires that the user be either a staff user or the owner of the review. The `review_edit` view controls the behavior of both review creation and review updates. The constraint that we are developing only applies to the case where an existing review is being updated. So, the place to add code is after a `Review` object has been successfully retrieved. If the user is not a staff account or the review's creator doesn't match the current user, we need to raise a `PermissionDenied` error:

```
...

from django.core.exceptions import PermissionDenied
from PIL import Image
from django.contrib import messages
...

@login_required
def review_edit(request, book_pk, review_pk=None):
    book = get_object_or_404(Book, pk=book_pk)

    if review_pk is not None:
        review = get_object_or_404(Review,
                                   book_id=book_pk,
                                   pk=review_pk)

        user = request.user
        if (not user.is_staff and
```

3. 리뷰 편집 및 도서 미디어 기능에 로그인 필수 조건을 추가하여 적절한 데코레이터를 추가하세요:

```
...

from django.contrib.auth.decorators import (
    login_required, user_passes_test)
...
@login_required
def review_edit(request, book_pk, review_pk=None):

@login_requireddef
    book_media(request, pk):...
```

4. review_edit 메서드에서 사용자가 반드시 다음 중 하나임을 요구하는 로직을 뷰에 추가하세요. 직원 사용자 또는 리뷰 소유자. review_edit 뷰는 리뷰 생성과 리뷰 업데이트의 동작을 제어합니다. 우리가 개발 중인 제약 조건 idx_ec47201c는 기존 리뷰가 업데이트되는 경우에만 적용됩니다. 따라서 코드를 추가할 위치는 Review 객체가 성공적으로 조회된 후입니다. 사용자가 직원 계정이 아니거나 리뷰 작성자가 현재 사용자와 일치하지 않는 경우, PermissionDenied 오류를 발생시켜야 합니다.

```
...

from django.core.exceptions import PermissionDenied
from PIL import Image
from django.contrib import messages
...

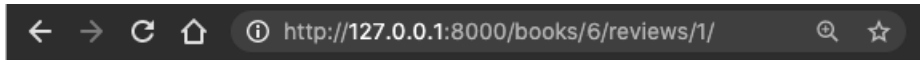
@login_required
def review_edit(request, book_pk, review_pk=None):
    book = get_object_or_404(Book, pk=book_pk)

    if review_pk is not None:
        review = get_object_or_404(Review,
                                   book_id=book_pk,
                                   pk=review_pk)

    user = request.user
    if (not user.is_staff and
```

```
        review.creator.id != user.id):  
            raise PermissionDenied  
    else:  
        review = None  
    ...
```

Now, when a non-staff user attempts to edit another user's review, a Forbidden error will be thrown, as in *Figure 9.7*. In the next section, we will look at applying conditional logic in templates so that users aren't taken to pages that they don't have sufficient permission to access:



403 Forbidden

Figure 9.7: Access is forbidden to non-staff users

In this exercise, we used authentication decorators to secure views in a Django app. The authentication decorators that were applied provided a simple mechanism to restrict views from users lacking necessary permissions, non-staff users, and unauthenticated users. Django's authentication decorators provide a robust mechanism that follows Django's role and permission framework, while the `user_passes_test` decorator provides an option to develop custom authentication.

Now that we have used decorators to control the authentication flow of the application, we can customize templates with user authentication data.

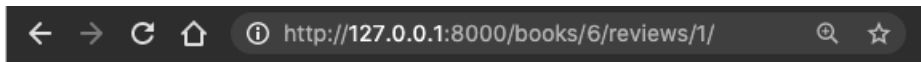
```

        review.creator.id !=
        user.id):raise PermissionDenied

    else:
        review = None
    ...

```

이제, idx_254f6d3enon-staff 사용자가 다른 사용자의 리뷰를 편집하려고 하면, 그림 9.7과 같이 Forbidden 오류가 발생합니다. 다음 섹션에서는 사용자가 충분한 권한이 없는 페이지로 이동하지 않도록 템플릿에서 조건 논리를 적용하는 방법을 살펴보겠습니다:



403 Forbidden

그림 9.7: 비직원 사용자는 접근이 금지됩니다

이 연습에서는 Django 앱의 뷰를 보호하기 위해 인증 데코레이터를 사용했습니다. 적용된 인증 데코레이터는 필요한 권한이 없는 사용자, 비직원 사용자 및 인증되지 않은 사용자의 뷰를 제한하는 간단한 메커니즘을 제공했습니다. Django의 인증 데코레이터는 Django의 역할 idx_4f715788 및 권한 프레임워크를 따르는 강력한 메커니즘을 제공하며, user_passes_test 데코레이터는 사용자 정의 인증을 개발할 수 있는 옵션을 제공합니다.

이제 우리는 애플리케이션의 인증 흐름을 제어하기 위해 데코레이터를 사용했으므로, 사용자 인증 데이터로 템플릿을 사용자 정의할 수 있습니다.

Enhancing templates with authentication data

In *Exercise 9.02 – Adding a profile page*, we saw that we can pass the `request.user` object to the template to render the current user's attributes in the HTML. We can also take the approach of giving different template renderings according to the user type or the permissions held by a user. Consider that we want to add an edit link that only appears to staff users. We might apply an `if` condition to achieve this:

```
{% if user.is_staff %}
    <p><a href="{% url 'review:edit' %}">Edit this Review</a>
</p>
{% endif %}
```

If we didn't take the time to conditionally render links based on permissions, users would have a frustrating experience navigating the application, as many of the links that they click on would lead to 403 Forbidden pages. The following exercise will show how we can use templates and authentication to present contextually appropriate links in our project.

Exercise 9.04 – Toggling login and logout links in the base template

In the bookr project's base template, located in `templates/base.html`, we have a placeholder logout link in the header. It is coded in HTML as follows:

```
<li class="nav-item">
    <a class="nav-link" href="#">Logout</a>
</li>
```

We don't want the logout link to appear after a user has logged out. So, this exercise aims to apply conditional logic to the template so that the Login and Logout links are toggled, depending on whether the user is authenticated:

1. Edit the `templates/base.html` file. Copy the structure of the Logout list element and create a Login list element. Then, replace the placeholder links with the correct URLs for the Logout and Login pages – `/accounts/logout` and `/accounts/login`, respectively – as follows:

```
<li class="nav-item">
    <a class="nav-link" href="/accounts/logout">Logout
</a>
```

인증 데이터로 템플릿 향상하기

연습 9.02 – 프로필 페이지 추가에서 우리는 request.user 객체 idx_763de513을 템플릿에 전달하여 현재 사용자의 속성을 HTML에 렌더링할 수 있음을 보았습니다. 우리는 또한 사용자 유형이나 사용자가 가진 권한에 따라 다른 템플릿 렌더링을 제공하는 접근 방식을 취할 수 있습니다. 직원 사용자에게만 나타나는 편집 링크를 추가하고 싶다고 가정해 보겠습니다. 이를 달성하기 위해 if 조건을 적용할 수 있습니다:

```
{% if user.is_staff %}
    <p><a href="{% url 'review:edit' %}">리뷰 편집</a>
</p>
{% endif %}
```

권한에 따라 링크를 조건부로 렌더링하는 데 시간을 할애하지 않았다면, 사용자는 애플리케이션을 탐색하는 데 불만을 느낄 것입니다. 그들이 클릭하는 많은 링크가 403 금지 페이지로 이어질 것이기 때문입니다. 다음 연습에서는 템플릿과 인증을 사용하여 프로젝트에서 맥락에 맞는 적절한 링크를 제공하는 방법을 보여줄 것입니다.

연습 9.04 – 기본 템플릿에서 로그인 및 로그아웃 링크 전환하기

책 프로젝트의 idx_23a30d61 기본 템플릿은 templates/base.html에 위치하고 있으며, 헤더에 placeholderidx_97af914 블로그 아웃 링크가 있습니다. 이는 다음과 같이 HTML로 코딩되어 있습니다:

```
<li class="nav-item">
    <a class="nav-link" href="#">로그아웃</a>
</li>
```

사용자가 로그아웃한 후 로그아웃 링크가 나타나지 않기를 원합니다. 따라서 이 연습의 목표는 사용자가 인증되었는지 여부에 따라 로그인 및 로그아웃 링크가 전환되도록 템플릿에 조건 논리를 적용하는 것입니다:

1. templates/base.html 파일을 편집합니다. 로그아웃 목록 요소의 구조를 복사하고 로그인 목록 요소를 만듭니다. 그런 다음, 자리 표시자 링크를 로그아웃 및 로그인 페이지의 올바른 URL로 교체합니다 – 각각 /accounts/logout 및 /accounts/login – 다음과 같이:

```
<li class="nav-item">
    <a class="nav-link" href="/accounts/logout">로그아웃
</a>
```

```
</li>
<li class="nav-item">
  <a class="nav-link" href="/accounts/login">Login</a>
</li>
```

2. Now, put our two `li` elements inside an `if ... else ... endif` conditional block. The logic condition that we are applying is `if user.is_authenticated`:

```
{% if user.is_authenticated %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/logout">Logout
  </a>
</li>
{% else %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/login">Login
  </a>
</li>
{% endif %}
```

3. Now, visit the user profile page at `http://localhost:8000/accounts/profile/`. When authenticated, you will see the **Logout** link:

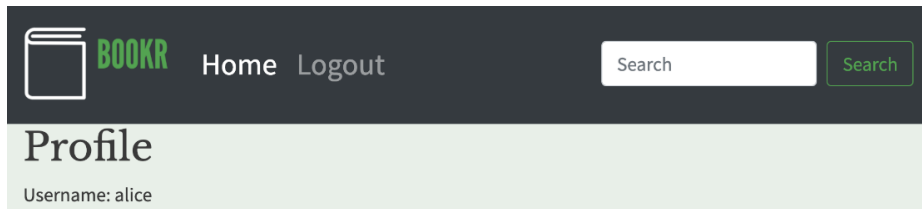


Figure 9.8: An authenticated user sees the Logout link

4. Now, click the **Logout** link; you will be taken to the `/accounts/logout` page. The **Login** link appears in the menu, confirming that the link is contextually dependent on the authentication state of the user:

```

</li>
<li class="nav-item">
  <a class="nav-link" href="/accounts/login">로그인</a>
</li>

```

2. 이제 idx_0b122644an 안에 두 개의 li 요소를 idx_1a25ed06our 넣고 if... else... endif 조건 idx_37bb2365block을 만듭니다. 그 논리 조건은 idx_da287cc7we가 적용하는 것입니다: user.is_authenticated인 경우:

```

{% if user.is_authenticated %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/logout">로그아웃
    </a>
  </li>
{% else %}
  <li class="nav-item">
    <a class="nav-link" href="/accounts/login">로그인
    </a>
  </li>
{% endif %}

```

3. 이제 <http://localhost:8000/accounts/profile/>에서 사용자 프로필 페이지를 방문하세요. 인증된 경우, 로그아웃 링크를 볼 수 있습니다:

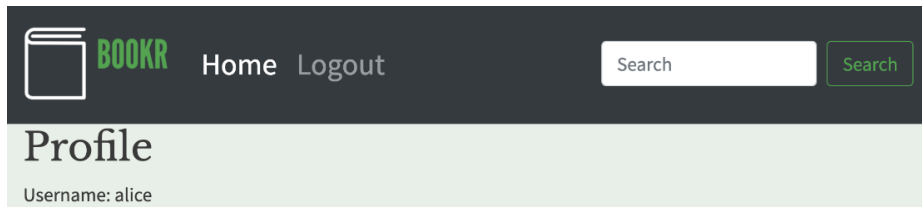


그림 9.8: 인증된 사용자가 로그아웃 링크를 봅니다

4. 이제 로그아웃 링크를 클릭하세요; /accounts/logout 페이지로 이동하게 됩니다. 메뉴에 로그인 링크가 나타나며, 이 링크가 사용자의 인증 상태에 따라 맥락적으로 의존적임을 확인합니다:

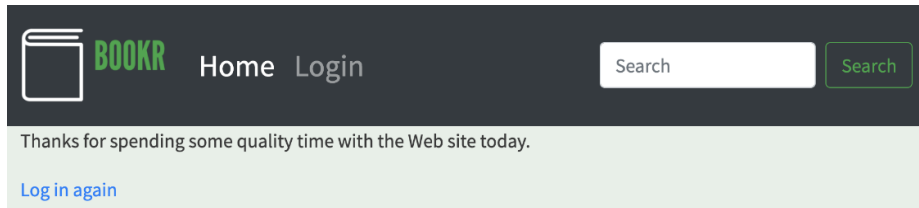


Figure 9.9: An unauthenticated user sees the Login link

Note

Requests to `/accounts/logout` require a POST request since Django 5.0. So, you will see that, in the example template in GitHub, there is a form with a button to handle this.

This exercise was a simple example of how Django templates can be used with authentication information to create a stateful and contextual user experience. We also do not want to provide links that a user does not have access to or actions that are not permissible for the user's permission level. The following activity will use this templating technique to fix some of these problems in Bookr.

Activity 9.01 – Authentication-based content using conditional blocks in templates

In this activity, you will apply conditional blocks to templates that modify content based on user authentication and user status. Users should not be presented with links that they are not permitted to visit or actions that they are not authorized to carry out. The following steps will help you complete this activity:

1. In the `book_detail` template, in the file at `reviews/templates/reviews/book_detail.html`, hide the **Add Review** and **Media** buttons from non-authenticated users.
2. Also, hide the heading that says, **Be the first one to write a review**, as that is not an option for non-authenticated users.
3. In the same template, make the **Edit Review** link only appear for the staff or the user who wrote the review. The conditional logic for the template block is very similar to the conditional logic that we used in the `review_edit` view in the previous section:

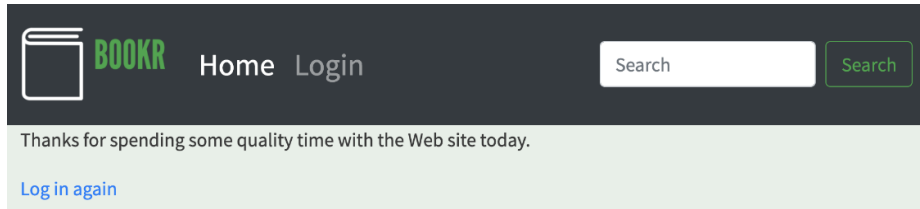


그림 9.9: 인증되지 않은 사용자가 로그인 링크를 봅니다

노트

/accounts/logout에 대한 요청은 Django 5.0부터 POST 요청이 필요합니다. 따라서 GitHub의 예제 템플릿에서 이를 처리하기 위한 버튼이 있는 양식을 볼 수 있습니다.

이 연습은 Django 템플릿이 인증 정보를 사용하여 상태 유지 및 맥락적 사용자 경험을 생성하는 방법에 대한 간단한 예였습니다. 또한 사용자가 접근할 수 없는 링크나 사용자의 권한 수준에 허용되지 않는 작업을 제공하고 싶지 않습니다. 다음 활동에서는 이 템플릿 기법을 사용하여 Bookr의 이러한 문제를 해결할 것입니다.

활동 9.01 – 템플릿에서 조건부 블록을 사용한 인증 기반 콘텐츠

이 활동에서는 사용자 인증 및 사용자 상태에 따라 콘텐츠를 수정하는 템플릿에 조건부 블록을 적용합니다. 사용자는 방문할 수 없는 링크나 수행할 수 없는 작업을 제시받아서 안 됩니다. 다음 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. book_detail 템플릿에서 reviews/templates/reviews/book_detail.html 파일에 비인증 사용자로부터 리뷰 추가 및 미디어 버튼을 숨깁니다.
2. 또한, 비인증 사용자에게는 옵션이 아니므로 '리뷰를 처음 작성하는 사람'이라는 제목을 숨기십시오.
3. 같은 템플릿에서 리뷰를 작성한 직원이나 사용자에게만 리뷰 수정 링크가 나타나도록 하십시오. 템플릿 블록에 대한 조건 논리는 이전 섹션의 review_edit 뷰에서 사용한 조건 논리와 매우 유사합니다:

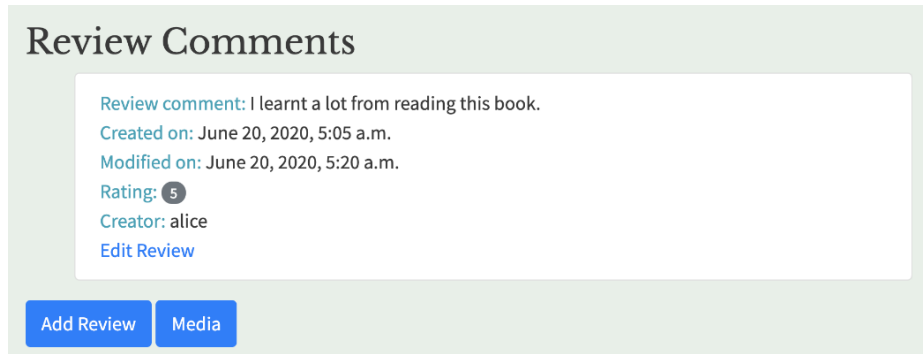


Figure 9.10: The **Edit Review** link appears on Alice's review when Alice is logged in

When a different user is logged in, the **Edit Review** link will not be present:

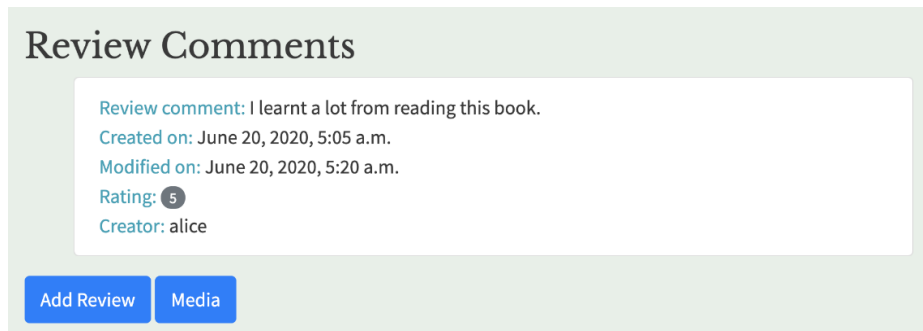


Figure 9.11: There is no **Edit Review** link on Alice's review when Bob is logged in

4. Modify `template/base.html` so that it displays the currently authenticated user's username to the right of the search form in the header, linking to the user profile page. By completing this activity, you will have added dynamic content to the template that reflects the authentication state and identity of the current user, as can be seen from the following screenshot:

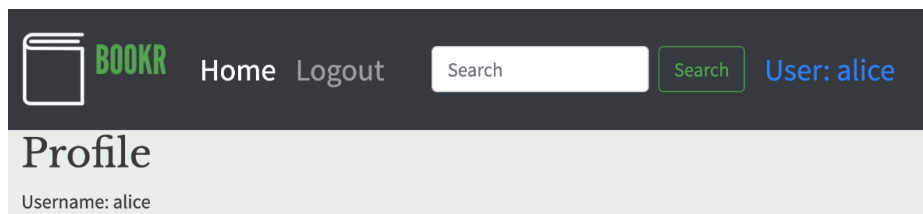


Figure 9.12: An authenticated user's name appears after the search form

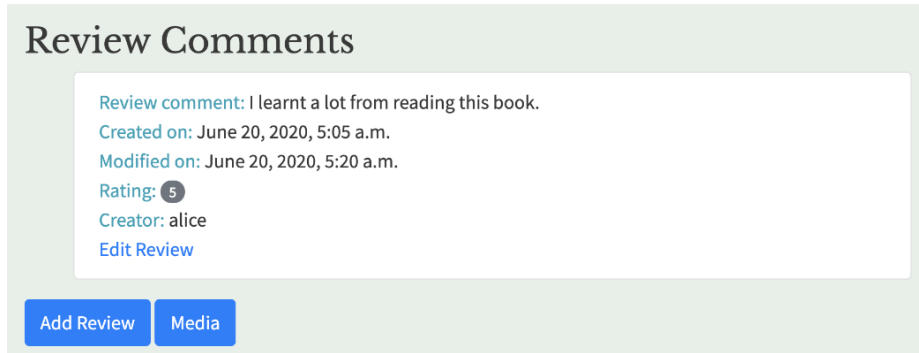


그림 9.10: 앨리스가 로그인했을 때 앨리스의 리뷰에 편집 리뷰 링크가 나타납니다

다른 사용자가 로그인하면 편집 리뷰 링크가 표시되지 않습니다:

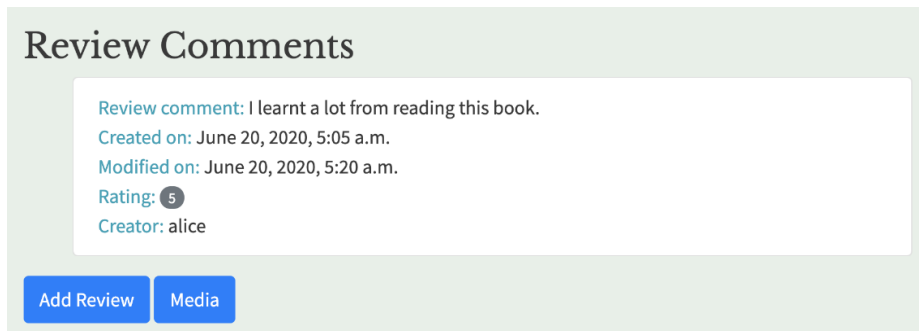


그림 9.11: 밥이 로그인했을 때 앨리스의 리뷰에는 편집 리뷰 링크가 없습니다

4. `template/base.html`을 수정하여 현재 인증된 사용자의 사용자 이름이 헤더의 검색 양식 오른쪽에 표시되고 사용자 프로필 페이지에 링크되도록 합니다. `idx_3970a5a2` 이 활동을 완료하면 현재 사용자의 인증 상태와 신원을 반영하는 동적 콘텐츠가 템플릿에 추가됩니다. 다음 스크린샷에서 볼 수 있습니다:

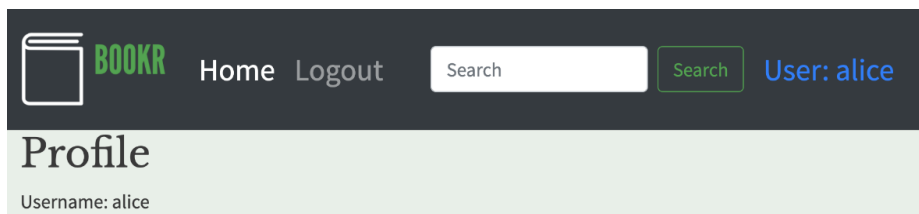


그림 9.12: 인증된 사용자의 이름이 검색 양식 뒤에 나타납니다

With that, we have reviewed the available authentication mechanisms in Django and can examine how Django implements sessions.

Sessions

It is worth looking at some theory to understand why sessions are a common solution in web applications for managing user content. The HTTP protocol defines the interactions between a client and a server. It is said to be a "stateless" protocol as no stateful information is retained by the server between requests. This protocol design worked well for delivering hypertextual information in the early days of the World Wide Web, but it did not suit the needs of secure web applications delivering customized information to specific users.

We are now acquainted with seeing websites adapt to our viewing habits. Shopping sites recommend similar products to the ones that we have recently viewed and tell us about products that are popular in our region. These features all required a stateful approach to website development. One of the most common ways to implement a stateful web experience is through **sessions**. A session refers to a user's current interaction with a web server or application and requires that data be persisted for the duration of the interaction. This may include information about the links that the user has visited, the actions that they have performed, and the preferences that they have made in their interactions.

If a user sets a blogging site to a dark theme on one page, there is an expectation that the next page will use the same theme as well. We describe this behavior as "maintaining state." A session key is stored client-side as a browser cookie, which can be identified with server-side information that persists while the user is logged in.

In Django, sessions are implemented as a form of middleware. When we initially created the app in *Chapter 4, Introduction to Django Admin*, session support was activated by default.

The session engine

Information about current and expired sessions needs to be stored somewhere. In the early days of the World Wide Web, this was done through saving session information in files on the server, but as web server architectures have become more elaborate and their performance demands have increased, other, more efficient strategies, such as a database or in-memory storage, have become the norm. By default, in Django, session information is stored in a project's database.

This is a reasonable default for most small projects. However, Django's middleware implementation of sessions gives us the flexibility to store our project's session information in a variety of ways to suit our system architecture and performance requirements. Each of these

이로써 우리는 Django에서 사용 가능한 인증 메커니즘을 검토하였고, Django가 세션을 어떻게 구현하는지 살펴볼 수 있습니다.

세션

사용자 콘텐츠 관리를 위한 웹 애플리케이션에서 세션이 일반적인 솔루션인 이유를 이해하기 위해 몇 가지 이론을 살펴볼 가치가 있습니다. HTTP 프로토콜은 클라이언트 idx_939d8884와 서버 간의 상호작용을 정의합니다. 이 프로토콜은 요청 간에 서버가 상태 정보를 유지하지 않기 때문에 "무상태" 프로토콜이라고 합니다. 이 프로토콜 설계는 초기 월드 와이드 웹에서 하이퍼텍스트 정보를 전달하는 데 잘 작동했지만, 특정 사용자에게 맞춤형 정보를 제공하는 보안 웹 애플리케이션의 요구에는 적합하지 않았습니다.

이제 우리는 웹사이트가 우리의 시청 습관에 맞춰 조정되는 것을 알고 있습니다. 쇼핑 사이트는 우리가 최근에 본 것과 유사한 제품을 추천하고, 우리 지역에서 인기 있는 제품에 대해 알려줍니다. 이러한 기능은 모두 웹사이트 개발에 대한 상태 기반 접근 방식을 필요로 했습니다. 상태 기반 웹 경험을 구현하는 가장 일반적인 방법 중 하나는 세션을 통해 이루어집니다. 세션 idx_f71b6004은 사용자의 웹 서버 또는 애플리케이션과의 현재 상호작용을 나타내며, 상호작용 기간 동안 데이터가 지속되어야 합니다. 여기에는 사용자가 방문한 링크, 수행한 작업 및 상호작용에서 설정한 기본 설정에 대한 정보가 포함될 수 있습니다.

사용자가 한 페이지에서 블로그 사이트를 어두운 테마로 설정하면, 다음 페이지도 동일한 테마를 사용할 것이라는 기대가 있습니다. 우리는 이 행동을 "상태 유지"라고 설명합니다. 세션 키는 클라이언트 측에서 브라우저 쿠키로 저장되며, 사용자가 로그인하는 동안 지속되는 서버 측 정보와 식별할 수 있습니다.

Django에서는 세션이 미들웨어의 한 형태로 구현됩니다. 우리가 4장에서 Django Admin 소개를 처음 만들었을 때, 세션 지원은 기본적으로 활성화되었습니다.

세션 엔진

현재 및 완료된 세션에 대한 정보는 어딘가에 저장되어야 합니다. 초기 idx_d9050f2d월드 와이드 웹에서는 서버의 파일에 세션 정보를 저장하는 방식으로 이루어졌지만, 웹 서버 아키텍처가 더 복잡해지고 성능 요구가 증가함에 따라 데이터베이스나 인메모리 저장소와 같은 다른 더 효율적인 전략이 표준이 되었습니다. 기본적으로 Django에서는 세션 정보가 프로젝트의 데이터베이스에 저장됩니다.

이것은 대부분의 소규모 프로젝트에 대한 합리적인 기본값입니다. 그러나 Django의 미들웨어 세션 구현은 시스템 아키텍처와 성능 요구 사항에 맞게 프로젝트의 세션 정보를 다양한 방법으로 저장할 수 있는 유연성을 제공합니다. idx_d1d3b479의 각

different implementations is called a **session engine**. If we want to change the session configuration, we need to specify the `SESSION_ENGINE` setting in the project's `settings.py` file:

- **Cached sessions:** In some environments, caching session information in memory or a database is an approach that is suited to high performance. Django provides the `django.contrib.sessions.backends.cache` and `django.contrib.sessions.backends.cached_db` session engines for this purpose.
- **File-based sessions:** As stated earlier, this is a somewhat antiquated way of maintaining session information, but it may suit some sites where performance is not an issue, and there are reasons not to store dynamic information in a database.
- **Cookie-based sessions:** Rather than keeping session information server-side, you can keep them entirely in the web browser client by serializing the content of the session as JSON and storing it in a browser-based cookie.

Do you need to flag cookie content?

All of Django's implementations of sessions require storing a session ID in a cookie on the user's web browser.

Regardless of the session engine employed, all these middleware implementations involve storing a site-specific cookie in the web browser. In the early days of web development, it was not uncommon to pass session IDs as URL arguments, but this approach has been eschewed in Django for reasons of security.

In many jurisdictions, including the European Union, websites are legally required to warn users if the site sets cookies in their browsers. If there are such legislative requirements in the region where you intend to operate your site, it is your responsibility to ensure that the code meets these obligations. Be sure to use up-to-date implementations and avoid using abandoned projects that have not kept pace with legislative changes.

Note

To cater to these changes and legislative requirements, there are many useful apps, such as **Django Cookie Consent** and **Django Cookie Law**, that are designed to work with several legislative frameworks. You can find out more by going to the following links:

- <https://github.com/jazzband/django-cookie-consent>
- <https://github.com/TyMaszWeb/django-cookie-law>

Many JavaScript modules exist that implement similar cookie consent mechanisms.

다양한 구현을 세션 엔진이라고 합니다. 세션 구성을 변경하려면 프로젝트의 `settings.py` 파일에서 `SESSION_ENGINE` 설정을 지정해야 합니다:

- 캐시된 세션: 일부 `idx_6e8ca0fbenvironments`에서는 메모리나 데이터베이스에 세션 정보를 캐시하는 것이 높은 성능에 적합한 접근 방식입니다. Django는 이를 위해 `django.contrib.sessions.backends.cache` 및 `django.contrib.sessions.backends.cached_db` 세션 엔진을 제공합니다.
- 파일 기반 세션: 앞서 언급했듯이, 이는 세션 정보를 유지하는 다소 구식 방법이지만, 성능이 문제가 되지 않는 일부 사이트에는 적합할 수 있으며, 동적 정보를 데이터베이스에 저장하지 않아야 하는 이유가 있을 수 있습니다.
- 쿠키 기반 세션: 세션 정보를 서버 측에 유지하는 대신, 세션의 내용을 JSON으로 직렬화하고 브라우저 기반 쿠키에 저장하여 웹 브라우저 클라이언트에 완전히 유지할 수 있습니다.

쿠키 콘텐츠에 플래그를 지정해야 하나?

Django의 모든 `idx_7c9575e9` 세션 구현은 사용자의 웹 브라우저에 쿠키에 세션 ID를 저장해야 합니다.

사용되는 세션 엔진에 관계없이, 이러한 모든 미들웨어 구현은 웹 브라우저에 사이트별 쿠키를 저장하는 것을 포함합니다. 웹 개발 초기에는 세션 ID를 URL 인수로 전달하는 것이 드문 일이 아니었지만, 보안상의 이유로 Django에서는 이 접근 방식을 피하고 있습니다.

유럽 연합을 포함한 많은 관할권에서는 웹사이트가 사용자의 브라우저에 쿠키를 설정하는 경우 사용자에게 경고할 법적 의무가 있습니다. 사이트를 운영할 지역에 이러한 법적 요구 사항이 있는 경우, 코드가 이러한 의무를 충족하는지 확인하는 것은 귀하의 책임입니다. 최신 구현을 사용하고 법적 변경 사항에 발맞추지 못한 버려진 프로젝트를 사용하지 않도록 하십시오.

메모

이러한 `idx_551754d6` 변경 사항 및 법적 요구 사항을 충족하기 위해 Django Cookie Consent 및 Django Cookie Law와 같이 여러 법적 프레임워크에서 작동하도록 설계된 유용한 앱이 많이 있습니다. 다음 링크로 이동하여 자세히 알아볼 수 있습니다:

- <https://github.com/jazzband/django-cookie-consent>
- <https://github.com/TyMaszWeb/django-cookie-law>

유사한 쿠키 동의 메커니즘을 구현하는 많은 JavaScript 모듈이 존재합니다.

With this in mind, we can examine how session data is implemented in Django.

Pickle or JSON storage?

Python provides the **pickle** module in its standard library for serializing Python objects into a byte stream representation. A pickle is a binary structure that has the benefit of being interoperable between different architectures and different versions of Python, so that a Python object can be serialized to a pickle on a Windows PC and deserialized to a Python object on a Linux Raspberry Pi.

This flexibility comes with security vulnerabilities, and it is not recommended that it be used to represent untrusted data. Consider the following Python object, which contains several types of data. It can be serialized using pickle:

```
import datetime
data = dict(
    viewed_books=[17, 18, 3, 2, 1],
    search_history=['1981', 'Machine Learning', 'Bronte'],
    background_rgb=(96, 91, 92),
    foreground_rgb=(17, 17, 17),
    last_login_login=datetime.datetime(2019,12,3,15,30,30),
    password_change=datetime.datetime(2019,9,2,8,41,25),
    user_class='Veteran',
    average_rating=4.75,
    reviewed_books={18, 3, 7}
)
```

Using the `dumps` (dump string) method of the `pickle` module, we can serialize the data object to produce a byte representation:

```
import pickle
data_pickle = pickle.dumps(data)
```

JSON stands for **JavaScript Object Notation**. The syntax of JSON is a small subset of the JavaScript language. It is a widespread standard for messaging and data exchange, commonly used for transferring data between web browsers and servers. We can serialize JSON with a similar approach to the one that we outlined with the `pickle` format:

```
import json
data_json = json.dumps(data)
```

이 점을 염두에 두고, 우리는 Django에서 세션 데이터가 어떻게 구현되는지 살펴볼 수 있습니다.

Pickle 또는 JSON 저장?

Python은 표준 라이브러리에서 Python 객체를 바이트 스트림 표현으로 직렬화하기 위해 pickle 모듈을 제공합니다. pickle은 서로 다른 아키텍처와 서로 다른 버전의 Python 간에 상호 운용성이 있는 이점이 있는 이진 구조입니다. 따라서 Windows PC에서 pickle로 직렬화된 Python 객체는 Linux Raspberry Pi에서 Python 객체로 역직렬화될 수 있습니다.

이 유연성은 보안 취약점과 함께 제공되며, 신뢰할 수 없는 데이터를 나타내는 데 사용되는 것은 권장되지 않습니다. 여러 유형의 데이터를 포함하는 다음 Python 객체를 고려하십시오. pickle을 사용하여 직렬화할 수 있습니다:

```
import datetime
data = dict(
    viewed_books=[17, 18, 3, 2, 1],
    search_history=['1981', 'Machine Learning', 'Bronte'],
    background_rgb=(96, 91, 92),
    foreground_rgb=(17, 17, 17),
    last_login_login=datetime.datetime(2019, 12, 3, 15, 30, 30),
    password_change=datetime.datetime(2019, 9, 2, 8, 41, 25),
    user_class='Veteran',
    average_rating=4.75,
    reviewed_books={18, 3, 7}
)
```

pickle 모듈의 dumps (dump string) 메서드를 사용하여 데이터 객체를 직렬화하여 바이트 표현을 생성할 수 있습니다:

```
import pickle
data_pickle = pickle.dumps(data)
```

JSON은 JavaScript 객체 표기법을 의미합니다. JSON의 구문은 JavaScript 언어의 작은 하위 집합입니다. 이는 메시징 및 데이터 교환을 위한 널리 사용되는 표준으로, 일반적으로 웹 브라우저와 서버 간의 데이터 전송에 사용됩니다. 우리는 피클 형식으로 설명한 것과 유사한 접근 방식을 사용하여 JSON을 직렬화할 수 있습니다:

```
import json
data_json = json.dumps(data)
```

Because data contains Python datetime and set objects, which aren't serializable with JSON, when we attempt to serialize the structure, a type error will be thrown:

```
TypeError: Object of type datetime is not JSON serializable
```

For serializing to JSON, we could convert the datetime objects into the string type and set them in a list:

```
data['last_login_login'] = \
    data['last_login_login'].strftime("%Y%d%m%H%M%S")
data['password_change'] = \
    data['password_change'].strftime("%Y%d%m%H%M%S")
data['reviewed_books'] = list(data['reviewed_books'])
```

As JSON data is human-readable, it is easy to examine:

```
{"viewed_books": [17, 18, 3, 2, 1], "search_history": ["1981", "Machine
Learning", "Bronte"], "background_rgb": [96, 91, 92], "foreground_rgb": [17, 17,
17], "last_login_login": "20190312153030", "password_change": "20190209084125",
"user_class": "Veteran", "average_rating": 4.75, "reviewed_books": [18, 3, 7]}
```

Note that we had to explicitly convert the datetime and set objects, but the tuple is automatically converted into a list by the JSON. Django ships with `PickleSerializer` and `JSONSerializer`. If the serializer needs to be altered, it can be changed by setting the `SESSION_SERIALIZER` variable in the project's `settings.py` file:

```
SESSION_SERIALIZER = 'django.contrib.sessions.serializers.JSONSerializer'
```

Exercise 9.06 – Examining the session key

The purpose of this exercise is to query the project's SQLite database and perform queries on the session table to become familiar with how session data is stored. You will then create a Django management command for examining session data that is stored using `JSONSerializer`:

1. Start the **DB Browser for SQLite** application and open your `db.sqlite3` database:

데이터에 Python datetime 및 set 객체가 포함되어 있어 JSON으로 직렬화할 수 없기 때문에 구조를 직렬화하려고 하면 유형 오류가 발생합니다:

```
TypeError: datetime 유형의 객체는 JSON으로 직렬화할 수 없습니다
```

JSON으로 직렬화하기 위해 datetime 객체를 문자열 유형으로 변환하고 목록에 설정할 수 있습니다:

```
data['last_login_login'] = \
    data['last_login_login'].strftime("%Y%d%m%H%M%S")
data['password_change'] = \
    data['password_change'].strftime("%Y%d%m%H%M%S")
data['reviewed_books'] = list(data['reviewed_books'])
```

JSON 데이터는 사람이 읽을 수 있으므로 쉽게 검사할 수 있습니다:

```
{ "viewed_books": [17, 18, 3, 2, 1], "search_history": ["1981", "MachineLearning", "Bronte"], "background_rgb": [96, 91, 92], "foreground_rgb": [17, 17, 17], "last_login_login": "20190312153030", "password_change": "20190209084125", "user_class": "Veteran", "average_rating": 4.75, "reviewed_books": [18, 3, 7]}
```

우리는 datetime과 set 객체를 명시적으로 변환해야 했지만, 튜플은 JSON에 의해 자동으로 리스트로 변환됩니다. Django는 PickleSerializer와 JSONSerializer를 제공합니다. 직렬 변환기를 변경해야 하는 경우, 프로젝트의 settings.py 파일에서 SESSION_SERIALIZER 변수를 설정하여 변경할 수 있습니다:

```
SESSION_SERIALIZER = 'django.contrib.sessions.serializers.JSONSerializer'
```

연습 9.06 – 세션 키 검사

이 연습의 목적은 프로젝트의 SQLite 데이터베이스를 쿼리하고 세션 테이블에 대한 쿼리를 수행하여 세션 데이터가 어떻게 저장되는지 익숙해지는 것입니다. 그런 다음 JSONSerializer를 사용하여 저장된 세션 데이터를 검사하는 Django 관리 명령을 생성합니다:

1. SQLite 애플리케이션의 DB 브라우저를 시작하고 db.sqlite3 데이터베이스를 엽니다:

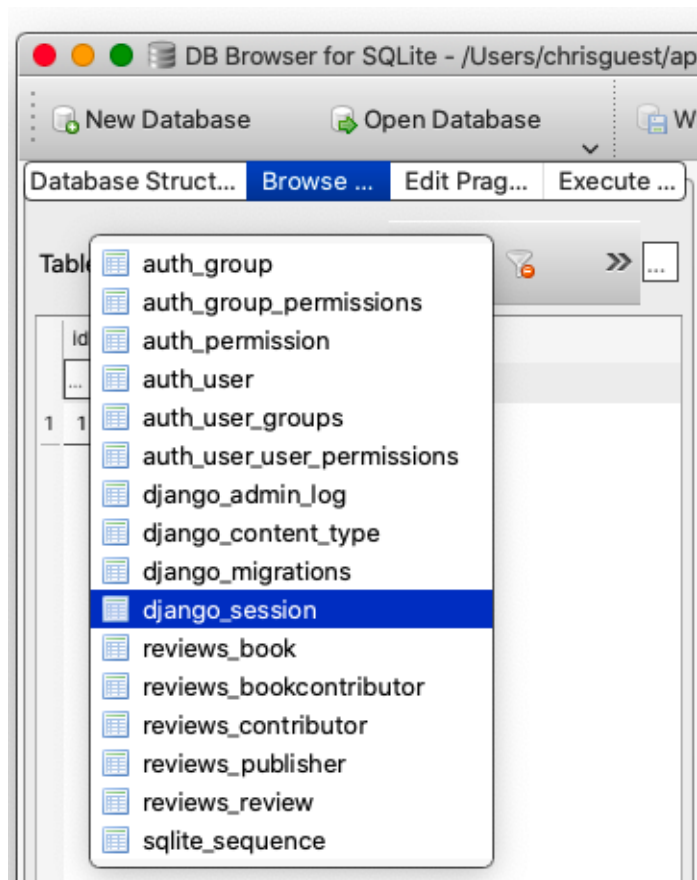


Figure 9.13: Table selection in DB Browser for SQLite

2. Under the **Database Structure** tab, expand the `django_session` table:

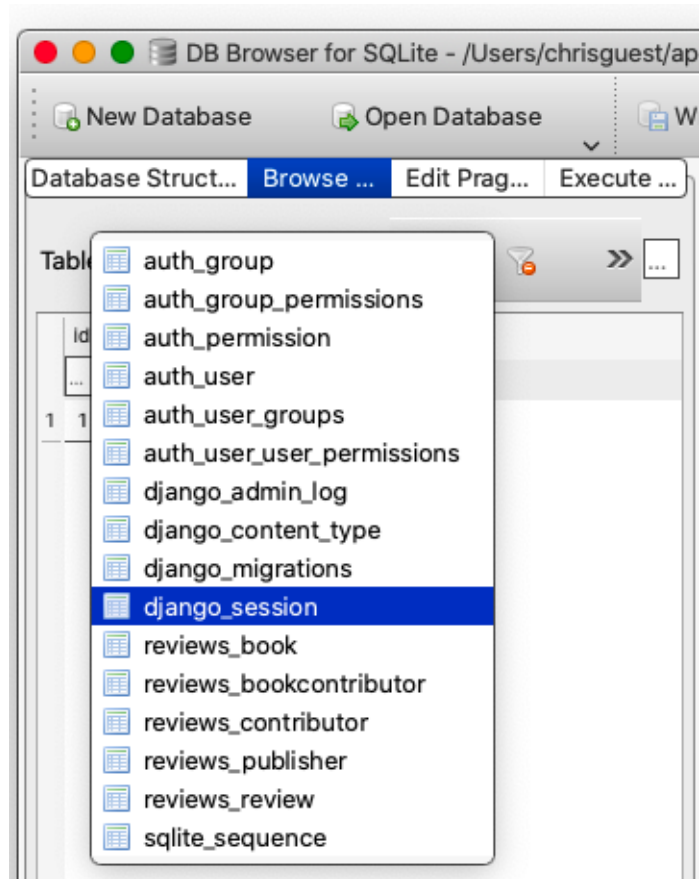


그림 9.13: SQLite용 DB 브라우저에서의 테이블 선택

2. idx_ae634ff4 아래의 데이터베이스 구조 탭에서 django_session 테이블을 확장합니다:

Name	Type	Schema
auth_group		CREATE TABLE "auth_group" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "name" va
auth_group_permissions		CREATE TABLE "auth_group_permissions" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT
auth_permission		CREATE TABLE "auth_permission" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "cont
auth_user		CREATE TABLE "auth_user" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "password"
auth_user_groups		CREATE TABLE "auth_user_groups" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "use
auth_user_user_permissions		CREATE TABLE "auth_user_user_permissions" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREM
django_admin_log		CREATE TABLE "django_admin_log" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "act
django_content_type		CREATE TABLE "django_content_type" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "i
django_migrations		CREATE TABLE "django_migrations" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "ap
django_session		CREATE TABLE "django_session" ("session_key" varchar(40) NOT NULL PRIMARY KEY, "session.d
session_key	varchar(40)	"session_key" varchar(40) NOT NULL
session_data	text	"session_data" text NOT NULL
expire_date	datetime	"expire_date" datetime NOT NULL
reviews_book		CREATE TABLE "reviews_book" ("id" Integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" va

Figure 9.14: Table definitions in DB Browser for SQLite

This reveals that the `django_session` table in the database stores session information in the `session_key`, `session_data`, and `expire_date` fields.

- Click on the **Browse Data** tab and select the `django_session` table:

	session_key	session_data	expire_date
1	ocgo7sy paz01tlw92dr1ftfkgoy0ux03	.eJxVJDEOWjAMRe-...	2022-06-19 07:28:26.390874
2	y2r3d73fwtpmsm2azc2a01ehdnbhr	.eJxVJDEOWIAQRe_C2pAyQAsu3XsGMSOAVA0kpV0Z765Nu...	2022-06-19 07:29:15.506410
3	gyfclclqm8ht8vms3c8l8m8m6zh1tgi	.eJxVJDEOWyAQBP9CHaGzDxCKTJ83olOD4CQCydIVib_Hil...	2022-06-19 07:30:25.165834

Figure 9.15: Querying data in the `django_session` table

- Django stores the session data using cryptographically signed values. Because, by default, we are using the database to store our sessions, we will import the `Session` model, as well as the `SessionStore` class of `django.contrib.sessions.backends.db`. `SessionStore` has a `decode` method that we can use to read the encoded `session_data` from the `Session` object.
- This code snippet can be executed using the Django shell by entering `python manage.py shell` at the command line to get an interactive prompt:

```
>>> from django.contrib.sessions.backends.db import SessionStore
>>> from django.contrib.sessions.models import Session
>>> session_store = SessionStore()
>>> sessions = Session.objects.all()
>>> session_store.decode(sessions[0].session_data)
```

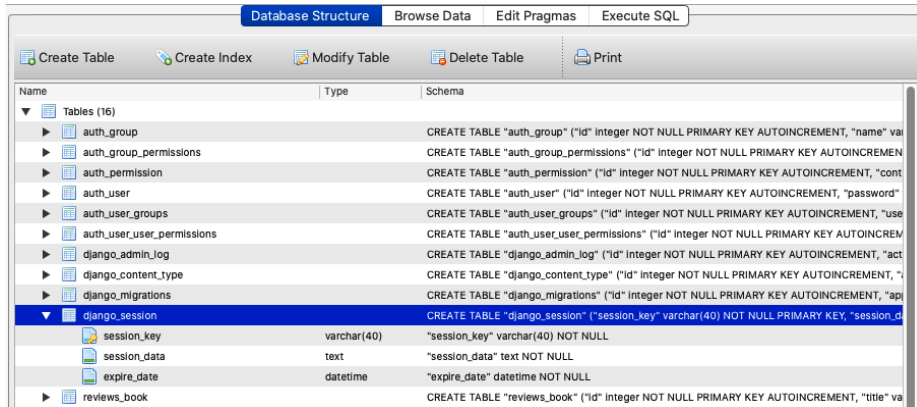


그림 9.14: DB Browser for SQLite의 테이블 정의

이것은 데이터베이스의 django_session 테이블이 session_key, session_data 및 expire_date 필드에 세션 정보를 저장하고 있음을 보여줍니다.

3. 데이터 탐색 탭을 클릭하고 django_session 테이블을 선택하세요:

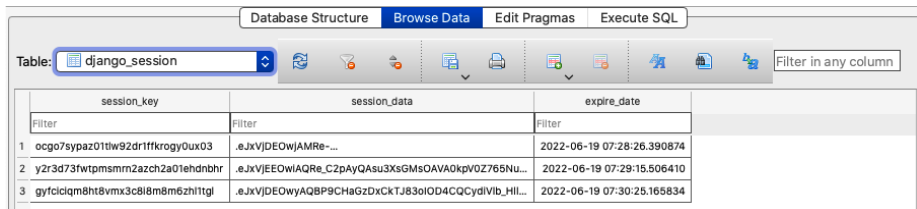


그림 9.15: django_session 테이블에서 데이터 쿼리하기

4. Django는 암호화된 서명 값을 사용하여 세션 데이터를 저장합니다. 기본적으로 데이터베이스를 사용하여 세션을 저장하고 있으므로, Session 모델과 django.contrib.sessions.backends.db.SessionStore의 SessionStore 클래스를 가져올 것입니다. 이 클래스에는 Session 객체에서 인코딩된 session_data를 읽는 데 사용할 수 있는 decode 메서드가 있습니다.
5. 이 코드 스니펫은 명령줄에 pythonmanage.py shell을 입력하여 Django 셸을 사용하여 실행할 수 있습니다. 인터랙티브 프롬프트를 얻으려면:

```
>>> from django.contrib.sessions.backends.db import
SessionStore>>> from django.contrib.sessions.models import
Session>>> session_store = SessionStore()
>>> sessions = Session.objects.all()>>>
session_store.decode(sessions[0].session_data)
```



```
{'_auth_user_id': '2', '_auth_user_backend':  
'django.contrib.auth.backends.ModelBackend', '_auth_user_hash':  
'eb5de6bf54460bafc9e0f555c65d16b17b3fd754617d1d73529d9605fea2f0c'}
```

6. Using this approach, we can create a management command for examining session data. Sensitive data may be stored in sessions, so creating a management command ensures that the data is only accessible by a user with console access to the server that is hosting Django.
7. Using **PyCharm**, in the `reviews/management/commands` directory of the `bookr` project, create a Python file called `sessioninfo.py`.
8. Let's start by importing the necessary modules. We will use the `pformat` command from the `pprint` module from Python's Standard Library to format session data. We will need the `User` model from `django.contrib.auth.models` and the `Session` model from `django.contrib.sessions.models` to query `User` and `Session` objects. The `BaseCommand` class from `django.core.management.base` provides the scaffolding for our Django Admin command:

```
from pprint import pformat  
  
from django.contrib.auth.models import User  
from django.contrib.sessions.models import Session  
from django.contrib.sessions.backends.db import SessionStore  
from django.core.management.base import BaseCommand
```

9. At a minimum, a user command needs to be defined by subclassing `BaseCommand` as the `Command` class and defining a `handle` method. Typically, additional command-line parameters would be defined in an `add_arguments` method, but we are aiming to create a very simple example. A `help` attribute of the `Command` class is used to provide information when the user enters `python manage.py --help` or `python manage.py sessioninfo --help`:

```
class Command(BaseCommand):  
    help = "List all user sessions and the data that they contain."
```

10. Write a `handle` method of the `Command` class. First, instantiate a `SessionStore` object, then iterate through the `Session` objects and decode the data for each using the `SessionStore.decode` method. The `_auth_user_id` key of the decrypted data

```
{'_auth_user_id': '2', '_auth_user_backend':
'django.contrib.auth.backends.ModelBackend', '_auth_user_hash':
'eb5de6bf54460bafc9e0f555c65d16b17b3fdf754617d1d73529d9605fea2f0c'}
```

6. 이 접근 방식을 사용하여 세션 데이터를 검사하는 관리 명령을 만들 수 있습니다. 민감한 `idx_6cec1fcfd` 데이터는 세션에 저장될 수 있으므로, 관리 명령을 생성하면 데이터에 대한 접근이 Django를 호스팅하는 서버에 대한 콘솔 접근 권한이 있는 사용자만 가능하도록 보장됩니다.
7. PyCharm을 사용하여 bookr 프로젝트의 `reviews/management/commands` 디렉토리에서 `sessioninfo.py`라는 `idx_752bec90a` Python 파일을 만듭니다.
8. 필요한 모듈을 가져오는 것부터 시작하겠습니다. 세션 데이터를 포맷하기 위해 Python의 표준 라이브러리에서 `pprint` 모듈의 `pformat` 명령을 사용할 것입니다. `User` 및 `Session` 객체를 쿼리하기 위해 `django.contrib.auth.models`에서 `User` 모델과 `django.contrib.sessions.models`에서 `Session` 모델이 필요합니다. `django.core.management.base`의 `BaseCommand` 클래스는 우리의 Django Admin 명령을 위한 골격을 제공합니다:

```
from pprint import pformat

from django.contrib.auth.models import User
from django.contrib.sessions.models import Session
from django.contrib.sessions.backends.db import SessionStore
from django.core.management.base import BaseCommand
```

9. 최소한 사용자 명령은 `BaseCommand`를 서브클래싱하여 `Command` 클래스를 정의하고 `handle` 메서드를 정의해야 합니다. 일반적으로 추가적인 명령줄 매개 변수는 `add_arguments` 메서드에서 정의되지만, 우리는 매우 간단한 예제를 만들고자 합니다. `Command` 클래스의 `help` 속성은 사용자가 `python manage.py --help` 또는 `python manage.py sessioninfo --help`를 입력할 때 정보를 제공하는 데 사용됩니다:

```
class Command(BaseCommand):
    help = "모든 사용자 세션과 그들이 포함하고 있는 데이터를 나열합니다."
```

10. `Command` 클래스의 `handle` 메서드를 작성하십시오. 먼저, `SessionStore` 객체를 인스턴스화한 다음, `Session` 객체를 반복하고 `SessionStore.decode` 메서드를 사용하여 `idx_9da79de2each`에 대한 데이터를 디코드합니다. 복호화된 데이터의 `_auth_user_id` 키

references a `User.id` from the database so that we can retrieve the user using the `User` model.

Now, for each session, write the session key, user ID, username, and email address to the console. We will develop a simple Python utility for decrypting this session information. Note that it is best practice to call `self.stdout.write` in a management command instead of `print` when writing to the console:

```
def handle(self, *args, **options):
    session_store = SessionStore()
    for session in Session.objects.all():
        data =
    session_store.decode(session.session_data)
    user =
    User.objects.get(id=data['_auth_user_id'])
    self.stdout.write(
        f"Session Key: {session.session_key} "
        f"User: {user.id} {user.username}
        {user.email} ")
    self.stdout.write(pformat(data))
```

11. Now, you can use this management command to examine session data that is stored in the database. You can call it on the command line as follows:

```
python manage.py sessioninfo
```

This will be useful for debugging session behavior when you attempt the final activity:

```
> python manage.py sessioninfo
Session Key: ocgo7sy paz01tlw92dr1ffkrogy0ux03 User: 2 carol carol.brown@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': 'eb5de6bf54460bafc9e0f555c65d16b17b3fdf754617d1d73529d9605fea2f0c',
 '_auth_user_id': '2'}
Session Key: y2r3d73fwtpmsmrn2azch2a01ehdnbhr User: 1 bob bob.black@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': '322625efba7788f598d02fca5ad86bb14db4893f3c92bc4099822ea3bf4d1e4f',
 '_auth_user_id': '1'}
Session Key: gyfciciqm8ht8vmx3c8i8m8m6zh11tgl User: 3 bookradmin bookradmin@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': 'a30d9315c9b5dda38802da5e8e64f94b0b063f0136fad8456205ed8cced8ff30',
 '_auth_user_id': '3'}
```

Figure 9.16: Python script

데이터베이스에서 User.id를 참조하여 Usermodel을 사용하여 사용자를 검색할 수 있습니다.

이제 각 세션에 대해 세션 키, 사용자 ID, 사용자 이름 및 이메일 주소를 콘솔에 기록하십시오. 이 세션 정보를 해독하기 위한 간단한 Python 유틸리티를 개발할 것입니다. 콘솔에 기록할 때 print 대신 management command에서 self.stdout.write를 호출하는 것이 모범 사례임을 유의하십시오:

```
def handle(self, *args, **options):
    session_store = SessionStore()
    for session in Session.objects.all():
        data =
    session_store.decode(session.session_data)
        user =
    User.objects.get(id=data['_auth_user_id'])
        self.stdout.write(
            f"세션 키: {session.session_key} "
            f"사용자: {user.id} {user.username}
            {user.email} ")
        self.stdout.write(pformat(data))
```

11. 이제 이 관리 명령을 사용하여 데이터베이스에 저장된 세션 데이터를 검사할 수 있습니다. 다음과 같이 명령줄에서 호출할 수 있습니다:

```
python manage.py sessioninfo
```

이것은 최종 활동을 시도할 때 세션 동작을 디버깅하는 데 유용할 것입니다:

```
> python manage.py sessioninfo
Session Key: ocgo7sy paz01tlw92dr1ffkrogy0ux03 User: 2 carol carol.brown@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': 'eb5de6bf54460bafc9e0f555c65d16b17b3fdf754617d1d73529d9605fea2f0c',
 '_auth_user_id': '2'}
Session Key: y2r3d73fwt pmsmrn2azch2a01ehdnbhr User: 1 bob bob.black@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': '322625efba7788f598d02fca5ad86bb14db4893f3c92bc4099822ea3bf4d1e4f',
 '_auth_user_id': '1'}
Session Key: gyfciciqm8ht8vmx3c8i8m8m6zh11tgl User: 3 bookradmin bookradmin@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': 'a30d9315c9b5dda38802da5e8e64f94b0b063f0136fad8456205ed8cced8ff30',
 '_auth_user_id': '3'}
```

그림 9.16: 파이썬 스크립트

This script outputs the decoded session information. At present, the session only contains three keys:

- `_auth_user_backend` is a string representation of the class of the user backend. As our project stores user credentials in the model, `ModelBackend` is used.
- `_auth_user_hash` is a hash of the user's password.
- `_auth_user_id` is the user ID obtained from the model's `User.id` attribute.

This exercise helped you become familiar with how session data is stored in Django. We will now turn our attention to adding additional information to Django sessions.

Storing data in sessions

We've already covered the way sessions are implemented in Django. Now, we are going to briefly examine some of the ways that we can make use of sessions to enrich our user experience. In Django, `session` is an attribute of the request object. It is implemented as a dictionary-like object. In our views, we can assign keys to the `session` object like a typical dictionary, as shown here:

```
request.session['books_reviewed_count'] = 39
```

But there are some restrictions. Firstly, the keys in the session must be strings, so integers and timestamps are not allowed. Secondly, keys starting with an underscore are reserved for internal system use. Data is limited to values that can be encoded as JSON, so some byte sequences that can't be decoded as UTF-8, such as `binary_key` listed previously, can't be stored as JSON data. The other warning is to avoid reassigning `request.session` to a different value. We should only assign or delete keys. So, *don't do* this:

```
request.session = {'books_read_count':30,  
                  'books_reviewed_count': 39}
```

Instead, *do* this:

```
request.session['books_read_count'] = 30  
request.session['books_reviewed_count'] = 39
```

With those restrictions in mind, we will investigate how we can make use of session data in our Reviews application.

이 스크립트는 디코딩된 세션 정보를 출력합니다. 현재 세션에는 세 개의 키만 포함되어 있습니다:

- `_auth_user_backend`는 사용자 백엔드 클래스의 문자열 표현입니다. 우리 프로젝트는 모델에 사용자 자격 증명을 저장하므로 `ModelBackend`가 사용됩니다. `_auth_user_hash`는 사용자의 비밀번호 해시입니다.
-

◦ `_auth_user_id`는 모델의 `User.id` 속성에서 얻은 `idx_3aed0ec9` 사용자 ID입니다. 이 연습은 Django에서 세션 데이터가 어떻게 저장되는지 익숙해지는 데 도움이 되었습니다. 이제 Django 세션에 추가 정보를 추가하는 데 주의를 기울이겠습니다.

세션에 데이터 저장하기

우리는 이미 Django에서 세션이 구현되는 방법을 다루었습니다. 이제 우리는 사용자 경험을 풍부하게 하기 위해 세션을 활용할 수 있는 몇 가지 방법을 간단히 살펴볼 것입니다. Django에서 세션은 요청 객체의 속성입니다. 사전과 유사한 객체로 구현됩니다. 우리의 뷰에서 우리는 일반적인 사전처럼 세션 객체에 키를 할당할 수 있습니다. 다음과 같이:

```
request.session['books_reviewed_count'] = 39
```

하지만 몇 가지 제한 사항이 있습니다. 첫째, 세션의 키는 문자열이어야 하므로 정수와 타임스탬프는 허용되지 않습니다. 둘째, 언더스코어로 시작하는 키는 내부 시스템 사용을 위해 예약되어 있습니다. 데이터는 JSON으로 인코딩할 수 있는 값으로 제한되므로, 이전에 나열된 `binary_key`와 같이 UTF-8로 디코딩할 수 없는 일부 바이트 시퀀스는 JSON 데이터로 저장할 수 없습니다. 또 다른 경고는 `request.session`을 다른 값으로 재할당하지 않는 것입니다. 우리는 키를 할당하거나 삭제하는 것만 해야 합니다. 그러므로, 이렇게 하지 마세요:

```
request.session = {'books_read_count':30,
                   'books_reviewed_count': 39}
```

대신 이렇게 하세요:

```
request.session['books_read_count'] = 30
request.session['books_reviewed_count'] = 39
```

이러한 제한 사항을 염두에 두고, 우리는 리뷰 애플리케이션에서 `idx_bacf7429` 세션 데이터를 어떻게 활용할 수 있는지 조사할 것입니다.

Exercise 9.06 – Storing recently viewed books in sessions

The purpose of this exercise is to use the session to keep information about the 10 books that have been most recently browsed by the authenticated user. This information will be displayed on the profile page of the bookr project. When a book is browsed, the `book_detail` view is called. In this exercise, we will edit `reviews/views.py` and add some additional logic to the `book_detail` method. We will add a key to the session called `viewed_books`. Using basic knowledge of HTML and CSS, the page can be created to show the profile details and viewed books stored in separate divisions of the page, as follows:

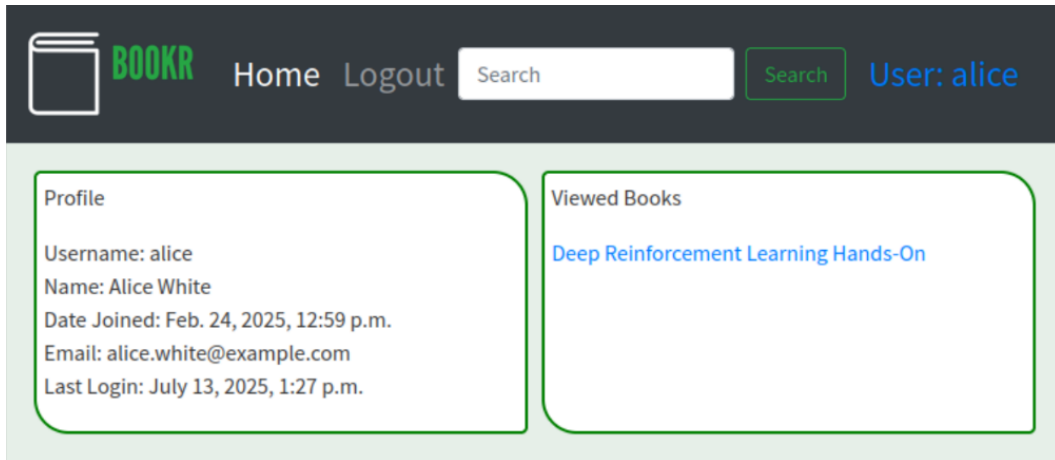


Figure 9.17: The Profile page incorporating Viewed Books

1. Edit `reviews/views.py` and the `book_detail` method. We are only interested in adding session information for authenticated users, so add a conditional statement to check whether the user is authenticated and set `max_viewed_books_length`, the maximum length of the viewed books list, to 10:

```
def book_detail(request, pk):
    ...
    if request.user.is_authenticated:
        max_viewed_books_length = 10
```

연습 9.06 – 최근에 본 책을 세션에 저장하기

이 idx_402baab2 연습의 목적은 세션을 사용하여 인증된 사용자가 가장 최근에 탐색한 10권의 책에 대한 정보를 유지하는 것입니다. 이 정보는 bookr 프로젝트의 프로필 페이지에 표시됩니다. 책이 탐색되면 book_detail 뷰가 호출됩니다. 이 연습에서는 reviews/views.py를 편집하고 book_detail 메서드에 추가 로직을 추가할 것입니다. 우리는 viewed_books라는 세션 키를 추가할 것입니다. HTML 및 CSS에 대한 기본 지식을 사용하여 페이지를 생성하여 프로필 세부정보와 페이지의 별도 구역에 저장된 본 책들을 표시할 수 있습니다. 다음과 같이:

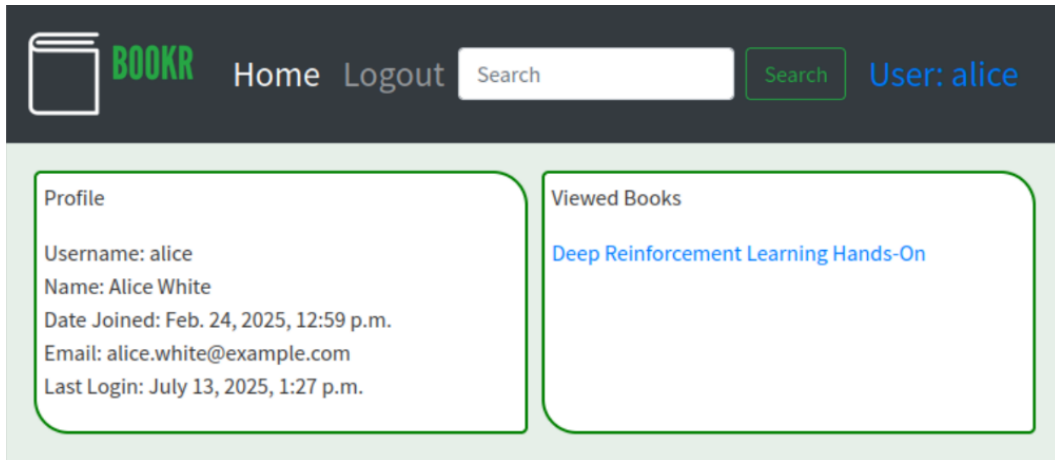


그림 9.17: 본 책을 포함한 프로필 페이지

1. reviews/views.py와 book_detail 메서드를 편집합니다. 우리는 인증된 사용자에게 대한 세션 정보 idx_c3638e2e를 추가하는 것에만 관심이 있으므로 사용자가 인증되었는지 확인하는 조건문을 추가하고 viewed_books 목록의 최대 길이인 max_viewed_books_length를 10으로 설정합니다:

```
def book_detail(request, pk):
    ...
    if request.user.is_authenticated:
        max_viewed_books_length = 10
```


2. Within the same conditional block, add code to retrieve the current value of `request.session['viewed_books']`. If this key isn't present in the session, start with an empty list:

```
viewed_books = request.session.get(
    'viewed_books', [])
```

3. If the current book's primary key is already present in `viewed_books`, the following code will remove it:

```
viewed_book = [book.id, book.title]
if viewed_book in viewed_books:
    viewed_books.pop(viewed_books.index(viewed_book))
```

4. The following code inserts the current book's primary key at the start of the `viewed_books` list:

```
viewed_books.insert(0, viewed_book)
```

5. Add the following key to only keep the first 10 elements of the list:

```
viewed_books = viewed_books[:
    max_viewed_books_length]
```

6. The following code will add our `viewed_books` back to `session['viewed_books']` so that it is available in subsequent requests:

```
request.session['viewed_books'] = viewed_books
```

7. As before, at the end of the `book_detail` function, render the `reviews/book_detail.html` template given the request and context data:

```
return render(request, "reviews/book_detail.html",
    context)
```

Once complete, the `book_detail` view will have this conditional block:

```
def book_detail(request, pk):
    ...
    if request.user.is_authenticated:
```

2. ... 같은 조건문 블록 내에서 `request.session['viewed_books']`의 현재 값을 가져오는 코드를 추가하세요. 이 키가 세션에 없으면 빈 리스트로 시작하세요:

```
viewed_books = request.session.get(
    'viewed_books', [])
```

3. ... 현재 책의 기본 키가 이미 `viewed_books`에 존재하는 경우, 다음 사항을 코드는 그것을 제거할 것입니다:

```
viewed_book = [book.id, book.title]
if viewed_book in viewed_books:
    viewed_books.pop(
        viewed_books.index(viewed_book))
```

4. ... 다음 코드는 현재 책의 기본 키를 `viewed_books` 목록의 시작 부분에 삽입합니다:

```
viewed_books.insert(0, viewed_book)
```

5. `idx_968ca2e5` 다음 키를 추가하여 목록의 처음 10개 요소만 유지합니다:

```
viewed_books = viewed_books[:
    max_viewed_books_length]
```

6. 다음 코드는 우리의 `viewed_books`를 `session['viewed_books']`에 다시 추가하여 후속 요청에서 사용할 수 있도록 합니다:

```
request.session['viewed_books'] = viewed_books
```

7. 이전과 마찬가지로, `book_detail` 함수의 끝에서 요청과 컨텍스트 데이터를 고려하여 `reviews/book_detail.html` 템플릿을 렌더링합니다:

```
return render(request, "reviews/book_detail.html",
    context)
```

완료되면, `book_detail` 뷰는 이 조건 블록을 가집니다:

```
def book_detail(request, pk):
    ...
    if request.user.is_authenticated:
```

```

max_viewed_books_length = 10
viewed_books =
    request.session.get('viewed_books', [])
viewed_book = [book.id, book.title]
if viewed_book in viewed_books:
    viewed_books.pop(viewed_books.index(viewed_book))
viewed_books.insert(0, viewed_book)
viewed_books = viewed_books[:max_viewed_books_length]
request.session['viewed_books'] = viewed_books
return render(request, "reviews/book_detail.html",
               context)

```

8. Modify the page layout and CSS of templates/profile.html to accommodate the viewed_books division. As we may add more divisions to this page in the future, one convenient layout concept is the **flexbox**. We will add this CSS and separate the content into nested div instances that will be arranged horizontally on the page. We will refer to the internal div instances as infocell instances and style them with green borders and rounded edges:

```

<style>
.flexrow { display: flex;
           border: 2px solid black;
        }
.flexrow > div { flex: 1; }

.infocell {
    border: 2px solid green;
    border-radius: 5px 25px;
    background-color: white;
    padding: 5px;
    margin: 20px 5px 5px 5px;
}

</style>

<div class="flexrow" >
    <div class="infocell" >
        <p>Profile</p>
        ...
    </div>

```

```

max_viewed_books_length = 10
viewed_books =
    request.session.get('viewed_books', [])
viewed_book = [book.id, book.title]
if viewed_book in viewed_books:
    viewed_books.pop(viewed_books.index(viewed_book))
viewed_books.insert(0, viewed_book)
viewed_books = viewed_books[:max_viewed_books_length]
request.session['viewed_books'] = viewed_books
return render(request, "reviews/book_detail.html",
               context)

```

8. 페이지 idx_d6b143adlayout 및 templates/profile.html의 CSS를 수정하여 viewed_books 구역을 수용합니다. 향후 이 페이지에 더 많은 구역을 추가할 수 있으므로, 편리한 레이아웃 개념은 flexbox입니다. 이 CSS를 추가하고 내용을 중첩된 div 인스턴스로 나누어 idx_f34dcec0가 수평으로 페이지에 배치되도록 할 것입니다. 내부 div 인스턴스를 infocell 인스턴스라고 부르고, 녹색 테두리와 둥근 모서리로 스타일링할 것입니다:

```

<style>
    .flexrow { display: flex;
                border: 2px black;
            }
    .flexrow > div { flex: 1; }

    .infocell {
        border: 2px solid green;
        border-radius: 5px 25px;
        background-color: white;
        padding: 5px;
        margin: 20px 5px 5px 5px;
    }
</style>

<div class="flexrow" >
    <div class="infocell" >
        <p>프로필</p>
        ...
    </div>

```

```

<div class="infocell" >
  <p>Viewed Books</p>
  ...
</div>
</div>

```

9. Modify the `viewed_books` div element in `templates/profile.html` so that if there are books present, their titles are displayed and linked to the individual book detail pages. This will be rendered as follows:

```

<a href="/books/1">Advanced Deep Learning with Keras </a><br>

```

A message should be displayed if the list is empty. The entire div, including the iteration through `request.session.viewed_books`, will look like this:

```

<div class="infocell" >
  <p>Viewed Books</p>
  <p>
    {% for book_id, book_title in
      request.session.viewed_books %}
    <a href="/books/{{ book_id }}">{{ book_title }}
    </a><br>
    {% empty %}
      No recently viewed books found.
    {% endfor %}
  </p>
</div>

```

You can find the complete code for the `templates/profile.html` in the `Chapter09` folder in this book's GitHub repo.

This exercise has enhanced the profile page by adding a list of recently viewed books. Now, when you visit the **Login** link at `http://127.0.0.1:8000/accounts/profile/`, you will see this page:

```

<div class="infocell" >
...
    <p>본 도서</p>
    ...
</div>
</div>

```

9. templates/profile.html의 viewed_books div 요소를 수정하여 다음과 같은 경우에 대비하십시오.
책이 있으면 제목이 표시되고 개별 책 상세 페이지에 연결됩니다.
이것은 다음과 같이 렌더링됩니다:

```
<a href="/books/1">Keras로 배우는 고급 딥러닝 </a><br>
```

목록이 비어 있으면 메시지 idx_0b947dd9가 표시되어야 합니다.
request.session.viewed_books를 통한 전체 div는 다음과 같이 보일 것입니다:

```

<div class="infocell" >
    <p>조회한 책</p>
    <p>
        {% for book_id, book_title in
            request.session.viewed_books %}
        <a href="/books/{{ book_id }}">{{ book_title }}
            </a><br>
        {% empty %}
            최근에 본 책이 없습니다.
        {% endfor %}
    </p>
</div>

```

이 책의 GitHub 저장소의 Chapter09 폴더에서 templates/profile.html의 전체 코드를 찾을 수 있습니다.

이 연습 idx_b33e246e는 최근에 본 책 목록을 추가하여 프로필 페이지를 향상시켰습니다. 이제 <http://127.0.0.1:8000/accounts/profile/>에서 로그인 링크를 방문하면 이 페이지를 볼 수 있습니다:

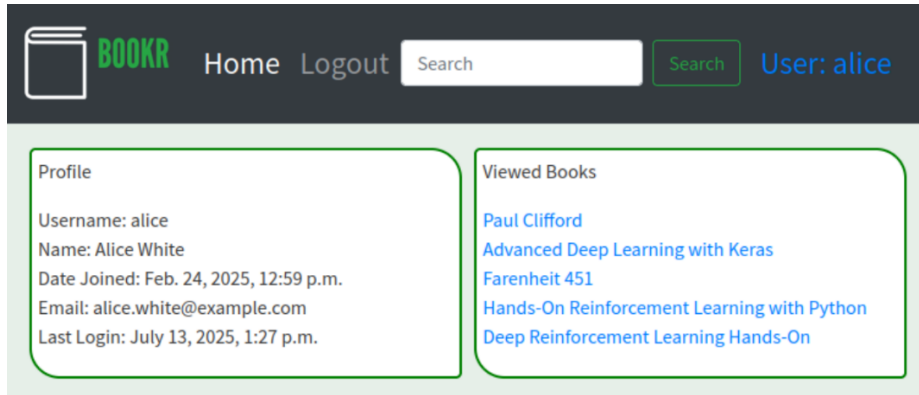


Figure 9.18: Recently viewed books

We can use the `sessioninfo` management command that we developed in *Exercise 9.06 – Examining the session key*, to examine the user's session once this feature is implemented. It can be called on the command line by passing the session data as an argument:

```
python manage.py sessioninfo
```

We can see that the book IDs and titles are listed in the `viewed_books` key. Remember that the encoded data is obtained by querying the `django_session` table in the SQLite database:

```
> python manage.py sessioninfo
Session Key: mjbppj6e6y0qayh1mj6sgw27kfugk1qc User: 4 alice alice.white@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': '384340e0724b154062dcad2c94a9fc94380cd323b9e1e433aff3aef799b99a3e',
 '_auth_user_id': '4',
 'viewed_books': [[17, 'Paul Clifford'],
                  [1, 'Advanced Deep Learning with Keras'],
                  [13, 'Fahrenheit 451'],
                  [6, 'Hands-On Reinforcement Learning with Python'],
                  [4, 'Deep Reinforcement Learning Hands-On']]}
```

Figure 9.19: The viewed books are stored in the session data

In this exercise, we used Django's session mechanism to store ephemeral information about user interactions with the Django project. We have learned how this information can be retrieved from the user session and displayed in a view that informs users about their recent activity.

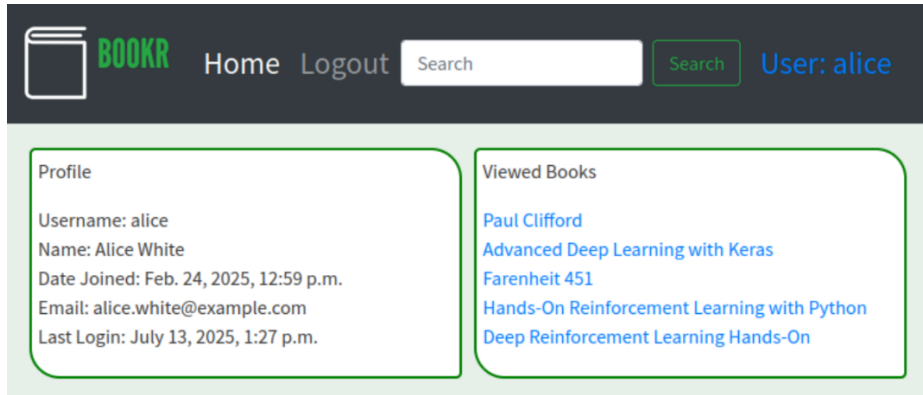


그림 9.18: 최근에 본 책들

이 기능이 구현되면 사용자의 세션을 검사하기 위해 우리가 연습 9.06에서 개발한 sessioninfo 관리 명령을 사용할 수 있습니다. 세션 데이터를 인수로 전달하여 명령줄에서 호출할 수 있습니다:

```
python manage.py sessioninfo
```

책 ID와 제목이 viewed_books 키에 나열되어 있는 것을 볼 수 있습니다.idx_d098011b 인코딩된 데이터는 SQLite 데이터베이스의 django_session 테이블을 쿼리하여 얻어진다는 것을 기억하세요:

```
> python manage.py sessioninfo
Session Key: mjbppj6e6y0qayh1mj6sgw27kfugk1qc User: 4 alice alice.white@example.com
{'_auth_user_backend': 'django.contrib.auth.backends.ModelBackend',
 '_auth_user_hash': '384340e0724b154062dcad2c94a9fc94380cd323b9e1e433aff3aef799b99a3e',
 '_auth_user_id': '4',
 'viewed_books': [[17, 'Paul Clifford'],
 [1, 'Advanced Deep Learning with Keras'],
 [13, 'Fahrenheit 451'],
 [6, 'Hands-On Reinforcement Learning with Python'],
 [4, 'Deep Reinforcement Learning Hands-On']]}
```

그림 9.19: 본 책들은 세션 데이터에 저장됩니다

이 연습에서는 Django의 세션 메커니즘을 사용하여 Django 프로젝트와의 사용자 상호작용에 대한 일시적인 정보를 저장했습니다. 우리는 이 정보가 사용자 세션에서 어떻게 검색되고 사용자의 최근 활동에 대해 알리는 뷰에 표시될 수 있는지 배웠습니다.

Exercise 9.07 – Using session storage for the Search for Books page

Sessions are a useful way to store short-lived information that assists in maintaining a stateful experience on a site. Users frequently revisit pages such as search forms, and it would be convenient to store their most recently used form settings when they return to those pages. In *Chapter 3, URL Mapping, Views, and Templates*, we developed a Book Search feature for the bookr project. The **Book Search** page has two options for **Search in: Title** and **Contributor**. Currently, each time the page is visited, it defaults to **Title**:

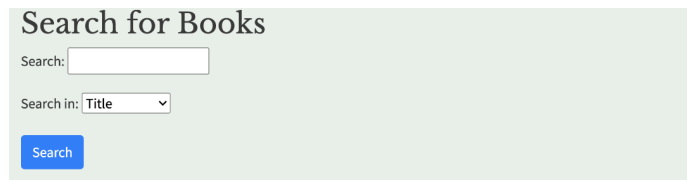
A screenshot of a web form titled "Search for Books". It features a text input field labeled "Search:" followed by a dropdown menu labeled "Search in:" with "Title" selected. Below the dropdown is a blue button labeled "Search".

Figure 9.20: The Search and Search in fields of the Book Search form

In this activity, you will use session storage so that when the book search page, `/book-search`, is visited, it will default to the most recently used search option. You will also add a third `infoCell` to the profile page that contains a list of links to the most recently used search terms. Follow these steps to complete this activity:

1. Edit the `book_search` view and retrieve `search_history` from the session.
2. When the form has received valid input and a user is logged in, append the search option and search text to the session's search history list.

If the form hasn't been filled (for example, when the page is first visited), render the form with the previously used **Search in** option selected – that is, either **Title** or **Contributor** (Figure 9.21):

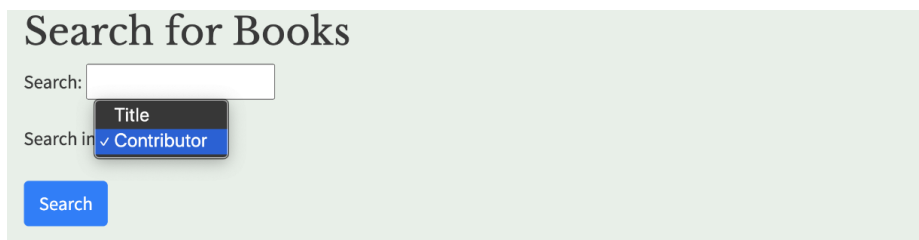
A screenshot of the same "Search for Books" form, but the "Search in:" dropdown menu is open, showing two options: "Title" and "Contributor". The "Contributor" option is highlighted with a blue background and a checkmark, indicating it is the selected option. The "Search" button remains visible below.

Figure 9.21: Selecting Contributor in the Search field

연습 9.07 – 도서 검색 페이지를 위한 세션 저장소 사용

세션은 사이트에서 상태 유지 경험을 유지하는 데 도움이 되는 단기 정보를 저장하는 유용한 방법입니다. 사용자는 검색 양식과 같은 페이지를 자주 다시 방문하며, 이러한 페이지로 돌아올 때 가장 최근에 사용한 양식 설정을 저장하는 것이 편리합니다. 3장, URL 매핑, 뷰 및 템플릿에서는 bookr 프로젝트를 위한 도서 검색 기능을 개발했습니다. 도서 검색 페이지에는 제목과 기여자라는 두 가지 검색 옵션이 있습니다. 현재 페이지를 방문할 때마다 기본값은 제목입니다:

The image shows a web form titled "Search for Books". It contains a text input field labeled "Search:" and a dropdown menu labeled "Search in:" with "Title" selected. Below the dropdown is a blue button labeled "Search".

그림 9.20: 도서 검색 양식의 검색 및 검색 필드

이 활동에서는 세션 저장소를 사용하여 도서 검색 페이지인 /book-search를 방문할 때 가장 최근에 사용한 검색 옵션으로 기본 설정되도록 합니다. 또한 가장 최근에 사용한 검색어 목록이 포함된 프로필 페이지에 세 번째 정보 셀을 추가합니다. 이 활동을 완료하려면 다음 단계를 따르세요:

1. book_search 뷰를 편집하고 세션에서 search_history를 검색합니다. 양식
2. 이 유효한 입력을 받았고 사용자가 로그인한 경우, 검색 옵션과 검색 텍스트를 세션의 검색 기록 목록에 추가합니다.

양식이 작성되지 않은 경우(예: 페이지를 처음 방문할 때), 이전에 사용한 검색 옵션이 선택된 상태로 양식을 렌더링합니다 – 즉, 제목 또는 기여자 중 하나입니다(그림 9.21):

The image shows the same "Search for Books" form as in Figure 9.20, but with the "Search in:" dropdown menu open. The menu shows two options: "Title" and "Contributor", with "Contributor" selected and highlighted in blue. The "Search" button remains below the dropdown.

그림 9.21: 검색 필드에서 기여자 선택하기

3. In the profile template, include an additional `infocell` division for Search History.
4. List the search history as a series of links to the book search page. The links will take this form: `/book-search?search=Python&search_in=title`.

This activity will challenge you to apply session data to solve a usability issue in a web form. This approach will have applicability in many real-world situations and will give you some idea of the use of sessions in creating a stateful web experience. After completing this activity, the profile page will contain the third `infocell`, as shown in *Figure 9.22*:

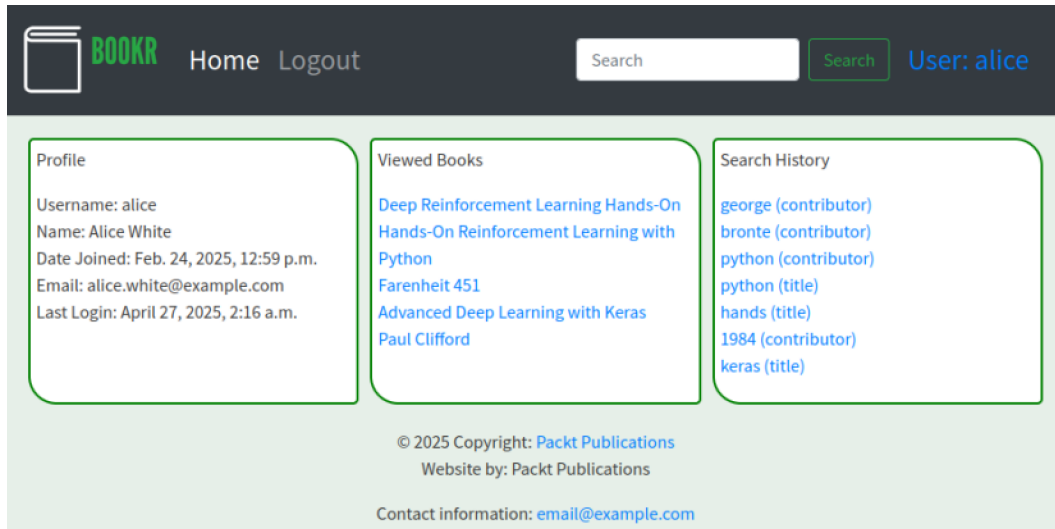


Figure 9.22: The profile page with the Search History infocell

Summary

In this chapter, we examined Django's middleware implementation of authentication and sessions. We also learned how to incorporate authentication and permission logic into views and templates. Now, we can set permissions on specific pages and limit their access to authenticated users. We also examined how to store data in a user's session and render it on subsequent pages.

With that, you have the skills to customize a Django project to deliver a personalized web experience. You can limit the content to authenticated or privileged users, and you can personalize a user's experience based on their prior interactions.

In the next chapter, we will revisit the Admin app and learn about some advanced techniques we can use to customize our user model and apply fine-grained changes to the admin interface for our models.

3. 프로필 템플릿에 검색 기록을 위한 추가 정보 셀 구분을 포함하십시오. 검색 기록을 책 검색 페이지에 대한 링크 시리즈로 나열하십시오. 링크는 다음 형식을 취합니다: `/book-search?search=Python&search_in=title`.

이 활동은 웹 양식의 사용성 문제를 해결하기 위해 세션 데이터를 적용하도록 도전할 것입니다. 이 접근 방식 `idx_49069a7d`는 많은 `idx_cd80d275`실제 상황에서 적용 가능성이 있으며, 상태 유지 웹 경험을 만드는 데 있어 세션 사용에 대한 아이디어를 제공할 것입니다. 이 활동을 완료한 후, 프로필 페이지에는 그림 9.22와 같이 세 번째 정보 셀이 포함될 것입니다:

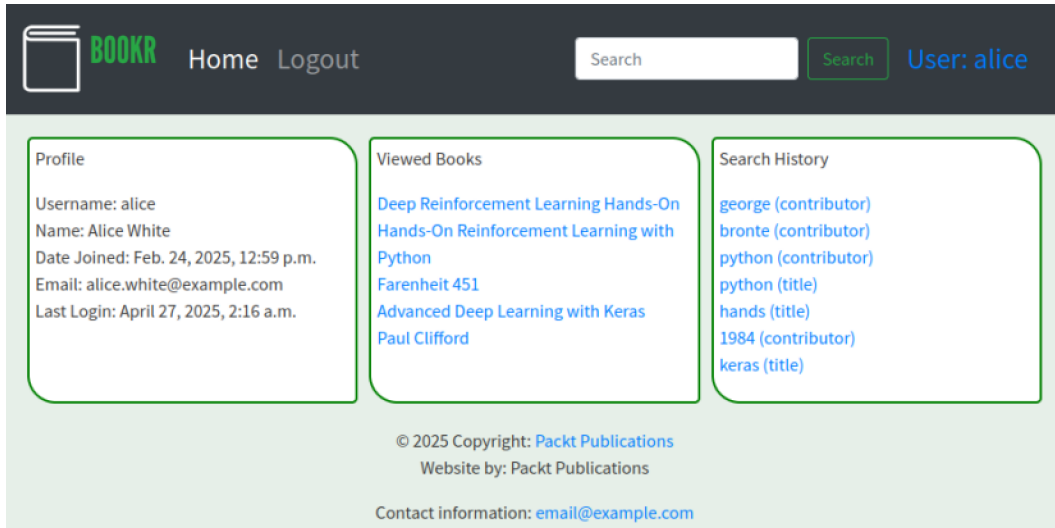


그림 9.22: 검색 기록 정보 셀을 포함한 프로필 페이지

요약

이번 장에서는 Django의 인증 및 세션 미들웨어 구현을 살펴보았습니다. 또한 인증 및 권한 논리를 뷰와 템플릿에 통합하는 방법을 배웠습니다. 이제 특정 페이지에 대한 권한을 설정하고 인증된 사용자에게만 접근을 제한할 수 있습니다. 또한 사용자의 세션에 데이터를 저장하고 이후 페이지에서 렌더링하는 방법을 살펴보았습니다.

이로써, 개인화된 웹 경험을 제공하기 위해 Django 프로젝트를 사용자 정의할 수 있는 기술을 갖추게 되었습니다. 인증된 사용자 또는 특권이 있는 사용자에게 콘텐츠를 제한할 수 있으며, 사용자의 이전 상호작용에 따라 사용자 경험을 개인화할 수 있습니다.

다음 장에서는 Admin 앱을 다시 살펴보고 사용자 모델을 사용자 정의하고 모델의 관리 인터페이스에 세밀한 변경을 적용하는 데 사용할 수 있는 몇 가지 고급 기술에 대해 배울 것입니다.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

이 책의 PDF 복사본, 코드 번들 등을 받으세요

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 가세요). 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 지침을 따르세요.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

10

Advanced Django Admin and Customizations

Let's say we want to customize the front page of a large organization's admin site. We want to show the health of the different systems in the organization and see any active high-priority alerts. If this were an internal website built on top of Django, we would need to customize it. Adding these kinds of functionalities will require the developers in the IT team to customize the default admin panel and create their own custom `AdminSite` module, which will render a different index page in comparison to what is provided by the default admin site. Fortunately, Django makes these kinds of customizations easy.

In this chapter, we will look at how we can leverage Django's framework and its extensibility to customize Django's default admin interface. We'll not just learn how to make the interface more personal; we will also learn how we can control the different aspects of the admin site to make Django load a custom admin site, instead of the one that ships with the default framework. Such customization can come in handy when we want to introduce features into the admin site that are not present by default.

This chapter builds upon the skills we practiced in *Chapter 4, Introduction to Django Admin*. Just to recap, we learned how to use the Django admin site to take control of the administration and authorization for our Bookr app. We also learned how to register models to read and edit their contents, and how to customize Django's admin interface using the `admin.site` properties. We will finish up by learning about how to configure an email server with Django and the use of the Django Tasks framework.

The following topics are covered in this chapter:

- Customizing the user model
- Form enhancements with `ModelAdmin` attributes

10

고급 Django 관리자 및 사용자 정의

대규모 조직의 관리 사이트의 첫 페이지를 사용자 정의하고 싶다고 가정해 보겠습니다. 우리는 조직의 다양한 시스템의 상태를 보여주고 활성화된 고우선 경고를 보고 싶습니다. 이것이 Django 위에 구축된 내부 웹사이트라면, 우리는 이를 사용자 정의해야 합니다. 이러한 기능을 추가하려면 IT 팀의 개발자들이 기본 관리자 패널을 사용자 정의하고 기본 관리자 사이트에서 제공하는 것과 비교하여 다른 인덱스 페이지를 렌더링하는 자신만의 사용자 정의 AdminSite 모듈을 만들어야 합니다. 다행히도 Django는 이러한 종류의 사용자 지정을 쉽게 만들어 줍니다.

이 장에서는 Django의 프레임워크와 확장성을 활용하여 Django의 기본 관리자 인터페이스를 사용자 정의하는 방법을 살펴보겠습니다. 우리는 인터페이스를 더 개인화하는 방법뿐만 아니라 기본 프레임워크와 함께 제공되는 것이 아닌 사용자 정의 관리자 사이트를 로드하도록 Django의 다양한 측면을 제어하는 방법도 배울 것입니다. 이러한 사용자 정의는 기본적으로 존재하지 않는 기능을 관리자 사이트에 도입하고자 할 때 유용할 수 있습니다.

이 장은 4장, Django 관리자 소개에서 연습한 기술을 바탕으로 합니다. 요약하자면, 우리는 Bookr 앱의 관리 및 권한을 제어하기 위해 Django 관리자 사이트를 사용하는 방법을 배웠습니다. 또한 모델을 등록하여 그 내용을 읽고 편집하는 방법과 admin.site 속성을 사용하여 Django의 관리자 인터페이스를 사용자 정의하는 방법을 배웠습니다. 우리는 Django로 이메일 서버를 구성하는 방법과 Django Tasks 프레임워크의 사용에 대해 배우며 마무리할 것입니다.

이 장에서는 다음 주제를 다룹니다: 사용자

- 모델 사용자 정의
- ModelAdmin 속성을 사용한 양식 개선

- Adding views to the Django admin app
- Configuring an email server with Django
- Automating admin with Django Tasks

Now, let's expand our knowledge further by taking a look at how we can start customizing the admin site by utilizing Django's `AdminSite` module to add powerful new functionalities to the admin portal of our web application.

Technical requirements

Find the solution in the `Chapter10` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Customizing the admin site

Django, as a web framework, provides a lot of customization options for building web applications. We will be using this same freedom provided by Django when we work on building the admin application for our project.

In *Chapter 4, Introduction to Django Admin*, we looked at how we can use the `admin.site` properties to customize the elements of Django's admin interface. However, what if we require more control over how our admin site behaves? For example, what if we wanted to show a custom template for the login page or the logout page when the user comes to the Bookr admin panel? In this case, the `admin.site` properties provided might not be enough, and we will need to build customizations that can extend the default admin site's behavior.

A customized user model

The user model that is provided with the authentication app comes with very general features. It is often the case that we want to retain more specific information about a user, such as address or phone number, or we may want to implement a different authentication pattern than the username/password approach. These are important design considerations. It is best to consider it now before we accumulate a lot of user data and need to replace the user model at a later date.

We will look at customizing the user model by either extending the user model or by subclassing one of its inherited abstract models.

- Django 관리 앱에 보기 추가하기
- Django로 이메일 서버 구성하기
- Django 작업으로 관리 자동화하기

이제 Django의 AdminSite 모듈을 활용하여 웹 애플리케이션의 관리 포털에 강력한 새로운 기능을 추가함으로써 관리 사이트를 사용자 정의하는 방법을 살펴보고 우리의 지식을 더욱 확장해 봅시다.

기술 요구 사항

이 책의 GitHub 리포지토리의 Chapter10 폴더에서 솔루션을 찾으세요. 리포지토리 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

관리 사이트 사용자 정의하기

Django는 웹 프레임워크로서 웹 애플리케이션을 구축하기 위한 많은 사용자 정의 옵션을 제공합니다. 우리는 프로젝트의 관리 애플리케이션을 구축할 때 Django가 제공하는 이러한 자유를 활용할 것입니다.

4장, Django 관리 소개에서는 admin.site 속성을 사용하여 Django의 관리 인터페이스 요소를 사용자 정의하는 방법을 살펴보았습니다. 그러나 관리 사이트의 동작에 대해 더 많은 제어가 필요하다면 어떻게 될까요? 예를 들어, 사용자가 Bookr 관리 패널에 올 때 로그인 페이지나 로그아웃 페이지에 대한 사용자 정의 템플릿을 표시하고 싶다면 어떻게 될까요? 이 경우 admin.site 속성만으로는 충분하지 않을 수 있으며, 기본 관리 사이트의 동작을 확장할 수 있는 사용자 정의를 구축해야 할 것입니다.

사용자 맞춤 모델

인증 앱과 함께 제공되는 isidx_239317aa 사용자 모델은 매우 일반적인 기능을 가지고 있습니다. 우리는 종종 주소나 전화번호와 같은 사용자에게 대한 더 구체적인 정보를 유지하고 싶거나, 사용자 이름/비밀번호 접근 방식과는 다른 인증 패턴을 구현하고 싶을 수 있습니다. 이는 중요한 설계 고려 사항입니다. 많은 사용자 데이터를 축적하고 나중에 사용자 모델을 교체해야 하기 전에 지금 고려하는 것이 가장 좋습니다.

우리는 사용자 모델을 확장하거나 상속된 추상 모델 중 하나를 서브클래싱하여 사용자 모델을 사용자 정의하는 방법을 살펴볼 것입니다.

Extending the user model

The first approach involves adding a model to contain supplementary information about our users. This doesn't involve modifications to the auth app or its user model. Instead, a model in a 1-to-1 relationship with the user model is created.

To illustrate this, we will add a `ReviewerProfile` model to the `reviews/models.py` file. This model contains additional details about our user, such as their favorite book and their profile picture. To define fields in models, we need to use the `settings.AUTH_USER_MODEL` attribute.

We will create a `BookrUser` model.

```
from django.contrib.auth.models import User
...

class ReviewerProfile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE)
    location = models.CharField(max_length=100)
    favourite_book = models.ForeignKey(
        Book, null=True, on_delete=models.SET_NULL,
        help_text="Your favourite book (if it is present in Bookr)")
    profile_photo = models.ImageField(
        null=True, blank=True, upload_to="profile_photos/")
```

Note that we have added a `favourite_book` attribute so that a reviewer can optionally select their favorite book from the reviews app if it is present. We've also added a `profile_photo` attribute, so that the user can add a display picture. We have set `on_delete=models.SET_NULL` because we want to ensure that the record deletion of that book would lead to the unsetting of the `favourite_book` attribute rather than the deletion of `ReviewerProfile`.

Currently, we are using the auth user model with our app, but we might want to future-proof the code so that our app does not make this assumption. This is particularly important if the app is intended to be reused in other projects where no assumptions about the user model can be made.

사용자 모델 확장

첫 번째 접근 방식은 보조 정보를 포함하는 모델을 추가하는 것입니다. 이는 인증 앱이나 그 사용자 모델에 대한 수정이 필요하지 않습니다. 대신, 사용자 모델과 1대1 관계에 있는 모델이 생성됩니다.

이를 설명하기 위해, 우리는 `reviews/models.py` 파일에 `ReviewerProfile` 모델을 추가할 것입니다. 이 모델은 사용자의 좋아하는 책과 프로필 사진과 같은 추가 세부 정보를 포함합니다. 모델의 필드를 정의하기 위해서는 `settings.AUTH_USER_MODEL` 속성을 사용해야 합니다.

우리는 `BookrUser` 모델을 생성할 것입니다.

```
from django.contrib.auth.models import User
...

class ReviewerProfile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE)
    location = models.CharField(max_length=100)
    favourite_book = models.ForeignKey(
        Book, null=True, on_delete=models.SET_NULL,
        help_text="당신의 좋아하는 책 (Bookr에 있을 경우)")
    profile_photo = models.ImageField(
        null=True, blank=True, upload_to="profile_photos/")
```

리뷰어가 리뷰 앱에서 좋아하는 책을 선택할 수 있도록 `favourite_book` 속성을 추가했습니다. 또한 사용자가 프로필 사진을 추가할 수 있도록 `profile_photo` 속성도 추가했습니다. 우리는 `on_delete=models.SET_NULL`을 설정했는데, 이는 해당 책의 기록 삭제가 `ReviewerProfile`의 삭제가 아닌 `favourite_book` 속성의 해제를 초래하도록 하려는 것입니다.

현재 우리는 `idx_4961b9e1auth` 사용자 모델을 앱과 함께 사용하고 있지만, 코드가 미래에도 유용하도록 하여 앱이 이 가정을 하지 않도록 할 수 있습니다. 이는 사용자 모델에 대한 가정을 할 수 없는 다른 프로젝트에서 앱이 재사용될 예정인 경우 특히 중요합니다.

To do so, we can use the `get_user_model` function from the `auth` app to determine which user model is being used in the project.

As we saw in the previous chapters, the `on_delete=models.CASCADE` parameter ensures that when the `User` object is deleted, `ReviewerProfile` will be deleted too. Similarly, in order to ensure that `ReviewerProfile` is created after a `User` object is created, we can employ an easy-to-code Django signal. This could be written to `signals.py`, which can then be imported in the `ready` function of the `AppConfig` class in `apps.py`.

```
from django.dispatch import receiver
from django.db.models.signals import post_save

...

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_reviewer_profile(sender, created, instance,
                           **kwargs):
    if created:
        profile = ReviewerProfile(user=instance)
        profile.save()
```

Here, we are using the `sender=settings.AUTH_USER_MODEL` argument to the receiver decorator. This is because we are not presuming that the `auth` user model will be employed in the app.

One slight disadvantage of a 1-to-1 extension of the `User` class is that by using two models for user and profile data, we no longer have a single place in the admin app to maintain this data.

Subclassing AbstractUser

There are situations where it is preferable to replace the user in the `auth` application with a more appropriate one. This can include the case where the primary fields used in authentication are different from the `username` and `password`, or where we need to override the password mechanism or permission structures.

It is advised to do this early in the development if it is apparent that such a need exists, as we don't want to create complicated migration scenarios if we don't have to.

이를 위해 auth 앱의 `get_user_model` 함수를 사용하여 프로젝트에서 어떤 사용자 모델이 사용되고 있는지 확인할 수 있습니다.

앞서의 장에서 보았듯이, `on_delete=models.CASCADE` 매개변수는 User 객체가 삭제될 때 `ReviewerProfile`도 삭제되도록 보장합니다. 마찬가지로, User 객체가 생성된 후 `ReviewerProfile`이 생성되도록 보장하기 위해, 쉽게 코딩할 수 있는 Django 신호를 사용할 수 있습니다. 이는 `signals.py`에 작성될 수 있으며, 이후 `apps.py`의 `AppConfig` 클래스의 `ready` 함수에서 가져올 수 있습니다.

```
from django.dispatch import receiver
from django.db.models.signals import post_save

...

@receiver(post_save, sender=settings.AUTH_USER_MODEL)
def create_reviewer_profile(sender, created, instance,
                            **kwargs):
    if created:
        profile = ReviewerProfile(user=instance)
        profile.save()
```

여기서 우리는 `receiverdecorator`에 `sender=settings.AUTH_USER_MODEL` 인수를 사용하고 있습니다. 이는 auth 사용자 모델이 앱에서 사용될 것이라고 가정하지 않기 때문입니다.

User 클래스의 1대1 확장의 한 가지 단점은 사용자 및 프로필 데이터를 위해 두 개의 모델을 사용함으로써 관리 앱에서 이 데이터를 유지할 수 있는 단일 장소가 더 이상 없다는 것입니다.

AbstractUser 서브클래싱

인증 애플리케이션에서 사용자를 더 적절한 것으로 교체하는 것이 바람직한 상황이 있습니다. 여기에는 인증에 사용되는 기본 필드가 사용자 이름 및 비밀번호와 다르거나 비밀번호 메커니즘 또는 권한 구조를 재정의해야 하는 경우가 포함될 수 있습니다.

이러한 필요가 명백하다면 개발 초기에 이를 수행하는 것이 좋습니다. 그렇지 않으면 복잡한 마이그레이션 시나리오를 만들고 싶지 않기 때문입니다.

First, we should look in more detail at how the user model is implemented in the auth app.

The auth app user model is implemented in `django/contrib/auth/models.py`. `User` is a subclass of `AbstractUser`, which in turn subclasses `AbstractBaseUser` (located in `django/contrib/auth/base_user.py`) and `PermissionMixin`.

Figure 10.1 depicts the attributes of these classes and where they come from. It is a useful guide to establishing how to approach overriding the user model without incurring any unnecessary coding complexity.

Before you implement your own alternative user or authentication protocols, it is worth reviewing what existing apps have already been developed for these needs. Many apps to meet common needs, such as OAuth2 client implementations, exist, as well as established Django apps. It is crucial to thoroughly review what options are available before devising your own customization, as adopting an established app will save a lot of unnecessary duplication of effort.

If you do start customizing the `User` subclass, you will be responsible for modifying it when the Authentication module is updated. You will also be responsible for ensuring that new Django security fixes are not blocked by your customizations.

먼저, auth 앱에서 사용자 모델이 어떻게 구현되어 있는지 더 자세히 살펴보아야 합니다.

auth 앱의 사용자 모델은 `django/contrib/auth/models.py`에 구현되어 있습니다.

User는 AbstractUser의 하위 클래스이며, AbstractUser는 다시 AbstractBaseUser (위치: `django/contrib/auth/base_user.py`)와 PermissionMixin의 하위 클래스입니다.

그림 10.1은 이러한 클래스의 속성과 그 출처를 나타냅니다. 이는 사용자 모델을 오버라이드하는 방법을 설정하는 데 유용한 가이드입니다. 불필요한 코딩 복잡성을 피할 수 있습니다.

자신만의 대체 사용자 또는 인증 프로토콜을 구현하기 전에, 이러한 요구를 위해 이미 개발된 기존 앱을 검토하는 것이 좋습니다. OAuth2 클라이언트 구현과 같은 일반적인 요구를 충족하는 많은 앱이 존재하며, 확립된 Django 앱도 있습니다. 자신의 사용자 정의를 구상하기 전에 사용 가능한 옵션을 철저히 검토하는 것이 중요합니다. 확립된 앱을 채택하면 불필요한 노력의 중복을 많이 줄일 수 있습니다.

사용자 하위 클래스를 사용자 정의하기 시작하면, 인증 모듈이 업데이트될 때 이를 수정할 책임이 있습니다. 또한, 새로운 Django 보안 수정이 사용자 정의로 인해 차단되지 않도록 할 책임도 있습니다.

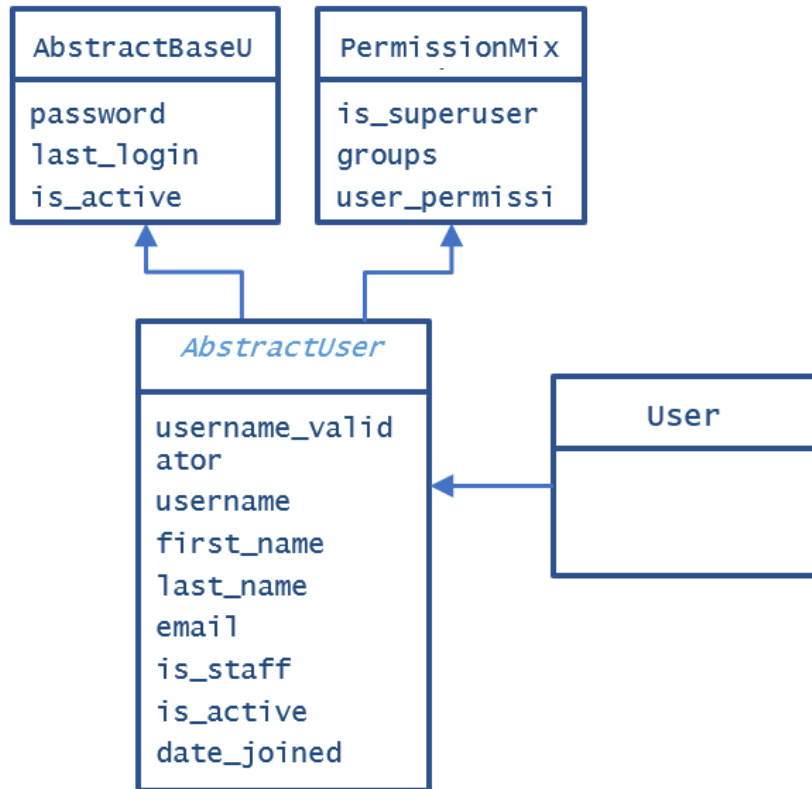


Figure 10.1: The user model's inheritance

Let us look at three different scenarios for subclassing `AbstractUser` or `AbstractBaseUser`.

Scenario 1: Adding additional fields to the User class

In this case, we would subclass `AbstractUser` and include the additional fields.

Sometimes the fields that define the user are insufficient for cultural or professional reasons. In some cultures, the first name/last name distinction is not a natural way to view a person's name. For example, in many parts of Russia and Eastern Europe, a middle name derived from a mother's or a father's first name, a matronym or patronym, respectively, is used. It may make more sense to store this in a separate field in the user model.

Similarly, if your project will be used to generate formal correspondence, it may make sense to store the honorifics Mrs, Ms, Mr, Miss, and so on in a separate honorific field. Professional titles such as Dr, Prof, and Colonel may warrant being stored in a separate field from the honorifics.

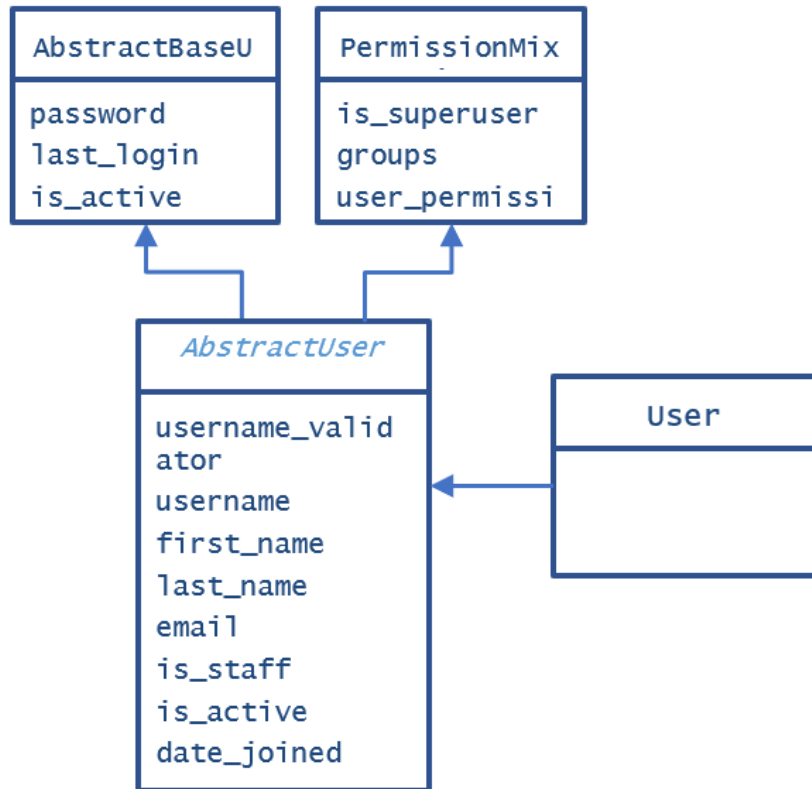


그림 10.1: 사용자 모델의 상속

AbstractUser 또는 AbstractBaseUser를 서브클래싱하기 위한 세 가지 다른 시나리오를 살펴보겠습니다.

시나리오 1: 사용자 클래스에 추가 필드 추가하기

이 경우, 우리는 AbstractUser를 서브클래싱하고 추가 필드를 포함할 것입니다.

때때로 사용자를 정의하는 필드는 문화적 또는 직업적 이유로 불충분할 수 있습니다. 일부 문화에서는 이름/성의 구분이 사람의 이름을 보는 자연스러운 방법이 아닙니다. 예를 들어, 러시아와 동유럽의 많은 지역에서는 어머니 또는 아버지의 이름에서 파생된 중간 이름, 즉 모계명 또는 부계명이 사용됩니다. 이를 사용자 모델의 별도 필드에 저장하는 것이 더 의미가 있을 수 있습니다.

마찬가지로, 귀하의 프로젝트가 공식적인 서신을 생성하는 데 사용될 경우, Mrs, Ms, Mr, Miss 등의 경칭을 별도의 경칭 필드에 저장하는 것이 의미가 있을 수 있습니다. Dr, Prof, Colonel과 같은 직업적 직함은 경칭과 별도의 필드에 저장될 필요가 있을 수 있습니다.

If a phone number is used as an essential part of the user registration process and is uniquely tied to the user's identity, it may be worth including it in this class too. Otherwise, consider adding a separate telephone model that is in a many-to-one relationship with the user profile method.

Scenario 2: Replacing the Username key

It is a frequent requirement to implement user authentication that consists of an email address and password, and to dispense with an additional username.

This would involve subclassing `AbstractUser` and replacing the username and username validation fields. New login-related methods may need to be devised.

Scenario 3: Modifying the password

While we can modify the way that the password is stored in the database by changing the middleware, changing the definition of the password field is a different proposition altogether.

If, for some reason, we needed to replace the 128-character password field or replace password authentication with a novel protocol, then it would make sense to override `AbstractBaseUser`. This approach would add a lot of complexity and is not recommended for novices.

Note

The scenarios presented are not mutually exclusive. There may be situations where the best course of action is to include an app-based user extension, such as the `ReviewerProfile` depicted, as well as sub-classing the `AbstractUser`.

Steps in replacing the auth user model

The following list of steps is used to create a custom user model. We will implement these in the next exercise.

1. Create a new app for the custom user model.
2. Define a custom user model class, inheriting from `User`, `AbstractUser`, or `AbstractBaseUser` as the case merits, based on the scenarios outlined previously.
3. Include the app in `INSTALLED_APPS`.
4. Set this model as `AUTH_USER_MODEL` in `<project>/settings.py`.
5. Ensure that any references to the user model are replaced with references to `settings.AUTH_USER_MODEL` or `get_user_model`.
6. Replace any templates or create appropriate extensions that are invalidated by the changes.

전화번호가 사용자 등록 프로세스의 필수 요소로 사용되고 사용자의 신원에 고유하게 연결되어 있다면, 이 클래스에 포함하는 것이 좋습니다. 그렇지 않다면, 사용자 프로필 메서드와 다대일 관계에 있는 별도의 전화 모델을 추가하는 것을 고려하십시오.

시나리오 2: 사용자 이름 키 교체

이메일 주소와 비밀번호로 구성된 사용자 인증을 구현하고 추가 사용자 이름을 생략하는 것은 자주 요구되는 사항입니다.

이는 `AbstractUser`를 서브클래싱하고 사용자 이름 및 사용자 이름 검증 필드를 교체하는 것을 포함합니다. 새로운 로그인 관련 메서드를 고안해야 할 수도 있습니다.

시나리오 3: 비밀번호 수정

우리는 미들웨어를 변경하여 비밀번호가 데이터베이스에 저장되는 방식을 수정할 수 있지만, 비밀번호 필드의 정의를 변경하는 것은 전혀 다른 제안입니다.

어떤 이유로 128자 비밀번호 필드를 교체하거나 비밀번호 인증을 새로운 프로토콜로 교체해야 한다면, `AbstractBaseUser`를 오버라이드하는 것이 의미가 있을 것입니다. 이 접근 방식은 많은 복잡성을 추가하며 초보자에게는 권장되지 않습니다.

참고

제시된 시나리오는 상호 배타적이지 않습니다. `ReviewerProfile`과 같은 앱 기반 사용자 확장을 포함하는 것이 최선의 행동 방식인 상황이 있을 수 있으며, `AbstractUser`를 서브클래싱하는 것도 포함됩니다.

인증 사용자 모델을 교체하는 단계

다음 `idx_88c7d8b3`단계 목록은 사용자 정의 사용자 모델을 생성하는 데 사용됩니다. 우리는 다음 연습에서 이를 구현할 것입니다.

1. 사용자 정의 사용자 모델을 위한 새 앱을 만듭니다.
2. 사용자 모델 클래스를 정의하고, 이전에 설명한 시나리오에 따라 `User`, `AbstractUser` 또는 `AbstractBaseUser`에서 상속합니다. 앱을 `INSTALLED_APPS`에 포함시킵니다.
- 3.
4. 이 모델을 `<project>/settings.py`의 `AUTH_USER_MODEL`로 설정합니다.
5. 사용자 모델에 대한 모든 참조가 `settings.AUTH_USER_MODEL` 또는 `get_user_model`에 대한 참조로 대체되었는지 확인합니다.
6. 변경으로 인해 무효화된 템플릿을 교체하거나 적절한 확장을 만듭니다.

7. Register the model with the admin app.
8. Create and run a migration script for these model changes.

Exercise 10.1: Replacing User with a TitledUser model

In this exercise, we will create a `TitledUser` class that includes an honorific and title field for each user.

These options are to be defined as `CharField` with fixed choice lists. The `TitledUser.get_full_name` method will override `AbstractUser.get_full_name`. It will follow using the following logic:

- If a title is present, return a string of "<title> <firstname> <secondname>".
- If an honorific is present, return a string of "<honorific> <firstname> <secondname>".
- Otherwise, return a string of "<firstname> <secondname>".

Create a separate app named `titledusers`.

```
python manage.py startapp titledusers
```

Subclass the User model.

```
class TitledUser(User):
```

Include a Meta class with `verbose_name` and `verbose_name_plural` attributes inside the `TitledUser` class.

```
class Meta:
    verbose_name = "titled user"
    verbose_name_plural = "titled users"
```

Define the `TextChoice` classes for `TitleChoice` and `HonorificChoice` inside the `TitledUser` class.

```
class TitleChoices(models.TextChoices):
    PROF = "PROF", "Prof"
    DR = "DR", "Dr"

class HonorificChoices(models.TextChoices):
    MR = "MR", "Mr"
    MISS = "MISS", "Miss"
```

7. 관리자 앱에 모델 등록하기.
8. 이 모델 변경 사항에 대한 마이그레이션 스크립트를 생성하고 실행하기.

연습 10.1: User를 TitledUser 모델로 교체하기

이 연습에서는 각 사용자에게 대한 경칭 및 직함 필드를 포함하는 TitledUser 클래스를 idx_928df150 생성할 것입니다.

이 옵션들은 고정 선택 목록이 있는 CharField로 정의되어야 합니다.

TitledUser.get_full_name 메서드는 AbstractUser.get_full_name을 재정의할 것입니다. 다음 논리를 사용하여 진행됩니다:

- 제목이 있는 경우, "<title> <firstname> <secondname>"의 문자열을 반환합니다. 존칭이 있는 경우, "<honorific> <firstname><secondname>"의 문자열을 반환합니다.
- 그렇지 않으면, "<firstname> <secondname>"의 문자열을 반환합니다. titledusers라는 이름의 별도 앱을 만듭니다.

```
python manage.py startapp titledusers
```

User 모델을 서브클래스합니다.

```
class TitledUser(User):
```

TitledUser 클래스 안에 verbose_name 및 verbose_name_plural 속성을 가진 Meta 클래스를 포함합니다.

```
class Meta:
    verbose_name = "titled user"
    verbose_name_plural = "titled users"
```

TitledUserclass 내에서 TitleChoice 및 HonorificChoice에 대한 TextChoice 클래스를 정의합니다.

```
class TitleChoices(models.TextChoices):
    PROF = "PROF", "Prof"
    DR = "DR", "Dr"
class HonorificChoices(models.TextChoices):
    MR = "MR", "Mr"
    MISS = "MISS", "Miss"
```

```
MRS = "MRS", "Mrs"
MS = "MS", "MS"
```

Add the two additional fields.

```
title = models.CharField(verbose_name="Title",
                          blank=True, choices=TitleChoices.choices)
honorific = models.CharField(
    verbose_name="Honorific",
    blank=True, choices=HonorificChoices.choices)
```

Add the `get_full_name` method.

```
def get_full_name(self):
    if self.title:
        return f'{self.title} '
            f'{self.firstname} {self.secondname}'
    elif self.honorific:
        return f'{self.honorific} '
            f'{self.firstname} {self.secondname}'
    else:
        return f'{self.firstname} '
            f'{self.secondname}'
```

After the preceding steps, our model in `titledusers/models.py` will look like this:

```
from django.db import models
from django.contrib.auth.models import User

class TitledUser(User):
    class Meta:
        verbose_name = _('titled user')
        verbose_name_plural = _('titled users')

    class TitleChoices(models.TextChoices):
        PROF = "PROF", "Prof"
        DR = "DR", "Dr"

    class HonorificChoices(models.TextChoices):
        MR = "MR", "Mr"
        MISS = "MISS", "Miss"
```

```
MRS = "MRS", "Mrs"
MS = "MS", "MS"
```

두 개의 추가 필드를 추가하세요.

```
title = models.CharField(verbose_name="Title",
                          blank=True, choices=TitleChoices.choices)
honorific = models.CharField(
    verbose_name="Honorific",
    blank=True, choices=HonorificChoices.choices)
```

get_full_name 메서드를 추가하세요.

```
def get_full_name(self):
    if self.title:
        return f'{self.title} '
        f'{self.firstname} {self.secondname}'
    elif self.honorific:
        return f'{self.honorific} '
        f'{self.firstname} {self.secondname}'
    else:
        return f'{self.firstname} '
        f'{self.secondname}'
```

이전 단계인 idx_35139e00 후, titledusers/models.py의 우리의 모델은 다음과 같을 것입니다:

```
from django.db import models
from django.contrib.auth.models import User

class TitledUser(User):
    class Meta:
        verbose_name = _('titled user')
        verbose_name_plural = _('titled users')

    class TitleChoices(models.TextChoices):
        PROF = "PROF", "Prof"
        DR = "DR", "Dr"

    class HonorificChoices(models.TextChoices):
        MR = "MR", "Mr"
        MISS = "MISS", "Miss"
```



```

MRS = "MRS", "Mrs"
MS = "MS", "MS"

title = models.CharField(verbose_name="Title",
                          blank=True, choices=TitleChoices.choices)
honorific = models.CharField(verbose_name="Honorific",
                              blank=True, choices=HonorificChoices.choices)

def get_full_name(self):
    if self.title:
        return f'{self.title} '
            f'{self.firstname} {self.secondname}'
    elif self.honorific:
        return f'{self.honorific} '
            f'{self.firstname} {self.secondname}'
    else:
        return f'{self.firstname} '
            f'{self.secondname}'

```

Include the new app in the list of installed apps and set `AUTH_USER_MODEL`.

```

INSTALLED_APPS = [
    ...
    'reviews',
    'titledusers',
]
AUTH_USER_MODEL = 'titledusers.TitledUser'

```

Ensure that any references to the user model are replaced with references to `settings.AUTH_USER_MODEL` or `get_user_model`.

A search of the code reveals that these only occur in `reviews/management/commands/loadcsv.py` and the original code for `reviews/models.py`.

Replace any templates or create appropriate extensions that are invalidated by the changes. Currently, since these changes do not affect any username, email, or login behaviors, we do not need to replace any templates.

Register the model with the admin app.

```

MRS = "MRS", "Mrs"
MS = "MS", "MS"

title = models.CharField(verbose_name="제목",
                          blank=True, choices=TitleChoices.choices)
honorific = models.CharField(verbose_name="칭호",
                              blank=True, choices=HonorificChoices.choices)

def get_full_name(self):
    if self.title:
        return f'{self.title} '
        f'{self.firstname} {self.secondname}'
    elif self.honorific:
        return f'{self.honorific} '
        f'{self.firstname} {self.secondname}'
    else:
        return f'{self.firstname} '
        f'{self.secondname}'

```

새 앱을 설치된 앱 목록에 포함하고 AUTH_USER_MODEL을 설정하세요.

```

INSTALLED_APPS = [
    ...
    '리뷰',
    '제목이 있는 사용자',
]
AUTH_USER_MODEL = 'titledusers.TitledUser'

```

사용자 모델에 대한 모든 참조가 settings.AUTH_USER_MODEL 또는 get_user_model에 대한 참조로 대체되도록 하십시오.

코드 검색 결과, 이러한 참조는 reviews/management/commands/loadcsv.py와 reviews/models.py의 원본 코드에만 발생합니다.

변경으로 인해 무효화된 anyidx_8881a1b0 템플릿을 교체하거나 적절한 확장을 만드십시오. 현재 이러한 변경이 사용자 이름, 이메일 또는 로그인 동작에 영향을 미치지 않으므로 템플릿을 교체할 필요가 없습니다.

모델을 관리자 앱에 등록하십시오.

At a minimum, the `titledusers/admin.py` file will look like this:

```
from django.contrib import admin

# Register your models here.
admin.site.register(TitledUser)
```

Create and run a migration script for these model changes.

We have now created a custom user, `TitledUser`.

Activity 10.1: Implementing the reviewer profile model

The aim of this activity is to follow through the necessary steps to implement the `ReviewerProfile` model that was listed in the *Extending the user model* section.

As we are keeping `auth.User` as the auth model, we do not have to create a new user. However, we need to explicitly define this in the project's settings file.

Replace any templates or create appropriate extensions that are invalidated by the changes. At this point, we will add custom admin templates for profile and registration. The existing templates will be modified to include the `ReviewerProfile` form.

1. Register the model with the admin app in `admin.py` by importing the model and then registering it with `admin.site.register(ReviewerProfile)`.
2. Create and run a migration script for these changes.
3. The outcome of this activity is that we have integrated our `ReviewerProfile` model into the Bookr project, and we can modify `ReviewerProfiles` through the admin interface.

최소한, `titledusers/admin.py` 파일은 다음과 같을 것입니다:

```
from django.contrib import admin

# 여기에 모델을 등록하세요.
admin.site.register(TitledUser)
```

이 모델 변경 사항에 대한 마이그레이션 스크립트를 생성하고 실행하세요. 이제 우리는 사용자 정의 사용자, `TitledUser`를 만들었습니다.

활동 10.1: 리뷰어 프로필 모델 구현

이 활동의 목표는 사용자 모델 확장 섹션에 나열된 `ReviewerProfile` 모델을 구현하기 위한 필요한 단계를 따르는 것입니다.

우리는 `auth.User`를 인증 모델로 유지하고 있으므로 새 사용자를 만들 필요가 없습니다. 그러나 프로젝트의 설정 파일에서 이를 명시적으로 정의해야 합니다.

변경으로 인해 무효화된 템플릿을 교체하거나 적절한 확장을 만드세요. 이 시점에서 프로필 및 등록을 위한 사용자 정의 관리자 템플릿을 추가할 것입니다. 기존 템플릿은 `ReviewerProfile` 양식을 포함하도록 수정될 것입니다.

1. `admin.py`에서 모델을 가져온 후 `admin.site.register(ReviewerProfile)`로 등록하여 관리자 앱에 모델을 등록하세요.
2. 이 변경 사항에 대한 마이그레이션 스크립트를 생성하고 실행하세요.
3. 이 활동의 결과는 `ReviewerProfile` 모델이 `idx_be495a6aBookr` 프로젝트에 통합되었으며, 관리자 인터페이스를 통해 `ReviwerProfiles`를 수정할 수 있다는 것입니다.

The screenshot shows the 'Bookr administration' interface. At the top, there's a dark blue header with the title 'Bookr administration' and links for 'WELCOME, BOOKRADMIN', 'VIEW SITE', 'CHANGE PASSWORD', and 'LOG OUT'. Below this is a breadcrumb trail: 'Home > Reviews > Reviewer profiles > Add reviewer profile'. The main content area is titled 'Add reviewer profile'. It contains four form fields: 'User:' with a dropdown menu showing 'bob' and edit/delete icons; 'Location:' with an empty text input; 'Favourite book:' with a dropdown menu showing '-----' and edit/delete icons, with a subtitle 'Your favourite book (if it is present in Bookr)'; and 'Profile photo:' with a 'Choose file' button and the text 'No file chosen'. At the bottom right, there are three buttons: 'Save and add another', 'Save and continue editing', and 'SAVE'.

Figure 10.2: Editing the reviewer profile in the admin app

Form enhancements with ModelAdmin attributes

This section addresses some of the additional attributes of the `ModelAdmin` class. We first encountered the `ModelAdmin` class in *Chapter 4* and learned how to subclass it to produce customizations in our admin app.

`filter_horizontal` / `filter_vertical`

Recall from the user change page that there was a widget consisting of two lists used for assigning permissions to the user. This kind of multiple selection is easier to use when navigating long lists of options than a typical dropdown. As the selected options shift from the left-hand list to the right-hand list, it is easy to keep track of which options have been selected.

If we want to assign this kind of widget to any of our multiple selection fields, we can set the `filter_horizontal` attribute in the `ModelAdmin` subclass. The `filter_horizontal` attribute takes a tuple of strings as arguments. The strings are references to fields in the model or fields in foreign keys.

Bookr administration

WELCOME, BOOKRADMIN / VIEW SITE / CHANGE PASSWORD / LOG OUT

Home > Reviews > Reviewer profiles > Add reviewer profile

Add reviewer profile

User:  

Location:

Favourite book:   
Your favourite book (if it is present in Bookr)

Profile photo: No file chosen

그림 10.2: 관리자 앱에서 리뷰어 프로필 편집하기

ModelAdmin 속성을 통한 양식 개선

이 섹션에서는 ModelAdmin 클래스의 추가 속성에 대해 다룹니다. 우리는 4장에서 ModelAdmin 클래스를 처음 접하고, 이를 서브클래싱하여 관리자 앱에서 사용자 정의를 만드는 방법을 배웠습니다.

filter_horizontal / filter_vertical

사용자 변경 페이지 idx_63d39080에서 권한을 사용자에게 할당하는 데 사용되는 두 개의 목록으로 구성된 위젯이 있었음을 기억하십시오. 이러한 종류의 다중 선택은 일반 드롭 다운보다 긴 옵션 목록을 탐색할 때 사용하기가 더 쉽습니다. 선택된 옵션이 왼쪽 목록에서 오른쪽 목록으로 이동함에 따라 어떤 옵션이 선택되었는지 쉽게 추적할 수 있습니다.

이러한 종류의 위젯을 다중 선택 필드에 할당하려면 ModelAdmin 서브클래스에서 filter_horizontal 속성을 설정할 수 있습니다. filter_horizontal 속성은 문자열의 튜플을 인수로 사용합니다. 문자열은 모델의 필드 또는 외래 키의 필드를 참조합니다.

There is also an option to display the two lists on top of each other.

If we don't want to display the two lists side by side, we have the option of displaying them as top and bottom. This might be the case if we want the selection to be displayed on a mobile phone. In this case, we add the field string to the `filter_vertical` attribute.

Inline forms

Sometimes we also want to represent models that are associated through a ManyToMany or OneToMany relationship.

For example, on the user page, we have a list of groups that the user is in and the option to assign them to additional groups. This is achieved using the `inlines` attribute.

There are two formats of inline form elements, `TabularInline` and `StackedInline`.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0

class BookAdmin(admin.ModelAdmin):
    date_hierarchy = 'publication_date'
    list_display = ('title', 'isbn')
    list_filter = ('publisher', 'publication_date')
    search_fields = ('title', 'isbn', 'publisher__name')
    inlines = (ContributorInline,)
```

두 목록을 서로 위에 표시하는 옵션도 있습니다.

두 목록을 나란히 표시하고 싶지 않다면, 위아래로 표시하는 옵션이 있습니다. 이는 선택 항목을 모바일폰에 표시하고 싶을 때 해당될 수 있습니다. 이 경우, 필드 문자열을 `filter_vertical` 속성에 추가합니다.

인라인 양식

때때로 우리는 ManyToMany 또는 OneToMany 관계를 통해 연결된 모델을 나타내고 싶습니다.

예를 들어, 사용자 페이지에는 사용자가 속한 그룹 목록과 추가 그룹에 할당할 수 있는 옵션이 있습니다. 이는 `inlines` 속성을 사용하여 달성됩니다.

인라인 양식 요소에는 TabularInline 및 StackedInline의 두 가지 형식이 있습니다.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0

class BookAdmin(admin.ModelAdmin):
    date_hierarchy = 'publication_date'
    list_display = ('title', 'isbn')
    list_filter = ('publisher', 'publication_date')
    search_fields = ('title', 'isbn', 'publisher__name')
    inlines = (ContributorInline,)
```


Change book

HISTORY

Title:1984

The title of the book.

Date the book was published.2017-04-04Today I

Note: You are 11 hours ahead of server time.

ISBN number of the book.9781328869333

Publisher:Houghton Mifflin Harcourt

Cover:Choose fileNo file chosen

Sample:Choose fileNo file chosen

BOOK CONTRIBUTORS

Book contributor: Orwell, G (AUTHOR)

Delete

Contributor:Orwell, G

The role this contributor had in the book.Author

Book contributor: #2

Contributor:

The role this contributor had in the book.

Book contributor: #3

Contributor:

The role this contributor had in the book.

Book contributor: #4

Contributor:

The role this contributor had in the book.

+ Add another Book contributor

Delete

Save and add another

Save and continue editing

SAVE

Figure 10.3: StackedInline forms

Change book

HISTORY

Title:
The title of the book.

Date the book was published: Today I
Note: You are 11 hours ahead of server time.

ISBN number of the book.

Publisher:

Cover: No file chosen

Sample: No file chosen

BOOK CONTRIBUTORS

Book contributor: Orwell, G (AUTHOR) ☐ Delete

Contributor:

The role this contributor had in the book.

Book contributor: #2

Contributor:

The role this contributor had in the book.

Book contributor: #3

Contributor:

The role this contributor had in the book.

Book contributor: #4

Contributor:

The role this contributor had in the book.

Add another Book contributor

Delete

Save and add another

Save and continue editing

SAVE

그림 10.3: StackedInline 양식

Perhaps having those blank extra inline forms looks too busy and confusing. They can be removed using the `extra` attribute.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0
```

BOOK CONTRIBUTORS

Book contributor: Straub, P (CO_AUTHOR) ☐ Delete

Contributor: Straub, P   

The role this contributor had in the book. Co-Author 

Book contributor: King, S (CO_AUTHOR) ☐ Delete

Contributor: King, S   

The role this contributor had in the book. Co-Author 

 Add another Book contributor

Figure 10.4: StackedInline forms without blank forms

Alternatively, if we want the parent form to enforce a maximum number of inline forms, we can add the `max_num` attribute. Observe that there will no longer be + Add another Book contributor at the bottom of the inline forms. Of course, setting a maximum number of contributors would not make sense for books, given that they can have multiple authors, editors, and other collaborators, but there are situations where this form constraint is perfectly reasonable.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0
    max_num = 2
```

아마도 idx_409130d2 그 빈 추가 인라인 양식이 너무 복잡하고 혼란스러워 보일 것입니다. extra 속성을 사용하여 제거할 수 있습니다.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0
```

그림 10.4: 빈 양식이 없는 StackedInline 양식

대안으로, 부모 양식이 최대 인라인 양식 수를 강제하도록 하려면 max_num 속성을 추가할 수 있습니다. 인라인 양식 하단에 더 이상 + 다른 Bookcontributor 추가가 없을 것입니다. 물론, 최대 기여자 수를 설정하는 것은 책에 대해 의미가 없을 것입니다. 책은 여러 저자, 편집자 및 기타 협력자가 있을 수 있기 때문입니다. 그러나 이 양식 제약이 완벽하게 합리적인 상황이 있습니다.

```
class ContributorInline(admin.StackedInline):
    model = BookContributor
    extra = 0
    max_num = 2
```

Figure 10.5: StackedInline forms with a maximum constraint

This StackedInline format emphasizes that the inline elements are separate elements. Sometimes, we might prefer the TabularInline rendering, which is more compact and arranges fields in columns. This is particularly useful for numerical data such as invoice lines.

```
class ContributorInline(admin.TabularInline):
    model = BookContributor
    extra = 0
```

Figure 10.6: TabularInline forms

Be mindful when adding inline (or `filter_horizontal` / `filter_vertical`) fields to an admin form, as this will increase the underlying SQL query complexity used to generate the page. It may have performance repercussions if you have large datasets.

BOOK CONTRIBUTORS

Book contributor: Straub, P (CO_AUTHOR) ☐ Delete

Contributor:

The role this contributor had in the book.

Book contributor: King, S (CO_AUTHOR) ☐ Delete

Contributor:

The role this contributor had in the book.

그림 10.5: 최대 제약 조건이 있는 StackedInline 양식

이 StackedInline 형식은 인라인 요소가 별개의 요소임을 강조합니다. 때때로, 우리는 필드를 옆에 배치하여 더 간결한 TabularInline 렌더링을 선호할 수 있습니다. 이 idx_35726978은 송장 항목과 같은 숫자 데이터에 특히 유용합니다.

```
class ContributorInline(admin.TabularInline):
    model = BookContributor
    extra = 0
```

BOOK CONTRIBUTORS

CONTRIBUTOR	THE ROLE THIS CONTRIBUTOR HAD IN THE BOOK.	DELETE?
Straub, P (CO_AUTHOR) Straub, P	Co-Author	☐
King, S (CO_AUTHOR) King, S	Co-Author	☐

Add another Book contributor

그림 10.6: TabularInline 양식

관리 양식에 인라인(또는 filter_horizontal / filter_vertical) 필드를 추가할 때는 주의해야 합니다. 이는 페이지 생성을 위해 사용되는 기본 SQL 쿼리의 복잡성을 증가시킬 수 있습니다. 대량의 데이터 세트가 있는 경우 성능에 영향을 미칠 수 있습니다.

radio_fields

By default, the admin app renders choices or foreign key relationships as dropdowns. Sometimes radio buttons are an appropriate way to represent a short list of discrete options. By using the `radio_fields` attribute, we render some fields as radio buttons.

We might consider rendering the `role` attribute of our `BookContributor` class using `radio_fields`. We need to specify `radio_fields` as a dictionary, specifying the field name as the key, and the orientation (either `admin.HORIZONTAL` or `admin.VERTICAL`) as the value.

```
class BookContributorAdmin(admin.ModelAdmin):
    role = {'rating': admin.HORIZONTAL}

admin.site.register(BookContributor, BookContributorAdmin)
```

This setting works well when there is a small list of options. Otherwise, the layout can look somewhat confusing or cluttered.

readonly_fields

Sometimes we want to protect specific fields from being modified in the admin interfaces. We can designate them as being read-only by using the `readonly_fields` attribute. For example, you might want to set the fields showing the date of creation and date of modification of a record as read-only. The following line of code shows how this can be done using `readonly_fields`.

```
readonly_fields = ('date_created', 'date_modified')
```

For example, in this situation, we have specified a tuple of read-only fields, but there are some fields that are only read-only when the object is being modified. Using the conditional is a common way to achieve this, and we will revisit this logic in the exercises and activities.

get_readonly_fields

Sometimes we need to assign more dynamic criteria to specify when a field is read-only. For this reason, there is a `get_readonly_fields` method that gives us the flexibility to include criteria such as a user's permissions or whether the object is being created or updated.

radio_fields

기본적으로 관리 앱은 `idx_0d3358e0choices` 또는 외래 키 관계를 드롭다운으로 렌더링합니다. 때때로 라디오 버튼은 짧은 목록의 개별 옵션을 나타내는 적절한 방법입니다. `radio_fields` 속성을 사용하여 일부 필드를 라디오 버튼으로 렌더링합니다.

우리는 `BookContributor` 클래스의 `role` 속성을 `radio_fields`를 사용하여 렌더링하는 것을 고려할 수 있습니다. 우리는 `radio_fields`를 사전으로 지정해야 하며, 필드 이름을 키로, 방향(`admin.HORIZONTAL` 또는 `admin.VERTICAL` 중 하나)을 값으로 지정해야 합니다.

```
class BookContributorAdmin(admin.ModelAdmin):
    role = {'rating': admin.HORIZONTAL}

admin.site.register(BookContributor, BookContributorAdmin)
```

이 설정은 옵션 목록이 작을 때 잘 작동합니다. 그렇지 않으면 레이아웃이 다소 혼란스럽거나 복잡해 보일 수 있습니다.

readonly_fields

때때로 우리는 관리 인터페이스에서 특정 `idx_92f2d6f2` 필드가 수정되지 않도록 보호하고 싶습니다. `readonly_fields` 속성을 사용하여 이를 읽기 전용으로 지정할 수 있습니다. 예를 들어, 레코드의 생성 날짜와 수정 날짜를 표시하는 필드를 읽기 전용으로 설정하고 싶을 수 있습니다. 다음 코드 줄은 `readonly_fields`를 사용하여 이를 수행하는 방법을 보여줍니다.

```
readonly_fields = ('date_created', 'date_modified')
```

예를 들어, 이 상황에서는 읽기 전용 필드의 튜플을 지정했지만, 객체가 수정될 때만 읽기 전용인 필드도 있습니다. 조건문을 사용하는 것은 이를 달성하는 일반적인 방법이며, `idx_047a0d2b`에서 이 논리를 연습 문제와 활동에서 다시 살펴볼 것입니다.

get_readonly_fields

때때로 우리는 필드가 읽기 전용일 때를 지정하기 위해 더 많은 `idx_da3866ba` 동적 기준을 할당해야 합니다. 이러한 이유로, 사용자의 권한이나 객체가 생성되거나 업데이트되고 있는지 여부와 같은 기준을 포함할 수 있는 유연성을 제공하는 `get_readonly_fields` 메서드가 있습니다.

Let's suppose we only wanted the creator and book fields of the review to be editable upon creation, but read-only fields when the review is being edited. We could achieve this by specifying an `if obj is None` condition.

```
def get_readonly_fields(self, request, obj=None):
    if obj is None or request.user.is_superuser:
        return self.readonly_fields
    return ['creator', 'book',] + self.readonly_fields
```

list_editable

There are occasions when you view a list of objects when it would be handy to be able to edit the display fields on the change list without having to edit a change page for each record.

Fortunately, the `ModelAdmin` object provides an option for this. We can assign fields to the `list_editable` attribute.

Exercise 10.2

In this exercise, we will use some of the `ModelAdmin` attributes that we have learned about to carry out some fine-grained customizations of the admin interface. In our modification to the `Review` model, reviews can be assigned a star ranking. We would like to modify the display of reviews in their respective admin form so that they appear as vertical radio fields.

Include the rating in `ReviewsAdmin list_display`. Note that the `CharFields` choices representation appears in `list_display`.

Add a method called `has_profile_photo` to `ReviewerProfile`. It should return an empty string if no profile photo has been uploaded or an appropriate emoji if one is available. We might use a camera, painting palette, or framed picture emoji (Unicode U+1F5BC, HTML 🖼️).

In order to display formatted HTML in `list_display` rather than escaping it, we need to use the `format_html` function in `reviews/admin.py`.

```
from django.utils.html import format_html
```

Create a `ReviewerProfileAdmin` class in `reviews/admin.py`, then adjust the model registration to include the custom admin class:

```
def has_profile_photo(self, obj):
    return obj.profile_photo and format_html(
        '<h1>🖼️ </h1>') or ''
```

리뷰의 생성 시에는 작성자와 책 필드만 편집 가능하고, 리뷰를 편집할 때는 읽기 전용 필드로 설정하고 싶다고 가정해 보겠습니다. 이는 obj가 None인 경우 조건을 지정하여 달성할 수 있습니다.

```
def get_readonly_fields(self, request, obj=None):
    if obj is None or request.user.is_superuser:
        return self.readonly_fields

    return ['creator', 'book',] + self.readonly_fields
```

list_editable

객체 목록 idx_4e3e8f99a를 볼 때 각 레코드에 대해 변경 페이지를 편집하지 않고도 변경 목록에서 표시 필드를 편집할 수 있으면 유용한 경우가 있습니다.

다행히도 ModelAdmin 객체는 이를 위한 옵션을 제공합니다. 우리는 list_editable 속성에 필드를 할당할 수 있습니다.

연습 10.2

이 연습에서는 관리 인터페이스의 세부적인 사용자 지정을 수행하기 위해 우리가 배운 ModelAdmin 속성 중 일부를 사용할 것입니다. Review 모델의 수정에서 리뷰는 별 등급을 부여받을 수 있습니다. 우리는 리뷰가 해당 관리 양식에서 수직 라디오 필드로 표시되도록 수정하고자 합니다.

ReviewsAdmin list_display에 등급을 포함합니다. CharFields 선택 항목의 표현이 list_display에 나타나는 것을 주목하세요.

ReviewerProfile에 has_profile_photo라는 메서드를 추가합니다. 프로필 사진이 업로드되지 않은 경우 빈 문자열을 반환하고, 사진이 있는 경우 적절한 이미지를 반환해야 합니다. 우리는 카메라, 화구, 또는 액자에 담긴 사진 이미지(유니코드 U+1F5BC, HTML)를 사용할 수 있습니다.

list_display에서 HTML을 이스케이프하지 않고 형식화된 HTML을 표시하려면 reviews/admin.py에서 format_html 함수를 사용해야 합니다.

```
from django.utils.html import format_html
```

reviews/admin.py에 ReviewerProfileAdmin 클래스를 생성한 다음, 모델 등록을 조정하여 사용자 정의 관리 클래스를 포함시킵니다:

```
def has_profile_photo(self, obj):
    return obj.profile_photo and format_html(
        '<h1> </h1>') or ''
```

Change lists on reviewer profiles should display username, location, and has_profile_photo.

Make the location field of ReviewerProfile editable from an admin display page.

The reviews/admin.py file should now look like this:

```
from django.utils.html import format_html

class ReviewerProfileAdmin(admin.ModelAdmin):
    list_display = ('user', 'location',
                   'has_profile_photo')
    list_editable= ('location',)

    def has_profile_photo(self, obj):
        return obj.profile_photo and format_html(
            '<h1>📷</h1>') or ''
    ...
admin.site.register(ReviewerProfile, ReviewerProfileAdmin)
```

Now, once you have created some reviewer profiles, the change list will look something like *Figure 10.7*.

☐	USER	LOCATION	HAS PROFILE PHOTO
☐	marksandler@test.com	Bukit Merah, Singapore	
☐	alice	Melbourne, Australia	
☐	bob	Shanghai, China	

3 reviewer profiles
Save

Figure 10.7: The reviewer profile change list

We've looked at how we can enhance form layout in the admin interface. Now we will examine pagination, search, and filter customizations.

검토자 프로필의 변경 목록은 사용자 이름, 위치 및 프로필 사진 여부를 표시해야 합니다. 관리자 표시 페이지에서 ReviewerProfile의 위치 필드를 편집 가능하게 만드세요. reviews/admin.py 파일은 이제 다음과 같아야 합니다:

```
from django.utils.html import format_html

class ReviewerProfileAdmin(admin.ModelAdmin):
    list_display = ('user', 'location',
                    'has_profile_photo')
    list_editable = ('location',)

    def has_profile_photo(self, obj):
        return obj.profile_photo and format_html(
            '<h1></h1>' ) or ''
    ...

admin.site.register(ReviewerProfile, ReviewerProfileAdmin)
```

이제 몇 개의 리뷰어 프로필을 생성했으므로 변경 목록은 Figure 10.7과 비슷하게 보일 것입니다.

☐ USER	LOCATION	HAS PROFILE PHOTO
☐ marksandler@test.com	Bukit Merah, Singapore	
☐ alice	Melbourne, Australia	
☐ bob	Shanghai, China	

3 reviewer profiles

Save

Figure 10.7: 리뷰어 프로필 변경 목록

우리는 관리 인터페이스에서 양식 레이아웃을 향상시키는 방법을 살펴보았습니다. 이제 페이지 매김, 검색 및 필터 사용자 지정을 살펴보겠습니다.

Pagination customizations

So far, we have been dealing with relatively small datasets in our example, but as our new data is accumulated, we have to render more and more information on our pages. Fortunately, the admin app implements options for pagination so that we can return small segments of retrieved data at a time. This is necessary for performance, as large query sets may lead to database performance issues and will slow down web server responsiveness.

By default, records are paginated into sets of 100. For many applications, we would prefer to have a smaller set that doesn't require page scrolling to navigate.

We can alter this by using the `list_per_page` attribute of the `ModelAdmin` class.

```
class ReviewAdmin(admin.ModelAdmin):
    list_display = ('creator', 'book', 'rating')
    list_per_page = 10
```

We can see in *Figure 10.8* that records are now paginated into 10 objects per page.

(Try this with 5 items.)

Action: 0 of 10 selected

☐	CREATOR	BOOK	RATING
☐	david	Paul Clifford (9781719053167)	2
☐	david	Fahrenheit 451 (9781451673319)	2
☐	david	The Catcher in the Rye (9780316769174)	3
☐	david	The Great Gatsby (9780743273565)	2
☐	david	To Kill A Mocking Bird (9780060935467)	3
☐	david	Brave New World (9780060850524)	3
☐	david	Deep Reinforcement Learning Hands-On (9781788834247)	3
☐	david	Architects of Intelligence (9781789954531)	3
☐	david	Hands-On Machine Learning for Algorithmic Trading (9781789346411)	4
☐	carol	Animal Farm: A Fairy Story (9780151002177)	5

1 2 3 4 40 reviews Show all

Figure 10.8: The ratings result frame

At the bottom of the result set, there is a Show all link. Clicking this link will return all records without pagination set. By default, the Show all link will only appear if there are 200 or fewer records to be returned. By using the `list_max_show_all` attribute, we can alter this number.

페이지 매김 사용자 정의

지금까지 우리는 예제에서 상대적으로 작은 데이터 세트를 다루어 왔지만, 새로운 데이터가 축적됨에 따라 페이지에 더 많은 정보를 렌더링해야 합니다. 다행히도, 관리 앱은 페이지 매김 옵션을 구현하여 한 번에 검색된 데이터의 작은 세그먼트를 반환할 수 있습니다. 이는 성능에 필요하며, 큰 쿼리 세트는 데이터베이스 성능 문제를 일으킬 수 있고 웹 서버 응답 속도를 저하시킬 수 있습니다.

기본적으로 레코드는 10개의 세트로 페이지 매김됩니다. 많은 애플리케이션에서는 페이지 스크롤 없이 탐색할 수 있는 더 작은 세트를 선호합니다.

우리는 `ModelAdmin` 클래스의 `list_per_page` 속성을 사용하여 이를 변경할 수 있습니다.

```
class ReviewAdmin(admin.ModelAdmin):
    list_display = ('creator', 'book', 'rating')
    list_per_page = 10
```

그림 10.8에서 레코드가 이제 페이지당 10개의 객체로 페이지 매김된 것을 볼 수 있습니다.(5개 항목으로 시도해 보세요.)

Action: Go 0 of 10 selected

☐	CREATOR	BOOK	RATING
☐	david	Paul Clifford (9781719053167)	2
☐	david	Fahrenheit 451 (9781451673319)	2
☐	david	The Catcher in the Rye (9780316769174)	3
☐	david	The Great Gatsby (9780743273565)	2
☐	david	To Kill A Mocking Bird (9780060935467)	3
☐	david	Brave New World (9780060850524)	3
☐	david	Deep Reinforcement Learning Hands-On (9781788834247)	3
☐	david	Architects of Intelligence (9781789954531)	3
☐	david	Hands-On Machine Learning for Algorithmic Trading (9781789346411)	4
☐	carol	Animal Farm: A Fairy Story (9780151002177)	5

1 2 3 4 40 reviews Show all

그림 10.8: 평점 결과 프레임

결과 집합의 하단에 모든 항목 보기 링크가 있습니다. 이 링크를 클릭하면 페이지 매김 없이 모든 레코드가 반환됩니다. 기본적으로 모든 항목 보기 링크는 반환할 레코드가 200개 이하일 때만 나타납니다. `list_max_show_all` 속성을 사용하여 이 숫자를 변경할 수 있습니다.

Search and filter customizations

We reviewed Django's search filter capability in *Chapter 4*. There are additional options available for customizing the filter's performance and appearance.

By default, when we retrieve a result set from a filter operation, the result set record count is displayed. What might not be apparent is that in order to display this number, Django has to make a secondary database query after the primary query that retrieves the dataset.

As pagination is an efficient way to retrieve a small portion of the results at a time, this second query that returns the record count can often be the one that creates performance issues for the site, especially with large record sets or datasets that involve multiple relations with other models. By default, `show_full_result_count=True`. If you are finding that Django is taking too long to retrieve and render your filtered result set, it may be worth setting `show_full_result_count=False` and seeing if this improves the filter's performance.

By default, after we edit an object that we retrieve from a filtered result set, we are returned to the filter set. This is usually the desired behavior, though sometimes it is a more natural workflow to return to the model index page without any filter settings. If this is the case, it is appropriate to specify the `preserve_filters=False` option in our `ModelAdmin` subclass.

These behaviors will work the same if the result set is derived from a search bar query or from a filter selection.

```
class ReviewAdmin(admin.ModelAdmin):
    ...
    list_filter = ('content', 'creator__username',
                  'book__title')
    search_fields = ('creator__username',)
    show_full_result_count = False
    preserve_filters = False
```

Activity 10.2

This activity will consolidate our knowledge of pagination, search, and filter customizations.

1. Standardize the pagination on the Reviews pages so that all objects are returned with a maximum of 10 records.
2. Remove the Show all link for result sets that have more than 50 records.
3. Add a search bar for the Review admin page. Allow searching on Book title, Creator username, and Review content.

검색 및 필터 사용자 정의

우리는 4장에서 Django의 검색 필터 `idx_06f6dbdbcapability`를 검토했습니다. 필터의 성능과 외관을 사용자 정의하기 위한 추가 옵션이 있습니다.

기본적으로 필터 작업에서 결과 집합을 검색할 때 결과 집합의 레코드 수가 표시됩니다. 이 숫자를 표시하기 위해 Django는 데이터 세트를 검색하는 기본 쿼리 후에 보조 데이터베이스 쿼리를 수행해야 한다는 점은 명확하지 않을 수 있습니다.

페이지 매김은 한 번에 결과의 작은 부분을 검색하는 효율적인 방법이므로, 레코드 수를 반환하는 이 두 번째 쿼리는 종종 사이트의 성능 문제를 일으키는 원인이 될 수 있습니다. 특히 대규모 레코드 집합이나 다른 모델과의 여러 관계가 포함된 데이터 세트의 경우 더욱 그렇습니다. 기본적으로 `show_full_result_count=True`입니다. Django가 필터링된 결과 집합을 검색하고 렌더링하는 데 너무 오랜 시간이 걸린다면, `show_full_result_count=False`로 설정하여 필터의 성능이 개선되는지 확인해 볼 가치가 있습니다.

기본적으로 필터링된 결과 집합에서 검색한 객체를 편집한 후에는 필터 집합으로 돌아갑니다. 이는 일반적으로 원하는 동작이지만, 때때로 필터 설정 없이 모델 인덱스 페이지로 돌아가는 것이 더 자연스러운 워크플로우일 수 있습니다. 이 경우 `ModelAdmin` 서브클래스에서 `preserve_filters=False` 옵션을 지정하는 것이 적절합니다.

이러한 동작은 결과 집합이 검색 바 쿼리에서 파생되었거나 필터 선택에서 파생되었는지에 관계없이 동일하게 작동합니다.

```
class ReviewAdmin(admin.ModelAdmin):
    ...
    list_filter = ('content', 'creator__username',
                  'book__title')
    search_fields = ('creator__username',)
    show_full_result_count = False
    preserve_filters = False
```

Activity 10.2

이 활동은 페이지 매김, 검색 및 필터 사용자 지정을 통합할 것입니다.

1. 리뷰 페이지의 페이지 매김을 표준화하여 모든 객체가 최대 10개의 레코드로 반환되도록 합니다.
2. 50개 이상의 레코드가 있는 결과 집합에 대한 모든 표시 링크를 제거합니다.
3. 리뷰 관리자 페이지에 검색 바를 추가합니다. 책 제목, 작성자 사용자 이름 및 리뷰 내용을 검색할 수 있도록 합니다.

4. Add a method to `ReviewAdmin` called `rating_stars`. It will return a number of stars (Unicode character U+2606) based on the value of rating. Use this method for display formatting of review ratings.
5. Add a filter on the review admin page for review rating.
6. Because the `Review` model has foreign key relations with `Book` and `Contributor`, the structure of the result count query may well cause performance issues for large query sets. Modify `ModelAdmin` for the `Review` class so that the full result count is not displayed.
7. By completing this activity, we have completed our customizations of the reviews app admin pages.

The modified change list page for Reviews will look like *Figure 10.9*.

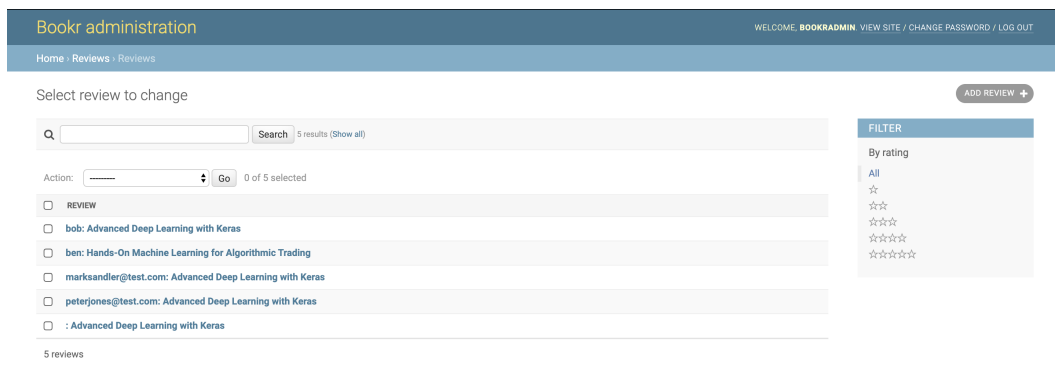


Figure 10.9: Search and filters on the review change list

Adding views to the admin site

Just like general applications inside Django, which can have multiple views associated with them, Django allows developers to add custom views to the admin site, also, which allows a developer to increase the scope of what the admin site interface can do.

The ability to add your own views to the admin site provides a lot of extensibility to the admin panel of the website, which can be leveraged for several additional use cases. For example, as we discussed at the start of the chapter, an IT team of a big organization can add a custom view to the admin site, which can then be used to monitor the health of the different IT systems at the organization, as well as to provide the IT team with the ability to quickly look at any urgent alerts that need to be addressed.

Now, the next question we need to answer is, *How can we add a custom view to the admin site?*

4. ReviewAdmin에 rating_stars라는 메서드를 추가하세요. 이 메서드는 rating의 값에 따라 별의 수(유니코드 문자 U+2606)를 반환합니다. 리뷰 평점의 표시 형식에 이 메서드를 사용하세요.
5. 리뷰 평점에 대한 필터를 리뷰 관리 페이지에 추가하세요.
6. Review 모델이 Book 및 Contributor와 외래 키 관계를 가지고 있기 때문에, 결과 수 쿼리의 구조는 대규모 쿼리셋에 대해 성능 문제를 일으킬 수 있습니다. Review 클래스의 ModelAdmin을 수정하여 전체 결과 수가 표시되지 않도록 하세요.
7. 이 활동을 완료함으로써 우리는 리뷰 앱 관리 페이지의 사용자 지정을 완료했습니다.

수정된 리뷰 변경 목록 페이지는 그림 10.9와 같을 것입니다.

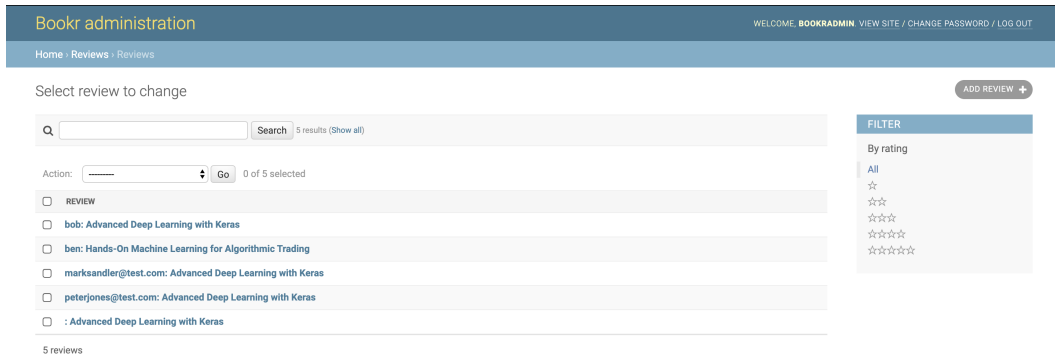


그림 10.9: 리뷰 변경 목록의 검색 및 필터

관리 사이트에 뷰 추가하기

Django 내부의 일반 `idx_6d7ad528applications`처럼, `idx_cdd53768`다수의 뷰가 연결될 수 있는 것처럼, Django는 개발자가 관리 사이트에 사용자 정의 뷰를 추가할 수 있도록 허용하며, 이를 통해 개발자는 관리 사이트 인터페이스의 기능 범위를 확장할 수 있습니다.

관리 사이트에 자신의 뷰를 추가할 수 있는 기능은 웹사이트의 관리 패널에 많은 확장성을 제공하며, 이는 여러 추가 사용 사례에 활용될 수 있습니다. 예를 들어, 장의 시작 부분에서 논의한 것처럼, 대규모 조직의 IT 팀은 관리 사이트에 사용자 정의 뷰를 추가할 수 있으며, 이를 통해 조직의 다양한 IT 시스템의 상태를 모니터링하고, IT 팀이 신속하게 해결해야 할 긴급 경고를 확인할 수 있는 기능을 제공합니다.

이제 우리가 답해야 할 다음 질문은, 관리 사이트에 사용자 정의 뷰를 어떻게 추가할 수 있는가입니다?

As it turns out, adding a new view inside the admin template is quite easy and follows the same approach we used while creating views for our applications, with some minor modifications. So, let's look at how we can add a new view to our Django admin dashboard.

Creating the view function

The first step to adding a new view to the Django application is to create a view function that implements the logic to handle the view. In the previous chapters, we created view functions inside a separate file known as `views.py`, which was used to hold all our method- and class-based views.

In the context of adding a new view to the Django admin dashboard, to create a new view, we need to define a new view function inside our custom `AdminSite` class. For example, to add a new view that renders a page showing the health of the different IT systems inside an organization, we can create a new view function named `system_health_dashboard()` inside our custom `AdminSite` class implementation, as shown in the following code snippet:

```
class SysAdminSite(admin.AdminSite):
    def system_health_dashboard(self, request):
        # View function logic
```

Inside the view function, we can perform any operations we want in order to generate a view and, finally, use that response to render a template. Inside this view function, there are some important pieces of logic we need to make sure are implemented correctly.

The first one is to set the `current_app` property for the request field inside the view function. This is required in order to allow Django's URL resolver inside the templates to correctly resolve the view functions for an application. To set this value inside the custom view function we just created, we need to set the `current_app` property, as shown in the following code snippet:

```
request.current_app = self.name
```

The `self.name` field is automatically populated by Django's `AdminSite` class, and we don't need to initialize it explicitly. With this, our minimal custom view implementation will appear, as shown in the following code snippet:

```
class SysAdminSite(admin.AdminSite):
    def system_health_dashboard(self, request):
        request.current_app = self.name
        # View function logic
```

알고 보니, 관리자 템플릿 안에 새로운 뷰를 추가하는 것은 꽤 쉽고, 우리의 애플리케이션을 위한 뷰를 생성할 때 사용했던 것과 동일한 접근 방식을 따르며, 약간의 수정만 필요합니다. 그러니, Django 관리자 대시보드에 새로운 뷰를 추가하는 방법을 살펴보겠습니다.

뷰 함수 생성

Django 애플리케이션에 새로운 뷰를 추가하는 첫 번째 단계는 뷰를 처리하기 위해 `idx_2be6ac0aologic`을 구현하는 뷰 함수를 만드는 것입니다. 이전 장에서는 `views.py`라는 별도의 파일 안에 뷰 함수를 생성했으며, 이 파일은 모든 메서드 및 클래스 기반 뷰를 보관하는 데 사용되었습니다.

Django 관리자 대시보드에 새로운 뷰를 추가하는 맥락에서, 새로운 뷰를 생성하기 위해서는 우리의 사용자 정의 `AdminSite` 클래스 안에 새로운 뷰 함수를 정의해야 합니다. 예를 들어, 조직 내 다양한 IT 시스템의 상태를 보여주는 페이지를 렌더링하는 새로운 뷰를 추가하기 위해, 우리는 사용자 정의 `AdminSite` 클래스 구현 안에 `system_health_dashboard()`라는 새로운 뷰 함수를 생성할 수 있습니다. 다음 코드 스니펫과 같이요:

```
class SysAdminSite(admin.AdminSite):
    def system_health_dashboard(self, request):
        # 뷰 함수 로직
```

뷰 함수 안에서, 우리는 뷰를 생성하기 위해 원하는 모든 작업을 수행할 수 있으며, 마지막으로 그 응답을 사용하여 템플릿을 렌더링합니다. 이 뷰 함수 안에는 우리가 올바르게 구현해야 할 몇 가지 중요한 로직이 있습니다.

첫 번째는 뷰 함수 내의 요청 필드에 대해 `current_app` 속성을 설정하는 것입니다. 이는 Django의 URL 해석기가 템플릿 내에서 애플리케이션의 뷰 함수를 올바르게 해석할 수 있도록 하기 위해 필요합니다. 방금 생성한 사용자 정의 뷰 함수 내에서 이 값을 설정하려면, 다음 코드 조각에 표시된 대로 `current_app` 속성을 설정해야 합니다:

```
request.current_app = self.name
```

`self.name` 필드는 Django의 `AdminSite` 클래스에 의해 자동으로 채워지며, 우리는 이를 명시적으로 초기화할 필요가 없습니다. 이렇게 하면 우리의 최소한의 사용자 정의 뷰 구현이 다음 코드 조각에 표시된 대로 나타납니다:

```
class SysAdminSite(admin.AdminSite):
    def system_health_dashboard(self, request):
        request.current_app = self.name
        # 뷰 함수 로직
```

Accessing common template variables

When creating a custom view function, we might want access to the common template variables, such as `site_header` and `site_title`, in order to render them correctly in the template associated with our view function. As it turns out, this is quite easy to achieve with the use of the `each_context()` method provided by the `AdminSite` class.

The `each_context()` method of the `AdminSite` class takes a single parameter, `request`, which is the current request context, and returns the template variables that are to be inserted in all the admin site templates.

For example, if we wanted to access the template variables inside our custom view function, we could implement code similar to the following code snippet:

```
def system_health_dashboard(self, request):
    request.current_app = self.name
    context = self.each_context(request)
    # view function logic
```

The value returned by the `each_context()` method is a dictionary containing the name of the variable and the associated value.

Mapping URLs for the custom view

Once the view function is defined, the next step involves mapping this view function to a URL so that a user can access it or allow the other views to link to it. For the views defined inside `AdminSite`, this URL mapping to views is controlled by the `get_urls()` method implemented by the `AdminSite` class. The `get_urls()` method returns the `urlpatterns` list that maps to the `AdminSite` views.

If we want to add a URL mapping for our custom view, the preferred approach includes overriding the implementation of `get_urls()` in our custom `AdminSite` class and adding the URL mapping there. This approach is demonstrated in the following code snippet:

```
class SysAdminSite(admin.AdminSite):
    def get_urls(self):
        base_urls = super().get_urls(). # Get the existing
                                         set of URLs

        # Define our URL patterns for custom views
        urlpatterns = [
            path("health_dashboard/",
                self.system_health_dashboard)
```

공통 템플릿 변수 접근하기

idx_8cdf77e9a 사용자 정의 뷰 함수를 생성할 때, `site_header` 및 `site_title`과 같은 공통 템플릿 변수에 접근하여 뷰 함수와 연결된 템플릿에서 이를 올바르게 렌더링하고자 할 수 있습니다. 사실, 이는 `AdminSite` 클래스에서 제공하는 `each_context()` 메서드를 사용하면 매우 쉽게 달성할 수 있습니다.

`AdminSite` 클래스의 `each_context()` 메서드는 현재 요청 컨텍스트인 단일 매개변수 `request`를 받아들이고, 모든 관리자 사이트 템플릿에 삽입될 템플릿 변수를 반환합니다.

예를 들어, 사용자 정의 뷰 함수 내에서 템플릿 변수에 접근하고자 한다면, 다음 코드 조각과 유사한 코드를 구현할 수 있습니다:

```
def system_health_dashboard(self, request):
    request.current_app = self.name
    context = self.each_context(request)
    # 뷰 함수 로직
```

`each_context()` 메서드가 반환하는 값은 변수의 이름과 관련된 값을 포함하는 딕셔너리입니다.

사용자 정의 뷰를 위한 URL 매핑

idx_29494be5function 뷰가 정의되면, 다음 단계는 idx_1ffed05b이 뷰 함수를 URL에 매핑하는 것입니다. 이를 통해 사용자가 접근하거나 다른 뷰가 이 뷰에 링크할 수 있습니다. `AdminSite` 내부에 정의된 뷰의 경우, 이 URL과 뷰 간의 매핑은 `AdminSite` 클래스에서 구현된 `get_urls()` 메서드에 의해 제어됩니다. `get_urls()` 메서드는 `AdminSite` 뷰에 매핑되는 `urlpatterns` 리스트를 반환합니다.

커스텀 뷰에 대한 URL 매핑을 추가하려면, 권장되는 방법은 커스텀 `AdminSite` 클래스에서 `get_urls()` 메서드의 구현을 재정의하고 그곳에 URL 매핑을 추가하는 것입니다. 이 방법은 다음 코드 스니펫에서 보여줍니다:

```
class SysAdminSite(admin.AdminSite):
    ...
    def get_urls(self):
        ...
        base_urls = super().get_urls(). # 기존 URL 가져오기
                                         URL 집합

        # 사용자 정의 뷰를 위한 URL 패턴 정의하기
        urlpatterns = [
            ...
            path("health_dashboard/",
                self.system_health_dashboard)
```

```
]
    return base_urls + urlpatterns. # Return the
                                    updated mapping
```

The `get_urls()` method is generally called automatically by Django, and there is no need to perform any manual processing on it.

Once this is done, the last step involves making sure that our custom admin view is only accessible through the admin site, and non-admin users should not be able to access it. Let's take a look at how that can be achieved.

Restricting custom views to the admin site

If you followed all the previous headings thoroughly, you will now have a custom `AdminSite` view ready for use. However, there is a small glitch. This view is also directly accessible to any user who is not on the admin site.

To ensure that such a situation does not arise, we need to restrict this view to the admin site. This can be achieved quite simply by wrapping our URL path inside the `admin_view()` call, as shown in the following code snippet:

```
urlpatterns = [
    self.admin_view(path("health_dashboard/",
                        self.system_health_dashboard))
]
```

The `admin_view` function makes sure the path provided to it is restricted just to the admin dashboard and that no non-admin-privilege user can access it.

Now, let's add a new custom view to our admin site.

Exercise 10.03 – adding custom views to the admin site

In this exercise, you will add a custom view to the admin site, which will render a user profile and will show the user the options to modify their email and add a new profile picture. To build this custom view, follow the steps described:

1. Open the `admin.py` file under the `bookr_admin` directory and add the following imports. These will be required to build our custom view inside the admin site application:

```
from django.template.response import TemplateResponse
from django.urls import path
```

```
]
return base_urls + urlpatterns. # 반환
```

업데이트된 매핑

get_urls() 메서드는 일반적으로 Django에 의해 자동으로 호출되며, 수동으로 처리할 필요가 없습니다.

이 작업이 완료되면 마지막 단계는 우리의 사용자 정의 관리자 뷰가 관리자 사이트를 통해서만 접근 가능하도록 하고 비관리자 사용자는 접근할 수 없도록 하는 것입니다. 이를 달성하는 방법을 살펴보겠습니다.

관리자 사이트에 사용자 정의 뷰 제한하기

이전 제목을 철저히 따랐다면 이제 사용할 준비가 된 사용자 정의 AdminSite 뷰가 있을 것입니다. 그러나 작은 문제가 있습니다. 이 뷰는 관리자 사이트에 없는 모든 사용자에게도 직접 접근 가능합니다.

이런 상황이 발생하지 않도록 하기 위해 이 뷰를 관리자 사이트에 제한해야 합니다. 이는 다음 코드 스니펫에 표시된 대로 admin_view() 호출 안에 URL 경로를 감싸는 것으로 간단히 달성할 수 있습니다:

```
urlpatterns = [
    self.admin_view(path("health_dashboard/",
                        self.system_health_dashboard))
]
```

admin_view 함수는 제공된 경로가 관리자 대시보드에만 제한되도록 하며 비관리자 권한 사용자가 접근할 수 없도록 합니다.

이제, 관리자 사이트에 새로운 사용자 정의 뷰를 추가해 보겠습니다.

연습 10.03 – 관리자 사이트에 사용자 정의 뷰 추가하기

이 연습에서는 관리자 사이트에 사용자 정의 뷰를 추가할 것입니다. 이 뷰는 사용자 프로필을 렌더링하고 사용자가 이메일을 수정하고 새로운 프로필 사진을 추가할 수 있는 옵션을 보여줍니다. 이 사용자 정의 뷰를 만들기 위해 다음 단계를 따르세요:

1. bookr_admin 디렉토리 아래의 admin.py 파일을 열고 다음 import를 추가합니다. 이는 관리자 사이트 애플리케이션 내에서 사용자 정의 뷰를 구축하는 데 필요합니다:

```
from django.template.response import TemplateResponse
from django.urls import path
```


2. Open the `admin.py` file under the `bookr_admin` directory and create a new method named `profile_view`, which takes in a request variable as its parameter, inside the `BookrAdmin` class:

```
def profile_view(self, request):
```

Next, inside the method, get the name of the current application and set that in the request context. For this, you can use the `name` property of the class, which is auto-populated by Django. To get this property and set it in your request context, you need to add the following line:

```
request.current_app = self.name
```

Once you have the application name populated in the request context, the next step is to fetch the template variables, which are required to render the contents, such as `site_title` and `site_header`, in the admin templates. For this, leverage the `each_context()` method of the `AdminSite` class, which provides the dictionary of the admin site template variables from the class:

```
context = self.each_context(request)
```

Once you have the data in place, the last step is to return a `TemplateResponse` object, which will render the custom profile template when someone visits the URL endpoint mapped to your custom view:

```
return TemplateResponse(request,  
    "admin/admin_profile.html", context)
```

3. With the view function now created, the next step is to make `AdminSite` return the URLs mapping the view to a path inside `AdminSite`. To do this, you need to create a new method with the name `get_urls()`, which overrides the `AdminSite.get_urls()` method and returns the mapping of your new view. This can be done by first creating a new method named `get_urls()` inside the `BookrAdmin` class you created for your custom admin site:

```
def get_urls(self):
```

Inside this method, the first thing you need to do is to get the list of the URLs that are already mapped to the admin endpoint. This is a required step; otherwise, your custom

2. bookr_admin 디렉토리 아래의 admin.py 파일을 열고, BookrAdmin 클래스 내부에 request 변수를 매개변수로 받는 profile_view라는 새 메서드를 만듭니다:

```
def profile_view(self, request):
```

다음으로, 메서드 내부에서 현재 애플리케이션의 이름을 가져와서 요청 컨텍스트에 설정합니다. 이를 위해 Django에 의해 자동으로 채워지는 클래스의 name 속성을 사용할 수 있습니다. 이 속성을 가져와서 요청 컨텍스트에 설정하려면 다음 줄을 추가해야 합니다:

```
request.current_app = self.name
```

요청 컨텍스트에 애플리케이션 이름이 채워지면, 다음 단계는 admin 템플릿에서 site_title 및 site_header와 같은 내용을 렌더링하는 데 필요한 템플릿 변수를 가져오는 것입니다. 이를 위해 AdminSite 클래스의 each_context() 메서드를 활용하여 클래스에서 admin 사이트 템플릿 변수의 사전을 제공합니다:

```
context = self.each_context(request)
```

데이터가 준비되면 마지막 단계는 TemplateResponse 객체를 반환하는 것입니다. 이는 누군가가 사용자 정의 뷰에 매핑된 URL 엔드포인트를 방문할 때 사용자 정의 프로필 템플릿을 렌더링합니다:

```
return TemplateResponse(request,
    "admin/admin_profile.html", context)
```

3. 이제 뷰 함수가 생성되었으므로 다음 단계는 AdminSite가 뷰를 AdminSite 내부의 경로에 매핑하는 URL을 반환하도록 만드는 것입니다. 이를 위해 idx_961f2abaneed는 AdminSite.get_urls() 메서드를 재정의하고 새 뷰의 매핑을 반환하는 get_urls()라는 새 메서드를 생성해야 합니다. 이는 사용자 정의 관리 사이트를 위해 생성한 BookrAdmin 클래스 내부에 get_urls()라는 새 메서드를 먼저 생성하여 수행할 수 있습니다:

```
def get_urls(self):
```

이 메서드 내부에서 가장 먼저 해야 할 일은 이미 관리 엔드포인트에 매핑된 URL 목록을 가져오는 것입니다. 이는 필수 단계입니다. 그렇지 않으면 사용자 정의

admin site will not be able to load any results associated with the model editing pages, the logout page, and so on if this mapping is lost. To get this mapping, call the `get_urls()` method of the base class from which the `BookrAdmin` class is derived:

```
urls = super().get_urls()
```

Once the URLs from the base class are captured, the next step is to create a list of URLs that map our custom view to a URL endpoint in the admin site. To do this, we will create a new list named `url_patterns` and map our `profile_view` method to the `admin_profile` endpoint. This is done by using the `path` utility function from Django, which allows us to map the view function with a string-based API endpoint path:

```
url_patterns = [  
    path("admin_profile", self.profile_view)  
]  
return url_patterns + urls
```

Save the `admin.py` file. It should look like `Exercise10.03/bookr/admin.py` in the `Chapter10` folder in this book's GitHub repo. .

Note that the value returned by `get_urls()` is a non-modifiable list. In the preceding code snippet, if we try to switch the order between the merging of the `url_patterns` and `urls` objects, the method won't work, and the `admin_profile` URL won't be created.

4. Now, with the `BookrAdmin` class configured for the new view, the next step is to create your template for the admin profile page. To do this, create a new file named `admin_profile.html` under the `templates/admin` directory of your project root. Inside this file, first, add an `extends` tag to make sure that you are extending from the default admin template:

```
{% extends "admin/index.html" %}
```

This step ensures that all of your admin template style sheets and HTML are available for use inside your custom view template. For example, without having this `extends` tag, your custom view will not show any specific content already mapped to the admin site, such as `site_header`, `site_title`, or any links to log out or go to another page.

Once the `extends` tag is added, add a `block` tag and provide it with the value of content. This makes sure that the code you add between the pair of `{% block content %}...{%`

이 매핑이 손실되면 관리 사이트는 모델 편집 페이지, 로그아웃 페이지 등과 관련된 결과를 로드할 수 없습니다. 이 매핑을 얻으려면 BookrAdmin 클래스가 파생된 기본 클래스의 `get_urls()` 메서드를 호출하십시오:

```
urls = super().get_urls()
```

기본 클래스에서 URL을 캡처한 후 다음 단계는 사용자 정의 보기를 관리 사이트의 URL 엔드포인트에 매핑하는 URL 목록을 만드는 것입니다. 이를 위해 `url_patterns`라는 새 목록을 만들고 `profile_view` 메서드를 `admin_profile` 엔드포인트에 매핑합니다. 이는 Django의 `path` 유틸리티 함수를 사용하여 문자열 기반 API 엔드포인트 경로와 보기 함수를 매핑함으로써 수행됩니다:

```
url_patterns = [
    path("admin_profile", self.profile_view)
]

return url_patterns + urls
```

`admin.py` 파일을 저장하세요. 이 파일은 이 책의 GitHub 리포지토리의 Chapter10 폴더에 있는 `Exercise10.03/bookr/admin.py`와 같아야 합니다.

`get_urls()`에서 반환되는 값은 수정할 수 없는 리스트입니다. 앞의 코드 조각에서 `url_patterns`와 `urls` 객체의 병합 순서를 바꾸려고 하면, 메서드가 작동하지 않으며 `admin_profile` URL이 생성되지 않습니다.

- 이제 새로운 뷰에 대해 BookrAdmin 클래스가 구성되었으므로, 다음 단계는 `admin_idx_ecf85b04` 프로필 페이지를 위한 템플릿을 `idx_36f4d59d` 생성하는 것입니다. 이를 위해 프로젝트 루트의 `templates/admin` 디렉토리 아래에 `admin_profile.html`이라는 새 파일을 만드세요. 이 파일 안에 먼저 `extends` 태그를 추가하여 기본 `admin` 템플릿에서 확장하고 있는지 확인합니다:

```
{% extends "admin/index.html" %}
```

이 단계는 모든 `admin` 템플릿 스타일 시트와 HTML이 사용자 정의 뷰 템플릿 내에서 사용 가능하도록 보장합니다. 예를 들어, 이 `extends` 태그가 없으면 사용자 정의 뷰는 `site_header`, `site_title` 또는 로그아웃하거나 다른 페이지로 이동하는 링크와 같이 이미 `admin` 사이트에 매핑된 특정 콘텐츠를 표시하지 않습니다.

`extends` 태그가 추가되면 `block` 태그를 추가하고 그 값으로 `content`를 제공합니다. 이렇게 하면 `{% block content %}...{%` 사이에 추가하는 코드가 확실히 포함됩니다.

`endblock %}` segments overrides whatever value is present in the `index.html` template that comes prepackaged with the Django admin module:

```
{% block content %}
```

Inside the `block` tag, add the HTML required to render the profile view that was created in *step 2* of this exercise:

```
<p>Welcome to your profile, {{ username }}</p>
<p>You can do the following operations</p>
<ul>
  <li><a href="#">Change E-Mail Address</a></li>
  <li><a href="#">Add Profile Picture</a></li>
</ul>
{% endblock %}
```

- Now, with the preceding steps complete, reload your application server by running `python manage.py runserver localhost:8000` and then visiting `http://localhost:8000/admin/admin_profile`.

When the page opens, you can expect to see something like the following screenshot:

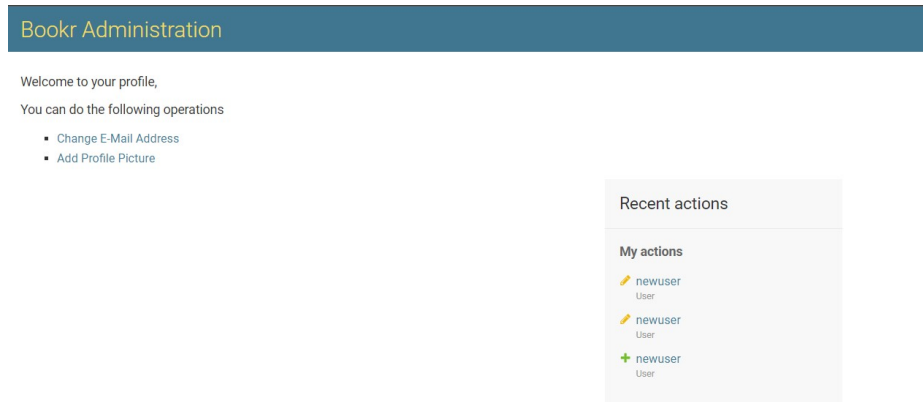


Figure 10.10: The profile page view on the Bookr Administration site

endblock %} 세그먼트는 Django 관리 모듈과 함께 제공되는 index.html 템플릿에 있는 값을 덮어씁니다:

```
{% block content %}
```

블록 태그 안에 이 연습의 2단계에서 생성된 프로필 뷰를 렌더링하는 데 필요한 HTML을 추가하세요:

```
<p>당신의 프로필에 오신 것을 환영합니다,
{{ username }}</p><p>다음 작업을 수행할 수 있습
니다</p><ul>

    <li><a href="#">이메일 주소 변경</a></li>

    <li><a href="#">프로필 사진 추가</a></li>

</ul>

{% endblock %}
```

5. 이제, idx_2d7019d4이전 단계가 완료되었으므로, python manage.py runserver localhost:8000을 실행하여 idx_9e7bfb21애플리케이션 서버를 다시 로드한 후 http://localhost:8000/admin/admin_profile을 방문하세요. 페이지가 열리면 다음 스크린샷과 유사한 것을 볼 수 있습니다:

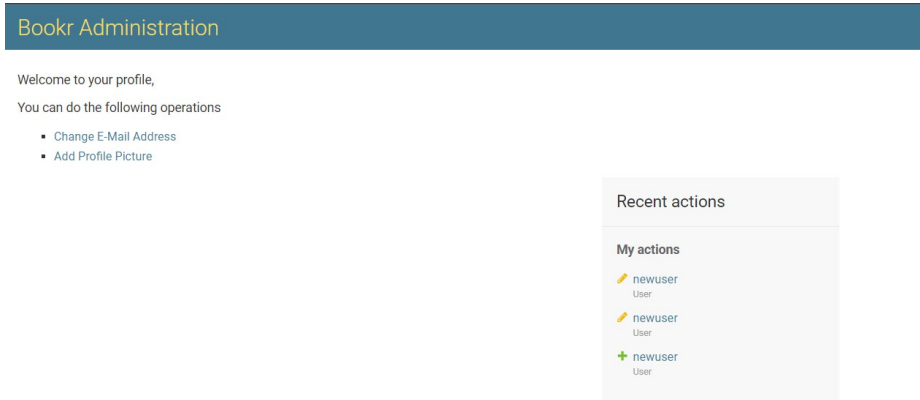


그림 10.10: Bookr 관리 사이트의 프로필 페이지 보기

Note

The view created so far will render just fine, irrespective of whether a user is logged into the admin application.

To make sure that this view is only accessible to the logged-in admins, you need to make a small modification inside your `get_urls()` method, which you defined in *step 2* of this exercise.

Inside the `get_urls()` method, modify the `url_patterns` list to look something like the one shown here:

```
url_patterns = [  
    path("admin_profile",  
        self.admin_view(self.profile_view)),  
]
```

In the preceding code, you wrapped your `profile_view` method inside the `admin_view()` method.

The `AdminSite.admin_view()` method causes the view to be restricted to those users who are logged in. If a user who is currently not logged into the admin site tries to visit the URL directly, they will be redirected to the login page, and only in the event of a successful login will they be allowed to see the contents of our custom page.

During this exercise, we leveraged our existing understanding of writing views for Django applications, merging it with the context of the `AdminSite` class to build a custom view for our admin dashboard. With this knowledge, we can now move on and add useful functionalities to our Django admin site to supercharge its usefulness.

Passing additional keys to templates using template variables

Inside the admin site, the variable values passed to the templates are done so by using template variables. These template variables are prepared and returned by the `AdminSite.each_context()` method.

Now, if there is a value that you want to pass to all the templates of your admin site, you can override the `AdminSite.each_context()` method and add the required fields to the request context. Let's look at an example to see how we can achieve this outcome.

메모

지금까지 생성된 뷰는 사용자가 관리자 애플리케이션에 로그인했는지 여부에 관계없이 잘 렌더링됩니다.

이 뷰가 로그인한 관리자만 접근할 수 있도록 하려면, 이 연습의 2단계에서 정의한 `get_urls()` 메서드 내에서 작은 수정을 해야 합니다.

`get_urls()` 메서드 내에서 `url_patterns` 목록을 다음과 같이 수정합니다:

```
url_patterns = [
    path("admin_profile",
        self.admin_view(self.profile_view)),
]
```

앞선 코드에서, 당신은 `profile_view` 메서드를 `admin_view()` 메서드 안에 감쌌습니다.

`AdminSite.admin_view()` 메서드는 뷰를 로그인한 사용자에게만 제한되도록 합니다. 현재 로그인하지 않은 사용자가 URL에 직접 방문하려고 하면 로그인 페이지로 리디렉션되며, 성공적으로 로그인한 경우에만 우리의 커스텀 페이지 내용을 볼 수 있습니다.

이번 연습에서는 Django 애플리케이션을 위한 뷰 작성에 대한 기존 이해를 활용하여 `AdminSite` 클래스의 맥락과 결합하여 우리의 관리자 대시보드를 위한 커스텀 뷰를 구축했습니다. 이 지식을 바탕으로 이제 유용한 기능을 Django 관리자 사이트에 추가하여 그 유용성을 극대화할 수 있습니다.

템플릿 변수를 사용하여 템플릿에 추가 키 전달하기

관리자 사이트 내에서 템플릿에 전달되는 변수 값은 템플릿 변수를 사용하여 이루어집니다. 이러한 템플릿 변수는 `AdminSite.each_context()` 메서드에 의해 준비되고 반환됩니다.

이제 관리자 사이트의 모든 템플릿에 전달하고 싶은 값이 있다면, `AdminSite.each_context()` 메서드를 오버라이드하고 요청 컨텍스트에 필요한 필드를 추가할 수 있습니다. 이 결과를 달성하는 방법을 보기 위해 예제를 살펴보겠습니다.

Consider the username field, which we passed to our `admin_profile` template earlier. If we want to pass it to every template inside our custom admin site, we first need to override the `each_context()` method inside our `BookrAdmin` class, as shown here:

```
def each_context(self, request):
    context = super().each_context(request)
    context['username'] = request.user.username
    return context
```

The `each_context()` method takes a single argument (we're not considering `self` here) of the `HttpRequest` type, which it uses to evaluate certain other values.

Now, inside our overridden `each_context()` method, we first make a call to the base class `each_context()` method to retrieve the context dictionary for the admin site:

```
context = super().each_context(request)
```

Once that is done, the next thing to do is to add our username field to context and set its value to the value of the `request.user.username` field:

```
context['username'] = request.user.username
```

Once this is done, the last thing that remains is to return this modified context.

Now, whenever a template is rendered by our custom admin site, the template will be passed with this additional username variable.

Configuring an email server with Django

The Django Admin app contains powerful features that enhance the rapid and robust development of applications. It also provides a mechanism to deal with forgotten passwords by emailing the user a secure link to reset the password.

In order to implement this email functionality in Django, we need to configure an SMTP server in the Django bookr/`settings.py` file.

It is recommended to install the `python-decouple` module so that sensitive mail server account details are not saved in the bookr/`settings.py` file. We can install it as follows.

```
pip install python-decouple
```

이전에 `admin_profile` 템플릿에 전달한 사용자 이름 필드를 고려해 보십시오. 이를 사용자 정의 관리 사이트의 모든 템플릿에 전달하려면 먼저 `BookrAdmin` 클래스 내에서 `each_context()` 메서드를 재정의해야 합니다. 다음과 같이:

```
def each_context(self, request):
    context = super().each_context(request)
    context['username'] = request.user.username
    return context
```

`each_context()` 메서드는 단일 인수(여기서 `self`는 고려하지 않음)인 `HttpRequest` 유형을 사용하여 특정 다른 값을 평가하는 데 사용됩니다.

이제 재정의된 `each_context()` 메서드 내에서 먼저 기본 클래스 `each_context()` 메서드를 호출하여 관리 사이트의 컨텍스트 사전을 검색합니다:

```
context = super().each_context(request)
```

`idx_c0be6110`이 완료되면, 다음으로 해야 할 일은 `context`에 사용자 이름 필드를 추가하고 그 값을 `request.user.username` 필드의 값으로 설정하는 것입니다:

```
context['username'] = request.user.username
```

이 작업이 완료되면, 남은 마지막 일은 이 수정된 `context`를 반환하는 것입니다.

이제, 우리의 커스텀 관리자 사이트에 의해 템플릿이 렌더링될 때마다, 템플릿은 이 추가된 사용자 이름 변수를 함께 전달받게 됩니다.

Django로 이메일 서버 구성하기

Django Admin 앱 `idx_bd141b38`은 애플리케이션의 빠르고 강력한 개발을 향상시키는 강력한 기능을 포함하고 있습니다. 또한 사용자가 비밀번호를 재설정할 수 있는 안전한 링크를 이메일로 보내는 방식으로 잊어버린 비밀번호를 처리하는 메커니즘을 제공합니다.

Django에서 이 이메일 기능을 구현하기 위해서는 Django `bookr/settings.py` 파일에서 SMTP 서버를 구성해야 합니다.

민감한 메일 서버 계정 세부정보가 `bookr/settings.py` 파일에 저장되지 않도록 `python-decouple` 모듈을 설치하는 것이 좋습니다. 우리는 다음과 같이 설치할 수 있습니다.

```
pip install python-decouple
```

Now we can add appropriate SMTP settings to the `<projectname>/setting.py` file. The settings that are applicable for SMTP configuration are `EMAIL_BACKEND`, `EMAIL_HOST`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, `EMAIL_PORT`, `EMAIL_USE_TLS`, and `EMAIL_USE_SSL`.

The following are the current settings that work with a Gmail account. Google accounts have recently stopped third-party apps from accepting email account passwords. Instead, they have introduced a system of generating "application passwords." Security policies are prone to frequent changes, but at the time of writing, application passwords can be generated for accounts with two-step verification enabled by visiting this link: <https://myaccount.google.com/apppasswords>.

```
from decouple import config

EMAIL_BACKEND = \
    'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_HOST_USER = config('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_USE_SSL = False
```

These changes to the `settings.py` file include calls to the `decouple.config` function. So, rather than including the email account username and password in the `settings.py` file (which might be included in the source code distribution or public repository), we can set these environment variables at runtime or by including them in the top-level project folder as either an `.env` or a `settings.ini` file. Invoking them at runtime could involve putting them in a `.bashrc` or `.profile` file, or defining them at the command line when the Django server is started:

```
EMAIL_HOST_USER=bookr2026 EMAIL_HOST_PASSWORD='vnpw zgkl ixpq pbcz' python
manage.py runserver
```

Alternatively, an `.env` or a `settings.ini` file can be added to the project directory. The `.env` format is more commonly used on macOS and Linux, while the `settings.ini` format is typically used in Windows environments. Decouple can read both formats on all platforms.

`EMAIL_HOST_USER` is set to the username on the SMTP server, and `EMAIL_HOST_PASSWORD` is set to either the email password or the app password (in the case of Gmail SMTP and other security-conscious email providers).

이제 <projectname>/setting.py 파일에 적절한 SMTP 설정을 추가할 수 있습니다. SMTP 구성에 적용 가능한 설정은 EMAIL_BACKEND, EMAIL_HOST, EMAIL_HOST_USER, EMAIL_HOST_PASSWORD, EMAIL_PORT, EMAIL_USE_TLS 및 EMAIL_USE_SSL입니다.

다음은 Gmail 계정과 함께 작동하는 현재 설정입니다. Google 계정은 최근에 타사 앱이 이메일 계정 비밀번호를 수락하는 것을 중단했습니다. 대신, "애플리케이션 비밀번호"를 생성하는 시스템을 도입했습니다. 보안 정책은 자주 변경될 수 있지만, 작성 시점에서 두 단계 인증이 활성화된 계정에 대해 이 링크를 방문하여 애플리케이션 비밀번호를 생성할 수 있습니다: <https://myaccount.google.com/apppasswords>.

```
from decouple import config

EMAIL_BACKEND = \
    'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_HOST_USER = config('EMAIL_HOST_USER')
EMAIL_HOST_PASSWORD = config('EMAIL_HOST_PASSWORD')
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_USE_SSL = False
```

settings.py 파일에 대한 이러한 변경 사항에는 decouple.config 함수 호출이 포함됩니다. 따라서 settings.py 파일에 이메일 계정 사용자 이름과 비밀번호를 포함하는 대신(소스 코드 배포 또는 공개 저장소에 포함될 수 있음), 런타임에 이러한 환경 변수를 설정하거나 최상위 프로젝트 폴더에 .env 또는 settings.ini 파일로 포함할 수 있습니다. 런타임에 호출하는 것은 .bashrc 또는 .profile 파일에 넣거나 Django 서버가 시작될 때 명령줄에서 정의하는 것을 포함할 수 있습니다:

```
EMAIL_HOST_USER=bookr2026 EMAIL_HOST_PASSWORD='vnpw zgkl ixpq pbcz' python
manage.py runserver
```

또는 .env 또는 settings.ini 파일을 프로젝트 디렉토리에 추가할 수 있습니다. .env 형식은 macOS 및 Linux에서 더 일반적으로 사용되며, settings.ini 형식은 일반적으로 Windows 환경에서 사용됩니다. Decouple은 모든 플랫폼에서 두 형식을 모두 읽을 수 있습니다.

EMAIL_HOST_USER는 SMTP 서버의 사용자 이름으로 설정되고, EMAIL_HOST_PASSWORD는 이메일 비밀번호 idx_307a3575 또는 앱 비밀번호 (Gmail SMTP 및 기타 보안 중심 이메일 제공자의 경우)로 설정됩니다.

.env

```
EMAIL_HOST_USER=bookr2026
EMAIL_HOST_PASSWORD=vnpw zgkl ixpq pbcz
```

The settings file is very similar to the .env format, but requires a tab.

settings.ini

```
[settings]
EMAIL_HOST_USER=bookr2026
EMAIL_HOST_PASSWORD=vnpw zgkl ixpq pbcz
```

Other SMTP servers will have different configuration settings.

Once these settings are added, the Django apps are able to use the email server to send emails to users.

Django provides functions such as `send_mail` and `send_mass_mail`, which can be used to send emails from within a Django app. You can read more about this functionality in the Django documentation at <https://docs.djangoproject.com/en/6.0/topics/email/> and learn by doing in the upcoming *Exercise 10.05*.

Automating admin with Django Tasks

Many jobs that we need to repeat in a web application do not suit the limitations of the HTTP request and response cycle. We do not want to limit a web server's capacity by keeping HTTP connections open while lengthy database updates or batches of network activity are occurring. There are many automated tasks involved in supporting a web application that are best carried out without the overhead of web interactions.

Until recently, these would typically have been implemented as cron jobs on Linux or Mac, or as scheduled tasks on Windows or using a standalone scheduling and task processor such as Celery.

Django 6.0 introduces the Django Tasks framework. Its functionality is somewhat limited at present, but it offers us a way to maintain scheduling tasks within a familiar Django paradigm.

The Django Tasks framework provides a mechanism to define units of work and their scheduling parameters, along with a mechanism to enqueue them. The API defines an interface to backends that actually carry the work.

.env

```
EMAIL_HOST_USER=
bookr2026EMAIL_HOST_PASSWORD=vnpw zgkl ixpq
pbcz
```

설정 파일은 .env 형식과 매우 유사하지만 탭이 필요합니다.

settings.ini

```
[settings]
EMAIL_HOST_USER=
bookr2026EMAIL_HOST_PASSWORD=vnpw zgkl ixpq
pbcz
```

다른 SMTP 서버는 다른 구성 설정을 가질 것입니다.

이 설정이 추가되면 Django 앱은 이메일 서버를 사용하여 사용자에게 이메일을 보낼 수 있습니다.

장고는 send_mail 및 send_mass_mail과 같은 함수를 제공하여 장고 앱 내에서 이메일을 보낼 수 있습니다. 이 기능에 대한 자세한 내용은

<https://docs.djangoproject.com/en/6.0/topics/email/>의 장고 문서에서 확인할 수 있으며, 다가오는 연습 문제 10.05에서 실습할 수 있습니다.

장고 작업으로 관리 자동화하기

웹 애플리케이션에서 반복해야 하는 많은 작업은 HTTP 요청 및 응답 주기의 제한에 적합하지 않습니다. 긴 데이터베이스 업데이트나 네트워크 활동 배치가 발생하는 동안 HTTP 연결을 열어 두어 웹 서버의 용량을 제한하고 싶지 않습니다. 웹 상호작용의 오버헤드 없이 수행하는 것이 가장 좋은 웹 애플리케이션 지원에 관련된 많은 자동화 작업이 있습니다.

최근까지 이러한 작업은 일반적으로 Linux 또는 Mac에서 cron 작업으로 구현되었거나 Windows에서 예약된 작업으로 구현되었거나 Celery와 같은 독립형 스케줄링 및 작업 프로세서를 사용하여 구현되었습니다.

장고 6.0은 장고 작업 프레임워크를 도입합니다. 현재 기능은 다소 제한적이지만, 친숙한 장고 패러다임 내에서 작업 예약을 유지할 수 있는 방법을 제공합니다.

장고 작업 프레임워크는 작업 단위와 그 스케줄링 매개변수를 정의하는 메커니즘과 이를 큐에 추가하는 메커니즘을 제공합니다. API는 실제로 작업을 수행하는 백엔드에 대한 인터페이스를 정의합니다.

A task can be defined in a Django application using the task decorator. This defines the properties of the task, including its priority, the queue that it will be scheduled in, and the backend that will be delegated to run the task.

If we had tasks to schedule that fell into groups such as AI image generation, database transformations, email batching, and data backup, it would make sense to configure a separate task queue for each of these task groups as a means of allocating resources. In the example and exercise that follow, we will only use the default queue and a trivial backend, but more complicated setups would be appropriate with a significant production deployment.

The Django Tasks application comes with two trivial backends: `ImmediateBackend` and `DummyBackend`. `ImmediateBackend` operates by running tasks as soon as they are enqueued, while `DummyBackend` returns a task result when enqueued without any work being passed to a worker. `DummyBackend` is useful for including in tests of a Django application.

The backend is specified in the `<project>/settings.py` file. A simple configuration with `ImmediateBackend` will look like this:

```
TASKS = {
    "default": {
        "BACKEND": "django.tasks.backends.immediate."
                    "ImmediateBackend"
    }
}
```

`DummyBackend` can be configured by specifying `"django.tasks.backends.dummy.DummyBackend"`.

Tasks are typically defined inside the app folder, in a file called `tasks.py`.

A task is defined by using the `@task` decorator. The decorator doesn't require any arguments, but has the optional arguments of:

- **priority:** An integer value that can be used by backends to determine the scheduling priority of the task, with a default of 0.
- **queue_name:** A string corresponding to a queue defined in the Tasks specification. The default value is "default".

작업은 task 데코레이터를 사용하여 `idx_41aa57d7Django` 애플리케이션에서 정의할 수 있습니다. 이 `idx_a896ebcf`는 작업의 속성을 정의하며, 여기에는 우선 순위, 작업이 예약될 큐 및 작업을 실행하도록 위임될 백엔드가 포함됩니다.

AI 이미지 생성, 데이터베이스 변환, 이메일 배치 및 데이터 백업과 같은 그룹에 속하는 작업을 예약해야 한다면, 이러한 작업 그룹 각각에 대해 별도의 작업 큐를 구성하는 것이 자원을 할당하는 방법으로 의미가 있을 것입니다. 다음의 예제와 연습에서는 기본 큐와 사소한 백엔드만 사용할 것이지만, 더 복잡한 설정은 중요한 프로덕션 배포에 적합할 것입니다.

Django Tasks 애플리케이션에는 두 개의 사소한 백엔드가 포함되어 있습니다: `ImmediateBackend`와 `DummyBackend`. `ImmediateBackend`는 작업이 큐에 추가되자마자 실행되도록 작동하며, `DummyBackend`는 작업이 큐에 추가될 때 작업 결과를 반환하지만 작업자가 수행할 작업은 전달되지 않습니다. `DummyBackend`는 Django 애플리케이션의 테스트에 포함하는 데 유용합니다.

백엔드는 `<project>/settings.py` 파일에 지정됩니다. `ImmediateBackend`로 간단한 구성은 다음과 같이 보일 것입니다:

```
TASKS = {
    "default": {
        "BACKEND": "django.tasks.backends.immediate."
        "ImmediateBackend"
    }
}
```

`DummyBackend`은 `"django.tasks.backends.dummy.DummyBackend"`를 지정하여 구성할 수 있습니다.

작업은 일반적으로 앱 폴더 내의 `tasks.py`라는 파일에 정의됩니다.

작업은 `@task` 데코레이터를 사용하여 정의됩니다. 데코레이터는 인수를 요구하지 않지만, 선택적 인수는 다음과 같습니다:

- `priority`: 작업의 스케줄링 우선 순위를 결정하는 데 백엔드에서 사용할 수 있는 정수 값으로, 기본값은 0입니다.
- `queue_name`: 작업 사양에 정의된 큐에 해당하는 문자열입니다. 기본값은 `"default"`입니다.

- **backend**: A string corresponding to a backend defined in the Tasks specification. The default value is "default".
- **takes_context**: A Boolean value specifying whether the first argument to the decorated task function is a TaskContext object. TaskContext has two attributes defined, the attempt number and task_result. The default is False.

Here is an example of a task that is used for generating an image and writing it to the file system.

myapp/tasks.py

```
from django.tasks import task
from .img_utils import _generate_cake_jpg

@task(priority=3, queue_name="img_generator")
def generate_birthday_greeting_jpg(user, background_img,
    favourite_cake):
    img = _generate_cake_jpg(user, background_img,
        favourite_cake)
    filename = f"media/{user.username}/birthday.jpg"
    img.save(filename)
```

The task will be scheduled to run by calling the enqueue method of the task. The enqueue method receives the arguments of the decorated task method.

```
>>> from myapp.tasks import generate_birthday_greeting_jpg
>>> generate_birthday_greeting_jpg.enqueue(user,
... "media/background/hundreds_and_thousands.jpg",
... "tiramisu")
```

Once the task runs, it will produce a TaskResult that will look something like that displayed in *Figure 10.12*.

- backend: 작업 사양에 정의된 백엔드에 해당하는 문자열. 기본값은 "default"입니다.
- takes_context: 장식된 작업 함수의 첫 번째 인수가 TaskContext 객체인지 여부를 지정하는 부울 값입니다. TaskContext에는 시도 번호와 task_result라는 두 개의 속성이 정의되어 있습니다. 기본값은 False입니다.

여기 이미지를 생성하고 파일 시스템에 idx_b6f6a5cd로 작성하는 데 사용되는 taskidx_823a6eba의 예가 있습니다.

myapp/tasks.py

```
from django.tasks import task
from .img_utils import _generate_cake_jpg

@task(priority=3, queue_name="img_generator")
def generate_birthday_greeting_jpg(user, background_img,
    favourite_cake):
    img = _generate_cake_jpg(user, background_img,
        favourite_cake)
    filename = f"media/{user.username}/birthday.jpg"
    img.save(filename)
```

작업은 작업의 enqueue 메서드를 호출하여 실행되도록 예약됩니다. enqueue 메서드는 장식된 작업 메서드의 인수를 받습니다.

```
>>> from myapp.tasks import
generate_birthday_greeting_jpg>>>
generate_birthday_greeting_jpg.enqueue(user,... "
media/background/hundreds_and_thousands.jpg",... "
tiramisu")
```

작업이 실행되면 Figure 10.12에 표시된 것과 유사한 TaskResult가 생성됩니다.

```
TaskResult(task=Task(priority=3, func=<function generate_birthday_greeting_jpg at 0x7d6cc5df3b00>, backend='default', queue_name='img_generator', run_after=None, takes_context=False), id='CoAhMGuktRkrYTciUqy5FoBWu6KAhp8R', status=TaskResultStatus.SUCCESSFUL, enqueued_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154716, tzinfo=datetime.timezone.utc), started_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154860, tzinfo=datetime.timezone.utc), finished_at=datetime.datetime(2026, 1, 29, 13, 25, 6, 288324, tzinfo=datetime.timezone.utc), last_attempted_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154860, tzinfo=datetime.timezone.utc), args=[<User: bob>, "media/background/hundreds_and_thousands.jpg", "tiramisu"], kwargs={}, backend='default', errors=[], worker_ids=['4mUrIPeXMGeZgU4l6AHUrgd5nbQEmK2m'], _return_value=1)
```

Figure 10.12: The TaskResult output

With a brief explanation of the Django Tasks framework, we will develop a functional task in the concluding exercise of this chapter.

Activity 10.03 – building a custom admin dashboard with a built-in search

In this activity, you will use the knowledge gained about the different aspects of creating a custom admin site to build a custom admin dashboard for Bookr. Inside this dashboard, you will introduce the capability of allowing a user to search for books by using either the name of the book or the name of the book publisher, and allowing the user to modify or delete these book records.

The following steps will help you build a custom admin dashboard and add the ability to search a book record by using the name of the publisher:

1. Create a new application inside the Bookr project named `bookr_admin`, if not created already. This is going to store the logic for our custom admin site.
2. Inside the `admin.py` file under the `bookr_admin` directory, create a new class, `BookrAdmin`, which inherits from the `AdminSite` class of Django's admin module.
3. Inside the newly created `BookrAdmin` class from *step 2*, add any customizations for the site title or any other branding component of the admin dashboard.
4. Inside the `apps.py` file under the `bookr_admin` directory, create a new `BookrAdminConfig` class, and inside this new `BookrAdminConfig` class, set the default site attribute to the fully qualified module name for our custom admin site class, `BookrAdmin`.
5. Inside the `settings.py` file of your Django project, add the fully qualified path of the `BookrAdminConfig` class created in *step 4* as the first installed application.

```
TaskResult(task=Task(priority=3, func=<function generate_birthday_greeting_jpg at 0x7d6cc5df3b00>, backend='default', queue_name='img_generator', run_after=None, takes_context=False), id='CoAhMGuktRkrYTciUqy5FoBWu6KAhp8R', status=TaskResultStatus.SUCCESSFUL, enqueued_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154716, tzinfo=datetime.timezone.utc), started_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154860, tzinfo=datetime.timezone.utc), finished_at=datetime.datetime(2026, 1, 29, 13, 25, 6, 288324, tzinfo=datetime.timezone.utc), last_attempted_at=datetime.datetime(2026, 1, 29, 13, 25, 1, 154860, tzinfo=datetime.timezone.utc), args=[<User: bob>, "media/background/hundreds_and_thousands.jpg", "tiramisu"], kwargs={}, backend='default', errors=[], worker_ids=['4mUrIPeXMGZgU4l6AHUrgd5nbQEmK2m'], _return_value=1)
```

그림 10.12: TaskResult 출력

Django Tasks 프레임워크에 대한 간단한 설명과 함께, 이 장의 마지막 연습에서 기능적인 작업을 개발할 것입니다.

활동 10.03 – 내장 검색 기능이 있는 사용자 정의 관리자 대시보드 구축

이 활동에서는 사용자 정의 관리자 사이트를 만드는 다양한 측면에 대해 얻은 지식을 사용하여 Bookr를 위한 사용자 정의 관리자 대시보드를 구축합니다. 이 대시보드 내에서 사용자가 책의 이름이나 출판사의 이름을 사용하여 책을 검색할 수 있는 기능을 도입하고, 사용자가 이러한 책 기록을 수정하거나 삭제할 수 있도록 합니다.

다음 단계는 사용자 정의 관리자 대시보드를 구축하고 출판사의 이름을 사용하여 책 기록을 검색할 수 있는 기능을 추가하는 데 도움이 될 것입니다:

1. Bookr 프로젝트 내에 bookr_admin이라는 새 애플리케이션을 생성합니다. 아직 생성되지 않았다면 말이죠. 이것은 우리의 맞춤형 관리자 사이트의 로직을 저장할 것입니다.
2. bookr_admin 디렉토리의 admin.py 파일 안에 Django의 admin 모듈의 AdminSite 클래스를 상속받는 새로운 클래스 BookrAdmin을 생성합니다. 2단계에서 새로 생성된 BookrAdmin 클래스 안에 사이트 제목이나 관리자 대시보드
3. 의 다른 브랜딩 구성 요소에 대한 사용자 정의를 추가합니다.
4. bookr_admin 디렉토리의 apps.py 파일 안에 새로운 BookrAdminConfig 클래스를 생성하고, 이 새로운 BookrAdminConfig 클래스 안에 기본 사이트 속성을 우리의 맞춤형 관리자 사이트 클래스인 BookrAdmin의 완전한 모듈 이름으로 설정합니다.
5. Django 프로젝트의 settings.py 파일 안에 4단계에서 생성된 BookrAdminConfig 클래스의 완전한 경로를 첫 번째 설치된 애플리케이션으로 추가합니다.

6. To register the Books model from the reviews application inside Bookr, open the `admin.py` file inside the reviews directory and make sure that the Books model is registered to the admin site by using `admin.site.register(ModelClass)`.
7. To allow a search of a book according to the name of the publisher, inside the `admin.py` file of the reviews application, modify the `BookAdmin` class and add to it a property named `search_fields`, which contains `publisher_name` as a field.
8. To get the publisher's name correctly for the `search_fields` property, introduce a new method named `get_publisher` inside the `BookAdmin` class, which will return the name field of the publisher from the Book model.
9. Make sure that the `BookAdmin` class is registered as a `Model` admin class for the Book model inside our Django admin dashboard by using `admin.site.register(Book, BookModel)`.

After completing this activity, once you start the application server and visit `http://localhost:8000/admin` and navigate to the Book model, you should be able to search for books by using the publisher's name and, in the event of a successful search, see a page that resembles the one shown in the following screenshot:

The screenshot displays the 'Bookr Administration Portal' interface. The top navigation bar includes the title 'Bookr Administration Portal' and user information 'WELCOME, TEST. VIEW SITE / CHANGE PASSWORD / LOG OUT'. Below this, a breadcrumb trail shows 'Home > Reviews > Books > Web Development with Django (1234-1234-1234)'. The left sidebar contains two main sections: 'AUTHENTICATION AND AUTHORIZATION' with links for 'Groups' and 'Users' (each with a '+ Add' button), and 'REVIEWS' with a link for 'Books' (with a '+ Add' button). The main content area is titled 'Change book' and features a 'HISTORY' button. The form contains several fields: 'Title' (text input with value 'Web Development with Django' and a note 'The title of the book.'), 'Date the book was published.' (date input with value '2021-01-29' and a note 'Note: You are 5.5 hours ahead of server time.'), 'ISBN number of the book.' (text input with value '1234-1234-1234'), 'Publisher.' (dropdown menu with value 'Packt Publishing'), 'Cover.' (file upload button 'Choose File' and text 'No file chosen'), and 'Sample.' (file upload button 'Choose File' and text 'No file chosen'). At the bottom of the form are four buttons: 'Delete' (red), 'Save and add another' (blue), 'Save and continue editing' (blue), and 'SAVE' (blue).

Figure 10.11: The book editing page inside the Bookr Administration dashboard

Note

The solution for this activity can be found in the Chapter09 folder in this book's GitHub repo.

6. Bookr의 리뷰 애플리케이션 내에서 Books 모델을 등록하려면 reviews 디렉토리 내의 admin.py 파일을 열고 admin.site.register(ModelClass)를 사용하여 Books 모델이 관리 사이트에 등록되어 있는지 확인하십시오.
7. 출판사의 이름에 따라 idx_c86d0dedbook을 검색할 수 있도록 리뷰 애플리케이션의 admin.py 파일 내에서 BookAdmin 클래스를 수정하고 publisher_name을 필드로 포함하는 search_fields라는 속성을 추가하십시오.
8. search_fields 속성을 위해 출판사의 이름을 올바르게 가져오기 위해 BookAdmin 클래스 내에 get_publisher라는 새 메서드를 도입하여 Book 모델의 출판사 이름 필드를 반환하도록 하십시오.
9. BookAdmin 클래스가 admin.site.register(Book, BookModel)을 사용하여 Django 관리 대시보드 내에서 Book 모델의 모델 관리 클래스로 등록되어 있는지 확인하십시오.

이 활동을 완료한 후 애플리케이션 서버를 시작하고 <http://localhost:8000/admin>을 방문하여 Book 모델로 이동하면 idx_c0760a36을 사용하여 책을 검색할 수 있어야 하며, 검색이 성공하면 다음 스크린샷에 표시된 것과 유사한 페이지를 볼 수 있습니다:

The screenshot shows the 'Bookr Administration Portal' interface. The top navigation bar includes 'Home', 'Reviews', 'Books', and 'Web Development with Django (1234-1234-1234)'. The left sidebar has sections for 'AUTHENTICATION AND AUTHORIZATION' (Groups, Users) and 'REVIEWS' (Books). The main content area is titled 'Change book' and contains a form with the following fields:

- Title:** Web Development with Django (with a subtext 'The title of the book.')
- Date the book was published:** 2021-01-29 (with a 'Today' button and a note: 'Note: You are 5.5 hours ahead of server time.')
- ISBN number of the book:** 1234-1234-1234
- Publisher:** Packt Publishing (dropdown menu)
- Cover:** Choose File (No file chosen)
- Sample:** Choose File (No file chosen)

At the bottom of the form, there are four buttons: 'Delete' (red), 'Save and add another' (blue), 'Save and continue editing' (blue), and 'SAVE' (blue).

그림 10.11: Bookr 관리 대시보드 내의 책 편집 페이지

참고

이 활동에 대한 솔루션은 이 책의 GitHub 저장소의 Chapter09 폴더에서 찾을 수 있습니다.

Exercise 10.5

In this exercise, we want to use the Django Tasks framework to create a periodic task that sends an email to subscribers telling them about recent reviews on the Bookr site.

We will write this task with some flexibility so that it can be configured to run daily, weekly, monthly, or annually.

First, we will add a function called `get_daterange_dt` to `review/utils.py`. This function will take two parameters: a date string in the `YYYY-mm-dd` format and a string to describe periodicity – daily, weekly, monthly, or annual. The function will require the `monthrange` function from the `calendar` module and the `datetime` and `timedelta` types from the `datetime` module.

```
from calendar import monthrange
from datetime import datetime, timedelta
```

Then we can define the `get_daterange_dt` function, which takes a `start_day` string and a period designator that is limited to the values `'daily'`, `'weekly'`, `'monthly'`, and `'annual'`.

```
def get_daterange_dt(start_day, period):
    assert period in ['daily', 'weekly', 'monthly', 'annual']
    start_day_dt = datetime.strptime(start_day, '%Y-%m-%d')
```

Our logic for calculating the end day of the date range will differ depending on whether we are calculating a day, week, month, or year-long range. Note that there is a `try/except` block that handles issues for leap days.

```
if period == 'daily':
    end_day_dt = start_day_dt
elif period == 'weekly':
    end_day_dt = start_day_dt + timedelta(days=6)
elif period == 'monthly':
    last_day_of_month = monthrange(
        start_day_dt.year, start_day_dt.month)[1]
    end_day_dt = start_day_dt.replace(
        day=last_day_of_month)
elif period == 'annual':
    try:
        end_day_dt = (start_day_dt.replace(
            year=start_day_dt.year+1) -
```

연습 10.5

이 연습에서는 Django Tasks 프레임워크를 사용하여 Bookr 사이트의 최근 리뷰에 대해 구독자에게 이메일을 보내는 주기적인 작업을 생성하고자 합니다.

이 작업은 매일, 매주, 매월 또는 매년 실행되도록 구성할 수 있도록 약간의 유연성을 가지고 작성할 것입니다.

먼저 review/utils.py에 get_daterange_dt라는 함수를 추가하겠습니다. 이 함수는 YYYY-mm-dd 형식의 날짜 문자열과 주기성을 설명하는 문자열 - 매일, 매주, 매월 또는 매년 - 두 개의 매개변수를 필요로 합니다. 이 함수는 calendar 모듈의 monthrange 함수와 datetime 모듈의 datetime 및 timedelta 타입을 필요로 합니다.

```
from calendar import monthrange
from datetime import datetime, timedelta
```

그런 다음 시작일 문자열과 'daily', 'weekly', 'monthly', 'annual' 값으로 제한된 주기 지정자를 받는 get_daterange_dt 함수를 정의할 수 있습니다.

```
def get_daterange_dt(start_day, period):
    assert period in ['daily', 'weekly', 'monthly', 'annual']
    start_day_dt = datetime.strptime(start_day, '%Y-%m-%d')
```

날짜 범위의 idx_025ab79a 종료일을 계산하는 우리의 논리는 우리가 일, 주, 월 또는 연간 범위를 계산하는지에 따라 다릅니다. 윤년 문제를 처리하는 try/except 블록이 있다는 점에 유의하세요.

```
    if period == 'daily':
        end_day_dt = start_day_dt
    elif period == 'weekly':
        end_day_dt = start_day_dt + timedelta(days=6)
    elif period == 'monthly':
        last_day_of_month = monthrange(
            start_day_dt.year, start_day_dt.month)[1]
        end_day_dt = start_day_dt.replace(
            day=last_day_of_month)
    elif period == 'annual':
        try:
            end_day_dt = (start_day_dt.replace(
                year=start_day_dt.year+1) -
```



```

        timedelta(days=1))
except ValueError:
    # catch error caused by 29th
    end_day_dt = start_day_dt.replace(
        year=start_day_dt.year+1,
        month=2, day=28)

```

Having calculated the end day, we should calculate the final microsecond of it, as this will be used in the database query. We return the datetime types `start_date_dt` and `end_day_dt`.

Create a file in the reviews folder called `tasks.py`. We will need to import the `groupby` function from `itertools`, the Django `send_mass_mail` function, the task decorator, and the `render_to_string` function from `django.template.loader`. We also need to import the `Review` model and the `get_daterange_dt` function that we just created.

```

from itertools import groupby
from django.core.mail import send_mass_mail
from django.tasks import task
from django.template.loader import render_to_string

from .models import Review
from .utils import get_daterange_dt

```

Now we can define a function with a task decorator. We are specifying a priority (but this will only be significant if we use an external queuing system with contending tasks) while accepting the default queue and backend. The `review_report` function takes arguments for the following:

- `emails`: A list of email recipients.
- `from_email`: The email address
- `period`: String values – 'daily', 'weekly', 'monthly', 'annual'.
- `start_day`: The start date of the reporting period in the 'YYYY-dd-mm' format.
- `site`: The URL of the Bookr website. The default is 'http://127.0.0.1:8000'.

```

@task(priority=2)
def review_report(emails, from_email, period, start_day,
    site="http://127.0.0.1:8000"):

```

```

        timedelta(days=1))
ValueError 예외:
    # 29일로 인한 오류 잡기
    end_day_dt = start_day_dt.replace
        (year=start_day_dt.year+1,
         month=2, day=28)

```

종료일을 계산한 후, 데이터베이스 쿼리에서 사용될 최종 마이크로초를 계산해야 합니다. 우리는 datetime 유형의 start_date_dt와 end_day_dt를 반환합니다.

리뷰 폴더에 tasks.py라는 파일을 만듭니다. 우리는 itertools에서 groupby 함수를 가져오고, Django의 send_mass_mail 함수, task 데코레이터, django.template.loader에서 render_to_string 함수를 가져와야 합니다. 또한 방금 생성한 Review 모델과 get_daterange_dt 함수를 가져와야 합니다.

```

from itertools import groupby
from django.core.mail import send_mass_mail
from django.tasks import task
from django.template.loader import render_to_string

from .models import Review
from .utils import get_daterange_dt

```

이제 우리는 작업 데코레이터가 있는 함수를 정의할 수 있습니다. 우리는 우선 순위를 지정하고 있습니다(하지만 이는 경쟁 작업이 있는 외부 큐 시스템을 사용할 경우에만 중요합니다) 기본 큐와 백엔드를 수용하면서. review_report 함수는 다음에 대한 인수를 받습니다:

- 이메일: 이메일 수신자 목록
- 록.from_email: 이메일 주소
- 기간: 문자열 값 - '일일', '주간', '월간', '연간'. 시작일: 'YYYY-dd-mm' 형
- 식의 보고 기간 시작 날짜. 사이트: Bookr 웹사이트의 URL. 기본값은 'http://127.0.0.1:8000'입니다.

```

@task(priority=2)
def review_report(emails, from_email, period, start_day,
                  site="http://127.0.0.1:8000"):

```

We call the `get_daterange_dt` utility function to derive a datetime start and end from a string-specified date, such as "2026-01-01", and a time period string. Then we create human-readable strings for the start and end day, using the locale-specific time specifier.

```
start_day_dt, end_day_dt = get_daterange_dt(
    start_day, period)
start_day_fmt = start_day_dt.strftime('%x')
end_day_fmt = end_day_dt.strftime('%x')
```

Use the reviews app's model to construct a query of the reviews that have been published in the given timeframe and order them by book title.

```
reviews = Review.objects.filter(
    date_created__range=[start_day_dt, end_day_dt]
).order_by('book__title')
```

Now we can construct a context of appropriate data that we will send to the Django template in order to create the email message. As the data from the Django model is already sorted by book name, we can use the `itertools.groupby` function so that this data is partitioned using the title as a key:

```
[('Title 1', [<Review 2>, <Review 3>]),
 ('Title 2', [<Review 1>]),
 ('Title 3', [<Review 6>, <Review 7>])
 ('Title 5', [<Review 4>, <Review 5>])]
```

The context will look like this:

```
context = dict(period=period, site=site,
    start_day_fmt=start_day_fmt,
    end_day_fmt=end_day_fmt,
    reviews_by_title=[(title, list(reviews))
        for title, reviews in
            groupby(reviews, lambda x: x.book.title)])
```

Next, we can generate a subject line for the email message. We will vary the message depending on whether it relates to reviews from a single day or a longer period of time.

```
subject = f"Your {period} Bookr reviews "
if period=='daily':
```

우리는 `get_daterange_dt` 유틸리티 함수를 호출하여 "2026-01-01"과 같은 문자열로 지정된 날짜와 기간 문자열에서 시작 및 종료의 `datetime`을 도출합니다. 그런 다음 지역별 시간 지정자를 사용하여 시작일과 종료일에 대한 사람이 읽을 수 있는 문자열을 생성합니다.

```
start_day_dt, end_day_dt = get_daterange_dt(
    start_day, period)
start_day_fmt = start_day_dt.strftime('%x')
end_day_fmt = end_day_dt.strftime('%x')
```

리뷰 앱의 모델을 사용하여 주어진 기간 내에 게시된 리뷰의 쿼리를 구성하고 책 제목별로 정렬합니다.

```
reviews = Review.objects.filter(
    date_created__range=[start_day_dt, end_day_dt]
).order_by('book__title')
```

이제 이메일 메시지를 생성하기 위해 Django 템플릿에 보낼 적절한 데이터의 컨텍스트를 구성할 수 있습니다. Django 모델의 데이터가 이미 책 이름으로 정렬되어 있으므로, `itertools.groupby` 함수를 사용하여 이 데이터를 제목을 키로 하여 분할할 수 있습니다:

```
[('Title 1', [<Review 2>, <Review 3>]),
 ('Title 2', [<Review 1>]),
 ('Title 3', [<Review 6>, <Review 7>])
 ('Title 5', [<Review 4>, <Review 5>])]
```

컨텍스트는 다음과 같이 보일 것입니다:

```
context = dict(period=period, site=site,
    start_day_fmt=start_day_fmt,
    end_day_fmt=end_day_fmt,
    reviews_by_title=[(제목, list(리뷰))
        for 제목, 리뷰 in
            groupby(리뷰, lambda x: x.book.title)])
```

다음으로, 우리는 이메일 메시지에 대한 제목을 생성할 수 있습니다. 우리는 메시지가 단일 날짜의 리뷰와 관련이 있는지 또는 더 긴 기간과 관련이 있는지에 따라 다르게 할 것입니다.

```
subject = f"당신의 {period} Bookr 리뷰 "
if period=='daily':
```

```

        subject += "for {start_day_fmt}"
    else:
        subject += "from {start_day_fmt} to {end_day_fmt}"

```

Finally, we will create the email message by rendering `review_report_email.txt` to a string. (We will create this template in the next step.) Then we create a list of email messages with one message for each of the emails passed to the function. We then use Django's `send_mass_mail`. Note that if we had deployed the message using the `send_mail` function and a list of email recipients, we would have had a situation where a single email went out with multiple recipients. In most situations, this would be deemed to be a privacy issue as the users had not requested to have their email addresses shared with the other recipients.

```

message = render_to_string(
    'reviews/review_report_email.txt', context)
email_messages = [(subject, message, from_email,
    [email]) for email in emails]
return send_mass_mail(
    email_messages, fail_silently=False)

```

Now this task requires a Django template to render the results. It will follow this structure:

```

{% if reviews_by_title %} <-- header --> {% endif %}
{% for title, reviews in reviews_by_title %}
    <- book title ->
    {% for review in reviews %}
        <- reviewer detail and link to review ->
    {% endfor %}
{% empty %}
    <-- Empty report message -->
{% endfor %}

```

A template for the email message needs be added to the application's templates directory. For the sake of clarity, we are using a text-only message without HTML formatting.

reviews/templates/reviews/review_report_email.txt

```

{% if reviews_by_title %}Here is your {{period}} report of Bookr reviews {% if
period == 'daily' %}on {{start_day_fmt}}. {% else %}from {{start_day_fmt}} to
{{end_day_fmt}}. {% endif %}
{% endif %}
{% for title, reviews in reviews_by_title %}

```

```

        subject += "for {start_day_fmt}"
    else:
        subject += "from {start_day_fmt} to {end_day_fmt}"

```

마지막으로, `review_report_email.txt`를 문자열로 렌더링하여 이메일 메시지를 생성합니다.(다음 단계에서 이 템플릿을 생성할 것입니다.) 그런 다음 함수에 전달된 각 이메일에 대해 `onemessage`로 이메일 메시지 목록을 생성합니다. 그런 다음 Django의 `send_mass_mail`을 사용합니다. `send_mail` 함수와 이메일 수신자 목록을 사용하여 메시지를 배포했다면, 여러 수신자가 있는 단일 이메일이 발송되는 상황이 발생했을 것입니다. 대부분의 경우, 이는 사용자가 다른 수신자와 이메일 주소를 공유하도록 요청하지 않았기 때문에 개인 정보 문제로 간주될 수 있습니다.

```

message = render_to_string(
    'reviews/review_report_email.txt',
    context)
email_messages = [(subject, message, from_email,
    [email]) for email in emails]
return send_mass_mail(
    email_messages, fail_silently=False)

```

이 작업은 결과를 렌더링하기 위해 Django 템플릿이 필요합니다. 다음 구조를 따릅니다:

```

{% if reviews_by_title %} <-- 헤더 --> {% endif %}
{% for title, reviews in reviews_by_title %}
    <- 책 제목 ->{% for review
    in reviews %}

        <- 리뷰어 세부정보 및 리뷰 링크 ->

    {% endfor %}
{% empty %}
    <-- 빈 보고서 메시지 -->{%
    endfor %}

```

이메일 메시지에 대한 템플릿은 애플리케이션의 템플릿 디렉토리에 추가해야 합니다. 명확성을 위해 HTML 형식 없이 텍스트 전용 메시지를 사용하고 있습니다.

reviews/templates/reviews/review_report_email.txt

```

{% if reviews_by_title %}여기 당신의 {{period}} Bookr 리뷰 보고서가 있습니다 {%
ifperiod == 'daily' %}{{start_day_fmt}}에.{% else %}{{start_day_fmt}}부터
{{end_day_fmt}}까지.{% endif %}

{% endif %}
{% for title, reviews in reviews_by_title %}

```

```
{{ title }}
{% for review in reviews %}
* {{ review.creator.username }} - {{ review.rating }}/5 - {{ site }}/reviews/
{{ review.id }}
{% endfor %}
{% empty %}
No reviews for {% if period == 'daily' %}{{start_day_fmt}}. {% else %} the period
{{start_day_fmt}} to {{end_day_fmt}}. {% endif %}
{% endfor %}
```

Now that we have finished developing the task, we can enqueue the task from the command line. This approach will work with DummyBackend or ImmediateBackend. The emails and from_email parameters should be the list of recipient emails and the email address configured in the settings.py file, respectively. If you aren't getting any data in the current time period, be sure to add your own.

```
>>> from reviews.tasks import review_report
>>> review_report.enqueue(
... ['alice@example.com', 'bob@example.com'],
...   'bookr2026@example.com', 'annual', '2026-01-01',
...   'bookr.example.com')
```

Your email recipients will receive an email similar to the one in *Figure 10.13*.

```

{{ title }}
{% for review in reviews %}
* {{ review.creator.username }} - {{ review.rating }}/5 - {{ site }}/reviews/
{{ review.id }}
{% endfor %}
{% empty %}
{% if period == 'daily' %}{{start_day_fmt}}에 대한 리뷰가 없습니다.{% else %} 기간에 대한 리뷰가 없습니다
{{start_day_fmt}}부터 {{end_day_fmt}}까지.{% endif %}
{% endfor %}

```

이제 idx_7e0b5f00 작업 개발이 완료되었으므로, 명령줄에서 작업을 큐에 추가할 수 있습니다. 이 방법은 DummyBackend 또는 ImmediateBackend와 함께 작동합니다. emails 및 from_email 매개변수는 각각 수신자 이메일 목록과 settings.py 파일에 구성된 이메일 주소여야 합니다. 현재 시간대에 데이터가 없으면, 자신의 데이터를 추가해야 합니다.

```

>>> from reviews.tasks import review_report
>>> review_report.enqueue(
... ['alice@example.com', 'bob@example.com'],
... 'bookr2026@example.com', 'annual', '2026-01-01',
... 'bookr.example.com')

```

귀하의 이메일 수신자는 그림 10.13과 유사한 이메일을 받을 것입니다.

bookr2026@gmail.com

2:49 PM (0 minutes ago)

to me, chrisguest, chrisguest+1 ▼

Here is your annual report of Bookr reviews from 01/01/26 to 12/31/26.

Advanced Deep Learning with Keras 2

- * alice - 4/5 - bookr.example.com/reviews/4
- * david - 4/5 - bookr.example.com/reviews/5

Architects of Intelligence 1

- * david - 5/5 - bookr.example.com/reviews/7

Deep Reinforcement Learning Hands-On 1

- * david - 5/5 - bookr.example.com/reviews/8

Hands-On Machine Learning for Algorithmic Trading 1

- * david - 4/5 - bookr.example.com/reviews/6

Hands-On Reinforcement Learning with Python 1

- * david - 4/5 - bookr.example.com/reviews/10

Natural Language Processing with TensorFlow 1

- * david - 5/5 - bookr.example.com/reviews/9

Figure 10.13: The review report email

Although it might seem like there is a lot of code in this exercise, this is a simple example of a time period report. It is better to handle this as a Django Task rather than as a Django request, as it may take a significant amount of time to generate the report and email a number of recipients, which could lead to web server timeouts during the execution.

Summary

In this chapter, we learned about advanced customization of our Django admin app. We delved deep into features that help us enhance the look and feel of our admin app, making the user interface more intuitive and useful, before moving on to the finer points of pagination and search results filtering. In the next chapter, we will be looking at how we can use templating and class-based views for our Django app.

bookr2026@gmail.com

2:49 PM (0 minutes ago)

to me, chrisguest, chrisguest+1 ▼

Here is your annual report of Bookr reviews from 01/01/26 to 12/31/26.

Advanced Deep Learning with Keras 2

* alice - 4/5 - bookr.example.com/reviews/4

* david - 4/5 - bookr.example.com/reviews/5

Architects of Intelligence 1

* david - 5/5 - bookr.example.com/reviews/7

Deep Reinforcement Learning Hands-On 1

* david - 5/5 - bookr.example.com/reviews/8

Hands-On Machine Learning for Algorithmic Trading 1

* david - 4/5 - bookr.example.com/reviews/6

Hands-On Reinforcement Learning with Python 1

* david - 4/5 - bookr.example.com/reviews/10

Natural Language Processing with TensorFlow 1

* david - 5/5 - bookr.example.com/reviews/9

그림 10.13: 검토 보고서 이메일

이 연습에는 많은 코드가 있는 것처럼 보일 수 있지만, 이는 시간 기간 보고서의 간단한 예입니다. 보고서를 생성하고 여러 수신자에게 이메일을 보내는 데 상당한 시간이 걸릴 수 있으므로 Django 요청보다는 Django 작업으로 처리하는 것이 좋습니다. 이는 실행 중에 웹 서버 타임아웃을 초래할 수 있습니다.

요약

이 장에서는 Django 관리 앱의 고급 사용자 정의에 대해 배웠습니다. 우리는 관리 앱의 모양과 느낌을 향상시키는 데 도움이 되는 기능을 깊이 파고들었고, 사용자 인터페이스를 더 직관적이고 유용하게 만든 후, 페이지 매김 및 검색 결과 필터링의 세부 사항으로 넘어갔습니다. 다음 장에서는 Django 앱을 위해 템플릿과 클래스 기반 뷰를 사용하는 방법을 살펴보겠습니다.

We also learned how to connect a Django project with an email server and how to develop non-HTTP services with the Django Tasks framework.

As we move on to the next chapter, we will get to build upon what we have learned thus far and extend that knowledge by being introduced to the concept of building our own custom tags and filters for templates, as well as the ability to build our views in an object-oriented style using the concept of class-based views.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

우리는 또한 Django 프로젝트를 이메일 서버에 연결하는 방법과 Django Tasks 프레임워크를 사용하여 비HTTP 서비스를 개발하는 방법을 배웠습니다.

다음 장으로 넘어가면서 지금까지 배운 내용을 바탕으로 템플릿을 위한 사용자 정의 태그와 필터를 만드는 개념을 소개받고, 클래스 기반 뷰 개념을 사용하여 객체 지향 스타일로 뷰를 구축하는 능력을 확장할 것입니다.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하거나 packtpub.com/unlock으로 이동하세요. 이 책의 이름으로 검색하고,판을 확인한 후 페이지의 지침을 따르세요.



UNLOCK NOW

참고: 송장 사본을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

11

Advanced Templating and Class-Based Views

In *Chapter 3, URL Mapping, Views, and Templates*, we learned how to build views and create templates in Django. Then, we learned how to use those views to render the templates we built. In this chapter, we will build upon our knowledge of developing views by using **class-based views**, allowing us to write views that can group logical methods into a single entity. This skill comes in handy when developing a view that maps to multiple HTTP request methods for the same **application programming interface (API)** endpoint. With method-based views, we may end up using a lot of the `if-else` conditions to successfully handle the different types of HTTP request methods. In contrast, class-based views allow us to define separate methods for every HTTP request method we want to handle. Then, based on the type of request received, Django takes care of calling the correct method in the class-based view.

Beyond the ability to build views based on different development techniques, Django also comes packed with a powerful templating engine. This engine allows developers to build reusable templates for their web applications. This reusability of the templating engine is further enhanced by using **template tags**, **filters**, and **template partials**, which help easily implement commonly used features inside templates, features such as iterating over lists of data, formatting the data in a given style, extracting a piece of text from a variable to display, and overriding the content in a specific block of a template. All these features also expand the reusability of a Django template.

As we go through this chapter, we will look at how we can expand the default set of template filters and template tags provided by Django by leveraging Django's ability to define our own custom template tags, filters, and template partials. These custom template tags, filters, and partials can then be used to implement some common features in a reusable fashion across our web application. For example, while building a user profile badge that can be shown in several

11

고급 템플릿 및 클래스 기반 뷰

3장, URL 매핑, 뷰 및 템플릿에서는 Django에서 뷰를 구축하고 템플릿을 생성하는 방법을 배웠습니다. 그런 다음, 우리는 이러한 뷰를 사용하여 우리가 만든 템플릿을 렌더링하는 방법을 배웠습니다. 이 장에서는 클래스 기반 뷰를 사용하여 뷰 개발에 대한 우리의 지식을 확장할 것입니다. 이를 통해 `idx_cca69a6fus`가 논리적 메서드를 단일 엔터티로 그룹화할 수 있는 뷰를 작성할 수 있습니다. 이 기술은 동일한 애플리케이션 프로그래밍 인터페이스(API) 엔드포인트에 대해 여러 HTTP 요청 메서드에 매핑되는 뷰를 개발할 때 유용합니다. 메서드 기반 뷰를 사용하면 다양한 HTTP 요청 메서드를 성공적으로 처리하기 위해 많은 `if-else` 조건을 사용해야 할 수 있습니다. 반면, 클래스 기반 뷰는 우리가 처리하고자 하는 각 HTTP 요청 메서드에 대해 별도의 메서드를 정의할 수 있게 해줍니다. 그런 다음 수신된 요청의 유형에 따라 Django는 클래스 기반 뷰에서 올바른 메서드를 호출하는 일을 처리합니다.

다양한 개발 기술을 기반으로 뷰를 구축하는 능력을 넘어, Django는 강력한 템플릿 엔진을 내장하고 있습니다. 이 엔진은 개발자가 웹 애플리케이션을 위한 재사용 가능한 템플릿을 구축할 수 있게 해줍니다. 템플릿 엔진의 이러한 재사용성은 템플릿 태그, 필터 및 템플릿 부분을 사용하여 더욱 향상됩니다. 이러한 요소들은 `idx_c0c02ca0`가 템플릿 내에서 일반적으로 사용되는 기능을 쉽게 구현하는 데 도움을 줍니다. 예를 들어, 데이터 목록을 반복하고, 주어진 스타일로 데이터를 포맷하고, 변수를 통해 텍스트의 일부를 추출하여 표시하며, 템플릿의 특정 블록 내에서 콘텐츠를 오버라이드하는 기능 등이 있습니다. 이러한 모든 기능은 Django 템플릿의 재사용성을 확장합니다.

이 장을 진행하면서 우리는 Django가 제공하는 기본 템플릿 필터 및 템플릿 태그 세트를 확장하는 방법을 살펴볼 것입니다. 이를 위해 Django의 기능을 활용하여 사용자 정의 템플릿 태그, 필터 및 템플릿 부분을 정의할 수 있습니다. 이러한 사용자 정의 템플릿 태그, 필터 및 부분은 웹 애플리케이션 전반에 걸쳐 재사용 가능한 방식으로 일반적인 기능을 구현하는 데 사용될 수 있습니다. 예를 들어, 여러 곳에 표시할 수 있는 사용자 프로필 배지를 구축하는 동안

places inside a web application, it is better to leverage the ability to write a custom template inclusion tag that just inserts the template of the badge in any of the views we desire, rather than rewriting the entire code for the badge template or by introducing additional complexity to the templates.

In this chapter, we will be covering the following topics:

- Template filters
- Custom template filters
- String filters
- Template tags
- Template partials
- Django views
- Class-based views

Technical requirements

Find the solution in the Chapter11 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Template filters

While developing templates, developers often just want to change the value of a template variable before rendering it to the user. For example, consider that we are building a profile page for a Bookr user. There, we want to show the number of books the user has read. Below that, we also want to show a table listing the books they have read.

To achieve this, we can pass two separate variables from our view to the HTML template. One can be named `books_read`, which denotes the number of books read by the user. The other can be `book_list`, containing the list of names of the books read by the user, for example:

```
<span class="books_read">You have read {{ books_read }}  
books</span>  
<ul>  
    {% for book in book_list %}  
        <li>{{ book }} </li>  
    {% endfor %}  
</ul>
```

웹 애플리케이션 내에서 배지를 삽입하는 경우, 배지 템플릿의 전체 코드를 다시 작성하거나 템플릿에 추가 복잡성을 도입하는 것보다 원하는 뷰에 배지 템플릿을 삽입하는 사용자 정의 템플릿 포함 태그를 작성하는 기능을 활용하는 것이 좋습니다.

이 장에서는 다음 주제를 다룰 것입니다:

- 템플릿 필터사용자 정의
- 의 템플릿 필터문자열
- 필터템플릿 태그템플릿
- 파설Django 뷰
-
-
- 클래스 기반 뷰

기술 요구 사항

이 책의 GitHub 저장소에 있는 Chapter11 폴더에서 해결책을 찾으세요. 저장소 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

템플릿 필터

템플릿을 개발하는 동안, 개발자들은 종종 사용자에게 렌더링하기 전에 템플릿 변수의 값을 변경하고 싶어합니다. 예를 들어, Bookr 사용자의 프로필 페이지를 만든다고 가정해 봅시다. 그곳에서는 사용자가 읽은 책의 수를 보여주고 싶습니다. 그 아래에는 사용자가 읽은 책 목록을 나열한 표도 보여주고 싶습니다.

이를 달성하기 위해, 뷰에서 HTML 템플릿으로 두 개의 별도 변수를 전달할 수 있습니다. 하나는 사용자가 읽은 책의 수를 나타내는 `books_read`라고 할 수 있습니다. 다른 하나는 사용자가 읽은 책의 이름 목록을 포함하는 `book_list`일 수 있습니다. 예를 들어:

```
<span class="books_read">당신은 {{ books_read }}권을 읽었습니다</span>
    책</span>
<ul>
    {% for book in book_list %}
        <li>{{ book }} </li>
    {% endfor %}
</ul>
```

Template filters in Django are simple Python-based functions that accept a variable as an argument (and any additional data in the context of the variable), change its value as per our requirements, and then render the changed value.

Now, the same outcome from writing the previous snippet can also be obtained without the use of two separate variables by using template filters in Django, as follows:

```
<span class="books_read">You have read
  {{ book_list|length }}</span>
<ul>
  {% for book in book_list %}
    <li>{{ book }}</li>
  {% endfor %}
</ul>
```

Here, we used the built-in length filter provided by Django. The use of this filter causes the length of the `book_list` variable to be evaluated and returned, which is then inserted into our HTML template during rendering.

Like length, there are a lot of other template filters that come pre-packaged with Django and that are ready to be used. For example, the lowercase filter converts the text to all lowercase format, the last filter can be used to return the last item in the list, and the `json_script` filter can be used to output a Python object passed to the template as a JSON value wrapped in a `<script>` tag in your template.

Note

You can refer to Django's official documentation for the complete list of template filters offered by Django here: <https://docs.djangoproject.com/en/5.2/ref/templates/builtins/>.

Now, with the knowledge of how to use template filters, let's jump into understanding how we can write our own custom filters.

Django의 템플릿 필터는 변수를 인수로 받아들이는 간단한 Python 기반 함수입니다 (변수의 맥락에서 추가 데이터를 포함하여), 우리의 요구 사항에 따라 값을 변경하고 변경된 값을 렌더링합니다.

이제 이전 코드 조각을 작성하여 얻은 동일한 결과를 Django의 템플릿 필터를 사용하여 두 개의 별도 변수를 사용하지 않고도 얻을 수 있습니다. 다음과 같이:

```
<span class="books_read">당신은 읽었습니다
  {{ book_list|length }}</span>
<ul>
  {% for book in book_list %}
    <li>{{ book }}</li>
  {% endfor %}
</ul>
```

여기서는 Django에서 제공하는 내장 길이 필터를 사용했습니다. 이 필터를 사용하면 `book_list` 변수의 길이가 평가되어 반환되며, 이는 렌더링 중에 우리의 HTML 템플릿에 삽입됩니다.

길이와 마찬가지로, Django에는 미리 패키지로 제공되어 바로 사용할 수 있는 많은 다른 템플릿 필터들이 있습니다. 예를 들어, `lowercase` 필터는 텍스트를 모두 소문자 형식으로 변환하고, `last` 필터는 리스트의 마지막 항목을 반환하는 데 사용할 수 있으며, `json_script` 필터는 템플릿에 전달된 파이썬 객체를 JSON 값으로 출력하여 템플릿 내 `<script>` 태그로 감싸는 데 사용할 수 있습니다.

노트

Django의 공식 문서에서 템플릿 필터의 전체 목록을 참조할 수 있습니다.
Django에서 제공하는 내용은 여기에서 확인할 수 있습니다:
<https://docs.djangoproject.com/en/5.2/ref/templates/builtins/>.

이제 템플릿 필터를 사용하는 방법을 알았으니, 직접 커스텀 필터를 작성하는 방법을 이해해 봅시다.

Custom template filters

Django supplies a lot of useful filters that we can use in our templates while we are working on our projects. But what if someone wants to format a specific piece of text and render it with different fonts? Or, say, someone wants to translate an error code to a user-friendly error message based on the mapping of the error code in the backend. In these cases, predefined filters do not suffice, and we would like to write our own filter that we can reuse across the project.

Luckily, Django supplies an easy-to-use API that we can use to write custom filters. This API provides developers with some useful decorator functions that can be used to quickly register a Python function as a custom template filter. Once a Python function is registered as a custom filter, a developer can start using the function in templates.

An instance of this template library method is required to access the filters. This instance can be created by instantiating the `Library()` class in Django from Django's `template` module, as shown here:

```
from django import template
register = template.Library()
```

Once the instance is created, we can use the filter decorator from the template library instance to register our filters.

Creating custom template filters

There are a couple of steps we need to take to create custom template filters. Let's try to understand what these steps are and how they help us with the creation of a custom template filter in the next subsection.

Setting up the directory for storing template filters

It is important to note that when creating a custom template filter or template tag, we need to put them in the `templatetags` directory under the application directory. This requirement arises because Django is internally configured to look for custom template tags and filters when loading a web application. A failure to name the directory `templatetags` will result in Django not loading the custom template filters and tags created by us.

To create this directory, first, navigate to the application folder inside which you want to create custom template filters, and then run the following command in the terminal:

```
mkdir templatetags
```

사용자 정의 템플릿 필터

장고는 프로젝트 작업 중 템플릿에서 사용할 수 있는 유용한 필터를 많이 제공합니다. 하지만 누군가 특정 텍스트 조각을 형식화하고 다른 글꼴로 렌더링하고 싶다면 어떻게 될까요? 또는, 누군가 오류 코드를 기반으로 사용자 친화적인 오류 메시지로 변환하고 싶다면 어떻게 될까요? 이러한 경우에는 미리 정의된 필터로는 부족하며, 프로젝트 전반에 걸쳐 재사용할 수 있는 사용자 정의 필터를 작성하고 싶습니다.

다행히도 장고는 사용자 정의 필터를 작성하는 데 사용할 수 있는 사용하기 쉬운 API를 제공합니다. 이 API는 개발자가 Python 함수를 사용자 정의 템플릿 필터로 빠르게 등록할 수 있도록 하는 유용한 데코레이터 함수를 제공합니다. Python 함수가 사용자 정의 필터로 등록되면, 개발자는 템플릿에서 해당 함수를 사용할 수 있습니다.

필터에 접근하기 위해 이 템플릿 라이브러리 메서드의 인스턴스가 필요합니다. 이 인스턴스는 장고의 템플릿 모듈에서 `Library()` 클래스를 인스턴스화하여 생성할 수 있습니다. 다음과 같이 보여줍니다:

```
from django import template
register = template.Library()
```

인스턴스가 생성되면, 템플릿 라이브러리 인스턴스의 필터 데코레이터를 사용하여 필터를 등록할 수 있습니다.

사용자 정의 템플릿 필터 만들기

커스텀 템플릿 필터를 만들기 위해 우리가 취해야 할 몇 가지 단계가 있습니다. 이러한 단계가 무엇인지, 그리고 다음 하위 섹션에서 커스텀 템플릿 필터의 생성에 어떻게 도움이 되는지 이해해 봅시다.

템플릿 필터를 저장할 디렉토리 설정하기

커스텀 템플릿 필터 또는 템플릿 태그를 만들 때는 반드시 애플리케이션 디렉토리 아래의 `templatetags` 디렉토리에 넣어야 한다는 점에 유의해야 합니다. 이 요구 사항은 Django가 웹 애플리케이션을 로드할 때 커스텀 템플릿 태그와 필터를 찾도록 내부적으로 구성되어 있기 때문입니다. 디렉토리 이름을 `templatetags`로 지정하지 않으면 Django가 우리가 만든 커스텀 템플릿 필터와 태그를 로드하지 않습니다.

이 디렉토리를 만들기 위해 먼저 커스텀 템플릿 필터를 만들고자 하는 애플리케이션 폴더로 이동한 다음, 터미널에서 다음 명령어를 실행합니다:

```
mkdir templatetags
```

Once the directory is created, the next step is to create a new file inside the `templatetags` directory to store the code for our custom filters. This can be done by executing the following command inside the `templatetags` directory:

```
touch custom_filter.py
```

Note

The aforementioned command won't work on Windows. You can, however, navigate to the desired directory and create a new file using Windows Explorer.

Alternatively, this can be done by using the GUI interface provided by PyCharm.

Setting up the template library

Once the file for storing the code for the custom filter is created, we can start working on implementing our custom filter code. For custom filters to work in Django, they need to be registered to Django's template library before they can be used inside templates. To that end, the first step is to set up an instance of the template library, which will be used to register our custom filters. For this, inside the `custom_filters.py` file, we created in the previous section, we first need to import the `template` module from the Django project:

```
from django import template
```

Once the import is resolved, the next step is to create an instance of the template library by adding the following line of code:

```
register = template.Library()
```

The `Library` class from Django's `template` module is implemented as a **singleton** class that returns the same object that is only initialized once at the start of the application.

Once the template library instance is set up, we can proceed with implementing our custom filter.

Implementing the custom filter function

Custom filters inside Django are nothing more than simple Python functions that essentially take the following parameters:

- The value on which the filter is being applied (mandatory)
- Any additional parameters (zero or more) that need to be passed to the filter (optional)

디렉토리가 생성되면 다음 단계는 `templatetags` 디렉토리 내에 새 파일을 생성하여 사용자 정의 필터 코드를 저장하는 것입니다. 이는 `templatetags` 디렉토리 내에서 다음 명령을 실행하여 수행할 수 있습니다:

```
touch custom_filter.py
```

참고

앞서 언급한 명령은 Windows에서 작동하지 않습니다. 그러나 원하는 디렉토리로 이동하여 Windows 탐색기를 사용하여 새 파일을 생성할 수 있습니다.

또는 PyCharm에서 제공하는 GUI 인터페이스를 사용하여 수행할 수 있습니다.

템플릿 라이브러리 설정

사용자 정의 필터 코드를 저장할 파일이 생성되면 사용자 정의 필터 코드를 구현하는 작업을 시작할 수 있습니다. Django에서 사용자 정의 필터가 작동하려면 템플릿 내에서 사용되기 전에 Django의 템플릿 라이브러리에 등록되어야 합니다. 이를 위해 첫 번째 단계는 사용자 정의 필터를 등록하는 데 사용할 템플릿 라이브러리의 인스턴스를 설정하는 것입니다. 이를 위해 이전 섹션에서 생성한 `custom_filters.py` 파일 내에서 먼저 Django 프로젝트의 템플릿 모듈을 가져와야 합니다:

```
from django import template
```

가져오기가 해결되면 다음 단계는 다음 코드 줄을 추가하여 템플릿 라이브러리의 인스턴스를 만드는 것입니다:

```
register = template.Library()
```

Django의 템플릿 모듈의 `Library` 클래스는 애플리케이션 시작 시 한 번만 초기화된 동일한 객체 `idx_154b0141`을 반환하는 싱글톤 클래스로 구현됩니다. `idx_446a1523` 템플릿 라이브러리 인스턴스가 설정되면 사용자 정의 필터를 구현할 수 있습니다.

사용자 정의 필터 함수 구현

Djangoidx_bef02201 내의 사용자 정의 필터는 본질적으로 다음 매개변수를 사용하는 간단한 Python 함수에 불과합니다:

- 필터가 적용되는 값(필수)
- 필터에 전달해야 하는 추가 매개변수(0개 이상, 선택 사항)

These functions need to be decorated with the `filter` attribute from Django's template library instance to behave as template filters. For example, the generic implementation of a custom filter will look like the following:

```
@register.filter
def my_filter(value, arg):
    # Implementation logic of the filter
```

With this, we have learned how to implement a custom filter that can be used inside templates. In the next section, we will learn how to use these custom filters.

Using custom filters inside templates

Once the filter is created, it's simple to start using it inside our templates. To do that, the filter must first be imported into the template. This can be easily done by adding the following line to the top of the template file:

```
{% load custom_filter %}
```

When Django's templating engine parses the template files, the preceding line is automatically resolved by Django to find the correct module specified under the `templatetags` directory. As a consequence, all the filters mentioned inside the `custom_filter` module are automatically made available inside the template.

Using our custom filter inside the template is as simple as adding the following line:

```
{{ some_value|generic_filter:"arg" }}
```

It is a simple process to create a custom filter inside Django and then use it in our templates. In the next section, let's take a look at another type of filter, namely, string filters, which work solely on string-type values.

String filters

In the general case, when building a custom filter, taking any type of variable argument and returning any type. For example, the `length` filter takes an iterable and returns an integer value. But what if we wanted to restrict our filter to work only with strings and not with any other type of values, such as integers?

We can use the `stringfilter` decorator provided by Django's template library to develop filters that work only on *strings*. When the `stringfilter` decorator is used to register a Python method as a filter in Django, the framework ensures that the value being passed to the filter is

이 함수들은 Django의 템플릿 라이브러리 인스턴스에서 필터 속성으로 장식되어야 템플릿 필터로 작동합니다. 예를 들어, 사용자 정의 필터의 일반적인 구현은 다음과 같이 보일 것입니다:

```
@register.filter
def my_filter(value, arg):
    # 필터의 구현 로직
```

이로써 우리는 템플릿 내에서 사용할 수 있는 사용자 정의 필터를 구현하는 방법을 배웠습니다. 다음 섹션에서는 이러한 사용자 정의 필터를 사용하는 방법을 배울 것입니다.

템플릿 내에서 사용자 정의 필터 사용하기

필터가 생성되면, 템플릿 내에서 사용하기가 간단합니다. 그렇게 하려면, 필터는 먼저 템플릿으로 가져와야 합니다. 이는 템플릿 파일의 맨 위에 다음 줄을 추가하여 쉽게 할 수 있습니다:

```
{% load custom_filter %}
```

장고의 템플릿 엔진이 템플릿 파일을 파싱할 때, 이전 줄은 자동으로 장고에 의해 해결되어 템플릿 태그 디렉토리 아래에 지정된 올바른 모듈을 찾습니다. 결과적으로, 사용자 정의 필터 모듈 내에 언급된 모든 필터는 템플릿 내에서 자동으로 사용 가능해집니다.

템플릿 내에서 사용자 정의 필터를 사용하는 것은 다음 줄을 추가하는 것만큼 간단합니다:

```
{{ some_value|generic_filter:"arg" }}
```

Django 내에서 사용자 정의 필터를 생성하고 이를 템플릿에서 사용하는 것은 간단한 과정입니다. 다음 섹션에서는 문자열 필터라는 또 다른 유형의 필터를 살펴보겠습니다. 이 필터는 오로지 문자열 유형 값에서만 작동합니다.

문자열 필터

일반적인 경우, 사용자 정의 필터를 만들 때 어떤 유형의 변수 인수를 받아들이고 어떤 유형이든 반환합니다. 예를 들어, 길이 필터는 반복 가능한 객체를 받아 정수 값을 반환합니다. 하지만 필터가 문자열과 같은 다른 유형의 값이 아닌 문자열에서만 작동하도록 제한하고 싶다면 어떻게 해야 할까요?

장고의 템플릿 라이브러리에서 제공하는 문자열 필터 데코레이터를 사용하여 문자열에서만 작동하는 필터를 개발할 수 있습니다. 문자열 필터 데코레이터를 사용하여 Python 메서드를 Django의 필터로 등록할 때, 프레임워크는 필터에 전달되는 값이

converted to a string before the filter executes. This reduces any potential issues that may arise when non-string values are passed on to our filter.

The steps to implement a **string filter** are similar to the ones we followed for building a custom filter with some minor changes.

Remember the `custom_filter.py` file we created in the *Setting up the directory for storing template filters* section? Let's now add a new Python function inside it that will act as our string filter.

Before we can implement a string filter, we first need to import the `stringfilter` decorator, which demarcates a custom filter function as a string filter. You can add this decorator by adding the following `import` statement inside the `custom_filters.py` file:

```
from django.template.defaultfilters import stringfilter
```

Now, to implement our custom string filter, the following syntax can be used:

```
@register.filter
@stringfilter
def generic_string_filter(value, arg):
    # Logic for string filter implementation
```

With this approach, we can build as many string filters as we want and use them just like any other filter.

Equipped with this knowledge, let's now create our first custom string filter.

Exercise 11.01 – Creating a custom template filter

In this exercise, we will write a custom filter named `link_worldcat`, which returns a URL to the WorldCat site when provided with an ISBN or title string along with an explicit argument to say if the string is either an ISBN or title.

Here's an example:

```
value = "9781727518603"
```

You will apply the following filter to this string:

```
{{ value|link_worldcat:"isbn" }}
```

필터가 실행되기 전에 문자열로 변환됩니다. 이는 비문자 값이 필터에 전달될 때 발생할 수 있는 잠재적인 문제를 줄입니다.

문자열 필터를 구현하는 단계는 약간의 변경 사항을 제외하고는 사용자 정의 필터를 구축할 때 따랐던 단계와 유사합니다.

템플릿 필터 저장을 위한 디렉토리 설정 섹션에서 우리가 만든 `custom_filter.py` 파일을 기억하시나요? 이제 그 안에 우리의 문자열 필터 역할을 할 새로운 Python 함수를 추가해 보겠습니다.

문자열 필터를 구현하기 전에 먼저 문자열 필터로 사용자 정의 필터 함수를 구분하는 `stringfilter` 데코레이터를 가져와야 합니다. 다음 가져오기 문을 `custom_filters.py` 파일 안에 추가하여 이 데코레이터를 추가할 수 있습니다:

```
from django.template.defaultfilters import stringfilter
```

이제 우리의 사용자 정의 문자열 필터를 구현하기 위해 다음 구문을 사용할 수 있습니다:

```
@register.filter@
stringfilter

def generic_string_filter(value, arg):
    # 문자열 필터 구현을 위한 로직
```

이 접근 방식을 사용하면 원하는 만큼의 문자열 필터 `idx_002eca20`를 만들고 다른 필터처럼 사용할 수 있습니다.

이 지식을 바탕으로 이제 첫 번째 사용자 정의 문자열 필터를 만들어 보겠습니다.

연습 11.01 – 사용자 정의 템플릿 필터 만들기

이 연습에서는 ISBN 또는 제목 문자열과 함께 문자열이 ISBN인지 제목인지 명시적으로 말하는 인수를 제공할 때 WorldCat 사이트의 URL을 반환하는 `link_worldcat`이라는 사용자 정의 필터를 작성할 것입니다.

예시입니다:

```
value = "9781727518603"
```

다음 필터를 이 문자열에 적용합니다:

```
{{ value|link_worldcat:"isbn" }}
```

The output after applying this filter should be as follows:

```
"https://www.worldcat.org/isbn/9781727518603"
```

Alternatively, there' the value was a book title:

```
value = "The Iliad"
```

The filter can be used to construct a book title query at worldcat.org such that:

```
{{ value|link_worldcat:"title" }}
```

would produce:

```
"https://www.worldcat.org/search?q=The+Iliad"
```

Let's get started with the steps:

1. Now, create a new directory named `templatetags` inside your reviews application directory to store the code for your custom template filters. To create the directory, run the following command from inside the `filter_demo` directory from the terminal app or command prompt:

```
mkdir templatetags
```

2. Once the directory is created, create a new file named `link_worldcat_filter.py` inside the `reviews/templatetags` directory, adding the following lines to it:

```
import urllib.parse
from django import template
from django.template.defaultfilters import stringfilter

register = template.Library()
```

The preceding code creates an instance of the Django library that can be used to register our custom filter with Django.

3. Add the following code to implement the `link_worldcat` filter:

```
@register.filter
@stringfilter
```

이 필터를 적용한 후의 출력은 다음과 같아야 합니다:

```
"https://www.worldcat.org/isbn/9781727518603"
```

또는, 값이 책 제목인 경우:

```
value = "The Iliad"
```

이 필터는 worldcat.org에서 책 제목 쿼리를 구성하는 데 사용할 수 있습니다:

```
{{ value|link_worldcat:"title" }}
```

다음과 같은 결과를 생성합니다:

```
"https://www.worldcat.org/search?q=The+Iliad"
```

단계에 대해 시작해 봅시다:

1. 이제, 사용자 정의 템플릿 필터의 코드를 저장하기 위해 reviews 애플리케이션 디렉토리 안에 templatetags라는 새 디렉토리를 만듭니다. 디렉토리를 만들기 위해 터미널 앱 또는 명령 프롬프트에서 filter_demo 디렉토리 안에서 다음 명령을 실행합니다:

```
mkdir templatetags
```

2. 디렉토리가 생성되면 reviews/templatetags 디렉토리 안에 link_worldcat_filter.py라는 새 파일을 만듭니다.

다음 줄을 추가합니다:

```
import urllib.parse
from django import template
from django.template.defaultfilters import stringfilter

register = template.Library()
```

이전 코드는 Django 라이브러리의 인스턴스를 생성하여 우리의 사용자 정의 필터를 Django에 등록하는 데 사용할 수 있습니다.

3. 다음 코드를 추가하여 link_worldcat 필터를 구현하십시오:

```
@register.filter(
    stringfilter
```

```
def link_worldcat (value, identifier):
    if not value:
        return ""
    if identifier=='title':
        link = urllib.parse.quote_plus(value)
        return f"https://www.worldcat.org/search?q={link}"
    else:
        return f"https://www.worldcat.org/isbn/{value}"
```

The `link_worldcat` filter takes two arguments; one is `value`, on which the filter was used, and the second is `identifier`, passed from the template to the filter. The filter will create an appropriate link depending on whether `identifier` is specified as `isbn` or `title`.

4. With the custom filter ready, we can revisit the existing book detail template where this filter can be applied. Add the `load link_worldcat_filter` directive at the start of the file after the `extends` directive and before the `block` declaration. Then add new spans that will display the ISBN and use the `link_worldcat` filter to provide a link to the book's WorldCat entry.

`reviews/templates/reviews/book_detail.html`

```
{% extends 'reviews/base.html' %}
{% load link_worldcat_filter %}
{% block content %}
    ...
    <span class="text-info">Publication Date: </span>
    <span>{{ book.publication_date }}</span>
    <br>
    <span class="text-info">ISBN: </span>
    <span><a href="{{ book.isbn|link_worldcat:'isbn' }}">
        {{ book.isbn }}</a></span>
    <br>
    ...
{% endblock %}
```

Likewise, we can add similar spans for ISBN and the WorldCat link in the book list template:

```
def link_worldcat (value, identifier):
    if not value:
        return ""
    if identifier=='title':
        link = urllib.parse.quote_plus(value)
        return f"https://www.worldcat.org/search?q={link}"
    else:
        return f"https://www.worldcat.org/isbn/{value}"
```

link_worldcat 필터는 두 개의 인수를 사용합니다. 하나는 필터가 사용된 value 이고, 두 번째는 템플릿에서 필터로 전달된 identifier입니다. 필터는 identifier가 isbn 또는 title로 지정되었는지에 따라 적절한 링크를 생성합니다.

4. 사용자 정의 필터가 준비되었으므로, 이 필터를 적용할 수 있는 기존 도서 세부 정보 템플릿을 다시 방문할 수 있습니다. 파일의 시작 부분에 extends 지시문 다음과 block 선언 이전에 load link_worldcat_filter 지시문을 추가합니다. 그런 다음 ISBN을 표시하고 link_worldcat 필터를 사용하여 책의 WorldCat 항목에 대한 링크를 제공하는 새로운 span을 추가합니다.

reviews/templates/reviews/book_detail.html

```
{% extends 'reviews/base.html' %}
{% load link_worldcat_filter %}
{% block content %}
    ...
    <span class="text-info">출판 날짜: </span>
    <span>{{ book.publication_date }}</span>
    <br>
    <span class="text-info">ISBN: </span>
    <span><a href="{{ book.isbn|link_worldcat:'isbn' }}">
        {{ book.isbn }}</a></span>
    <br>
    ...
{% endblock %}
```

마찬가지로, 책 목록 템플릿에 ISBN 및 WorldCat 링크에 대한 유사한 span을 추가할 수 있습니다:

reviews/templates/reviews/book_list.html

```
{% extends 'reviews/base.html' %}
{% load link_worldcat_filter %}
{% block content %}
<ul class="list-group">
  {% for item in book_list %}
  <li class="list-group-item">
    ...
    <span class="text-info">ISBN: </span>
    <span><a
      href="{{ item.book.isbn|link_worldcat:'isbn' }}">
        {{ item.book.isbn }}</a></span>
    <br>
    ...
  </li>
  {% endfor %}
</ul>
{% endblock %}
```

After the changes have been made, the code should resemble that in the bookr project directory in the Chapter11 folder in this book's GitHub repo.

5. To view whether the custom filter works, run the following command:

```
python manage.py runserver localhost:8000
```

Now, navigate to the following page in the browser: <http://localhost:8000/books/2>.

6. This page should appear as shown in *Figure 11.1*:

reviews/templates/reviews/book_list.html

```
{% extends 'reviews/base.html' %}
{% load link_worldcat_filter %}
{% block content %}
<ul class="list-group">{% for
  item in book_list %}<li
  class="list-group-item">

  ...
  <span class="text-info">ISBN: </span>
  <span><a
    href="{{ item.book.isbn|link_worldcat:"isbn" }}">
    {{ item.book.isbn }}</a></span>
  <br>
  ...
  </li>
  {% endfor %}
</ul>
{% endblock %}
```

변경사항이 적용된 후, 코드는 이 책의 GitHub 저장소의 Chapter11 폴더에 있는 bookr 프로젝트 디렉토리의 코드와 유사해야 합니다.

5. 사용자 정의 필터가 작동하는지 확인하려면 다음 명령을 실행하십시오:

```
python manage.py runserver localhost:8000
```

이제 브라우저에서 다음 페이지로 이동하십시오: <http://localhost:8000/books/2>.

6. 이 페이지는 그림 11.1과 같이 표시되어야 합니다:

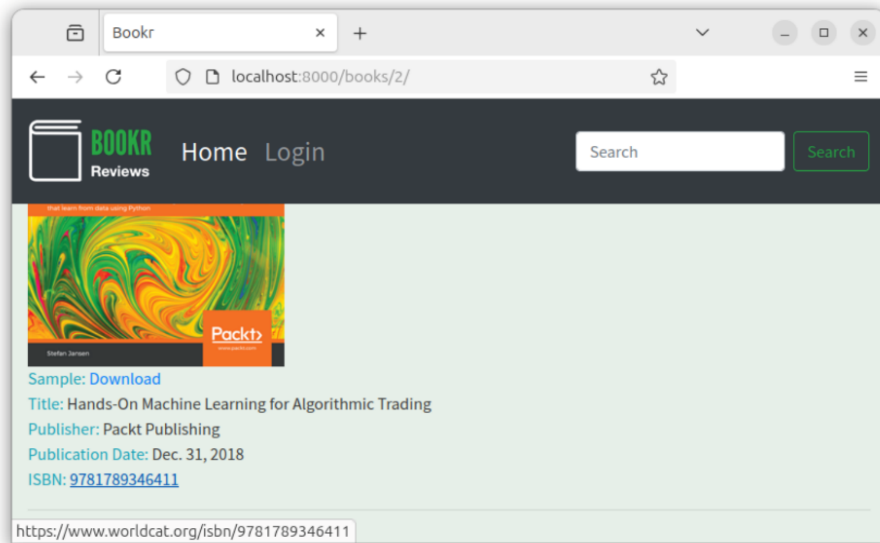


Figure 11.1 – Book detail page with ISBN link using the `link_worldcat` filter

Template tags

Template tags are a powerful feature of Django's templating engine. They allow developers to build powerful templates by generating HTML by evaluating certain conditions and help avoid the repetitive writing of common code.

One example where we may use template tags is the sign-up/login options in the navigation bar of a website. In this case, we can use template tags to evaluate whether the visitor on the current page is logged in. Based on that, we can render either a profile banner or a sign-up/login banner.

Tags are also a common occurrence while developing templates. For example, consider the following line of code, which we used to import the custom filters inside our templates in the previous section:

```
{% load explode_filter %}
```

This uses a template tag known as `load`, responsible for loading the `explode` filter into the template. Template tags are much more powerful compared to filters. While filters have access only to the values they are operating on, template tags have access to the context of the whole template and hence they can be used to build a lot of complex functionalities inside a template.

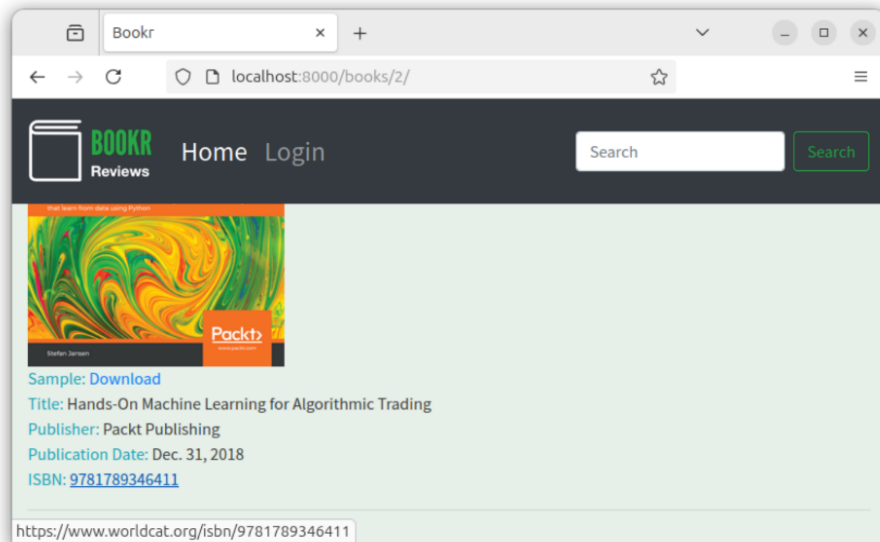


그림 11.1 - 링크 월드캣 필터를 사용한 ISBN 링크가 있는 도서 세부 정보 페이지

템플릿 태그

템플릿 태그는 Django의 템플릿 엔진의 강력한 기능입니다. 이들은 개발자가 특정 조건을 평가하여 HTML을 생성함으로써 강력한 템플릿을 구축할 수 있게 해주며, 일반 코드의 반복 작성을 피하는 데 도움을 줍니다.

템플릿 태그를 사용할 수 있는 한 가지 예는 웹사이트의 탐색 바에 있는 가입/로그인 옵션입니다. 이 경우, 우리는 템플릿 태그를 사용하여 현재 페이지의 방문자가 로그인했는지 평가할 수 있습니다. 이를 바탕으로 프로필 배너 또는 가입/로그인 배너를 렌더링할 수 있습니다.

태그는 템플릿을 개발할 때 흔히 발생합니다. 예를 들어, 이전 섹션에서 템플릿 내에서 사용자 정의 필터를 가져오는 데 사용한 다음 코드 줄을 고려해 보십시오:

```
{% load explode_filter %}
```

이것은 explode 필터를 템플릿에 로드하는 역할을 하는 load라는 템플릿 태그를 사용합니다. 템플릿 태그는 필터에 비해 훨씬 더 강력합니다. 필터는 자신이 작동하는 값에만 접근할 수 있는 반면, 템플릿 태그는 전체 템플릿의 컨텍스트에 접근할 수 있으므로 템플릿 내에서 많은 복잡한 기능을 구축하는 데 사용할 수 있습니다.

Let's now look at the different types of template tags supported by Django and how we can build our own custom template tags.

The types of template tags

Django mainly supports two types of template tags:

- **Simple tags:** These are the tags that operate on the variable data provided (and any additional variables to them) and render in the same template they have been called in. For example, one such use case can include rendering a custom welcome message to the user based on their username or displaying the user's last login time based on their username.
- **Inclusion tags:** These tags take in the provided data variables and generate an output by rendering another template. For example, the tag can take in a list of objects and iterate over them to generate an HTML list.

In the next sections, we will look at how we can create these different types of tags and use them in our application.

Simple tags

Simple tags provide a way for developers to build template tags that take in one or more variables from the template, process them, and return a response. The response returned from the template tag replaces the template tag definition provided inside the HTML template. These kinds of tags can be used to build several useful functionalities – for example, the parsing of dates or displaying any active alerts, if there are any, that we want to show to the user.

Simple tags can be created easily using the `simple_tag` decorator provided by the template library, by decorating the Python method that should act as a template tag. Now, let's look at how we can implement a custom simple tag using Django's template library.

Creating a simple template tag

Creating simple template tags follows the same conventions we discussed in the *Custom template filters* section, with some subtle differences. First, let's go over understanding how template tags can be created for use in our Django templates.

이제 Django에서 지원하는 다양한 유형의 템플릿 태그와 우리가 어떻게 사용자 정의 템플릿 태그를 만들 수 있는지 살펴보겠습니다.

템플릿 태그의 유형

Django는 주로 두 가지 유형의 템플릿 태그를 지원합니다:

- 간단한 태그: 이러한 태그는 제공된 변수 데이터(idx_b866a78b)와 그에 추가된 변수들에 대해 작동하며, 호출된 동일한 템플릿에서 렌더링됩니다. 예를 들어, 이러한 사용 사례 중 하나는 사용자의 사용자 이름에 따라 사용자에게 맞춤 환영 메시지를 렌더링하거나 사용자 이름에 따라 사용자의 마지막 로그인 시간을 표시하는 것입니다.
- 포함 태그: 이러한 태그는 제공된 데이터 변수(idx_41d6b202)를 받아들이고 다른 템플릿을 렌더링하여 출력(idx_9b898fe2)을 생성합니다. 예를 들어, 태그는 객체 목록을 받아들이고 이를 반복하여 HTML 목록을 생성할 수 있습니다.

다음 섹션에서는 이러한 다양한 유형의 태그를 만드는 방법과 이를 애플리케이션에서 사용하는 방법을 살펴보겠습니다.

간단한 태그

간단한 태그는 개발자(idx_7b136bfc)가 템플릿에서 하나 이상의 변수를 받아들이고 이를 처리하여 응답을 반환하는 템플릿 태그를 구축할 수 있는 방법을 제공합니다. 템플릿 태그에서 반환된 응답은 HTML 템플릿 내에 제공된 템플릿 태그 정의를 대체합니다. 이러한 종류의 태그는 날짜 파싱이나 사용자에게 표시하고 싶은 활성 알림이 있는 경우 이를 표시하는 등 여러 유용한 기능을 구축하는 데 사용할 수 있습니다.

간단한 태그는 템플릿 라이브러리에서 제공하는 `simple_tag` 데코레이터를 사용하여 쉽게 생성할 수 있으며, 템플릿 태그로 작동해야 하는 Python 메서드를 장식함으로써 가능합니다. 이제 Django의 템플릿 라이브러리를 사용하여 사용자 정의 간단한 태그를 구현하는 방법을 살펴보겠습니다.

간단한 템플릿 태그 만들기

간단한 템플릿 태그 만들기는 사용자 정의 템플릿 필터 섹션에서 논의한 것과 동일한 규칙을 따르며, 몇 가지 미세한 차이가 있습니다. 먼저, Django 템플릿에서 사용할 수 있도록 템플릿 태그를 만드는 방법을 이해해 보겠습니다.

Setting up the directory

Just like custom filters, custom template tags also need to be created inside the same `templatetags` directory to make them discoverable by Django's templating engine. The directory can be created either directly using the PyCharm GUI or by running the following command inside the application directory where we want to create our custom tags:

```
mkdir templatetags
```

Once this is done, we can now create a new file that will store the code for our custom template tags by using the following command:

```
touch custom_tags.py
```

Note

The aforementioned command won't work on Windows. You can, however, create a new file using Windows Explorer.

Setting up the template library

Once the directory structure is set up and we have a file in place for keeping the code for our custom template tags, we can now start creating our template tags. But before that, we need to set up an instance of Django's template library as we did earlier. This can be done by adding the following lines of code to our `custom_tag.py` file:

```
from django import template
register = template.Library()
```

Like custom filters, the template library instance is used here to register the custom template tags for use inside Django templates.

Implementing a simple template tag

Simple template tags inside Django are Python functions that can take any number of arguments as we desire. These Python functions need to be decorated with the `simple_tag` decorator from the template library to register those functions as simple template tags. The following snippet of code shows how a simple template tag is implemented:

```
@register.simple_tag
def generic_simple_tag(arg1, arg2):
    # Logic to implement a generic simple tag
```

디렉토리 설정

사용자 정의 필터와 마찬가지로, 사용자 정의 `templateidx_e3b7a8a1` 태그도 Django의 템플릿 엔진이 발견할 수 있도록 `templatetags` 디렉토리 내에서 생성해야 합니다. 디렉토리는 PyCharm GUI를 사용하여 직접 생성하거나, 사용자 정의 태그를 생성하려는 애플리케이션 디렉토리 내에서 다음 명령어를 실행하여 생성할 수 있습니다:

```
mkdir templatetags
```

이 작업이 완료되면, 이제 다음 명령어를 사용하여 사용자 정의 `templatetags`의 코드를 저장할 새 파일을 생성할 수 있습니다:

```
touch custom_tags.py
```

참고

앞서 언급한 명령어는 Windows에서 작동하지 않습니다. 그러나 Windows 탐색기를 사용하여 새 파일을 생성할 수 있습니다.

템플릿 라이브러리 설정

디렉토리 구조가 설정되고 우리 커스텀 템플릿 `idx_fe535b2a` 태그의 코드를 유지할 파일이 준비되면, 이제 템플릿 태그를 생성할 수 있습니다. 하지만 그 전에, 이전에 했던 것처럼 Django의 템플릿 라이브러리 인스턴스를 설정해야 합니다. 이는 우리 `custom_tag.py` 파일에 다음 코드 줄을 추가함으로써 수행할 수 있습니다:

```
from django import template
register = template.Library()
```

커스텀 필터와 마찬가지로, 템플릿 라이브러리 인스턴스는 여기서 Django 템플릿 내에서 사용할 커스텀 템플릿 태그를 등록하는 데 사용됩니다.

간단한 템플릿 태그 구현하기

Django `idx_78855261` 내의 간단한 템플릿 태그는 우리가 원하는 만큼의 인수를 받을 수 있는 Python 함수입니다. 이러한 Python 함수는 템플릿 라이브러리의 `simple_tag` 데코레이터로 장식되어야 하며, 이를 통해 해당 함수가 간단한 템플릿 태그로 등록됩니다. 다음 코드 조각은 간단한 템플릿 태그가 어떻게 구현되는지를 보여줍니다:

```
@register.simple_tag
def generic_simple_tag(arg1, arg2):
    # 일반적인 간단한 태그를 구현하는 로직
```


Next, let's use these simple tags inside the templates.

Using simple tags inside templates

Using simple tags inside Django templates is quite easy. Inside the template file, we need first to make sure that we have the tag imported inside the template by adding the following to the top of the template file:

```
{% load custom_tag %}
```

The preceding statement will load all the tags from the `custom_tag.py` file we defined earlier and make them available inside our template. Then we can use our custom simple tag by adding the following command:

```
{% custom_simple_tag "argument1" "argument2" %}
```

Wasn't that easy?! Now, let's put this knowledge into practice and create our first custom simple tag.

Exercise 11.02 – Creating a custom simple tag

Much of the existing templating code is somewhat cluttered and difficult to read because it contains control and conditional code. Consider this section from the search results template:

`reviews/templates/reviews/search-results.html`

```
{% for contributor in book.contributors.all %}
  {{ contributor.first_names }} {{ contributor.last_names }}
  {% if not forloop.last %}, {% endif %}
{% endfor %}
```

It makes sense to use simple tags as a way of keeping this conditional and control code in its own method. Now the code in the template can be replaced with two calls:

```
{% load list_items %}
{% list_items book.contributors.all 'first_names' 'last_names' %}
```

The `list_items` simple tag can be reused in our templates when we need to represent another list as a comma delimited string.

In this exercise, we will create a simple tag that takes multiple arguments: the first will be an iterable (such as a list of books or contributors), and the following arguments will be attributes

다음으로, 템플릿 안에 이러한 간단한 태그를 사용해 봅시다.

템플릿 안에 간단한 태그 사용하기

Djangoidx_e92e3dfc 템플릿 안에 간단한 태그를 사용하는 것은 매우 쉽습니다. 템플릿 파일 안에서, 먼저 템플릿에 태그가 임포트되었는지 확인해야 하며, 다음을 템플릿 파일의 맨 위에 추가합니다:

```
{% load custom_tag %}
```

앞서 언급한 문장은 우리가 이전에 정의한 custom_tag.py 파일의 모든 태그를 로드하고, 이를 idx_ce3b3182 템플릿 안에서 사용할 수 있도록 합니다. 그런 다음, 다음 명령어를 추가하여 우리의 사용자 정의 간단한 태그를 사용할 수 있습니다:

```
{% custom_simple_tag "argument1" "argument2" %}
```

그렇게 쉽지 않았나요?! 이제 이 지식을 실천에 옮겨 우리의 첫 번째 사용자 정의 간단한 태그를 만들어 봅시다.

연습 11.02 – 사용자 정의 간단한 태그 만들기

기존의 템플릿 idx_1d6ad6a0 코드는 제어 및 조건 코드가 포함되어 있어 다소 복잡하고 읽기 어렵습니다. 검색 결과 템플릿의 이 섹션을 고려해 보십시오:

```
reviews/templates/reviews/search-results.html
```

```
{% for contributor in book.contributors.all %}
  {{ contributor.first_names }} {{ contributor.last_names }}
  {% if not forloop.last %}, {% endif %}
{% endfor %}
```

이 조건 및 제어 코드를 자체 메서드로 유지하는 방법으로 간단한 태그를 사용하는 것이 합리적입니다. 이제 템플릿의 코드는 두 개의 호출로 대체될 수 있습니다:

```
{% load list_items %}
{% list_items book.contributors.all 'first_names' 'last_names' %}
```

list_items 간단한 태그는 다른 목록을 심표로 구분된 문자열로 표현해야 할 때 템플릿에서 재사용할 수 있습니다.

이 연습에서는 여러 인수를 받는 간단한 idx_10ea70d8 태그를 만들 것입니다: 첫 번째는 반복 가능한 객체(예: 책 또는 기여자의 목록)이고, 다음 인수는 속성이 될 것입니다.

that belong to the list item. The tag will return a string rendering of the specified attributes separated by commas.

1. First, create a new file named `simple_tag.py` under the `reviews/templatetags` directory. Inside this file, add the following code:

```
from django import template

register = template.Library()

@register.simple_tag
def list_items(iterable, *fields):
    return ', '.join(
        [ ' '.join([getattr(item, field, '')
                    for field in fields])
          for item in iterable])
```

Here, you have created a new Python method, `list_items`, which takes an iterable argument followed by multiple arguments for the fields to be referenced. This method is then decorated with `@register.simple_tag`, indicating that this method is a simple tag and can be used as a template tag in the templates.

2. Now, edit the search results template that will use your simple tag. `reviews/templates/reviews/search-results.html`

```
{% extends 'base.html' %}
{% load simple_tag %}
{% block title %}
...
<ul class="list-group">
    {% for book in books %}
        <li class="list-group-item">
            ...
            <span class="text-info">Contributors: </span>
            {% list_items book.contributors.all
                'first_names' 'last_names' %}
        </li>
    ...
</ul>
{% endif %}
{% endblock %}
```

목록 항목에 속하는. 태그는 지정된 속성을 심표로 구분하여 문자열 렌더링을 반환합니다.

1. 먼저 reviews/templatetags 디렉토리 아래에 simple_tag.py라는 새 파일을 만듭니다. 이 파일 안에 다음 코드를 추가합니다:

```
from django import template

register = template.Library()

@register.simple_tag
def list_items(iterable, *fields):
    return ', '.join(
        [' '.join([getattr(item, field, '')
                    for field in fields])
         for item in iterable])
```

여기에서 iterable 인수를 받고 참조할 필드에 대한 여러 인수를 따르는 새로운 Python 메서드 list_items를 생성했습니다. 이 메서드는 @register.simple_tag로 장식되어 있으며, 이 메서드는 간단한 태그임을 나타내며 템플릿에서 템플릿 태그로 사용할 수 있습니다.

2. 이제 간단한 태그를 사용할 검색 결과 템플릿을 수정하세요.
reviews/templates/reviews/search-results.html

```
{% extends 'base.html' %}
{% load simple_tag %}
{% block title %}
...
<ul class="list-group">
    {% for book in books %}
    <li class="list-group-item">
        ...
        <span class="text-info">기여자: </span>
        {% list_items
            book.contributors.all'first_names'
            'last_names' %}
    </li>
    ...
    </ul>
{% endif %}
{% endblock %}
```

In the preceding code snippet, we just created a bare-bones HTML page that will use your custom simple tag. The semantics of loading a custom template tag is similar to that of loading a custom template filter and requires the use of a `{% load %}` tag in the template. The process will look for the `simple_tag.py` module under the `templatetags` directory and, if found, will load the tags that have been defined under the module.

The final set of changes should resemble the ones in the `Chapter11` folder in this book's GitHub repo.

3. To see the custom tag in action, start your web server by running the following command:

```
python manage.py runserver localhost:8000
```

When using the search form, the contributors will still be rendered as before, but the simple tag greatly simplifies the Django template.

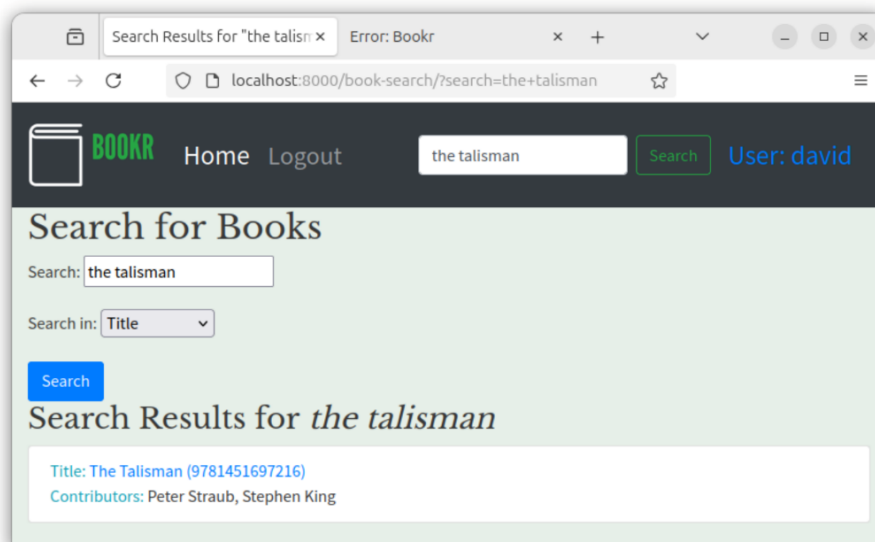


Figure 11.2 – Search form results

With this, we created our first custom template tag and used it successfully to render our template, as shown in *Figure 11.2*. Now, let's look at another important aspect of simple tags, which is associated with passing the context variables available in the template to the template tag.

앞서의 코드 idx_361957b6snippet에서는 사용자 정의 간단 태그를 사용할 수 있는 기본 HTML 페이지를 만들었습니다. 사용자 정의 템플릿 태그를 로드하는 의미는 사용자 정의 템플릿 필터를 로드하는 것과 유사하며, 템플릿에서 {% load %} 태그를 사용해야 합니다. 이 과정은 templatetags 디렉토리 아래의 simple_tag.py 모듈을 찾고, 발견되면 모듈 아래에 정의된 태그를 로드합니다.

최종 변경 사항 세트는 이 책의 GitHub 리포지토리의 Chapter11 폴더에 있는 것과 유사해야 합니다.

3. 사용자 정의 태그를 작동시키려면 다음 명령어를 실행하여 웹 서버를 시작하세요:

```
python manage.py runserver localhost:8000
```

검색 양식을 사용할 때 기여자는 이전과 같이 렌더링되지만, 간단 태그는 Django 템플릿을 크게 단순화합니다.

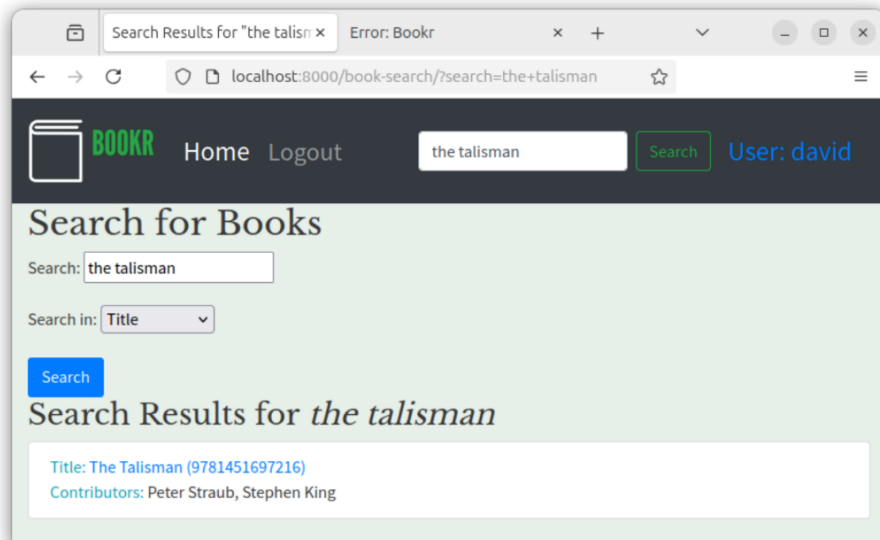


그림 11.2 - 검색 양식 결과

이로써 우리는 첫 번째 사용자 정의 템플릿 태그 idx_397403ea를 만들고, 그림 11.2에 표시된 대로 템플릿을 성공적으로 렌더링하는 데 사용했습니다. 이제 템플릿에서 사용할 수 있는 컨텍스트 변수를 템플릿 태그에 전달하는 것과 관련된 간단 태그의 또 다른 중요한 측면을 살펴보겠습니다.

Passing the template context in a custom template tag

In the previous exercise, we created a simple tag to which we passed two arguments, the greeting message and the username. But what if we wanted to pass a large number of variables to the tag? Or, simply, what if we did not want to pass the user's username explicitly to the tag?

There are times when developers would like to have access to all the variables and data that is present in the template to be available inside the custom tag. Fortunately for us, this is easy to implement.

Let's create a new tag named `contextual_greet_user` and see how we can pass the data available in the template directly to the tag instead of passing it manually as an argument.

Set up the tag as follows:

```
@register.simple_tag(takes_context=True)
def contextual_greet_user(context, message):
    username = context['username']
    return f'{message}, {username}'
```

With this, we tell Django that when our `contextual_greet_user` tag is used, Django should also pass it the template context, which has all the data passed from the view to the template. It accepts the added context as an argument and renders a greeting message to the current user.

In the preceding code example, we can see how the `contextual_greet_user()` method was modified to accept the passed context as the first argument, followed by the greeting message passed by the user.

To use this, we need the following call to the `contextual_greet_user` tag inside `simple_tag_template.html` under `filter_demo` to look like this:

```
{% contextual_greet_user "Hey" %}
```

Then, when we reload our Django web application, the output at `http://localhost:8000/filter_demo/greet` should look the same as was shown in step 5 of *Exercise 11.02, Creating a custom simple tag*.

With this, we learned how we can build a simple tag and handle passing the template context to the tag. Now, let's take a look at how we can build an inclusion tag that can be used to render data in a certain format, as described by another template.

사용자 정의 템플릿 태그에서 템플릿 컨텍스트 전달하기

이전 연습에서는 인사 메시지와 사용자 이름이라는 두 개의 인수를 전달한 간단한 태그를 만들었습니다. 하지만 태그에 많은 변수를 전달하고 싶다면 어떻게 해야 할까요? 또는, 단순히 사용자의 사용자 이름을 태그에 명시적으로 전달하고 싶지 않다면 어떻게 해야 할까요?

개발자들이 템플릿에 있는 모든 변수와 데이터에 접근하고 싶어하는 경우가 있습니다. 다행히도, 이는 구현하기 쉽습니다.

`contextual_greet_user`라는 새 태그를 만들어 템플릿에서 사용할 수 있는 데이터를 태그에 직접 전달하는 방법을 살펴보겠습니다. 인수를 수동으로 전달하는 대신 다음과 같이 태그를 설정합니다:

```
@register.simple_tag(takes_context=True)
def contextual_greet_user(context, message):
    username = context['username']
    return f"{message}, {username}"
```

이로써 우리는 Django에 `contextual_greet_user` 태그가 사용될 때 Django가 템플릿 컨텍스트를 전달해야 한다고 알립니다. 이 컨텍스트는 뷰에서 템플릿으로 전달된 모든 데이터를 포함하고 있습니다. 추가된 컨텍스트를 인수로 받아 현재 사용자에게 인사 메시지를 렌더링합니다.

앞선 코드 예제에서 우리는 `contextual_greet_user()` 메서드가 전달된 컨텍스트를 첫 번째 인수로 받아들이도록 수정된 방법을 볼 수 있으며, 그 뒤에는 사용자가 전달한 인사 메시지가 있습니다.

이를 사용하기 위해서는 `filter_demo` 아래의 `simple_tag_template.html`에서 `contextual_greet_user` 태그에 대한 다음 호출이 다음과 같아야 합니다:

```
{% contextual_greet_user "Hey" %}
```

그런 다음, `idx_6e28d95e`를 다시 로드하면 Django `idx_7a0eed9e` 웹 애플리케이션에서 `http://localhost:8000/filter_demo/greet`의 출력은 연습 11.02의 5단계에서 보여준 것과 같아야 합니다. 사용자 정의 간단 태그 만들기.

이로써 우리는 간단한 태그를 만들고 템플릿 컨텍스트를 태그에 전달하는 방법을 배웠습니다. 이제 다른 템플릿에 의해 설명된 특정 형식으로 데이터를 렌더링하는 데 사용할 수 있는 포함 태그를 만드는 방법을 살펴보겠습니다.

Inclusion tags

Simple tags allow us to build tags that accept one or more input variables, do some processing on them, and return an output. This output is then inserted where the simple tag was used.

But what if we wanted to build tags that, instead of returning text output, return an HTML template, which can then be used to render the parts of the page? For example, many web applications allow users to add custom widgets to their profiles. These individual widgets can be built as an inclusion tag and rendered independently. This kind of approach keeps the code for the base page template and the individual templates separate and hence allows for easy reuse and refactoring.

Developing custom inclusion tags is a similar process to how we develop our simple tags. This involves the use of the `inclusion_tag` decorator provided by the template library. So, let's take a look at how we can do it.

Implementing inclusion tags

Inclusion tags are those tags used for rendering a template as a response to their usage inside a template. These tags can be implemented in a similar manner to how other custom template tags are implemented, with some minor modifications.

Inclusion tags are also simple Python functions that can take multiple parameters, where each parameter maps to an argument passed from the template where the tag was called. These tags are decorated using the `inclusion_tag` decorator from Django's template library. The `inclusion_tag` decorator takes a single parameter, the name of the template, which should be rendered as a response to the processing of the inclusion tag.

A generic implementation of an inclusion tag will look like the one shown in the following code snippet:

```
@register.inclusion_tag('template_file.html')
def my_inclusion_tag(arg):
    # Logic for processing
    return {'key1': 'value1'}
```

Notice the return value in this case. An inclusion tag is supposed to return a dictionary of values that will be used to render the `template_file.html` file specified as an argument in the `inclusion_tag` decorator.

포함 태그

간단한 태그는 하나 이상의 입력 변수를 수용하고, 이를 처리한 후 출력을 반환하는 `tagidx_e68ec84c`를 구축할 수 있게 해줍니다. 이 출력은 간단한 태그가 사용된 곳에 삽입됩니다.

하지만 텍스트 출력을 반환하는 대신 HTML 템플릿을 반환하는 태그를 구축하고 싶다면 어떻게 될까요? 예를 들어, 많은 웹 애플리케이션은 사용자가 프로필에 사용자 정의 위젯을 추가할 수 있도록 허용합니다. 이러한 개별 위젯은 포함 태그로 구축되고 독립적으로 렌더링될 수 있습니다. 이러한 접근 방식은 기본 페이지 템플릿과 개별 템플릿의 코드를 분리하여 쉽게 재사용하고 리팩토링할 수 있게 해줍니다.

사용자 정의 포함 태그를 개발하는 과정은 간단한 태그를 개발하는 과정과 유사합니다. 여기에는 템플릿 라이브러리에서 제공하는 `inclusion_tag` 데코레이터를 사용하는 것이 포함됩니다. 그러니, 우리가 어떻게 할 수 있는지 살펴보겠습니다.

포함 태그 구현하기

포함 태그는 템플릿 내에서 사용될 때 응답으로 템플릿을 렌더링하는 데 사용되는 `tagidx_f17f10cb`입니다. 이러한 태그는 다른 사용자 정의 템플릿 태그가 구현되는 방식과 유사하게 약간의 수정으로 구현될 수 있습니다.

포함 태그는 여러 매개변수를 받을 수 있는 간단한 Python 함수이기도 하며, 각 매개변수는 태그가 호출된 템플릿에서 전달된 인수에 매핑됩니다. 이러한 태그는 Django의 템플릿 라이브러리에서 `inclusion_tag` 데코레이터를 사용하여 장식됩니다. `inclusion_tag` 데코레이터는 포함 태그 처리에 대한 응답으로 렌더링되어야 하는 템플릿의 이름이라는 단일 매개변수를 받습니다.

포함 태그의 일반적인 구현은 다음 코드 스니펫에 표시된 것과 유사하게 보일 것입니다:

```
@register.inclusion_tag('template_file.html')
def my_inclusion_tag(arg):
    # 처리 로직
    return {'key1': 'value1'}
```

이 경우 반환 값을 주목하세요. 포함 태그는 인수로 지정된 `template_file.html` 파일을 렌더링하는 데 사용될 값의 사전을 반환해야 합니다. `idx_c0b5ed70` in the `inclusion_tag` decorator.

Using an inclusion tag inside a template

An inclusion tag can easily be used inside a template file. This can be done by first importing the tag as follows:

```
{% load custom_tags %}
```

Then, by using the tag like any other tag:

```
{% my_inclusion_tag "argument1" %}
```

The response of the rendering of this tag will be a sub-template that will be rendered inside our primary template where the inclusion tag was used.

Exercise 11.03 – Building a custom inclusion tag

In this exercise, we are going to build a custom inclusion tag, called `book_item`, which will render the details of a book, simplifying the code in the existing book list template of the reviews app.

1. For this exercise, you will continue to use the same demo folders as in earlier exercises. First, create a new file named `inclusion_tag.py` under the `filter_demo/templatetags` directory and write the following code inside it:

```
from django import template

register = template.Library()

@register.inclusion_tag('book_list_item.html')
def book_list_item(book):
    return {'item': book}
```

The `@register.inclusion_tag` decorator is used to mark the method as a custom inclusion tag. This decorator takes the template's name as an argument that should be used to render the data returned by the tag function.

After the decorator, we defined a function that implements the logic of our custom inclusion tag. This function takes a single argument called `book`. This argument will be passed from the template file and contain a `Book` object.

템플릿 내에서 포함 태그 사용하기

포함 태그는 템플릿 파일 내에서 쉽게 사용할 수 있습니다. 이는 다음과 같이 태그를 먼저 가져오는 것으로 할 수 있습니다:

```
{% load custom_tags %}
```

그런 다음, 다른 태그처럼 태그를 사용하여:

```
{% my_inclusion_tag "argument1" %}
```

이 태그의 렌더링 응답은 포함 태그가 사용된 기본 템플릿 내에서 렌더링될 하위 템플릿이 될 것입니다.

연습 11.03 – 사용자 정의 포함 태그 만들기

이 연습에서는 `book_item`이라는 사용자 정의 포함 태그를 만들 것입니다. 이 태그는 리뷰 앱의 기존 책 목록 템플릿에서 책의 세부 정보를 렌더링하여 코드를 단순화합니다.

1. 이 연습을 위해, 이전 연습에서와 동일한 데모 폴더를 계속 사용합니다. 먼저, `filter_demo/templatetags` 디렉토리 아래에 `inclusion_tag.py`라는 새 파일을 만들고 그 안에 다음 코드를 작성합니다:

```
from django import template

register = template.Library()

@register.inclusion_tag('book_list_item.html')
def book_list_item(book):
    return {'item': book}
```

`@register.inclusion_tag` 데코레이터는 메서드를 사용자 정의 포함 태그로 표시하는 데 사용됩니다. 이 데코레이터는 태그 함수가 반환하는 데이터를 렌더링하는 데 사용해야 하는 템플릿의 이름을 인수로 받습니다.

데코레이터 이후에, 우리는 사용자 정의 포함 태그의 논리를 구현하는 함수를 정의했습니다. 이 함수는 `book`이라는 단일 인수를 받습니다. 이 인수는 템플릿 파일에서 전달되며 `Book` 객체를 포함합니다.

This object is then returned as a context for the `book_list_item.html` template:

```
return {'item': book}
```

The value returned by this method will be passed by Django to the `book_list_item.html` template, and the contents will then be rendered.

2. Next, create the actual template, which will contain the rendering structure for the template tag. For this, create a new template file, `book_list_item.html`, under the `reviews/templates` directory, and add the following content to it:
`reviews/templates/book_list_item.html`

```
{% load link_worldcat_filter %}
<li class="list-group-item">
    <span class="text-info">Title: </span>
    <span>{{ item.book.title }}</span>
    <br>
    <span class="text-info">Publisher: </span>
    <span>{{ item.book.publisher }}</span>
    <br>
    <span class="text-info">Publication Date: </span>
    <span>{{ item.book.publication_date }}</span>
    <br>
    <span class="text-info">ISBN: </span>
    <span>
        <a href="{% item.book.isbn|link_worldcat:'isbn' %}">
            {{ item.book.isbn }}</a></span>
    <br>
    <span class="text-info">Rating: </span>
    <span class="badge badge-primary badge-pill">
        {{ item.book_rating }}</span>
    <br>
    <span class="text-info">Number of reviews: </span>
    <span>{{ item.number_of_reviews }}</span>
    <br>
    {% if not item.book_rating %}
    <span class="text-secondary">
        Provide a rating and write the first review
        for this book.</span>
    <br>
    {% endif %}
```

이 객체는 book_list_item.html 템플릿의 컨텍스트로 반환됩니다:

```
return {'item': book}
```

이 메서드에 의해 반환된 값은 Django에 의해 book_list_item.html 템플릿으로 전달되며, 그 내용은 렌더링됩니다.

2. 다음으로, 템플릿 태그의 렌더링 구조를 포함할 실제 템플릿을 만듭니다. 이를 위해 reviews/templates 디렉토리 아래에 새로운 템플릿 파일인 book_list_item.html을 생성하고 다음 내용을 추가합니다: reviews/templates/book_list_item.html

```
{% load link_worldcat_filter %}
<li class="list-group-item">
    <span class="text-info">제목: </span>
    <span>{{ item.book.title }}</span>
    <br>
    <span class="text-info">출판사: </span>
    <span>{{ item.book.publisher }}</span>
    <br>
    <span class="text-info">출판 날짜: </span>
    <span>{{ item.book.publication_date }}</span>
    <br>
    <span class="text-info">ISBN: </span>
    <span>
        <a href="{% item.book.isbn|link_worldcat:'isbn' %}">
            {{ item.book.isbn }}</a></span>
    <br>
    <span class="text-info">평점: </span>
    <span class="badge badge-primary badge-pill">
        {{ item.book_rating }}</span>
    <br>
    <span class="text-info">리뷰 수: </span>
    <span>{{ item.number_of_reviews }}</span>
    <br>
    {% if not item.book_rating %}
    <span class="text-secondary">
        이 책에 대한 평가를 제공하고 첫 리뷰를 작성하
        세요.</span>
    <br>
    {% endif %}
```

```

        <a class="btn btn-primary btn-sm active" role="button"
          aria-pressed="true"
          href="/books/{{ item.book.id }}/">Reviews</a>
    </li>

```

Here, in the new template file, we used the contents of the `<li>` tag from the `reviews/templates/reviews/book_list.html` template. Note that we also included the `load link_worldcat_filter` directive as this filter is now included within the inclusion tag.

3. With the template defined for the `book_list_item` tag, modify the existing `book_list.html` template to make this tag available inside it and replace the contents of the `<li>` tag with a call to `book_list_item`.
- `reviews/templates/reviews/book_list.html`

```

{% extends 'reviews/base.html' %}

{% load inclusion_tag %}

{% block content %}
    <ul class="list-group">
        {% for item in book_list %}
            {% book_list_item item %}
        {% endfor %}
    </ul>
{% endblock %}

```

In this snippet, the first thing you did was load the `inclusion_tag` module by writing the following:

```

{% load inclusion_tag %}

```

Once the tag is loaded, you can use it anywhere in the template. To use it, you added the `book_list_item` tag in the following format:

```

{% book_list_item item %}

```

This tag takes a single argument, a `Book` object.

Once the changes have been made, the files should resemble the ones hosted in the `Chapter11` folder in this book's GitHub repo. .

```
<a class="btn btn-primary btn-sm active" role="button"
    aria-pressed="true"
    href="/books/{{ item.book.id }}/">리뷰</a>
</li>
```

여기, 새로운 템플릿 파일에서 우리는 reviews/templates/reviews/book_list.html 템플릿의 태그 내용을 사용했습니다. 이 필터가 이제 포함 태그 내에 포함되었으므로 loadlink_worldcat_filter 지시어도 포함했음을 주목하세요.

3. templateidx_a56e0084가 book_list_item 태그에 대해 정의되었으므로 기존 book_list.html 템플릿을 수정하여 이 태그를 내부에서 사용할 수 있도록 하고 태그의 내용을 book_list_item 호출로 교체하세요.
.reviews/templates/reviews/book_list.html

```
{% extends 'reviews/base.html' %}

{% load inclusion_tag %}

{% block content %}
<ul class="list-group">{% for
    item in book_list %}{%
    book_list_item item %}{%
    endfor %}

</ul>

{% endblock %}
```

이 코드 조각에서, 당신이 처음 한 일은 다음과 같이 inclusion_tag 모듈을 로드하는 것이었습니다:

```
{% load inclusion_tag %}
```

태그가 로드되면, 템플릿의 어디에서나 사용할 수 있습니다. 사용하기 위해, 다음 형식으로 book_list_item 태그를 추가했습니다:

```
{% book_list_item item %}
```

이 태그는 단일 인수, 즉 Book 객체를 받습니다.

변경이 완료되면, 파일은 이 책의 GitHub 저장소에 있는 Chapter11 폴더에 호스팅된 파일과 유사해야 합니다.

4. With the preceding changes implemented, it's time to render the modified template. To do this, restart your Django application server by running the following command:

```
python manage.py runserver localhost:8080
```

5. Navigate to `http://localhost:8080/books`, which should now render a page similar to the following screenshot:

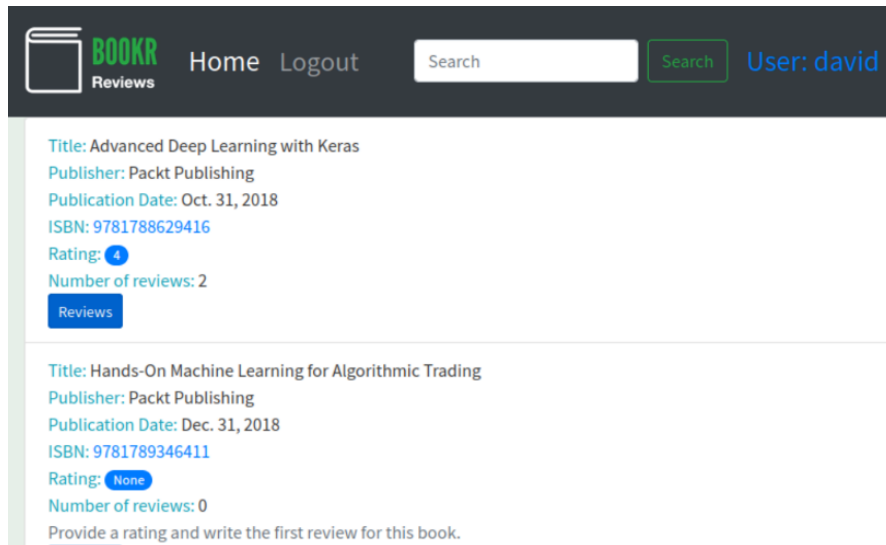


Figure 11.3 – List of books after refactoring the template code with inclusion_tag

The books endpoint still renders the same, but we have simplified the `book_list.html` template by using an inclusion tag. By modularizing the template code with inclusion tags, we make it easy to reuse components of pages without wholesale repetition of common code blocks.

Activity 11.01 – Rendering details on the user profile page using inclusion tags

In this activity, you will create a custom inclusion tag that helps to develop a user profile page that renders not only the users' details but also lists the books they have read.

The following steps should help you to complete this activity successfully:

1. Create a new `templatetags` directory under the reviews application inside the bookr project to provide a place where you can create your custom template tags.

4. 앞서 변경된 `idx_ed80e546`이 적용되었으므로, 수정된 템플릿을 렌더링할 시간입니다. 이를 위해 다음 명령어를 실행하여 Django 애플리케이션 서버를 재시작하세요:

```
python manage.py runserver localhost:8080
```

5. `http://localhost:8080/books`로 이동하면 이제 다음 스크린샷과 유사한 페이지가 렌더링되어야 합니다:

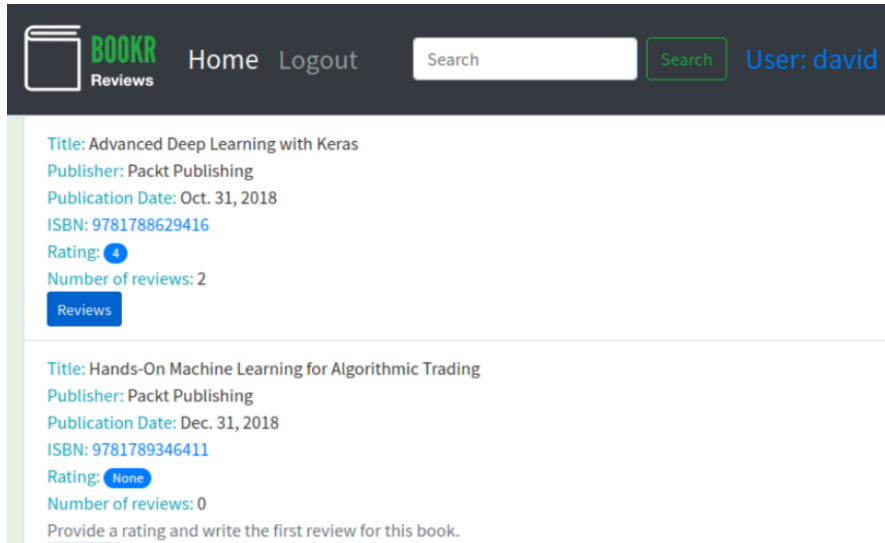


그림 11.3 - 템플릿 코드를 `inclusion_tag`로 리팩토링한 후의 책 목록

책 엔드포인트는 여전히 동일하게 렌더링되지만, `inclusion` 태그를 사용하여 `book_list.html` 템플릿을 단순화했습니다. `inclusion` 태그로 템플릿 코드를 모듈화함으로써, 공통 코드 블록의 전체 반복 없이 페이지 구성 요소를 쉽게 재사용할 수 있습니다.

활동 11.01 – `inclusion` 태그를 사용하여 사용자 프로필 페이지의 세부정보 렌더링

이 활동에서는 사용자 프로필 페이지를 개발하는 데 도움이 되는 사용자 정의 포함 태그를 생성할 것입니다. 사용자 세부정보뿐만 아니라 그들이 읽은 책 목록도 렌더링합니다. 다

음 단계는 이 활동을 성공적으로 완료하는 데 도움이 될 것입니다:

1. `bookrproject`의 `reviews` 애플리케이션 아래에 새로운 `templatetags` 디렉토리를 생성하여 사용자 정의 템플릿 태그를 생성할 수 있는 장소를 제공합니다.

2. In the `inclusion_tag.py` file, which you created in *Exercise 11.03*, add the following functionality:
3. Import the `Review` model from the `reviews` application to fetch the reviews written by a user. This will be used to filter the reviews for the current user to render on the user profile page.
4. Next, create a new Python function named `review_list`, which will contain the logic for your inclusion tag. This function should only take a single parameter, the username of the currently logged-in user.
5. Inside the body of the `review_list` function, add the logic for fetching the reviews for the given user and extract the review IDs and titles of the books this user has reviewed.
6. Decorate this `review_list` function with the `inclusion_tag` decorator and provide it with a template name, `review_list.html`.
7. Create a new template file named `review_list.html`, which was specified to the inclusion tag decorator in step 4. Inside this file, add code to render a list of books with hyperlinks to the user's reviews. This can be done by using a for loop construct and rendering HTML anchor and line break tags – `<a>`, `<br>` – for every item inside the list provided. The for loop will contain an empty clause that will display, "No reviews found".
8. Modify the existing `profile.html` file under the `templates` directory, which will be used to render the user profile. Inside this template file, load the `inclusion_tag` library and use `review_list` to render the list of books reviewed by the user. The code will be wrapped inside a new `div` of class `infoCell`.

Once you complete all these steps, starting the application server and visiting the user profile page should render a page that is similar to the one shown in *Figure 11.10*:

2. Exercise 11.03에서 생성한 `inclusion_tag.py` 파일에 다음 기능을 추가하세요:
 3. 리뷰 애플리케이션에서 Review 모델을 가져와 사용자가 작성한 리뷰를 가져옵니다. 이는 현재 사용자에게 대한 리뷰를 필터링하여 사용자 프로필 페이지에 렌더링하는 데 사용됩니다.
 4. 다음으로, `review_list`라는 새로운 Python 함수를 생성합니다. 이 함수는 포함 태그에 대한 논리를 포함할 것입니다. 이 함수는 현재 로그인한 사용자의 사용자 이름이라는 단일 매개변수만 받아야 합니다.
 5. `review_list` 함수의 본문 안에 주어진 사용자의 리뷰를 가져오는 논리를 추가하고 이 사용자가 리뷰한 책의 리뷰 ID와 제목을 추출합니다. 이 `review_list` 함수를
 6. `inclusion_tag` 데코레이터로 장식하고 template name인 `review_list.html`을 제공합니다.
 7. `review_list.html`이라는 새로운 템플릿 파일을 생성합니다. 이 파일은 4단계에서 `inclusion` 태그 데코레이터에 지정되었습니다. 이 파일 안에 사용자의 리뷰에 대한 하이퍼링크가 있는 책 목록을 렌더링하는 코드를 추가합니다. 이는 for 루프 구조를 사용하고 제공된 목록의 각 항목에 대해 HTML 앵커 및 줄 바꿈 태그 – `<a>`, `<br>` –를 렌더링하여 수행할 수 있습니다. for 루프에는 "리뷰가 없습니다"라는 메시지를 표시하는 빈 절이 포함됩니다.
 8. `templates` 디렉토리 아래에 있는 기존 `profile.html` 파일을 수정합니다. 이 파일은 사용자 프로필을 렌더링하는 데 사용됩니다. 이 템플릿 파일 안에 `inclusion_tag` 라이브러리를 로드하고 `review_list`를 사용하여 사용자가 리뷰한 책 목록을 렌더링합니다. 코드는 `infocell` 클래스의 새로운 div 안에 감싸집니다.
- 이 모든 단계를 완료하면, 애플리케이션 서버를 시작하고 사용자 프로필 페이지를 방문하면 그림 11.10에 표시된 것과 유사한 페이지가 렌더링되어야 합니다:

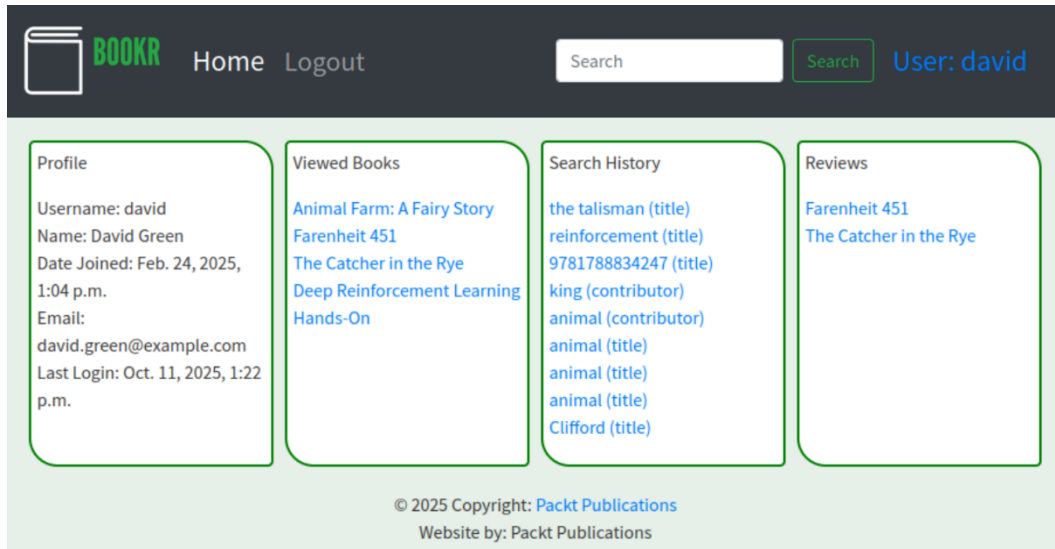


Figure 11.10 – The user profile page with the list of books reviewed by the user

Template partials

As we code our templates in a Django app, we tend to see a lot of repetition in the features that we have coded. Using inheritance from base templates is a good way to reuse common features such as headers, menus and footers but sometimes we want to reuse components within a page. **Template partials** are similar to functions that are defined within a template to define a reusable template block.

In order to create a template partial, we first define the template partial content using between the `partialdef` and `endpartialdef` template tags. We can reference context variables in the template partial as we do in the rest of a template.

```
{% partialdef "chapter" %}
    <h2>
    <a href="{{chapter.url}}" >{{chapter.number}}:
        {{chapter.title}}</a></h2>
{% endpartialdef %}
```

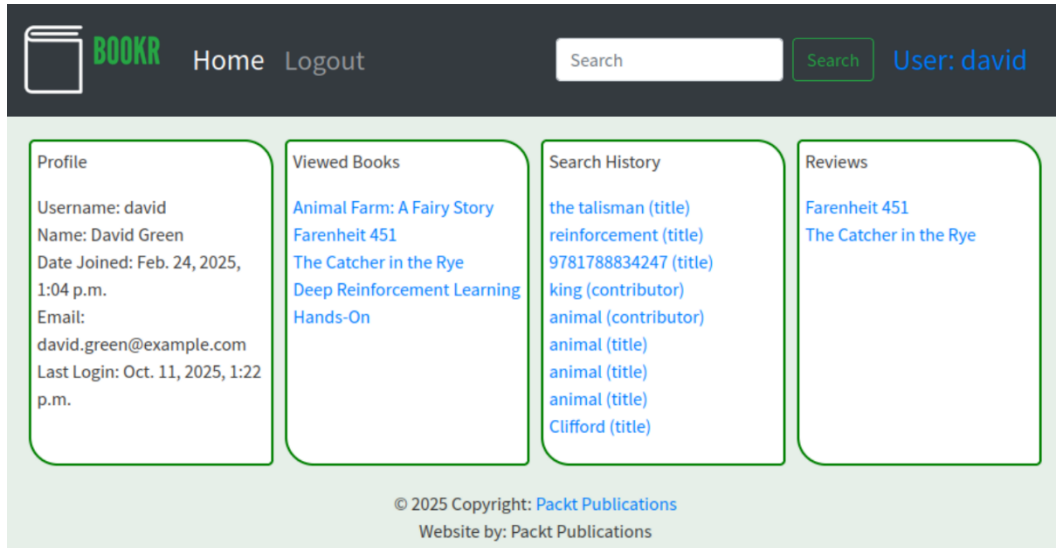


그림 11.10 - 사용자가 리뷰한 책 목록이 있는 사용자 프로필 페이지

템플릿 부분

Django 앱에서 템플릿을 코딩할 때, 우리가 코딩한 기능에서 많은 반복을 보게 됩니다. 기본 템플릿에서 상속을 사용하는 것은 헤더, 메뉴 및 푸터와 같은 공통 기능을 재사용하는 좋은 방법이지만, 때때로 페이지 내에서 구성 요소를 재사용하고 싶습니다. 템플릿 부분은 재사용 가능한 템플릿 블록을 정의하기 위해 템플릿 내에서 정의된 `idx_177af22a` 함수와 유사합니다.

템플릿 부분을 만들기 위해, 먼저 `partialdef`와 `endpartialdef` 템플릿 태그 사이에 템플릿 부분 내용을 정의합니다. 나머지 템플릿에서처럼 템플릿 부분에서 컨텍스트 변수를 참조할 수 있습니다.

```
{% partialdef "chapter" %}
    <h2>
    <a href="{{chapter.url}}" >{{chapter.number}}:
        {{chapter.title}}</a></h2>
{% endpartialdef %}
```

The template partial is called by using the partial template tag.

```
{% for chapter in chapters %}
    {% partial "chapter" %}
{% endfor %}
```

Let's say the Django view parses a context variable for chapters like this:

```
context = dict(chapters=(
    dict(url='/chap/1', number=1, title='Introduction'),
    dict(url='/chap/2', number=2,
        title='Historical Background'),
    dict(url='/chap/3', number=3,
        title='Technical Information'),
    dict(url='/chap/4',
        number=4, title='Conclusion'),
))
```

The template will be rendered as follows:

```
<h2><a href="/chap/1">1: Introduction</a></h2>
<h2><a href="/chap/2">2: Historical Background</a></h2>
<h2><a href="/chap/3">3: Technical Information</a></h2>
<h2><a href="/chap/4">4: Conclusion</a></h2>
```

This approach offers a simpler approach to reuse templating code than using an inclusion tag as the template partial is defined in the template file and doesn't require the overhead of an additional function in the `views.py` file. However, there are some cases, such as the `review_list` inclusion tag in *Activity 11.01 – Rendering details on the user profile page using inclusion tags*, where it makes sense to encapsulate database or control logic in the inclusion tag so that it can be shareable across multiple views. In the next activity, we will replace the `book_list_item` inclusion tag from *Exercise 11.03 – Building a custom inclusion tag* with a template partial.

템플릿 부분은 부분 템플릿 태그를 사용하여 호출됩니다.

```
{% for chapter in chapters %}
    {% partial "chapter" %}
{% endfor %}
```

장고 뷰가 다음과 같이 장에 대한 컨텍스트 변수를 구문 분석한다고 가정해 보겠습니다:

```
context = dict(chapters=(
    dict(url='/chap/1', number=1, title='Introduction'),
    dict(url='/chap/2', number=2,
        title='Historical Background'},
    dict(url='/chap/3', number=3,
        title='기술 정보'},
        dict(url='/chap/4',
number=4, title='결론'},
    )
)
```

템플릿은 다음과 같이 렌더링됩니다:

```
<h2><a href="/chap/1">1: 소개</a></h2>
<h2><a href="/chap/2">2: 역사적 배경</a></h2>
<h2><a href="/chap/3">3: 기술 정보</a></h2>
<h2><a href="/chap/4">4: 결론</a></h2>
```

이 접근 방식은 템플릿 파일에 템플릿 부분이 정의되어 있고 views.py 파일에 추가 함수의 오버헤드가 필요하지 않기 때문에 포함 태그를 사용하는 것보다 템플릿 코드를 재사용하는 더 간단한 방법을 제공합니다. 그러나 Activity 11.01 – 포함 태그를 사용하여 사용자 프로필 페이지에 세부정보 렌더링에서 review_list 포함 태그와 같은 경우에는 데이터베이스 또는 제어 논리를 포함 태그에 캡슐화하여 여러 뷰에서 공유할 수 있도록 하는 것이 합리적입니다. 다음 활동에서는 Exercise 11.03 – 사용자 정의 포함 태그 만들기에서 book_list_item 포함 태그를 템플릿 부분으로 교체할 것입니다.

Activity 11.02 – Rendering a book list item with template partials

In this activity, we will convert the inclusion tag defined in *Exercise 11.03* into a template partial.

1. Use the code in `reviews/templates/book_list_item.html` to form the body of a template partial definition in `reviews/templates/reviews/book_list.html`. Call the template partial `book_list_item` and include the `load link_worldcat_filter` directive and the contents of the `li` tag.
2. Now, in the content block in `reviews/templates/reviews/book_list.html`, replace the call to the inclusion tag `book_list_item` with a call to the template partial `book_list_item` using a Django template directive, `{% partial "book_list_item" %}`.
3. We can now delete the `reviews/templates/book_list_item.html` file as well as the inclusion tag, `book_list_item`, that is defined in `reviews/templatetags/inclusion_tag.py`.

The book list review will continue to render as it did in *Exercise 11.03*, but we have simplified the coding of the template.

With this, we now have the foundations on which we can build highly complex template filters, custom tags, and template partials that can be helpful in the development of the projects we want to work on.

Now, let's take a look at Django views and dive into a new territory of class-based views provided by Django to help us leverage the power of object-oriented programming that allows the reuse of code for the rendering of a view.

Django views

To recall, a view in Django is a piece of Python code that allows a request to be taken in, performs an action based on the request, and then returns a response to the user, hence forming an important part of Django applications.

Inside Django, we have the option of building our views by following two different methodologies, one of which we have already seen in the preceding examples and is known as

활동 11.02 – 템플릿 부분으로 책 목록 항목 렌더링하기

이 활동에서는 연습 11.03에서 정의된 포함 태그를 템플릿 부분으로 변환할 것입니다.

1. `reviews/templates/book_list_item.html`의 코드를 사용하여 `reviews/templates/reviews/book_list.html`에서 템플릿 부분 정의의 본문을 형성합니다. 템플릿 부분의 이름을 `book_list_item`으로 하고 `load link_worldcat_filter` 지시문과 `li` 태그의 내용을 포함합니다.
2. 이제 `reviews/templates/reviews/book_list.html`의 콘텐츠 블록에서 포함 태그 `book_list_item`에 대한 호출을 Django 템플릿 지시문 `{% partial "book_list_item" %}`를 사용하여 템플릿 부분 `book_list_item`에 대한 호출로 교체합니다.
3. 이제 `reviews/templates/book_list_item.html` 파일과 `reviews/templatetags/inclusion_tag.py`에 정의된 포함 태그 `book_list_item`을 삭제할 수 있습니다.

책 목록 리뷰는 연습 11.03에서와 같이 계속 렌더링되지만, 템플릿 코딩이 간소화되었습니다.

이로써 우리는 복잡한 템플릿 필터, 사용자 정의 태그 및 템플릿 부분을 구축할 수 있는 기초를 마련했습니다. 이는 우리가 작업하고자 하는 프로젝트의 개발에 도움이 될 수 있습니다.

이제 Django 뷰를 살펴보고 Django가 제공하는 클래스 기반 뷰의 새로운 영역으로 들어가 보겠습니다. 이는 객체 지향 프로그래밍의 힘을 활용하여 뷰 렌더링을 위한 코드 재사용을 가능하게 합니다.

장고 뷰

상기하자면, 장고에서 뷰는 요청을 받아들이고, 요청에 따라 작업을 수행한 다음 사용자에게 응답을 반환하는 파이썬 코드의 일부로, 장고 애플리케이션의 중요한 부분을 형성합니다.

장고 내부에서 우리는 두 가지 다른 방법론을 따르면서 뷰를 구축할 수 있는 옵션이 있으며, 그 중 하나는 이전 예제에서 이미 본 것으로 알려져 있습니다.

function-based views, while the other one, which we will be covering soon, is known as class-based views:

- **Function-based views (FBVs):** FBVs inside Django are nothing more than generic Python functions that are supposed to take an `HttpRequest` type object as their first positional parameter and return an `HttpResponse` type object, which corresponds to the action the view wants to perform once the request is processed by it. In the preceding exercise, `index()` and `greeting_view()` were examples of FBVs.
- **Class-based views (CBVs):** CBVs are views that closely adhere to the Python object-oriented principles and allow mapping of the view calls in a class-based representation. These views are specialized in nature, and a given type of CBV performs a specific operation. The benefits that CBVs provide include easy extensibility of the view and the reuse of code, which may turn out to be a complex task with FBVs.

Now, with the basic definitions clear and with knowledge of FBVs already in our arsenal, let's look at CBVs and see what they have in store for us.

Class-based views

Django provides different ways in which developers can write views for their applications. One way is to map a Python function to act as a view function to create FBVs. Another way of creating views is to use Python object instances (based on top of Python classes). These are known as CBVs. An important question that arises is, what is the need for a CBV when we can already create views using the FBV approach?

When creating FBVs, the idea is that, at times, we may be replicating the same logic again and again – for example, the processing of certain fields or logic for handling certain request types. Although it is completely possible to create logically separate functions that handle a particular piece of logic, the task becomes difficult to manage as the complexity of the application increases.

This is where CBVs come in handy, as they abstract away implementation of the common repetitive code that we need to write to handle certain tasks, such as the rendering of templates, while also making it easy to reuse pieces of code through the use of inheritance and mix-ins. For example, the following code snippet shows the implementation of a CBV:

```
from django.http import HttpResponse
from django.views import View

class IndexView(View):
```

함수 기반 뷰, 그리고 우리가 곧 다룰 다른 하나는 클래스 기반 뷰로 알려져 있습니다:

- **함수 기반 뷰(FBVs):** Django 내의 FBVs는 `HttpRequest` 유형 객체를 첫 번째 위치 매개변수로 받아들이고, 요청이 처리된 후 뷰가 수행하고자 하는 작업에 해당하는 `HttpResponse` 유형 객체를 반환해야 하는 일반적인 Python 함수에 불과합니다. 이전 연습에서 `index()`와 `greeting_view()`는 FBVs의 예였습니다.
- **클래스 기반 뷰(CBVs):** CBVs는 Python 객체 지향 원칙을 밀접하게 준수하며, 클래스 기반 표현에서 뷰 호출을 매핑할 수 있게 해주는 뷰입니다. 이러한 뷰는 본질적으로 전문화되어 있으며, 특정 유형의 CBV는 특정 작업을 수행합니다. CBVs가 제공하는 이점에는 뷰의 용이한 확장성과 FBVs로는 복잡한 작업이 될 수 있는 코드 재사용이 포함됩니다.

이제 기본 정의가 명확해지고 FBVs에 대한 지식이 우리 무기고에 있으므로, CBVs를 살펴보고 그들이 우리에게 무엇을 제공하는지 알아보시다.

클래스 기반 뷰

Django는 개발자가 애플리케이션을 위한 뷰를 작성할 수 있는 다양한 방법을 제공합니다. 한 가지 방법은 Python 함수를 뷰 함수로 매핑하여 FBVs를 생성하는 것입니다. 뷰를 생성하는 또 다른 방법은 Python 클래스 위에 기반한 Python 객체 인스턴스를 사용하는 것입니다. 이를 CBVs라고 합니다. CBV가 필요한 이유는 무엇일까요? 이미 FBV 접근 방식을 사용하여 뷰를 생성할 수 있는데 말입니다.

FBVs를 생성할 때, 때때로 동일한 논리를 반복하고 있을 수 있다는 아이디어가 있습니다. 예를 들어, 특정 필드의 처리 또는 특정 요청 유형을 처리하는 논리입니다. 특정 논리를 처리하는 논리적으로 분리된 함수를 만드는 것은 완전히 가능하지만, 애플리케이션의 복잡성이 증가함에 따라 관리하기 어려워집니다.

여기서 CBVs가 유용하게 사용됩니다. CBVs는 특정 작업을 처리하기 위해 작성해야 하는 일반적인 반복 코드를 추상화하여 템플릿 렌더링과 같은 작업을 처리하는 데 필요한 코드를 쉽게 재사용할 수 있도록 상속 및 믹스인을 통해 코드 조각을 재사용할 수 있게 해줍니다. 예를 들어, 다음 코드 스니펫은 CBV의 구현을 보여줍니다:

```
from django.http import HttpResponse
from django.views import View

class IndexView(View):
```

```
def get(self, request):  
    return HttpResponse("Hey there!")
```

In the preceding example, we built a simple CBV by inheriting from the built-in `View` class that Django provides.

Using these CBVs is also quite easy. For example, let's say we wanted to map `IndexView` to a URL endpoint in our application. In this case, all we need to do is to add the following line to our `urlpatterns` list inside the `urls.py` file of the application:

```
urlpatterns = [  
    path('my_path', IndexView.as_view(), name='index_view')  
]
```

In this, as we can observe, we used the `as_view()` method of the CBV we created. Every CBV implements the `as_view()` method, which allows the `View` class to be mapped to a URL endpoint by returning the instance of the view controller from the view class.

Django provides a couple of built-in CBVs that provide the implementation of a lot of common tasks, such as how to render a template or how to process a particular request. The built-in CBVs help avoid rewriting code from scratch when handling basic functionality, thereby enabling the reusability of code. Some of these in-built views include the following:

- **View:** This is the base class for all CBVs available in Django that allows a custom CBV to be written with all the features provided and overridable. A user can implement their own definitions for different HTTP Request methods, such as GET, POST, PUT, and DELETE, and the view will automatically delegate the call to the method responsible for handling the request based on the type of request received.
- **TemplateView:** This is a view that can be used to render a template based on the parameters for the template data provided in the URL of the call. This allows developers to easily render a template without writing any logic related to how the rendering should be handled.
- **RedirectView:** This is a view that can automatically redirect a user to the correct resource based on the request they have made.
- **DetailView:** This is a view that is mapped to a Django model and can be used to render the data obtained from the model using a template of choice.

The preceding views are just some of the built-in views that Django provides by default, and we will cover more of them as we move through the chapter.

```
def get(self, request):
    return HttpResponse("안녕하세요!")
```

앞서의 예제에서는 Django가 제공하는 내장 View 클래스를 상속받아 간단한 CBV를 만들었습니다.

이 CBV를 사용하는 것도 매우 쉽습니다. 예를 들어, IndexView를 애플리케이션의 URL 엔드포인트에 매핑하고 싶다고 가정해 보겠습니다. 이 경우, 우리가 해야 할 일은 애플리케이션의 `urls.py` 파일 안에 있는 `urlpatterns` 목록에 다음 줄을 추가하는 것입니다:

```
urlpatterns = [
    path('my_path', IndexView.as_view(), name='index_view')
]
```

여기서 볼 수 있듯이, 우리는 우리가 만든 CBV의 `as_view()` 메서드를 사용했습니다. 모든 CBV는 `as_view()` 메서드를 구현하여 View 클래스가 뷰 클래스로부터 뷰 컨트롤러의 인스턴스를 반환함으로써 URL 엔드포인트에 매핑될 수 있도록 합니다.

장고는 템플릿을 렌더링하는 방법이나 특정 요청을 처리하는 방법과 같은 많은 일반 작업의 구현을 제공하는 몇 가지 내장 CBV를 제공합니다. 내장 CBV는 기본 기능을 처리할 때 코드를 처음부터 다시 작성하는 것을 피할 수 있도록 도와주며, 코드의 재사용성을 가능하게 합니다. 이러한 내장 뷰 중 일부는 다음과 같습니다:

- **View:** 이것은 Django에서 사용할 수 있는 모든 CBV의 기본 클래스이며, 모든 제공된 기능을 사용하여 사용자 정의 CBV를 작성할 수 있도록 합니다. 사용자는 GET, POST, PUT 및 DELETE와 같은 다양한 HTTP 요청 메서드에 대한 자신의 정의를 구현할 수 있으며, 뷰는 수신된 요청의 유형에 따라 요청을 처리하는 메서드에 자동으로 호출을 위임합니다.
- **TemplateView:** 이것은 호출의 URL에 제공된 템플릿 데이터의 매개변수를 기반으로 템플릿을 렌더링하는 데 사용할 수 있는 뷰입니다. 이를 통해 개발자는 렌더링이 어떻게 처리되어야 하는지에 대한 로직을 작성하지 않고도 쉽게 템플릿을 렌더링할 수 있습니다.
- **RedirectView:** 이것은 사용자가 요청한 내용을 기반으로 올바른 리소스로 자동으로 리디렉션할 수 있는 뷰입니다.
- **DetailView:** 이것은 Django 모델에 매핑된 뷰로, 선택한 템플릿을 사용하여 모델에서 얻은 데이터를 렌더링하는 데 사용할 수 있습니다.

앞서 언급한 뷰는 Django가 기본적으로 제공하는 내장 뷰 중 일부에 불과하며, 장을 진행하면서 더 많은 뷰를 다룰 것입니다.

Now, to better understand how CBVs work inside Django, let's try to build our first CBV.

Exercise 11.04 – Creating a book catalog using a CBV

In this exercise, we will create a class-based form view that will help build a book catalog. This catalog will consist of the name of the book and the name of the author of the book:

1. To get started, create a new application inside the bookr project and name it `book_management`. This can be done by simply running the following command:

```
python manage.py startapp book_management
```

2. Now, before building the book catalog, you first need to define a Django model that will help you store the records inside the database. To do this, open the `models.py` file under the `book_management` app you just created and define a new model named `Book`, as shown here:

```
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=255)
    author = models.CharField(max_length=50)
```

The model contains two fields, the name of the book and the name of the author.

3. Once all the preceding steps are complete, add your `book_management` application to the `INSTALLED_APPS` list such that Django can discover it and you can use your model properly. For this, open the `settings.py` file under the `bookr` directory and add the following code at the final position in the `INSTALLED_APPS` section:

```
INSTALLED_APPS = [
    ...,
    'book_management'
]
```

4. After the changes have been made, the `settings.py` file should look like in the `Chapter11` folder in this book's GitHub repo.

이제 CBV가 Django 내부에서 어떻게 작동하는지 더 잘 이해하기 위해, 첫 번째 CBV를 만들어 보겠습니다.

연습 11.04 – CBV를 사용하여 도서 카탈로그 만들기

이 연습에서는 도서 카탈로그를 만드는 데 도움이 되는 클래스 기반 폼 뷰를 생성할 것입니다. 이 카탈로그는 책의 이름과 저자의 이름으로 구성됩니다:

1. 시작하려면, bookr 프로젝트 내에 새로운 애플리케이션을 생성하고 이름을 book_management로 지정합니다. 이는 다음 명령어를 실행하여 간단히 수행할 수 있습니다:

```
python manage.py startapp book_management
```

2. 이제 도서 카탈로그를 만들기 전에, 먼저 데이터베이스에 레코드를 저장하는 데 도움이 되는 Django 모델을 정의해야 합니다. 이를 위해 방금 생성한 book_management 앱의 models.py 파일을 열고 다음과 같이 Book이라는 새 모델을 정의합니다:

```
from django.db import models

class Book(models.Model):
    name = models.CharField(max_length=255)
    author = models.CharField(max_length=50)
```

모델에는 책의 이름과 저자의 이름이라는 두 개의 필드가 포함되어 있습니다.

3. 모든 이전 단계가 완료되면, book_management 애플리케이션을 INSTALLED_APPS 목록에 추가하여 Django가 이를 발견하고 모델을 제대로 사용할 수 있도록 합니다. 이를 위해 bookr 디렉토리 아래의 settings.py 파일을 열고 INSTALLED_APPS 섹션의 마지막 위치에 다음 코드를 추가합니다:

```
INSTALLED_APPS = [
    ...,
    'book_management'
]
```

4. 변경이 완료되면, settings.py 파일은 이 책의 GitHub 저장소의 Chapter11 폴더에 있는 것처럼 보여야 합니다.

5. Migrate your model to the database by running the following two commands. These will first create a Django migrations file and then create a table in your database:

```
python manage.py makemigrations
python manage.py migrate
```

6. Now, with the database model in place, let's create a new form that we will use to capture information pertaining to the books, such as the book title, author, and ISBN. For this, create a new file named `forms.py` under the `book_management` directory and add the following code inside it:

```
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['name', 'author']
```

In the preceding code snippet, we first imported Django's `forms` module, which will allow us to easily create forms and also provide the form's rendering capability. The next line imports the model that will store the data for the form:

```
from django import forms
from .models import Book
```

In the next line, we created a new class named `BookForm`, which inherits from `ModelForm`. This is nothing but a class that maps the fields of the model to the form. To successfully achieve this mapping between the model and the form, we defined a new subclass named `Meta` under the `BookForm` class and set the attribute `model` to point to the `Book` model and the attribute `fields` to the list of fields that you want to display in the form:

```
class Meta:
    model = Book
    fields = ['name', 'author']
```

5. 다음 두 명령어를 실행하여 모델 `idx_cd4ca9a8`을 데이터베이스로 마이그레이션합니다. 이 명령어는 먼저 Django 마이그레이션 파일을 생성하고, 그 다음 데이터베이스에 테이블을 생성합니다:

```
python manage.py makemigrations
python manage.py migrate
```

6. 이제 데이터베이스 모델이 준비되었으므로, 책 제목, 저자 및 ISBN과 같은 책에 대한 정보를 수집하는 데 사용할 새 양식을 만들어 보겠습니다. 이를 위해 `book_management` 디렉토리 아래에 `forms.py`라는 새 파일을 만들고 그 안에 다음 코드를 추가합니다:

```
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model = Book
        fields = ['name', 'author']
```

위의 코드 조각에서는 먼저 Django의 `forms` 모듈을 가져왔습니다. 이 모듈은 우리가 쉽게 양식을 만들 수 있게 해주고 양식의 렌더링 기능도 제공합니다. 다음 줄에서는 양식의 데이터를 저장할 모델을 가져옵니다:

```
from django import forms
from .models import Book
```

다음 줄에서는 `ModelForm`에서 상속받는 새로운 클래스 `BookForm`을 만들었습니다. 이것은 모델의 필드를 양식에 매핑하는 클래스입니다. 모델과 양식 간의 매핑을 성공적으로 달성하기 위해, `BookForm` 클래스 아래에 `Meta`라는 새로운 하위 클래스를 정의하고 속성 `model`을 `Book` 모델을 가리키도록 설정하고 속성 `fields`를 양식에 표시할 필드 목록으로 설정했습니다:

```
class Meta:
    model = Book
    fields = ['name', 'author']
```

This allows `ModelForm` to render the correct form of HTML when expected to do so. The `ModelForm` class provides a built-in `Form.save()` method, which, when used, writes the data in the form to the database and helps avoid having to write redundant code.

7. Now that you have both your model and the form ready, go ahead and implement a view that will render the form and accept input from the user. For this, open `views.py` under the `book_management` directory and add the following lines of code to the file:

```
from django.http import HttpResponseRedirect
from django.views.generic.edit import FormView
from django.views import View
from .forms import BookForm

class BookRecordFormView(FormView):
    template_name = 'book_form.html'
    form_class = BookForm
    success_url = '/book_management/entry_success'

    def form_valid(self, form):
        form.save()
        return FormView.form_valid(self, form)

class FormSuccessView(View):
    def get(self, request, *args, **kwargs):
        return HttpResponseRedirect(
            "Book record saved successfully")
```

In the preceding code snippet, we created two major views, `BookRecordFormView`, which is also responsible for rendering the book catalog entry form, and `FormSuccessView`, which you will use to render the success message if the form data is saved successfully. Let's now look at both views individually and understand what we are doing.

First, we created a new view named the `BookRecordFormView` CBV, which inherits from `FormView`:

```
class BookRecordFormView(FormView)
```

The `FormView` class allows us to easily create views that deal with forms. To this class, we need to provide certain parameters, such as the name of the template it will render

이것은 ModelForm이 예상할 때 올바른 HTML 양식을 렌더링할 수 있도록 합니다. TheModelForm 클래스는 내장된 Form.save() 메서드를 제공하며, 이를 사용하면 양식의 데이터를 데이터베이스에 기록하고 중복 코드를 작성하지 않도록 도와줍니다.

7. 이제 모델과 양식이 모두 준비되었으므로, 양식을 렌더링하고 사용자로부터 입력을 받을 수 있는 뷰를 구현하세요. 이를 위해 book_management 디렉토리 아래의 views.py를 열고 파일에 다음 코드 줄을 추가하세요:

```
from django.http import HttpResponseRedirect
from django.views.generic.edit import FormView
from django.views import View
from .forms import BookForm

class BookRecordFormView(FormView):
    template_name = 'book_form.html'
    form_class = BookForm
    success_url = '/book_management/entry_success'

    def form_valid(self, form):
        form.save()
        return FormView.form_valid(self, form)

class FormSuccessView(View):
    def get(self, request, *args, **kwargs):
        return HttpResponseRedirect(
            "책 기록이 성공적으로 저장되었습니다")
```

앞서의 코드 조각에서는 두 개의 주요 뷰인 BookRecordFormView와 FormSuccessView를 생성했습니다. BookRecordFormView는 책 카탈로그 항목 양식을 렌더링하는 역할도 하며, FormSuccessView는 양식 데이터가 성공적으로 저장되면 성공 메시지를 렌더링하는 데 사용됩니다. 이제 두 뷰를 개별적으로 살펴보고 우리가 무엇을 하고 있는지 이해해 봅시다.

먼저, FormView에서 상속받는 BookRecordFormView라는 새 뷰를 생성했습니다:

```
class BookRecordFormView(FormView)
```

FormView 클래스는 양식과 관련된 뷰를 쉽게 생성할 수 있도록 해줍니다. 이 클래스에는 렌더링할 템플릿의 이름과 같은 특정 매개변수를 제공해야 합니다.

to show the form, the form class that it should use to render the form, and the success URL to redirect to when the form is processed successfully:

```
template_name = 'book_form.html'
form_class = BookForm
success_url = '/book_management/entry_success'
```

The `FormView` class also provides a `form_valid()` method, which is called when the form successfully finishes the validation. Inside the `form_valid()` method, we can decide what we want to do. For our use case, when the form validation completes successfully, we first call the `form.save()` method, which persists the data for our form into the database, and then call the base class `form_valid()` method, which will cause the form view to redirect to the successful URL in the event that form validation was a success:

```
def form_valid(self, form):
    form.save()
    return FormView.form_valid(self, form)
```

Note

The `form_valid()` method should always return an `HttpResponse` object.

This completes the implementation of `BookRecordFormView`. The next thing we have to do is to build a view named `FormSuccessView`, which we will use to render the success message once the data is saved successfully for the book record form we just created. This is done by creating a new view class named `FormSuccessView`, which inherits from the `View` base class of Django CBVs:

```
class FormSuccessView(View)
```

Inside this class, we override the `get()` method, which will be rendered when the form is saved successfully. Inside the `get()` method, we render a simple success message by returning a new `HttpResponse`:

```
def get(self, request, *args, **kwargs):
    return HttpResponse("Book record saved successfully")
```

양식을 표시하고, 양식을 렌더링하는 데 사용할 양식 클래스와 양식이 성공적으로 처리되었을 때 리디렉션할 successURL을 설정합니다:

```
template_name = 'book_form.html'
form_class = BookForm
success_url = '/book_management/entry_success'
```

FormView 클래스는 또한 form_valid() 메서드를 제공하는데, 이는 theformidx_68a7f98a가 유효성 검사를 성공적으로 마쳤을 때 호출됩니다 idx_ffd24fa4. form_valid() 메서드 내부에서 우리는 무엇을 할지 결정할 수 있습니다. 우리의 사용 사례에서는 양식 유효성 검사가 성공적으로 완료되면 먼저 form.save() 메서드를 호출하여 양식의 데이터를 데이터베이스에 저장하고, 그 다음에 기본 클래스의 form_valid() 메서드를 호출하여 양식 유효성 검사가 성공적일 경우 양식 뷰가 성공적인 URL로 리디렉션되도록 합니다:

```
def form_valid(self, form):
    form.save()
    return FormView.form_valid(self, form)
```

메모

form_valid() 메서드는 항상 HttpResponseRedirect 객체를 반환해야 합니다.

이것으로 BookRecordFormView의 구현이 완료되었습니다. 다음으로 해야 할 일은 FormSuccessView라는 뷰를 만드는 것입니다. 이 뷰는 방금 생성한 책 기록 양식의 데이터가 성공적으로 저장되면 성공 메시지를 렌더링하는 데 사용됩니다. 이는 FormSuccessView라는 새로운 뷰 클래스를 생성하여 Django CBVs의 View 기본 클래스를 상속받아 수행됩니다:

```
class FormSuccessView(View)
```

이 클래스 내부에서 우리는 get() 메서드를 재정의합니다. 이 메서드는 양식이 성공적으로 저장될 때 렌더링됩니다. get() 메서드 내부에서 우리는 새로운 HttpResponseRedirect를 반환하여 간단한 성공 메시지를 렌더링합니다:

```
def get(self, request, *args, **kwargs):
    return HttpResponseRedirect("책 기록이 성공적으로 저장되었습니다")
```

8. Now, create a template that will be used to render the form. For this, create a new `templates` folder under the `book_management` directory and a new file named `book_form.html`. Add the following lines of code inside the file:

```
<html>
<head>
  <title>Book Record Insertion</title>
</head>
<body>
  <form method="POST">
    {% csrf_token %}
    {{ form.as_p }}
    <input type="submit" value="Save record" />
  </form>
</body>
</html>
```

9. With the CBVs now built, go ahead and map them to URLs so that you can start using them to add new book records. To do this, create a new file named `urls.py` under the `book_management` directory and add the following code to it:

```
from django.urls import path

from .views import BookRecordFormView, FormSuccessView

urlpatterns = [
    path('new_book_record',
          BookRecordFormView.as_view(),
          name='book_record_form'),
    path('entry_success', FormSuccessView.as_view(),
          name='form_success'),
]
```

Most parts of the preceding snippet are similar to the ones you wrote earlier, but there is one thing different in the way we map our CBVs under the URL patterns. When using CBVs, instead of directly adding the function name, we use the class name and its `as_view` method, which maps the class object to the view. For example, to map `BookRecordFormView` as a view, we will use `BookRecordFormView.as_view()`.

8. 이제 양식을 렌더링하는 데 사용할 `templateidx_51c4baac`를 생성하십시오. 이를 위해 `book_management` 디렉토리 아래에 새로운 `templates` 폴더와 `book_form.html`이라는 새 파일을 만드십시오. 파일 안에 다음 코드 줄을 추가하십시오:

```
<html>
<head>
    <title>도서 기록 삽입</title>
</head>
<body>
    <form method="POST">
        {% csrf_token %}
        {{ form.as_p }}
        <input type="submit" value="기록 저장" />
    </form>
</body>
</html>
```

9. `CBVsidx_f6de6885`가 이제 구축되었으므로, 새로운 책 기록을 추가하기 위해 사용할 수 있도록 `URLsidx_9c9c6a81`에 매핑하세요. 이를 위해 `book_management` 디렉토리 아래에 `urls.py`라는 새 파일을 만들고 다음 코드를 추가하세요:

```
from django.urls import path

from .views import BookRecordFormView, FormSuccessView

urlpatterns = [
    path('new_book_record',
        BookRecordFormView.as_view(),
        name='book_record_form'),
    path('entry_success', FormSuccessView.as_view(),
        name='form_success'),
]
```

앞서의 코드 조각의 대부분은 당신이 이전에 작성한 것과 유사하지만, URL 패턴 아래에서 CBV를 매핑하는 방식에서 한 가지 다른 점이 있습니다. CBV를 사용할 때는 함수 이름을 직접 추가하는 대신 클래스 이름과 그 `as_view` 메서드를 사용하여 클래스 객체를 뷰에 매핑합니다. 예를 들어, `BookRecordFormView`를 뷰로 매핑하려면 `BookRecordFormView.as_view()`를 사용합니다.

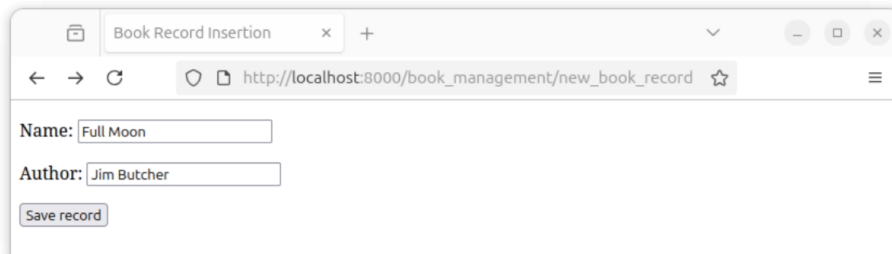
10. With the URLs added to our `urls.py` file, the next thing is to add the application URL mapping to the `bookr` project. To do this, open the `urls.py` file under the `bookr` application and add the following line to `urlpatterns`:

```
urlpatterns = [  
    path('book_management/',  
        include('book_management.urls')),  
    ...  
]
```

11. Now, start your development server by running the following command:

```
python manage.py runserver localhost:8080
```

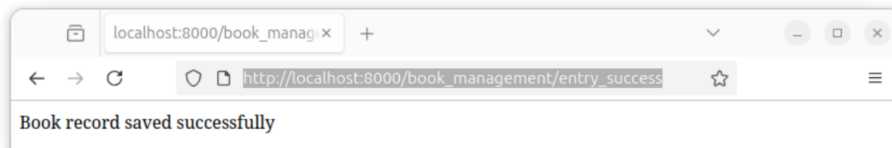
12. Then, visit `http://localhost:8080/book_management/new_book_record`.
If everything works fine, you will see a page like the one shown here:



The screenshot shows a web browser window with the title 'Book Record Insertion'. The address bar displays 'http://localhost:8000/book_management/new_book_record'. The form contains two text input fields: 'Name:' with the value 'Full Moon' and 'Author:' with the value 'Jim Butcher'. Below these fields is a 'Save record' button.

Figure 11.4 – The view for adding a new book to the database

Upon clicking **Save record**, your record will be written to the database, and the following page will appear:



The screenshot shows a web browser window with the title 'localhost:8000/book_manag x'. The address bar displays 'http://localhost:8000/book_management/entry_success'. The page content shows the message 'Book record saved successfully'.

Figure 11.5 – The template is rendered when the record is successfully inserted

With this, we have created our own CBV, which allows us to save records for new books. With our knowledge of CBVs in tow, let's now look at how we can perform **create, read, update, and delete (CRUD)** operations with the help of CBVs.

10. urls.py 파일에 추가된 idx_edd8d2a4와 함께, 다음으로 할 일은 idx_0f09b6da 애플리케이션 URL bookr 프로젝트에 매핑하는 것입니다. 이를 위해 bookr 애플리케이션 아래의 urls.py 파일을 열고 urlpatterns에 다음 줄을 추가하세요:

```
urlpatterns = [
    path('book_management/', include
        ('book_management.urls'))
    ...
]
```

11. 이제 다음 명령어를 실행하여 개발 서버를 시작하세요:

```
python manage.py runserver localhost:8080
```

12. 그런 다음, http://localhost:8080/book_management/new_book_record를 방문하세요. 모든 것이 잘 작동하면, 여기 표시된 것과 같은 페이지를 볼 수 있습니다:

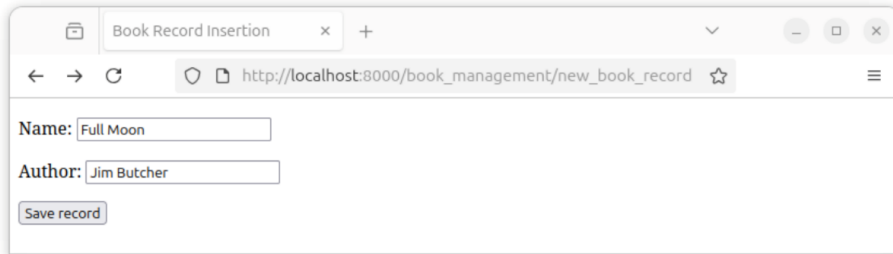


그림 11.4 - 데이터베이스에 새 책을 추가하는 뷰

저장 기록을 클릭하면, 귀하의 기록이 데이터베이스에 작성되고 다음 페이지가 나타납니다:

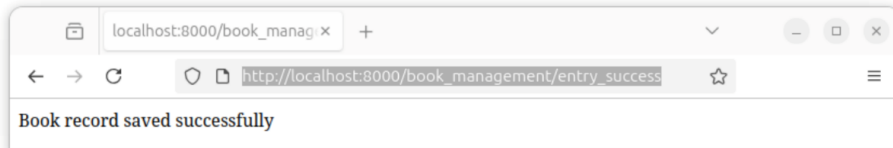


그림 11.5 - 기록이 성공적으로 삽입되면 템플릿이 렌더링됩니다

이로써 우리는 새 책에 대한 기록을 저장할 수 있는 자체 CBV를 생성했습니다. CBV에 대한 지식을 바탕으로, 이제 CBV의 도움으로 생성, 읽기, 업데이트 및 삭제(CRUD) 작업을 수행하는 방법을 살펴보겠습니다.